

目录

第一章 介绍.....	1
1.1 关于 PROTEUS VSM.....	1
1.2 关于本文档.....	1
第二章 使用指南.....	1
2.1 交互式仿真/调试.....	1
介绍	1
2.2 工程设置.....	2
编译器配置	3
运行仿真	5
编写固件	7
活动弹窗	10
基本调试	13
监视窗口	16
硬件断点	18
交互式测量	20
图表测量	23
诊断信息	28
2.3 图表仿真.....	30
介绍	30
开始	31
激励源	32
探针	33
图表	34
仿真	37
测量	38
电流探针	40
频率分析	42
扫描变量分析	44
噪声分析	46
2.4 新建工程.....	47
介绍	47
新建工程向导	48
新建工程	50
绘制电路图	51
编程	54
仿真与调试	58
活动弹窗	61

2.5 导入旧工程	63
介绍	63
导入原理图	63
创建 VSM Studio 项目	64
导入源文件	66
工程设置	67
编译并运行	68
第三章 交互式仿真	70
3.1 基本技能	70
仿真控制按钮	70
指示及执行	70
建立一个交互式仿真	71
3.2 测量	73
概况	73
动画效果	73
指示和显示参数信息	75
虚拟仪器	76
3.3 动态步进控制	76
概况	76
仿真帧率	76
帧步进时间	77
单步时间	77
3.4 亮点和提示	77
电路时间尺度	77
电压范围	78
接地	78
高阻抗点	78
第四章 虚拟仪器	79
4.1 虚拟示波器	79
总述	79
4.2 使用示波器	80
使用模式	80
触发	80
输入耦合	81
测量	81
打印	81
色彩	81
4.3 逻辑分析仪	82
概况	82
缩放和全局显示	83

使用逻辑分析仪	83
测量	84
打印	85
色彩	85
4.4 虚拟信号发生器	85
总述	85
使用信号发生器	86
使用 AM、FM 调制输入	87
4.5 模式发生器	87
总述	87
使用模式发生器	88
模式发生器的引脚说明	90
时钟模式	91
触发模式	91
外部保持	93
附加功能	94
4.6 定时/计数器	95
概述	95
使用定时模式	96
使用频率模式	97
使用计数模式	98
4.7 虚拟终端	99
概述	99
使用虚拟终端	99
MAX232 模型	101
4.8 I2C 调试器	102
概述	102
原理图部件	102
I2C 调试窗口	102
使用 I2C 调试器	104
模型特性	105
语法参考	105
4.9 SPI 调试器	106
简述	106
原理图部件	106
SPI 调试窗口	107
模型的特性	108
使用 SPI 调试器	109
4.10 电压表和电流表	110
第五章 微处理器仿真	111

5.1 介绍	111
概述	111
5.2 VSM Studio	111
介绍	111
指南	112
受支持的编译器	119
Arduino 开发	125
固件开发	130
测试和调试	135
图标参考	141
5.3 基于 Proteus VSM 源代码调试	142
概述	142
源代码窗口	142
支持的仿真文件格式	143
单步执行	145
使用断点	145
变量窗口	146
多个 CPU 仿真	147
5.4 源代码控制系统	148
概述	148
创建源代码文件	148
在源代码上工作	148
安装第三方目标代码生成工具	149
使用 MAKE 程序	150
使用第三方源代码编辑器	150
使用第三方 IDE	150
调试工具	151
5.5 观测窗口	156
给观测窗口增加项目	156
观测点条件	157
5.6 断点触发	158
建立硬件断点	158
电压断点触发 (RTVBREAK)	159
电流断点触发 (RTIBREAK)	159
数字断点触发 (RTDBREAK)	159
电压监测器 (RTVMON)	159
电流监测器 (RTIMON)	160
第六章 图表仿真	161
6.1 介绍	161
6.2 建立一个基于图表的仿真	161

概况	161
绘制电路	161
放置探针和发生器.....	161
放置图表	162
在图表中添加轨迹.....	163
仿真过程	163
6.3 图表对象.....	164
概况	164
当前图表	164
放置图表	164
编辑图表	164
在图表中添加轨迹.....	164
添加曲线命令对话框.....	165
图表轨迹编辑	166
改变轨迹的顺序或颜色.....	166
手工 Y 轴设置.....	166
6.4 仿真过程.....	167
命令驱动仿真	167
执行仿真	167
仿真开始后发生的事件.....	168
第七章 分析类型.....	170
7.1 介绍.....	170
7.2 模拟信号瞬态分析	171
概述	171
计算方法	171
使用模拟图表	171
定义模拟轨迹表达式.....	173
7.3 数字信号瞬态分析	174
概述	174
计算方法	174
使用数字图表	175
数字数据的显示	175
7.4 混合模式瞬态分析	176
概述	176
计算方法	176
使用混合图表	177
7.5 频率分析.....	177
概述	177
计算方法	177
使用频率图表	178

7.6 DC 扫描分析	180
概述	180
计算方法	180
使用 DC 扫描分析图表	180
7.7 AC 扫描分析	182
概述	182
计算方法	182
使用 AC 扫描图表	182
7.8 DC 转移曲线分析	183
概述	183
计算方法	183
使用 AC 扫描图表	183
7.9 噪声分析	185
概述	185
计算方法	185
使用噪声分析图表	185
7.10 失真分析	186
概述	186
计算方法	186
使用失真分析图表	187
7.11 傅立叶分析	187
概述	187
分析方法	187
使用傅立叶分析图表	187
7.12 音频分析	188
概述	188
分析方法	188
使用音频分析图表	188
7.13 交互式分析	189
概述	189
计算方法	189
使用交互式分析图表	189
7.14 数字一致性分析	190
概述	190
分析方法	190
使用数字一致性分析图表	190
第八章 激励源和探针	194
8.1 激励源	194
概述	194
放置激励源	194

编辑激励源	195
直流激励源	195
正弦激励源	195
脉冲激励源	196
指数激励源	196
单频率调频激励源	197
分段线性激励源	197
文件激励源	198
音频激励源	199
数字激励源	199
8.2 探针	200
概述	200
探针放置	200
探针设置	200
第九章 脚本化信号源	202
9.1 前言	202
9.2 创建、编辑及调试 EasyHDL 脚本	202
9.3 程序结构	206
9.4 语法图解析	207
9.5 使用 EASYHDL 规定模拟信号	208
声明输出	208
程序流	208
规定输出转换	210
9.6 使用 EASYHDL 规定数字信号	211
声明输出	211
程序流	211
规定输出转换	212
9.7 变量	213
变量声明	213
数据类型	214
字符串类型	214
BOOL 及 PIN 类型	215
节点类型	216
指定变量	216
9.8 表达式	218
操作符	218
值	219
函数	220
9.9 系统常量及变量	221
数据常量	221

事件 ID 常量	221
网络状态常量	221
PIN 及 BOOL Pin 活动常量	221
TDSCALE 系统变量	222
RANDOM 系统变量	222
REALTIME 及 EVTTIME 系统变量	222
9.10 命令参考	222
ABORT 命令	222
ALIAS 命令	223
BREAK 命令	224
CALLBACK 命令	224
CONTINUE 命令	225
DATA 命令	226
END 命令	226
FOR...TO...STEP...NEXT 命令	227
GOTO 命令	228
IF...ELIF...ELSE...ENDIF 命令	228
MAP ON...ENDMAP 命令	229
ON BOOT...ENDON 命令	231
ON EVENT...ENDON 命令	231
ON TIMER...ENDON 命令	232
PRINT 命令	233
EAD 命令	234
REM 命令	234
REPEAT...UNTIL 命令	235
RESTORE 命令	236
SEED 命令	236
SLEEP 命令	237
SWITCH...ENDSW 命令	238
TIMER 命令	239
WAIT 命令	240
WHILE...WEND 命令	240
第十章 SPICE 模型的应用	242
10.1 前言	242
10.2 使用 SPICE 子电路 (SUBCKT 定义)	242
10.3 使用一个 SPICE 模型 (模型卡)	244
10.4 SPICE 模型库	245
10.5 *SPICE 脚本	246
10.6 模型仿真失败的处理	246
第十一章 进阶	247

11.1 温度模型	247
11.2 固化模型数据	247
11.3 地和电源范围	247
为什么需要一个地	247
电源	249
11.4 初始条件	249
前言	249
设定一个网络的初始条件	250
设定器件的初始条件	250
NS (NODESET)属性	251
PRECHARGE 属性	251
11.5 数字仿真范例	251
九态逻辑	251
不确定状态	252
浮空输入行为	252
毛刺的处理	253
11.6 混合模式接口模型 (ITFMOD)	255
概述	255
使用 ITFMOD 属性	256
11.7 录音机和电路分区	257
概述	257
单个部分的仿真	257
录音机对象	257
录音机模式	258
11.8 仿真控制属性	259
概述	259
误差属性	259
MOSFET 属性	259
迭代属性	259
温度属性	260
数字仿真器属性	260
11.9 仿真模型的类型	261
如何描述器件含有模型	261
原型模型 (Primitive Models)	262
原理图型模型 (Schematic Models)	262
VSM 模型 (VSM Models)	262
SPICE 模型 (SPICE Models)	263
11.10 如何提高交互仿真的速度	263
前言	263
使用数字的电阻和二极管模型	264

优化外部 RAM 和 ROM 的访问	266
第十二章 错误处理.....	267
12.1 仿真日志	267
12.2 网络表错误	267
12.3 链接错误	267
12.4 分区错误	268
12.5 电源范围定义错误	268
12.6 仿真错误	268
12.7 收敛问题	269

日期/版本	修改内容	作者
20200713 V8.10	增加第二章 4 篇使用指南； 新增第四章内容图片，修改格式； 新增第五章“VSM Studio 介绍” 新增第十章“模型仿真失败的处理”。	赵也兰翻译 梁继超校对

第一章 介绍

1.1 关于 PROTEUS VSM

传统的电路仿真是一个非交互性事件，网络表需要人工准备，得到的仿真结果也仅仅是大量的数字代码，最多可以得到用符号构成的伪图形输出来显示电压和电流的波形。

现在的原理图输入及图形输出都已经有了标准可循，但是仿真的过程仍然是一个非交互的过程——绘制电路，运行并得到一些仿真输出结果。如果你测试的电路是一个静态过程的话，这是一个非常好的方式；比如 1MHZ 的振荡器。然而现在的很多设计是交互式的过程，比如说一个报警器，（当键盘上有错误的 PIN 输入时，报警器能进行报警工作）。这种仿真是没有办法实现的。

现在教育行业一直在进行一个尝试，就是将电路像现实一样做交互式仿真。其中最大的困难是用于交互的动态器件很难建模。现在只有一些简单的模型，比如开关、灯、电机等，在专业电路设计中这些模型还远远满足不了要求。再者，电路仿真缺少许多的信息，比如核心器件数字模型中的定时信息。

PROTEUS VSM 很好的提供了解决方案。它基于工业标准 SPICE3F5，含有动态器件的高级混合电路仿真器。另外还提供一个结构，用户可以据此创建动态模型。很多种类的动态模型无需编程直接建模。有了动态模型，使用者可以像现实设计一样对电路进行交互式仿真。

PROTEUS 有一大批仿真模型，包含现在主流的处理器、一系列接近现实元件的动态模型（比如 LCD、LED 显示，键盘，RS232 终端等）。这样，使用者无需架构硬件电路及仿真设备，直接设计的单片机系统，编写程序，运行仿真，减少产品的开发周期。

现在的 PC 的处理能力已经得到了飞速发展。一个 300MHZ 的奔腾 II 的 PC 可以实时的仿真一个简单的单片机系统。当然有些稍微复杂一些的设计对仿真环境要求苛刻一些，这时使用者应清楚仿真与现实实时性的差别。如果你注重仿真的实时性，你可以购买一个 2GHz 双核 PC 机。但是这仍然不足以能完全实现实时交互仿真。

1.2 关于本文档

这个手册包括一些背景知识及操作指南。帮助使用者更快的掌握 VSM 仿真。

PROTEUS VSM 有两种仿真方式：交互式仿真和基于图表仿真。一般来说，交互式仿真主要用来验证设计电路是否正常工作。高级图形仿真主要用来测量一些电路细节。当然设计中也可以同时使用两种仿真方式。

手册中有一些例子，使用者可以按照说明按部就班来学习软件的基本使用。

注：对于 VSM 模型构建，在 VSM SOFTWARE DEVELOPMENT KIT (SDK) 中有详细的说明，它作为一个在线资源提供给正版用户。

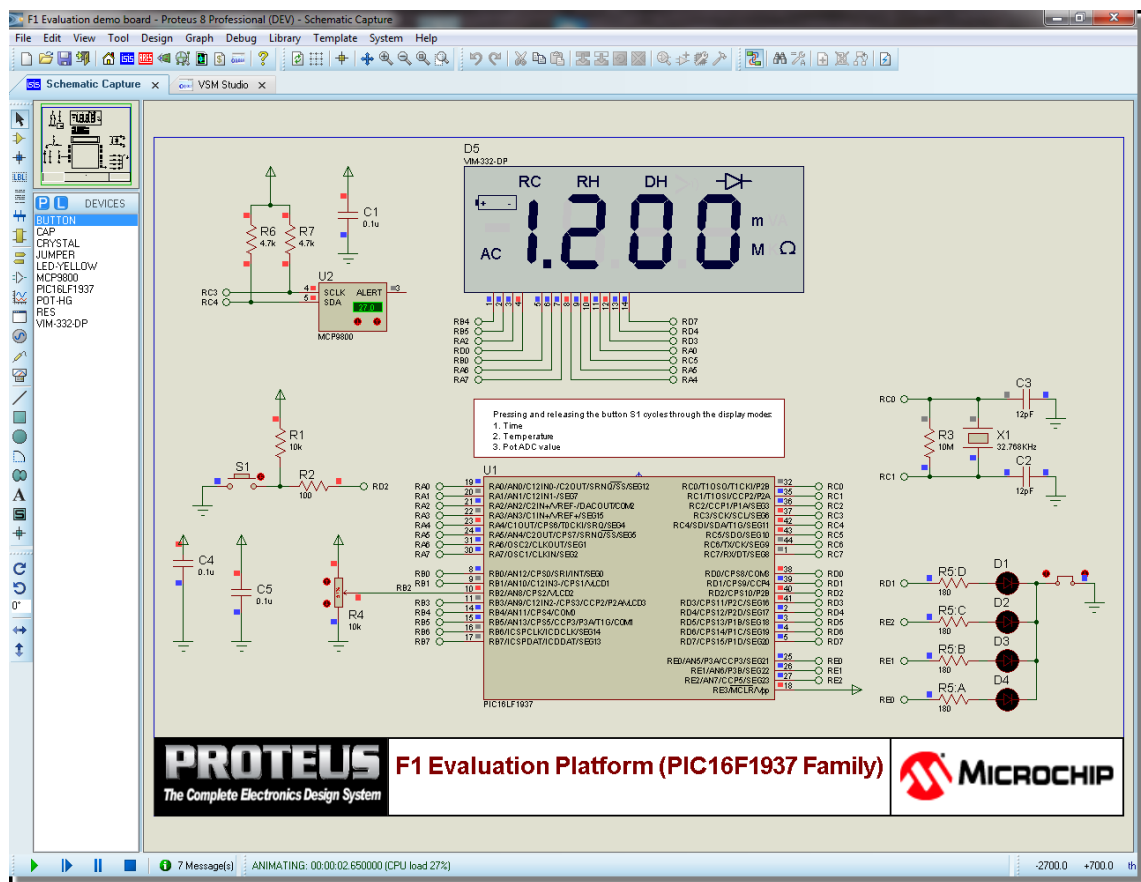
第二章 使用指南

2.1 交互式仿真/调试

介绍

本教程的目的是向您展示如何使用 Proteus VSM 和 VSM Studio IDE 与微控制器之间进行交互式仿真。教程重点将放在电路原理图仿真和 IDE 的实践使用上，参考手册中提供了每个主题的详细说明。本教程不涉及 ISIS 原理图设计；如果您对在 ISIS 中绘图并不熟悉，请阅读 ISIS 参考手册中的教程内容。

如下图所示，我们将使用 Microchip F1 评估板的预绘制原理图作为演示例程。



我们从 VSM Studio IDE 最基础的驱动仿真开始,介绍 Proteus VSM 提供的各种调试和测量工具。

要求

为完成本教程,您需要:

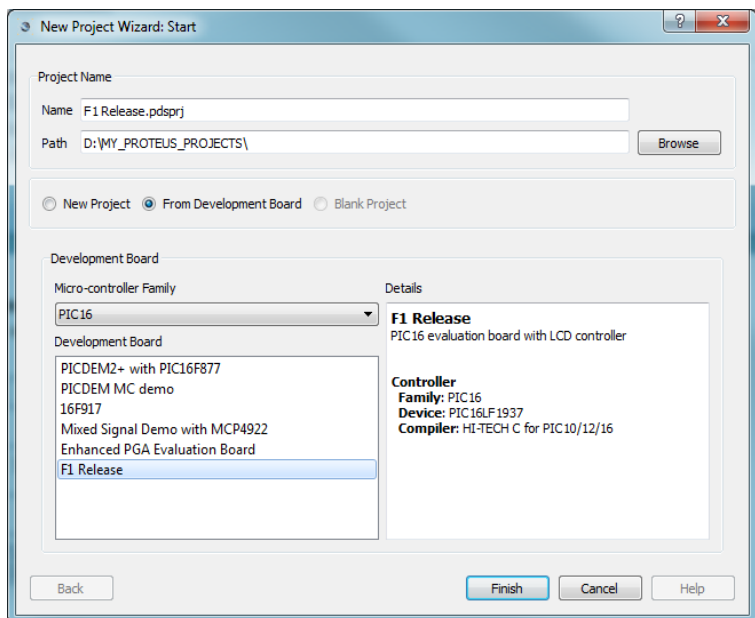
- ◆ 安装 Proteus 软件 8.0 版本或更高版本。如果您没有专业版本,可以从 Labcenter 网站上免费下载该软件的演示版本。
- ◆ 安装 9.8 版或更高版本的 Hi-Tech PIC16 编译器。可以在 Proteus 软件 (VSM Studio IDE) 中下载和安装该编译器。在接下来的章节“工程设置”中将介绍此过程。

即使您没有安装工具,依然建议您通读本教程。大部分资料 and 所有调试技术都是通用的,且对您以后的设计有帮助。

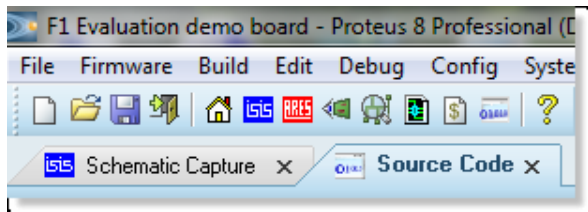
2.2 工程设置

首先我们需要新建一个 Proteus 8 工程。由于我们使用的是虚拟开发板形式的预绘制原理图,因极大地简化了此过程:

从 Proteus 的主页中启动新的工程向导,选择开发板类型。然后从底部的列表框中将微控制器系列改成 PIC16,最后选择开发板为 F1 评估板 (F1 发行版)。单击完成以导入项目。



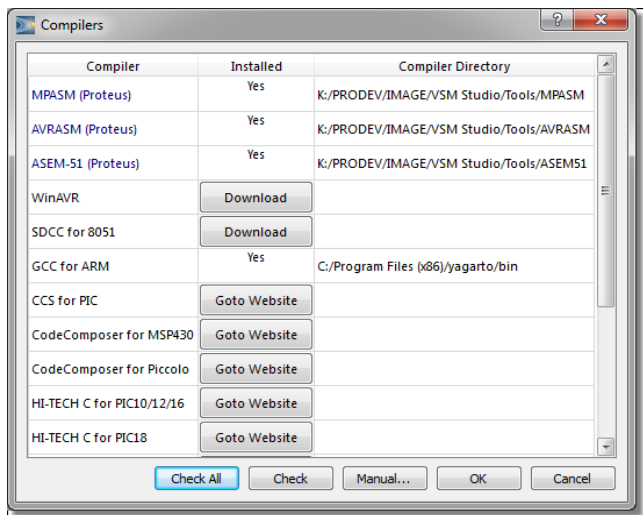
成功导入项目后,您将会应用界面上看到两个选项卡,具有 F1 Release 设计的原理图选项卡和包含能驱动硬件的标准 Microchip 源代码的 VSM Studio IDE 选项卡。



接下来我们需要编译源代码并产生相应的固件，可以在仿真中进行调试。

编译器配置

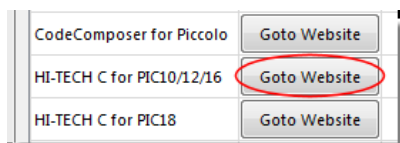
Microchip 源代码是使用 Hi-Tech PIC16 编译器编写的，所以我们得先安装此编译器才能编译固件。切换到“VSM Studio”选项卡即 Source Code 选项卡，然后从“配置”菜单中打开编译器配置对话框。



对话框中列出了所有受支持的编译器以及是否已安装和配置的路径指示。点击对话框底部的“全部检查”，将扫描计算机中所有编译器，找到后自动配置它并可直接在 Proteus 中使用。

开源编译器可以直接从 Labcenter 的服务器下载并安装。可以在供应商网站的下载页面上找到所需编译器的链接，我们要用到的 Hi-Tech 编译器需要从 Microchip 官网下载，因此，如果您未安装此编译器，则必须：

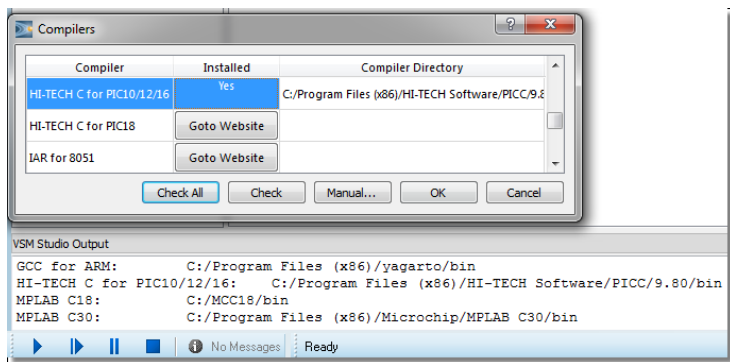
- 1) 单击编译器旁边的“Goto Website”按钮。



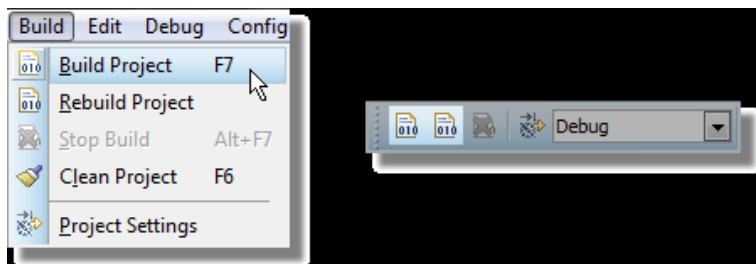
- 2) 下载并安装编译器。
- 3) 单击对话框底部的“全部检查”按钮，以配置编译器在 Proteus 中使用。



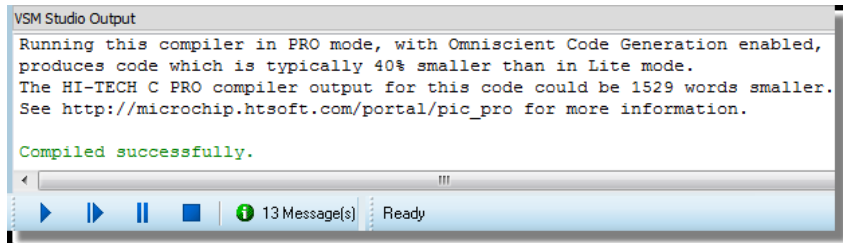
在运行此命令后您可以注意到，对话框上的 Hi-Tech 编译器行被标记为已安装。IDE 底部的输出窗口还将报告编译器检查的结果。



到这一步，我们已经准备好了编译器，源代码和原理图。点击 VSM Studio 中的“构建”菜单上的“构建项目”命令（构建图标）来编译固件。



编译器的输出写在 IDE 的底部窗格，完成后将收到一条成功编译的消息。



下一步是运行仿真并在虚拟硬件上测试我们已编译好的代码。

- ② 注意：VSM Studio 会自动配置常用的编译器选项，以正确构建您的程序在 Proteus 中进行仿真。如果您需要调整设置（比如说，连接外部的固件库），则可以通过“构建”菜单上的“工程设置”

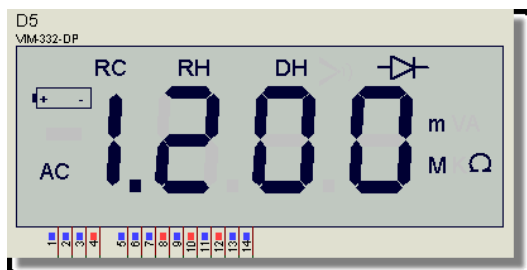
来进行。

运行仿真

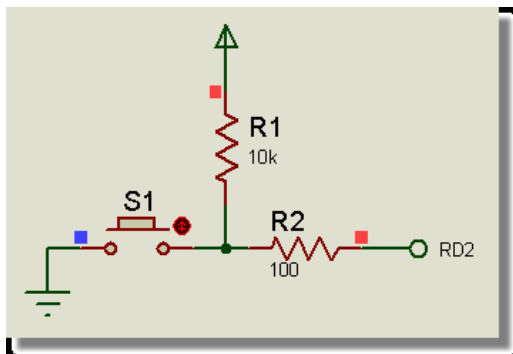
运行仿真十分简单只需在 Proteus 左下方的动画控制面板上点击播放按钮 。



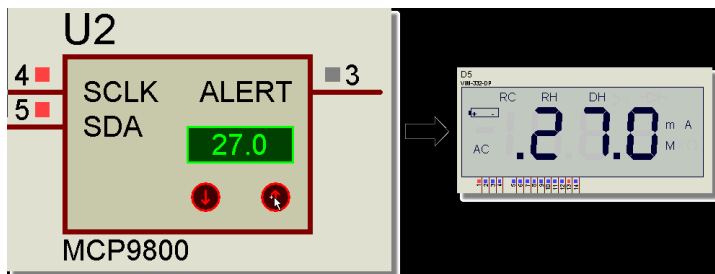
您可以看到原理图选项卡位于前台，并且 LCD 面板显示默认的时间。



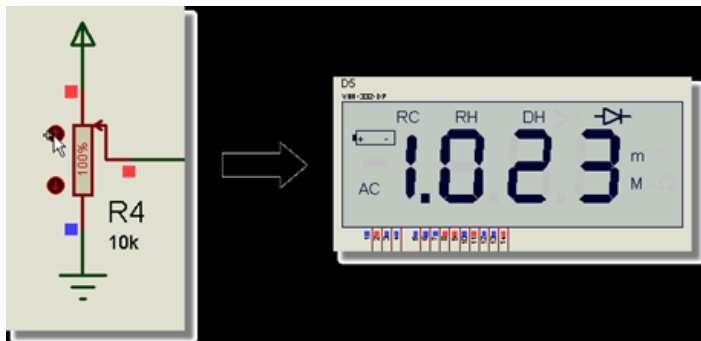
F1 评估板的 Microchip 演示代码具有三种模式，第一种是时间模式，第二种是温度模式，第三种是 ADC 采集 POT 的模拟电压值。您可以通过按下原理图上的控制按钮进行交互式仿真并在几种操作模式之间循环。



在温度模式下，您可以通过单击增量或减量按钮来调整 MCP9800 温度传感器上的温度。当前温度通过 I2C 总线传输到 PIC 处理器，然后转化数据格式并显示在 LCD 上。



在电压模式下，您可以调整 POT 的值。读取的电压值由处理器转换，然后显示在 LCD 面板上。



PIC 微控制器具有 10 位 LCD，因此在 0 至 3.3V 的电压采集范围内，LCD 显示的值范围为 0 至 1023。

调试完成后，点击动画控制面板上的“停止”按钮来停止仿真。

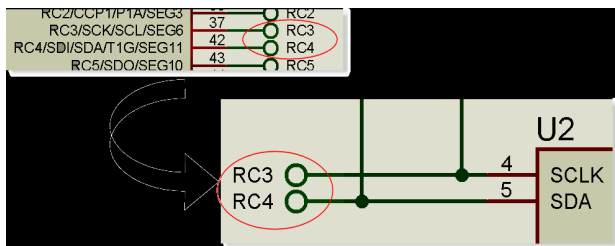


重要笔记

当我们在 Proteus 中运行仿真时，后台实际上经过了一系列的操作处理，但结果并不会立即显示在你面前。

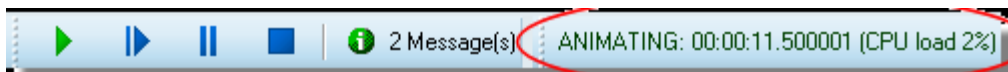
首先，原理图上的微控制器执行编译的固件文件的方式与物理 PIC 执行其编程固件的方式完全相同。在成功编译后，通过将输出文件发送到原理图，使用 VSM Studio IDE 可以简单地自动执行“编程”步骤。

您会注意到，示意图上的几乎所有导线都在带有网络标号的端子中。对于不熟悉终端连接的人，可以了解一下它是如何工作的。在原理图上，任何两个具有相同网络标号的端子都被视为已经连接起来（就像是“虚拟”电线）。例如，温度传感器侧面的 RC3 和 RC4 端子对应于连接到 PIC 处理器的 I2C 引脚的 RC3 和 RC4 端子。



绘制原理图时我们倾向于这样布局，以避免意大利面条式的混乱布线，且能将原理图拆分为易于识别的逻辑块。需要了解的一点是，所有信号都在可见线上传播。这意味着您可以在看得到电线的任何地方连接电压探头或测量仪器（我们将在本教程的后面部分进行介绍）。ISIS 所做的只是在两个终端之间“跳跃”信号，就好像只有一条电线连接一样。您可以在 ISIS 参考手册中阅读有关此类连接的更多信息。

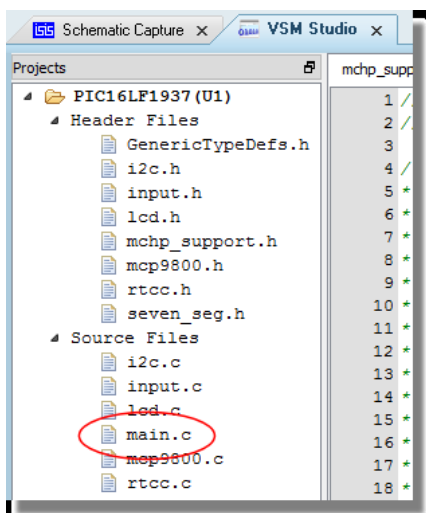
最后，应用程序底部的状态栏将报告执行的模拟时间。根据您的计算机的能力、处理器的时钟速度和原理图的复杂性，某些特定的仿真可能不会实时运行，但始终会同步运行，并且在状态栏上报告执行时间。



例如，在一台特别慢的计算机上，时钟模式下的仿真可能不会随着墙上时钟的推进而前进，但它将始终随着模拟时间的推进而前进。

编写固件

现在，我们已经通过仿真完成了必需的流程，让我们为程序再添加一些功能。我们知道，按下按钮可在时间显示，温度显示和 POT 电压值显示之间切换，因此我们可以在此处轻松地添加另一种操作模式。首先切换到 VSM Studio IDE 选项卡并打开 main.c 文件（双击项目树中的项目）。



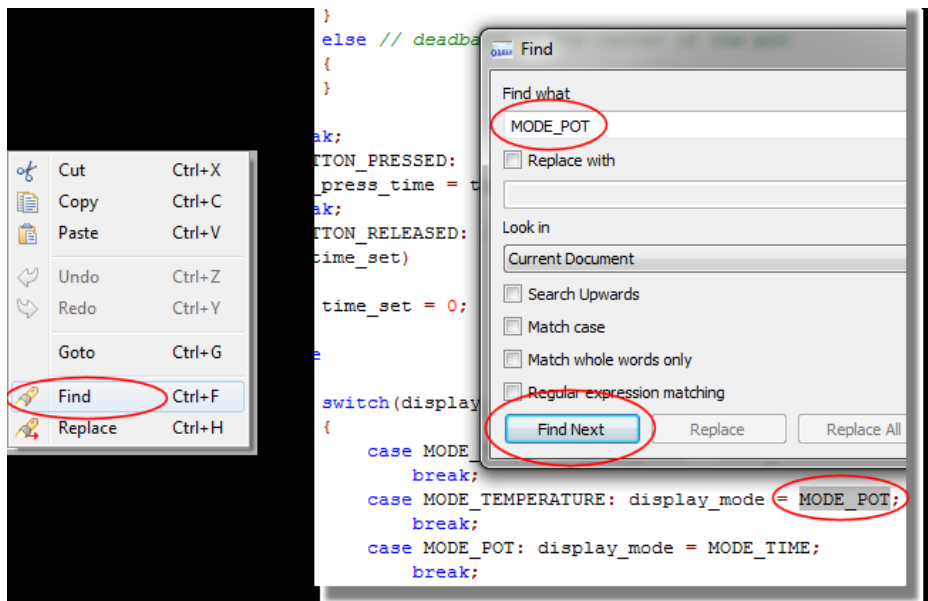
在文件顶部附近，您可以看到一些函数原型和操作模式的枚举。我们需要为我们的测试模式声明一个新的测试函数原型，并在枚举函数中添加相应的值。您可以添加以下内容：

```
40 void display_time(void);
41 void display_temp(int t);
42 void display_int(int t);
43 void display_rpm(int r);
44 void display_pot(int p);
45 void display_test(void);
46
47 volatile char blink;
48
49 typedef enum { MODE_TIME, MODE_TEMPERATURE, MODE_POT, MODE_TEST } mode_t;
```

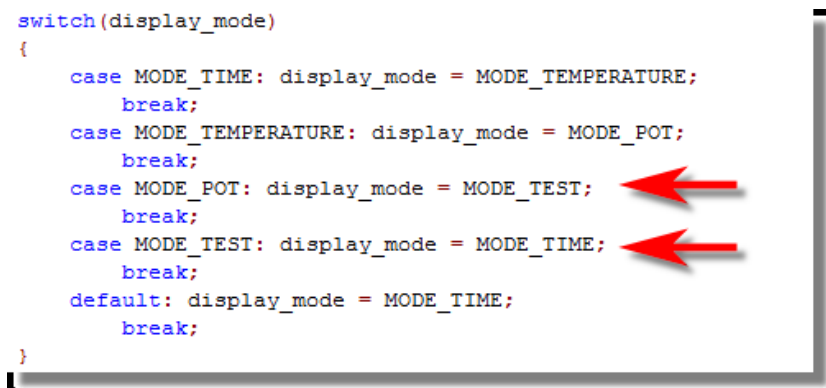


接下来，我们需要找到按钮切换逻辑，以便添加新的操作模式。最简单的方法是搜索其他模式

之一。右击 VSM Studio 中的编辑窗口，从出现的菜单栏中选择“查找”，然后输入“MODE_POT”作为搜索词。



第一个匹配项正是我们在寻找的匹配项，即按钮释放的 switch 语句在显示模式设置之间循环。我们需要做的就是添加新模式。



请注意，您还需要更改前一个 case 语句的显示模式。

在代码的下面，还有另一个 switch 语句，用于操作当前的显示模式。同样，我们需要在这里添加 case 语句并调用我们刚刚声明的新函数。

```
switch(display_mode)
{
    case MODE_TIME:
        display_time();
        break;
    case MODE_TEMPERATURE:
        display_temp(mcp9800_get_temp());
        break;
    case MODE_POT:
        display_pot(input_pot());
        break;
    case MODE_TEST:
        display_test();
        break;
    default: display_mode = MODE_TIME; break;
}
```

最后一步是编写这个测试函数的内容，该函数在进入操作模式时会执行相关操作。我们将其添加到文件的底部。

```
336 void display_test(void)
337 { // Do Something...
338 }
```

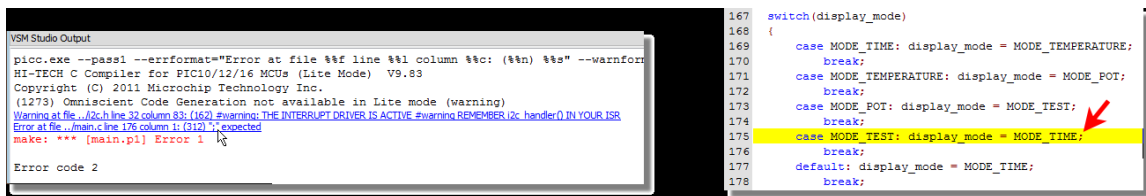
您可以在此处进行测试或添加任何您想要它实现的功能，但在这里我们仅做一些简单的事情，在显示屏上写入一个值，并在 LED 上添加一个小的二进制计数器。

```
343 void display_test (void)
344 { static int i,d;
345
346     display_int(1111);
347     if (++d > 400)
348     { d = 0;
349       PORTE=i;
350       i = (i+1) % 8;
351     }
352 }
```

由于我们在这里使用 PORTE 来写入 LED，因此还需要配置对应的端口引脚。我们将在主函数上方的外部接口配置部分中设置 ANSELE。

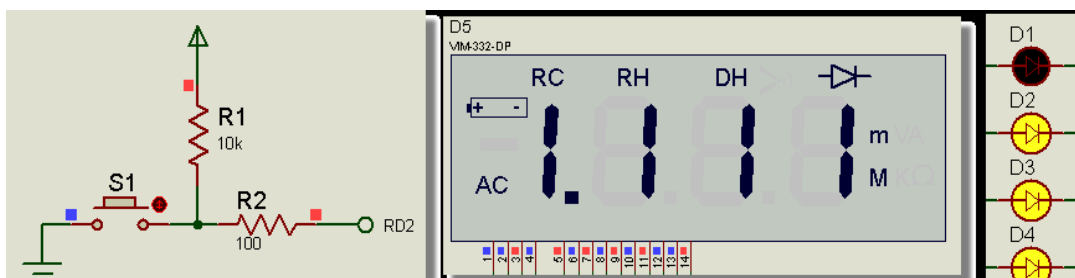
```
78     ANSA1 = 1;
79     ANSA3 = 1;
80     ANSB3 = 1;
81     ANSB1 = 1;
82     ANSELE= 0;
```

最后一步是之前已经讨论过的构建项目并启动 ISIS。如果您的代码中有一个错误（比如说下面这个例子），您将在输出窗口中看到编译器报错。单击此错误将导航到有问题的那一行代码。在这个例子中，我们错过了行尾的分号。



❗注意：某些错误（例如工具链错误）显然将无法导航。在这些情况下，您可能需要在工程设置中更改选项/路径。

在这次的仿真中，我们可以通过按钮在操作模式之间循环，直到到达我们自己新增的模式（时间->温度->测试）。此时，我们应该在显示屏上看到显示的数字，并且 LED 上的字节周期计数



按动画控制面板上的“停止”按钮，停止仿真。

在这个示例中，虽然需要我们动手实践的操作并不多，但它确实说明了如何在虚拟硬件上快速编写和仿真代码。

活动弹窗

到目前为止，我们一直在 VSM Studio 选项卡内中编写代码，并在原理图捕获选项卡上进行仿真。这样做并没有什么问题，但如果将其中一个窗口拖到另一个屏幕上，就可以在两台显示器上很好的配合使用。

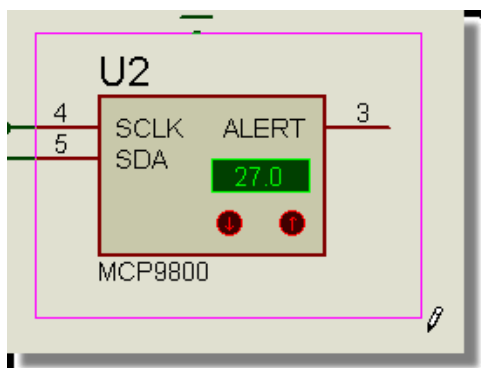
但是，在进行调试时，通常我们更感兴趣的是逐步执行代码并查看电路板上的某处细节进行验证。Proteus 中的活动弹窗旨在完全做到这一点-在仿真过程中将原理图中所圈选的部分引入 VSM Studio。

在这个案例中，温度传感器是调试过程中我们可能希望看到并与之交互的一个很好的例子。可将其添加至活动弹窗中：

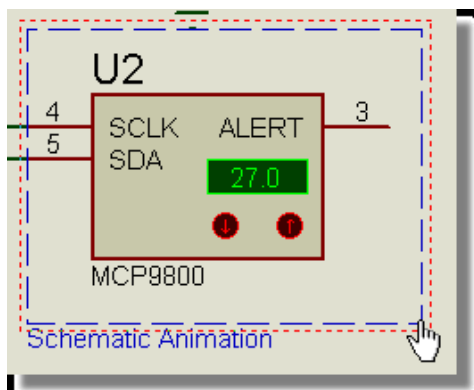
1. 切换到 ISIS 选项卡，然后点击“活动弹窗”图标。



2. 从左上方开始，并在其周围拖动形成一个框。

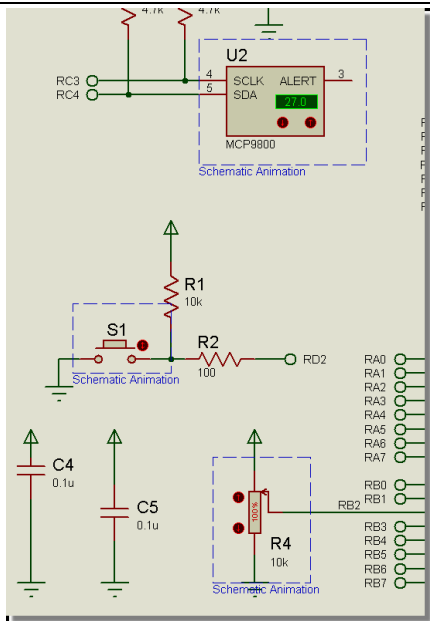


3. 完成后，您应该可以在该区域周围看到一条蓝色虚线。

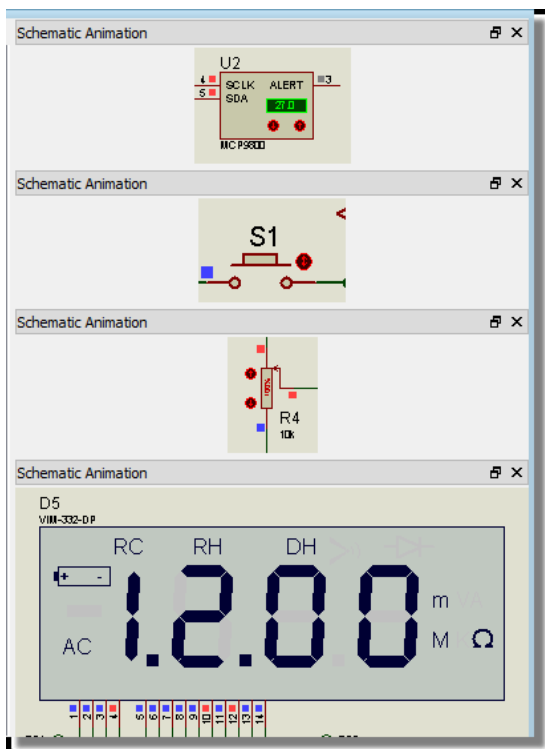


❗注意：如果圈选有误，可以右击虚线并删除。同样，如果位置错误，则可以右击虚线并拖动。

活动弹窗的其他候选对象可以是模式选择按钮，POT 和显示器本身，因此我们可以重复上述过程，在这些项目周围拖动框选。完成后，原理图应类似于以下内容。

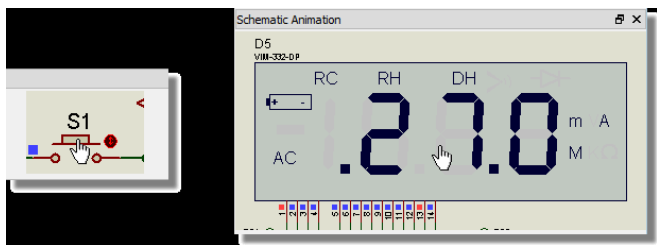


由于我们已经指定了活动弹窗的内容，因此假设我们将在 VSM Studio 中进行工作（调试）。如果您按下播放按钮以运行仿真，现在应该会发现您已切换到 VSM Studio 选项卡，并且活动弹窗排列在 IDE 的右侧。



注意：当我们自由执行代码时，源和变量区域将在此时显示仿真运行消息。我们将在本教程的下一部分中介绍调试。

活动弹窗很聪明，不仅因为它们将原理图的区域带入调试环境，而且还因为您可以与之交互。例如，如果您单击 S1 按钮以切换模式，您应该会看到显示屏更改为 I2C 温度传感器上的温度，如果您随后调整了传感器上的温度，显示屏上将会反映出更改。



您可以切换回“原理图”选项卡，将看到与活动弹窗完全相同的状态。当您停止仿真时（从动画控制面板），活动弹窗将消失，VSM Studio 将从其“调试布局”切换为“设计布局”，您可以在其中再次编辑和编译源代码。

❗注意：仅当仿真停止时，才能在原理图上创建或调整活动弹窗。但是，您可以在仿真运行时通过拖拽来调整 VSM Studio 内部活动弹窗的大小。

现在，我们已经配置了活动弹出窗口，我们可以在某些功能第一次无法正常运行时用调试的方式研究这些问题。

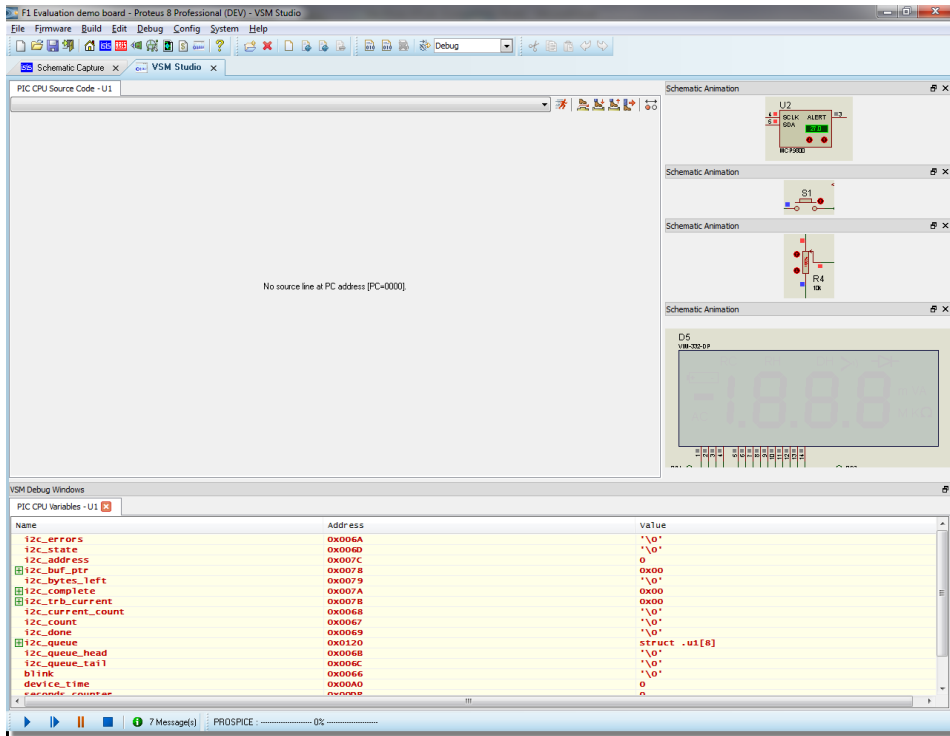
基本调试

Proteus VSM 功能的真正强大在于其调试功能。我们已经了解了如何在自由运行的仿真过程中编写代码并进行测试，现在我们将研究如何在仿真模拟时间内逐步完成代码。

如下图所示，使用动画控制面板上的暂停按钮开始仿真。



程序将切换到 VSM Studio 选项卡，活动弹窗将显示在右侧，主面板将报告“无源代码行信息”，并且程序变量列表将出现在屏幕底部。

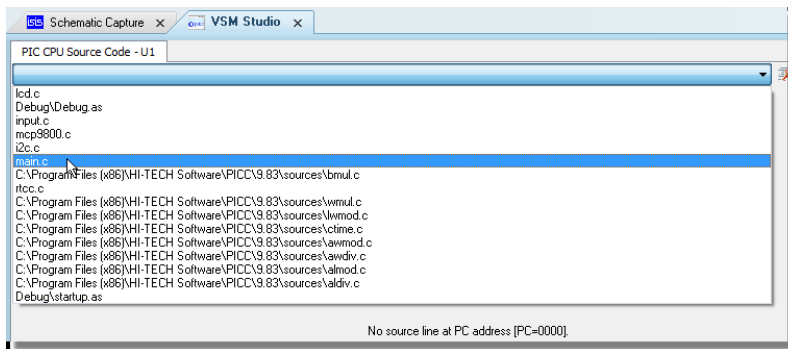


在此阶段，仿真已“启动”，并且找到了稳定的工作点，但没有执行任何指令，也没有经过任何实时操作。

没有显示源代码，因为当前程序计数器值处没有源代码行。

您可以从窗口顶部的组合框中选择任意“

程序源文件，但我们将从包含主程序控制循环的 main.c 文件开始。

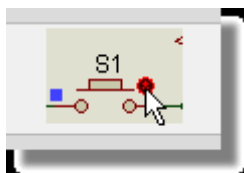


举一个简单的例子，让我们在释放按钮时在主事件循环中设置一个断点。设置/切换断点的最简单方法是双击要中断的代码行。断点指示器应出现在源代码窗口左侧相关代码行的旁边。如果您错误地设置了一个断点，则可以通过再次双击该行（或从右侧菜单中）将其删除或将其关闭。

```

----          case BUTTON_PRESSED:
01EF          btn_press_time = time(0);
01FC          break;
----          case BUTTON_RELEASED: // on relea
01FD          if(time_set)
0201          {
0202              time_set = 0;
0203          }
0204          else
          {
              switch(display_mode)
              {
                  case MODE_TIME: disp1
    
```

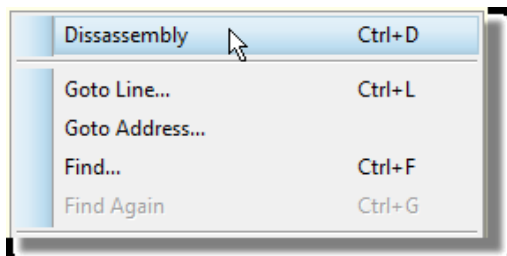
如果现在运行仿真（播放按钮），它将在自由运行模式下愉快地切换执行，直到释放模式切换按钮（S1），此时将按下按钮释放代码，并且模拟将暂停。通过单击活动弹出窗口中的按钮，您可以轻松地做到这一点。



达到断点后，我们可以通过常用的命令单步执行代码（和系统），这些命令可以在源代码窗口的右上角或 VSM Studio 中的调试菜单上找到。另外，您可以使用 F10 和 F11 快捷键分别“跳越（子函数）”和“跳进（子函数）”。



如果您确实需要详细检查行为，甚至可以在汇编级别上逐步执行代码。右击源菜单，然后从出现的菜单栏中选择“反汇编”，然后使用与之前相同的单步执行命令。



右击并再次选择反汇编，以返回高级源代码单步执行。

注意：您可能会注意到，在单步调试过程中，显示屏的活动弹窗未显示完整的清晰输出。这个现象是正常的，并且显示被多路复用的结果。

如果要浏览下一次按钮释放，只需按动动画控制面板上的播放按钮，然后单击活动弹窗上的按钮即可再次触发断点。

完成操作后，右击并选择清除所有断点，然后按动动画控制面板上的“停止”按钮结束调试过程。

重要笔记

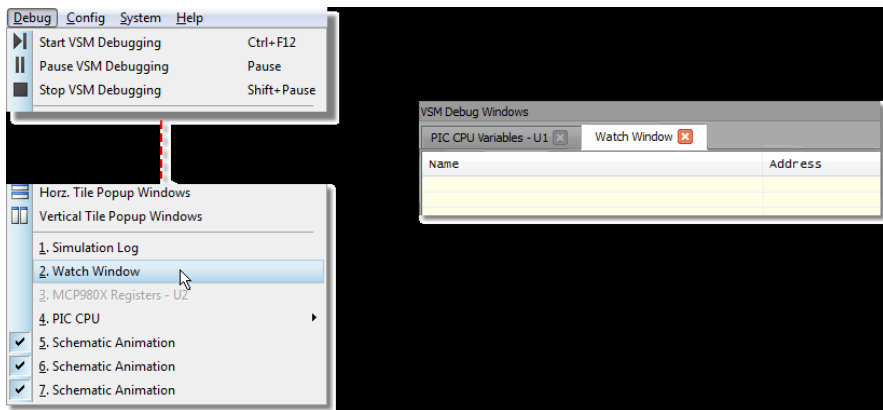
当您在 Proteus 中进行断点或单步调试时，重要的是要了解整个系统都处于时间控制之下。这意味着，例如在系统暂停时电容器不会放电或电机失去动量。每当您执行命令时，指令将工作，这些指令的效果会在系统中传播，然后系统将再次暂停。在调试期间，您可以通过状态栏上的输出看到时间在您的控制下推进。

还有许多其他可用的调试窗口，本教程中暂不作讨论。所有这些都可以从 VSM Studio 中的调试窗口启动，并将显示在 IDE 的底部。每当暂停仿真时，数据将显示在调试窗口上。监视窗口（一种特殊情况）在下一小节中将单独讨论。

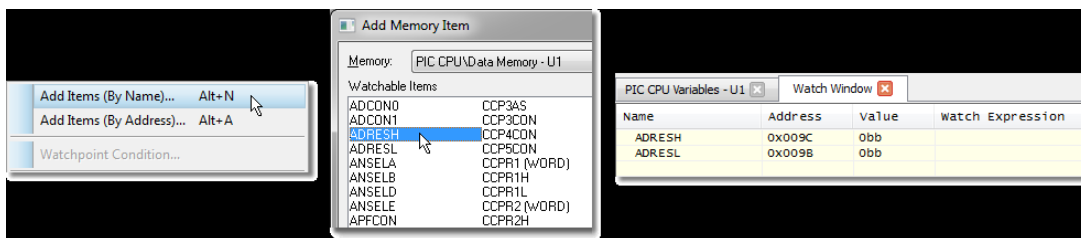
监视窗口

监视窗口是一个调试窗口，可在自由运行的仿真过程中提供实时数据，它还为我们提供了一种不同的方法来确定断点。首先，让我们使用监视窗口监视来自 POT 的 ADC 转换。

使用 Proteus 底部的播放按钮开始仿真，然后从 VSM Studio 中的调试菜单栏中启动监视窗口。启动后您可以看到它位于 IDE 底部。



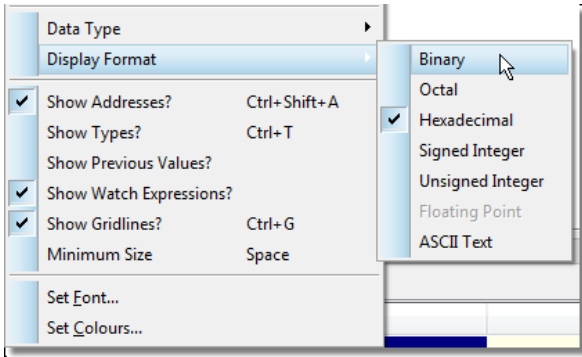
接下来，右击监视窗口，然后从菜单栏中选择“按名称添加项目”。我们要添加 ADC 寄存器 ADRESH 和 ADRESL，您可以通过双击条目来完成。完成后退出对话框，您可以看到两个项目都出现在监视窗口中。



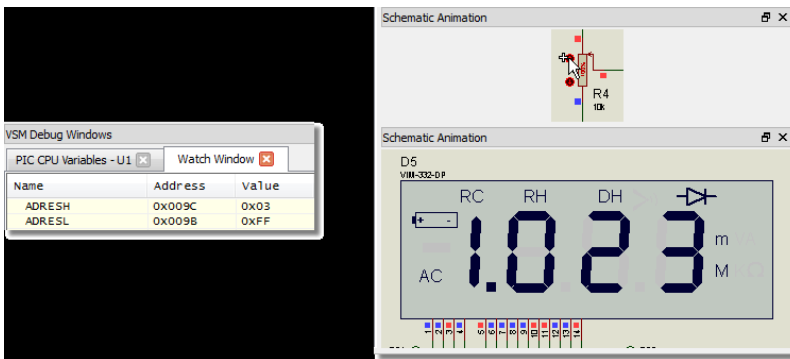
现在，我们需要使用活动弹窗（或切换到原理图选项卡）切换到 POT 读取模式。单击按键三下以循环模式，然后使用滑动变阻器左侧的加减按钮更改 POT 上的值。



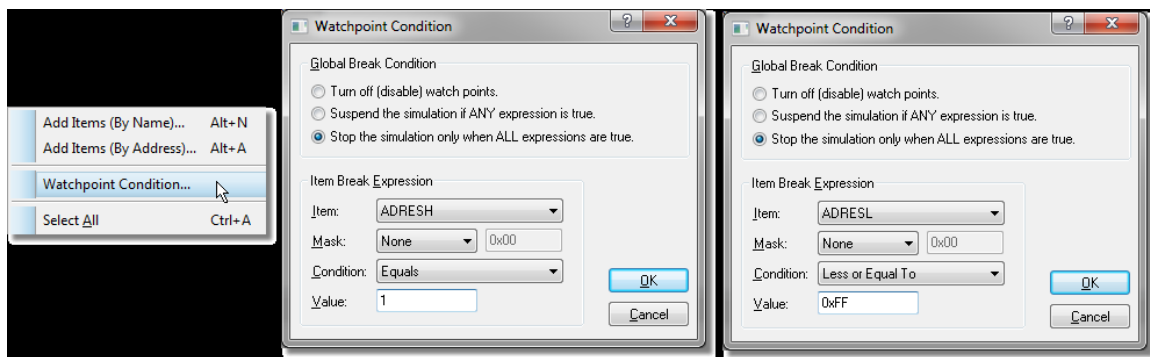
如果您觉得这更易于理解结果，可以将项目的显示格式更改为二进制。您可以通过依次右击监视窗口中的每个项目并将显示格式更改为二进制来实现。



由于这是 10 位 ADC，因此我们希望最大值显示为 1023 或 0x03FF，并且当 POT 完全上升时我们能看到该值。



现在假设我们需要在回退的路上在中点停下来。我们知道中点位置的值应该在 0x1FF 左右，因此我们可以在监视项上设置条件以触发断点。我们通过右击监视项并从菜单栏中选择监视点条件来执行此操作。我们希望，当两个 ADRESH 都等于 0x01 且 ADRESL 小于或等于 0xFF 时，暂停仿真。这将是我们的第一次从最高点下降到中点电压时达到此值。



请注意，您需要分别配置 ADRESH 和 ADRESL。完成后，监视窗口的显示如下图所示。

VSM Debug Windows			
PIC CPU Variables - U1		Watch Window	
Name	Address	Value	Watch Expression
ADRESH	0x009C	0x03	= 1
ADRESL	0x009B	0xFF	<= 0xFF

设置好监视条件后，我们现在需要做的就是向下调整 POT，直到满足条件时才产生断点。与之前一样，您可以通过 VSM Studio 中的 POT 活动弹窗或切换到原理图选项卡来执行此操作。触发后，您应该看到监视项上的值为 0x1FF。如果需要的话，您可以在触发条件的位置开始单步执行代码。

VSM Debug Windows			
PIC CPU Variables - U1		Watch Window	
Name	Address	Value	Watch Expression
ADRESH	0x009C	0x01	= 1
ADRESL	0x009B	0xFF	<= 0xFF

② 注意：当您想捕获溢出条件时，监视条件对于调试计时器代码之类的工作特别有用。

② 注意：您可以通过右击监视项目，选择监视条件，然后从出现的对话框中选择“关闭监视点”选项来禁用监视点表达式。

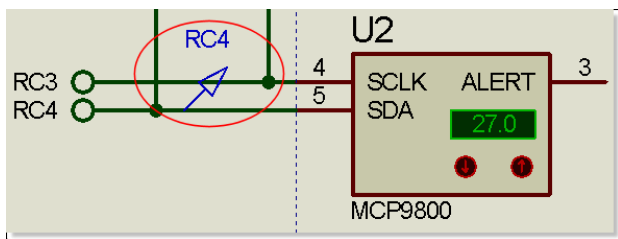
在继续讨论硬件断点之前，请先禁用监视点条件（如上所述）或完全删除监视项，然后按动画控制面板上的“停止”按钮以结束模拟。

硬件断点

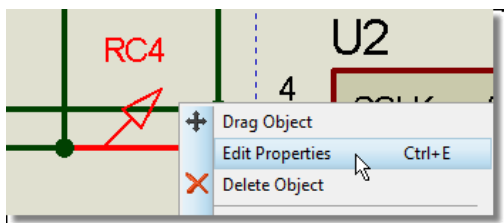
到目前为止，我们已经研究了如何基于软件条件（观察点和断点）进入仿真。其实当发生硬件状况时，也可以使用硬件断点对象来检查代码。例如，如果我们想捕获 I2C 总线的开始信号，我们

可以在 I2C 线上设置硬件断点，如下所示。

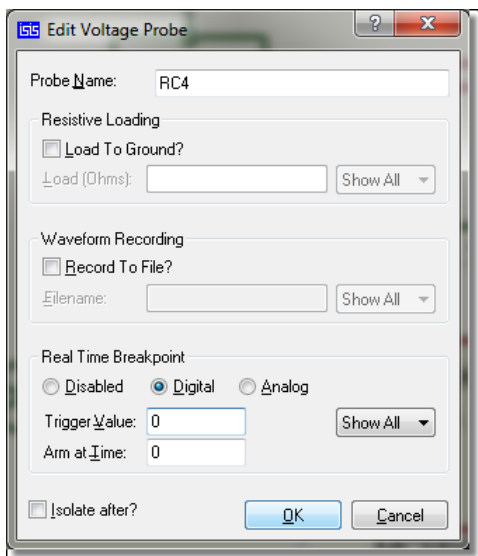
切换到原理图选项卡，然后从左侧的模式选择器中选择电压探针。接下来，单击编辑窗口开始放置，将鼠标移到总线上的 SDA 线上，然后再次单击鼠标左键将探针放到电线上。



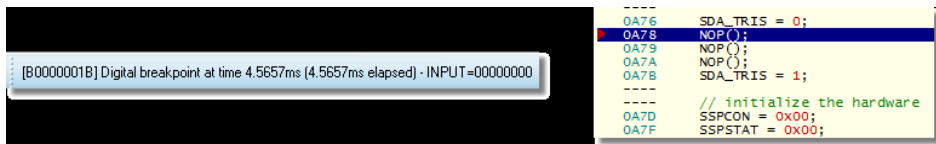
现在，右键单击探针，然后从出现的菜单栏中选择编辑属性。



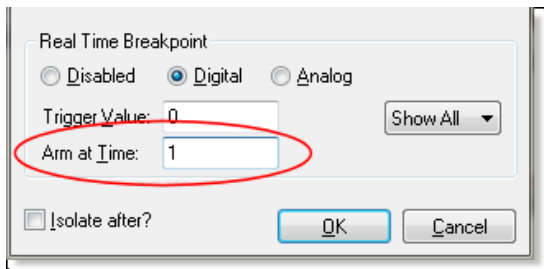
选择数字断点，我们想在低电平有效时触发，因此输入 0 作为触发值。



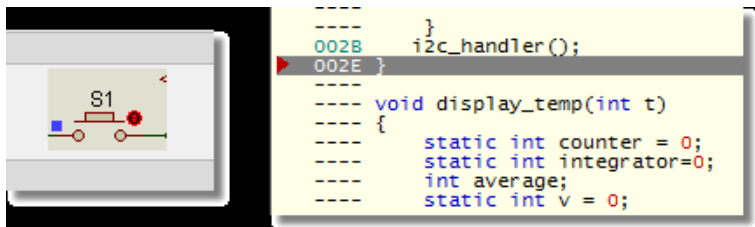
单击确定，然后按播放以运行仿真。您应该发现仿真几乎会立即暂停，并且 VSM Studio 中的源代码窗口将标注刚刚执行的将 SDA 设置为低电平的指令。



实际上该断点发生在初始化例程中。如果我们要跳过此操作并仅在进入 I2C 模式时激活，则需要断点上设置时间进行布防。为此，请停止仿真，切换到原理图选项卡，然后像以前一样进行编辑。将布防时间设置为 1 秒。



这次运行仿真时，直到按下模式选择按钮进入温度模式（在 I2C 总线上传输温度）后，断点才会触发。

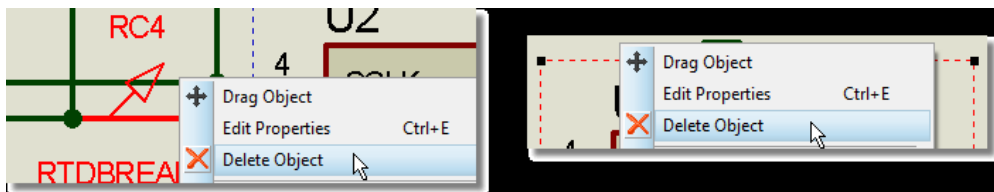


您应该发现自己最终进入了源代码中的 I2C 例程之一。您可以像前面各节中介绍的那样，从这行代码开始，进行单步调试。完成后，再次停止仿真。

交互式测量

想要检查设计上的信号和波形是很常见的，Proteus 提供了两种主要的方法。作为示例，假设我们需要查看来自 MCP9800 温度传感器的 I2C 流量。我们可以通过交互式仿真或更传统的图表仿真的方法来做到这一点。

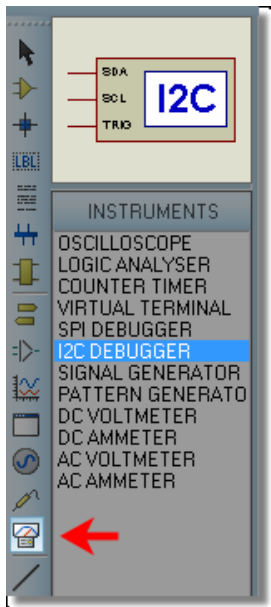
我们需要做的第一件事是整理原理图并删除我们之前使用的调试项。您可以通过右键单击原理图上的探针并从菜单栏中选择“删除”来删除电压探针。同样，我们可以右击活动弹窗对象的边框并将其删除。



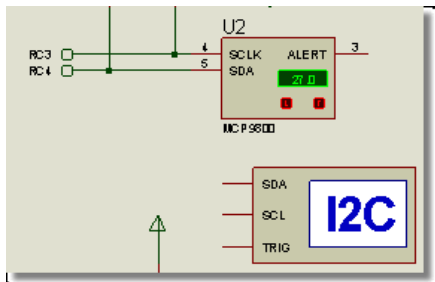
注意：在原理图上定义了活动弹窗后，仿真开始时 Proteus 会自动切换到 VSM Studio，因为假定

您主要是调试源代码。且由于我们现在正在分析信号，因此我们希望删除活动弹窗，以便仿真从原理图选项卡开始。

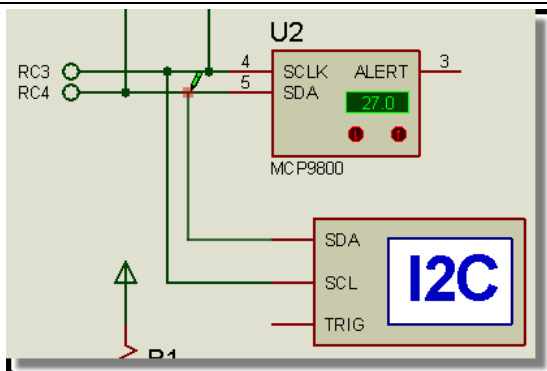
接下来要做的是放置并连接 I2C 调试器。选择仪器图标，该图标将在零件箱中生成一个交互式测量工具的列表。在我们的案例中，显而易见的候选者是 I2C 调试器。



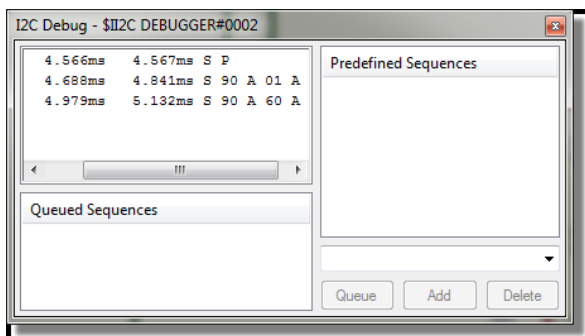
单击零件箱中的 I2C 调试器以将其选中，然后将其放在原理图上。为了便于接线，您需要将其放在温度传感器下方。



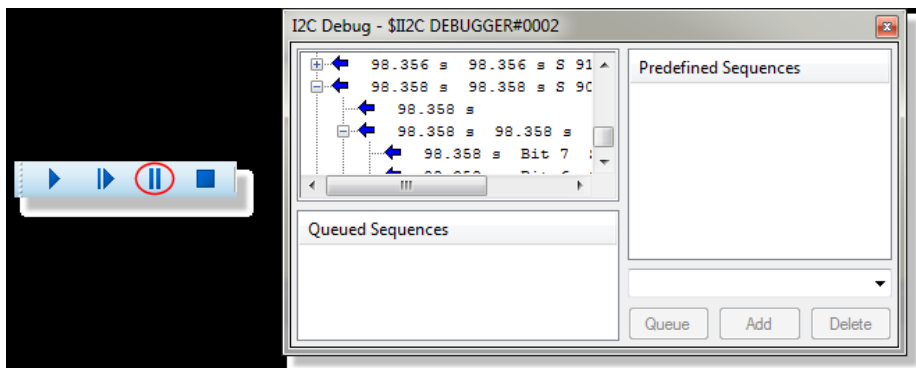
接下来，我们需要连接引脚。请注意，光标在电线的起点和终点都变为绿色。因此，接线过程是将鼠标置于起始点（I2C 调试器上的引脚）上方，直到光标变为绿色，然后单击鼠标左键以开始放置，将鼠标移至目标点（光标变为绿色），最后单击鼠标左键以终止放置。使用此连线方法将 I2C 分析仪上的 SCL 和 SDA 都连接到 I2C 总线上。



我们已经连接了调试器，现在可以开始仿真了。您应该可以看到 I2C 调试器还有一个额外的窗口，并且似乎有一些初始化/识别流量 initialization/identification traffic。要查看 I2C 总线是否正常传输数据，我们先要点击原理图上的按钮切换到温度模式。



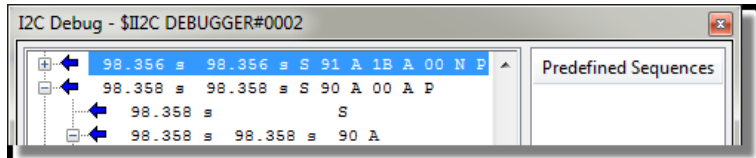
由于代码一直在循环，因此您应该从动画控制面板中暂停仿真，以便我们花一些时间来分析流量。



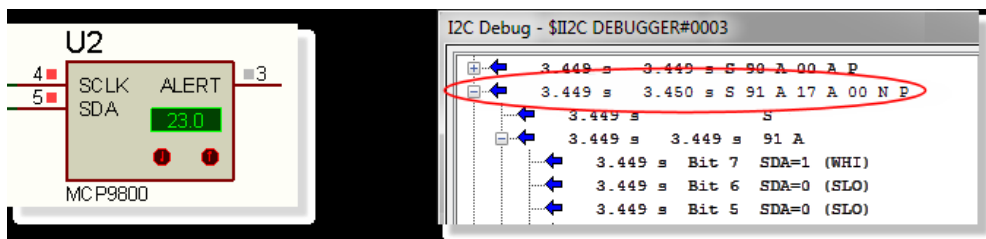
❗注意：如果您在单个框架内工作，假定您对等待执行的源代码感兴趣，在仿真暂停时 VSM Studio 选项卡将出现在前台，但是分析仪将保持在顶部。如果要同时查看原理图和源代码，则可以使用活动弹窗，也可以将 VSM Studio 选项卡拖动到一些可用的屏幕空间以将其分开。

协议分析器中使用的语法是标准的，应该熟悉。您可以通过左侧的“+”框扩展任何序列。外设（MCP9800）的标准地址为 0x90，因此我们可以从读取请求中看到序列，先是开始信号（S），然后

是读取请求 0x91（读取请求，第 0 位置 1），然后是 0x1B（数据），依此类推。可以预期，接收到的数据（0x1B）为 27 位十进制数 27 decimal，对应于在外围设备上实时显示的温度。



您可以通过运行仿真来进行实验，更改 MCP9800 原理图部件上的温度，然后再次暂停以查看 I2C 调试器上的显示输出。



请注意，在不断写入文本数据到显示器的过程中，诸如此类的持续轮询固件会在性能方面有很高的要求。但是在大多数情况下，您使用仪器进行测试或调试时，仿真速度仅次于解决问题的速度。与所有调试窗口一样，您可以在不使用调试器时将其关闭，然后从 VSM Studio 或 ISIS 中的调试菜单中将其重新打开。

进一步阅读：

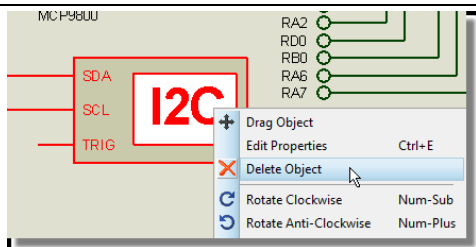
- ◆ Proteus VSM 参考手册中提供了有关调试器以及其他仪器的更多信息。特别要注意的是，您可以将分析仪用作 I2C 主（或从）设备，也可以仅用作监视器。
- ◆ 有关在原理图上进行选择器件，放置和布线的更多信息，请参见 ISIS 教程文档，可从主页打开。

图表测量

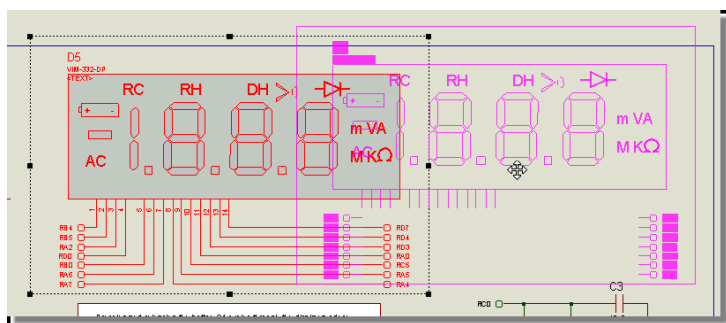
通过使用图表仿真，我们可以以不同的方式查看相同的数据传输。但是，有一些重要差异会影响我们建立仿真，即：

- ◆ 在图表仿真期间，您无法与电路交互。
- ◆ 图表仿真运行在指定的时间段内。
- ◆ 在这段时间结束并且仿真停止之前，结果是不可见的。

鉴于以上所述，让我们看一下如何使用数字图表分析 I2C 数据。首先右击交互式 I2C 调试器，并从菜单栏中选择“删除”来删除它。



接下来，使用鼠标右键框选显示器和电线，左键按下鼠标将所框选的内容批次拖至原理图的右侧。为图表留出一些空间，以使它们全部突出显示

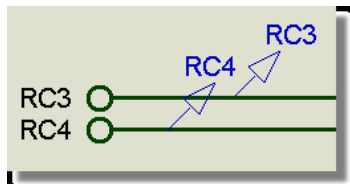


现在选择图表图标，然后从零件箱中选择数字图表。左键单击鼠标后，向右下角移动，再次左键单击放置图表。

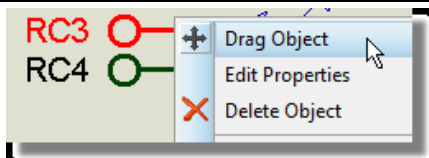


注意：原理图上的图表不必特别大。当我们要分析仿真结果时，我们再最大化它。

添加完图表后，我们现在需要告诉它要跟踪的导线。因此我们将探针附加到导线上，然后将该探针指定为图表上的迹线。选择电压探针图标，然后将探针放置在两条 I2C 线上。完全按照我们之前在硬件断点部分中所讨论的那样进行。

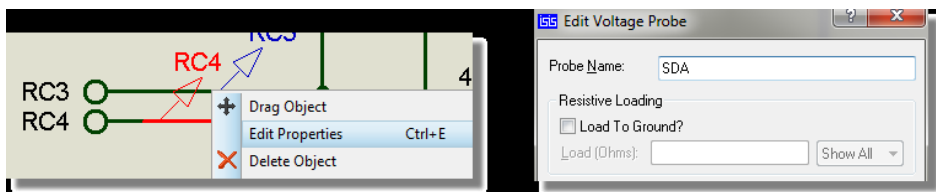


如果需要为探针留出空间，请右键单击导线末端的端子，右键单击并从菜单栏中选择拖动对象，然后向左移动以为探针留出空间。

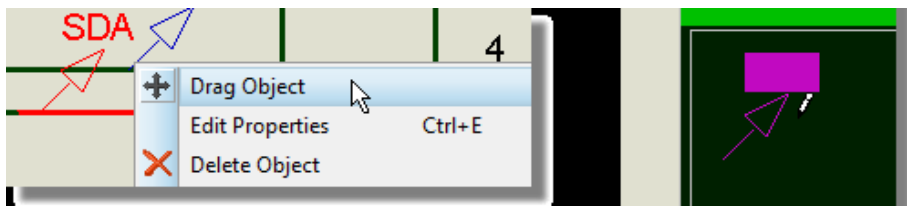


❗注意：如果需要靠近，可以使用鼠标中间的滑轮或 F6 / F7 键在鼠标位置周围放大和缩小。按 F8 键将返回到原理图的默认视图。

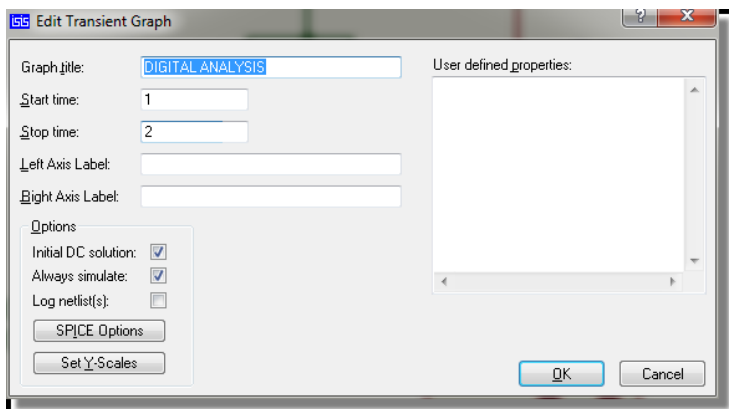
默认情况下，电压探针将选择端子标号。重命名它们以在我们的仿真环境中更直观明了。您可以通过右键单击探针，编辑其属性，然后将标号分别更改为 SCL 或 SDA 来执行此操作。



现在我们在导线上有探针，在原理图上有图表，我们需要将探针添加到图表中。有几种方法可以执行此操作，但最简单的方法是将探针直接拖到图表上。为此，右键单击探针，选择拖动对象，将鼠标移到图表上，然后再次单击鼠标左键以拖放。对两个探针都执行此操作。

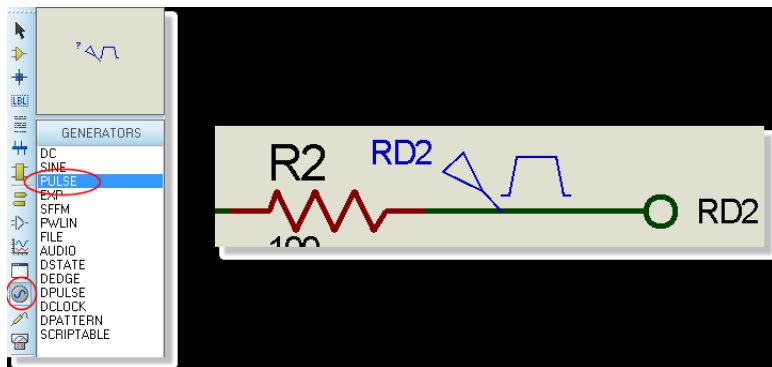


默认情况下，图表仿真将从仿真时间的零时开始运行到一秒钟。在我们的情况下，从一秒钟运行到两秒钟会更有意义，从而跳过了初始化流量。我们可以通过编辑图表（右键单击和编辑属性）并更改开始和停止时间来做到这一点。

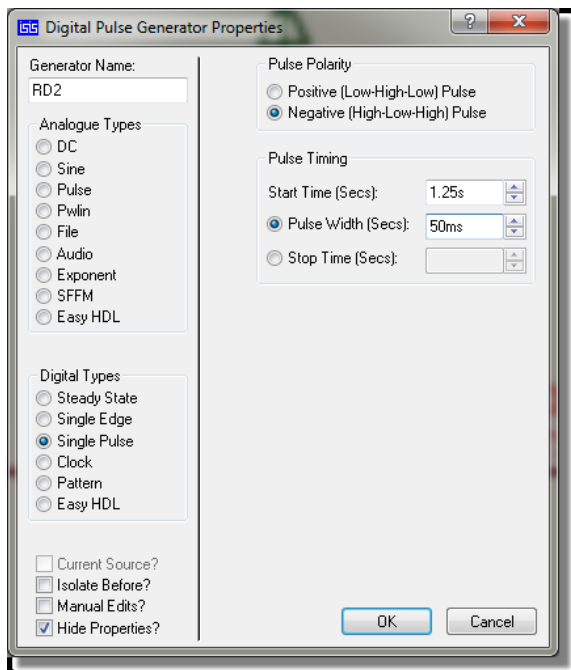


❗注意：左键单击原理图的空白区域将取消选择所有当前选定的对象。通常，当您退出编辑属性对话框时，该对象仍将突出显示，需要左击原理图以取消选择。

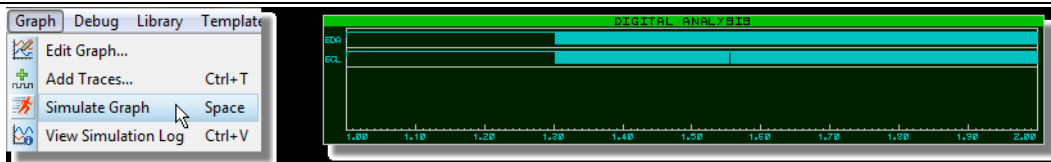
现在我们可以开始仿真，除了一个重大问题。如前所述，您无法在图表仿真过程中与电路进行交互，但是我们需要切换模式以轮询温度传感器并触发 I2C 流量。由于无法按下按钮，我们需要向 PIC 输入等效信号，以便固件切换到温度模式。我们用激励源来做。选择激励源图标，然后从零件箱中选择脉冲信号。将脉冲信号放置在 RD2 端子旁边的按钮右侧-您可能需要先移动该端子为脉冲信号放在线上留出一些空间。



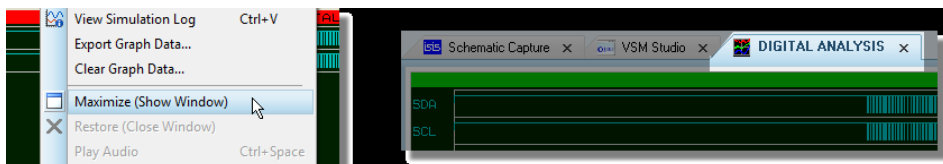
现在，像平常那样编辑脉冲信号发生器（右键单击-编辑属性）。从数字信号类型中选择单个脉冲，并将脉冲极性更改为负，这样脉冲信号便表示了按钮按下。由于我们是从 1s 到 2s 进行仿真，因此将开始时间设置为 1.25s，并将脉冲宽度设置为 50ms。



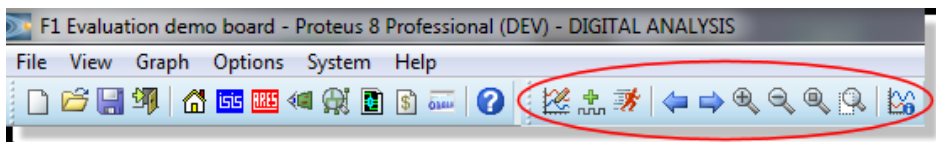
现在，我们可以运行仿真。与交互式仿真使用动画控制面板不同（我们在固定时间进行仿真，而不一定要从时间零开始）。而是使用图表上的“仿真图表”命令。然后，您应该看到如下所示的结果。



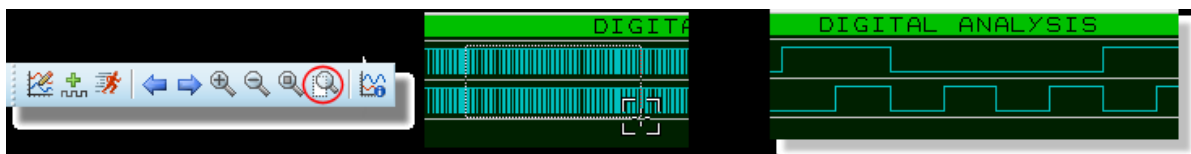
除了流量直到大约 1.25 秒（由脉冲信号指定）才开始的事实之外，我们在这里看不到太多信息。右键单击图表菜单，然后从菜单栏中选择最大化，以便在新的图表界面中启动程序。



当图表界面位于前台时，显示的菜单和图标将更改为我们可以对图表执行的操作。特别注意，应用程序顶部的新图表工具栏。



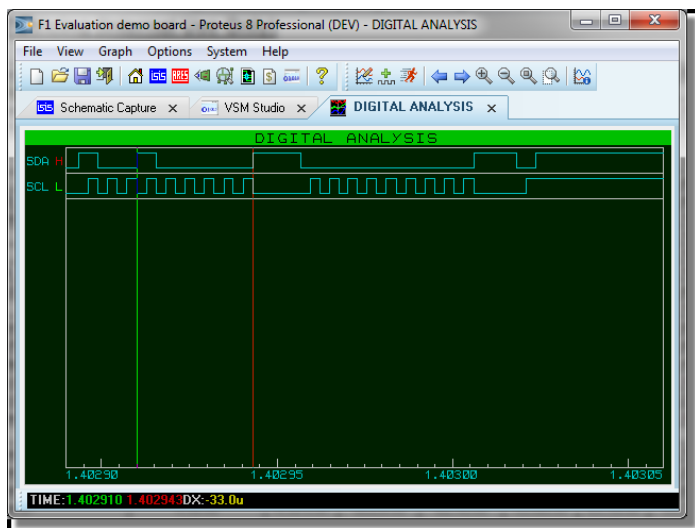
可以在此处找到缩放命令，此处需要用到“缩放到区域”图标。单击此图标，然后在小区域周围拖动以定位显示。如果重复此操作，您很快就能放大单个 I2C 序列。



❗注意：您可以按住 **SHIFT** 键并用鼠标拖动一个框来放大框内的区域。

那些熟悉 I2C 协议的人可以根据我们先前在协议分析器上看到的排序，来验证图中的原始数据。最后，您可以在图表上进行以下测量：

- ◆ 在图表窗口上单击鼠标左键将在图表上放置一个（绿色）光标。
- ◆ 向左拖动鼠标可让您定位光标
- ◆ 按住 **CTRL** 键并单击鼠标左键将放置一个（红色）光标。
- ◆ 按住 **CTRL** 键的同时向左拖动鼠标，可以定位（红色）光标。
- ◆ 每个光标的数据和它们之间的时间增量将显示在图表窗口底部的显示屏上。



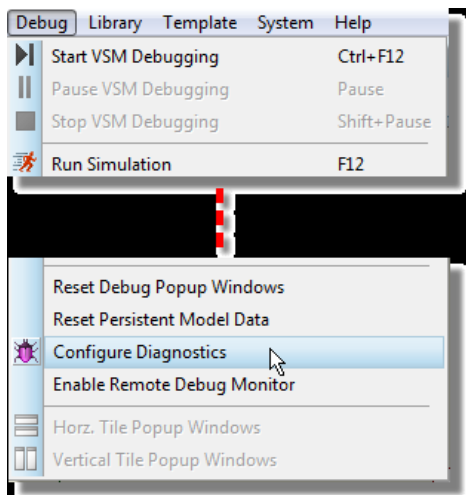
进一步阅读

- ◆ ISIS 教程文档中更详细地介绍了所有常规放置和编辑技术，此处简要讨论。
- ◆ 本文档后面将提供有关图表的更多信息，有关这些激励源，探针和图表仿真的完整讨论，请参见 Proteus VSM 参考手册。

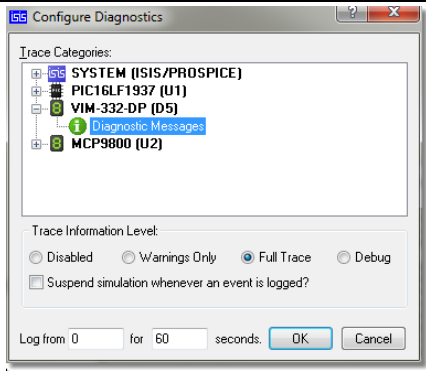
诊断信息

Proteus 中提供的另一个经常被忽略的工具是诊断信息传递。此界面使您可以要求系统提供指定外围设备中活动的纯文本报告。为此，我们将打开 LCD 面板的诊断消息。

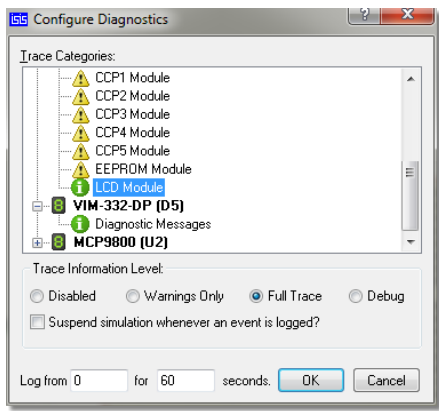
切换到原理图选项卡，然后从“调试”菜单中启动配置诊断命令。



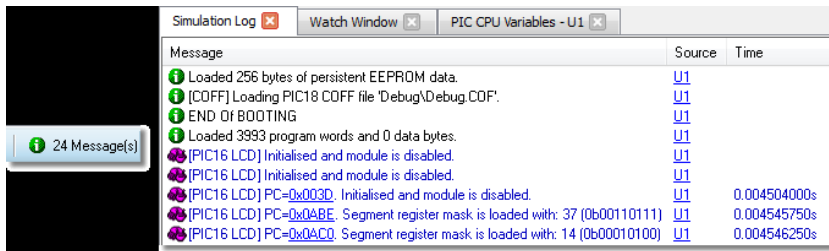
展开显示面板的条目，单击条目，然后将跟踪信息级别更改为完全跟踪。



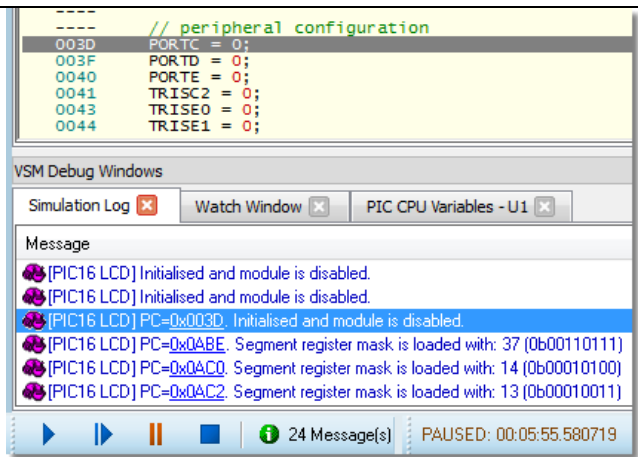
现在，展开 PIC16 并向下滚动至 LCD 模块条目，再次将跟踪信息级别更改为“全跟踪”。



从动画控制面板启动交互式仿真，然后单击仿真顾问以查看诊断信息。这会将 Proteus 切换到 VSM Studio 选项卡，并且仿真顾问的窗口将显示在框架底部。



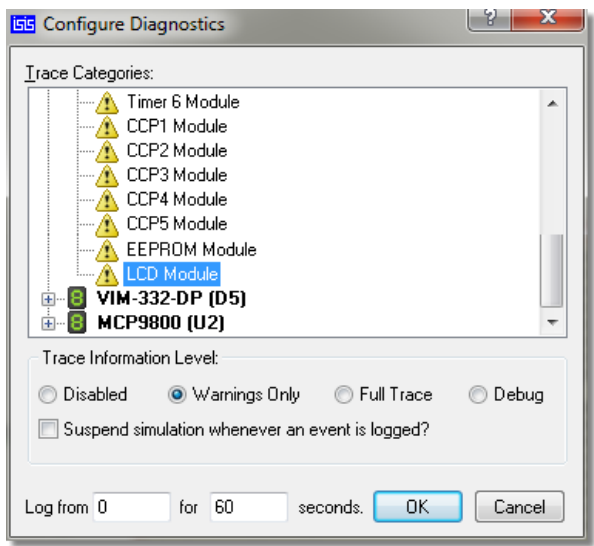
如果您暂停仿真，请注意 LCD 信息包含 PC 值，您可以单击这些值在源代码中找到生成 PC 值的指令。



❗注意：仅在仿真暂停时才可以导航到源代码。

最后，停止仿真，用启用它们的相同方式关闭诊断，把跟踪信息级别更改回仅警告。

诊断信息在出现问题的情况下特别有用，在这种情况下，您需要先收集信息，然后再决定在哪里寻找错误。但它确实会严重影响性能（例如，在温度模式下与 PIC MSSP 外设和 MCP9800 一起尝试），因此仅在需要时才启用。如果您清楚知道您想要了解的活动所处的时间段范围，也可以选择仅在有限的时间段内登录。



2.3 图表仿真

介绍

本教程的目的是通过简单的放大器电路向您展示如何使用 Proteus VSM 执行图表仿真。它会逐

步引导您完成以下任务：

- ◆ 放置图表，探针和激励源。
- ◆ 执行实际的仿真。
- ◆ 使用图表显示结果并进行测量。
- ◆ 对某些可用分析类型的调查。

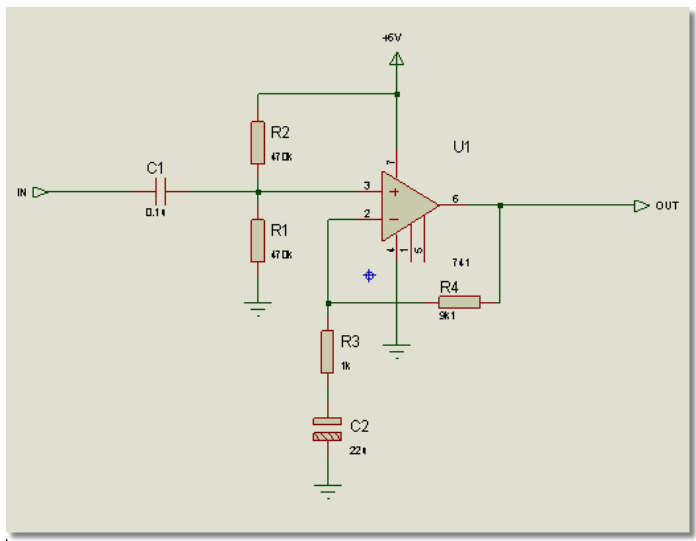
本教程未涵盖一般意义上的 ISIS 的使用--即放置组件、对其进行布线、标记对象等的过程。交互式仿真教程涵盖了 ISIS 的部分内容，ISIS 手册本身对此进行了更详细的介绍。如果尚未熟悉 ISIS 的使用，则必须在使用本教程前去了解如何使用 ISIS。

我们强烈建议您在进行图表仿真之前，按照本教程的方法进行操作：掌握概念后，可以更轻松地吸收参考章节中的内容，并在以后的长期工作中节省大量时间，减少沮丧的体验。

开始

我们将要模拟的电路是基于 741 运算放大器的音频放大器，如下所示。它显示了 741 的不寻常配置，由 5V 单电源供电。反馈电阻 R3 和 R4 决定了增益为 10 倍。输入偏置器件 R1, R2 和 C1 在同相输入处提供了虚拟的参考地，该参考地与输入信号间通过 C1 去耦。

通常情况下，我们将对电路进行瞬态分析。这种分析形式最有用，可以提供有关电路的大量信息。在完成了对带有瞬态分析的仿真之后，将对其他形式的分析进行对比。

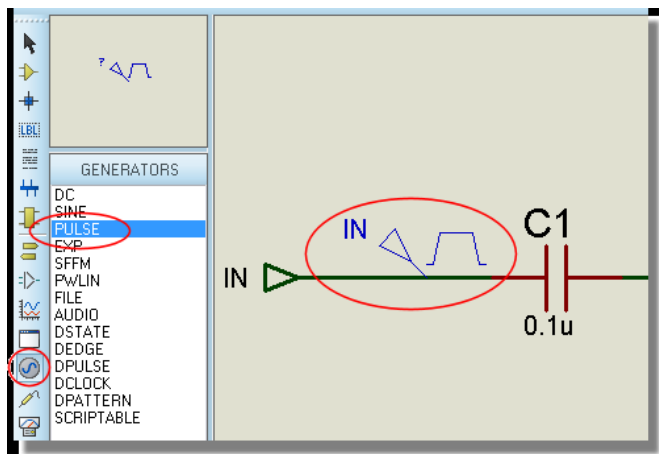


如果需要，您可以自己绘制电路，也可以通过 Proteus 中的“文件菜单”->“打开示例”->“教程”->“模拟仿真教程”（第 1 部分）来加载现成的设计文件。无论您选择什么，此时都要确保已运行 ISIS 并绘制了电路。

激励源

为了测试电路，我们需要为其提供合适的输入。我们将使用具有方波输出的电压源作为测试信号。激励源对象将用于生成所需的信号。

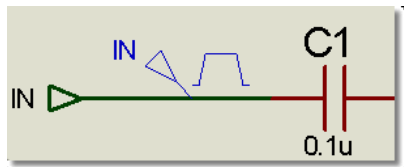
单击激励源图标：对象选择栏显示可用激励源类型的列表。对于我们的仿真，我们需要一个脉冲发生器。选择脉冲类型，将鼠标移到 IN 端子右侧的编辑窗口，然后在导线上单击左以放置脉冲发生器。



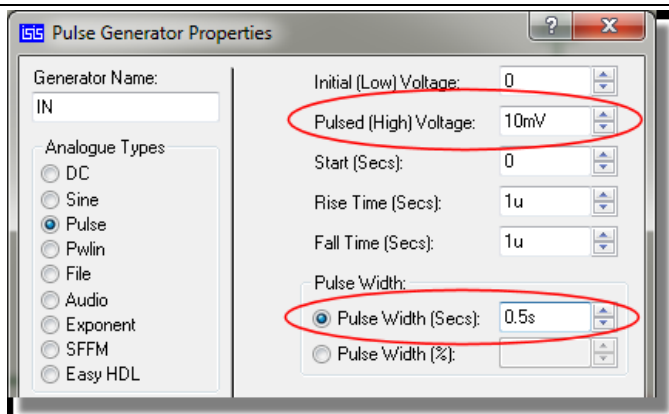
激励源对象与 ISIS 中的其他大多数对象一样。进行相同的步骤，放置之前预览和选定激励源，放置之后编辑、移动、重新选定或删除对象（请参阅参考手册中的“激励源和探针”）。

除了像我们刚才放置到现有的电线上之外，还可以先将激励源直接放置在原理图上，再以常规方式进行布线。如果将激励源从导线上拖出，则 ISIS 会假定您要分离导线，就不会像将导线与组件一样拖曳导线。

请注意，如何为激励源自动分配参考-端子标号 IN。只要将激励源连接到对象（或直接放置在现有电线上），便会为其分配与其连接的网络标号。如果没有网络标号，则默认使用最近的组件引脚的标号。



最后，我们必须编辑激励源来定义所需的脉冲形状。要编辑激励源，请右击它，然后从出现的菜单栏中选择“编辑属性”。选择“高电平”字段并将其值设置为 10mV。同时将脉冲宽度设置为 0.5s。



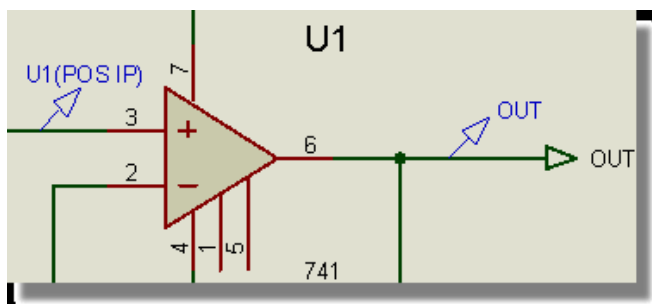
选择确定按钮以接受更改。

❗注意：对于该电路，仅需要一个激励源，但实际上对放置的数量没有限制。

探针

使用激励源作为电路的输入后，我们现在必须将探针放置在我们希望监视的点上。我们显然对输出感兴趣，对输入进行偏置后，输入也是值得监测的点。如果需要，可以在关键点均添加探针，并重复进行仿真。

要放置探针，请在“电压探针”图标上单击鼠标左键（确保您没有意外选择电流探针-我们稍后会介绍）。可以按照与放置激励源相同的方式将探针放置在电线上，或者先放置再布线。将鼠标移到 U1 引脚 3 左侧的编辑窗口，然后左击将探针放在将引脚 3 连接到 R1 和 R2 的电线上。请确保将探针放在电线上，因为它不能放在针脚上。请注意，它获取的标号是它所连接的最近设备的标号，并在小括号中标明了引脚标号。现在，在 OUT 端子左侧的导线上，左键单击放置第二个探针。



探针对象就像 ISIS 中的激励源和大多数其他对象一样。可以使用相同的过程，与放置探针之前预览和选定探针，在放置之后编辑、移动、重新选定或删除探针（有关更多信息，请参阅参考手册中的“探针”部分）。可以编辑探针以更改其标号。在这里的情况下，默认分配的标号是可以的，值得注意，标记探针是针对探针的尖端，而不是主体或参考标号。

现在，我们已经准备好了需要进行仿真的电路，接下来我们将放置一个图表显示结果。

图表

图表在仿真中起着重要作用：图表不仅充当结果的显示介质，而且实际上定义了要执行的仿真。通过放置一个或多个图表并指示您希望在图表上看到的数据类型（数字，电压，阻抗等），ISIS 知道要执行哪种类型的仿真以及哪部分电路需要被包括在仿真内。对于瞬态分析，我们需要一个模拟类型图表。为了将其与数字图表类型区分开来，将其称为模拟而不是瞬态，后者用于显示数字分析的结果，而数字分析实际上是瞬态分析的一种特殊形式。可以使用混合图表在相同的时间轴上显示两者。

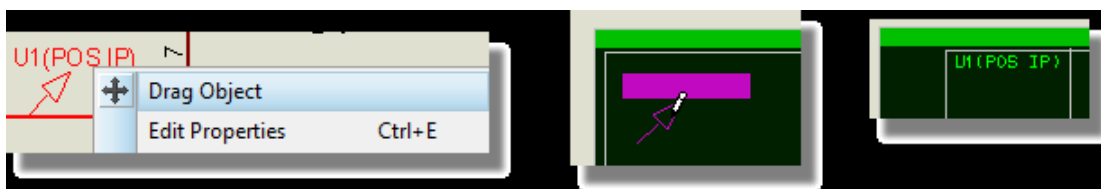
要放置图表，请首先选择“图表”图标：对象选择器显示可用图表类型的列表。选择模拟类型，将鼠标移至编辑窗口，单击第一次开始放置，拖出适当大小的矩形，然后单击第二次以放置图表。



图表的操作类似于 ISIS 中的大多数对象，尽管它们确实有一些微妙的差异。我们将介绍与本教程相关的功能，但是图表的参考章节非常值得一读。您可以使用鼠标左键以常规方式选中图表，然后（使用鼠标左键）拖动其中一个角或图表，以调整图表的大小或重新放置图表。

现在，我们需要将激励源和探针添加到图表上。每个激励源都有一个与之相连的探头，因此无需将探头直接放在激励源上即可查看输入波形。有三种向图表添加探针和激励源的方法：

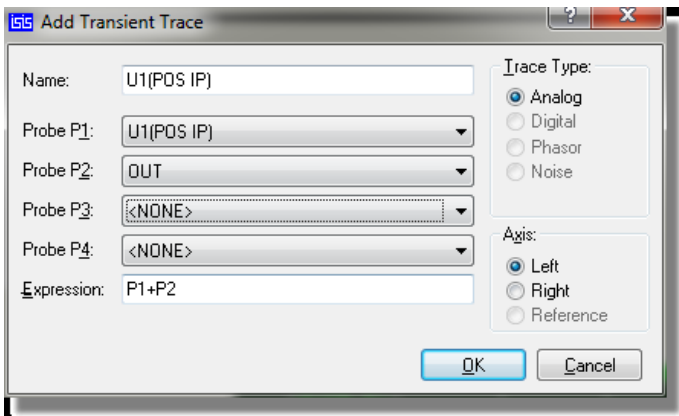
第一种方法是标记探针/激励源，然后将其拖动到图表上并释放它-就像我们在重新放置对象一样，对每个探针/激励源重复此操作。ISIS 检测到您正在尝试将探针/激励源放置在图表上方，将探针/激励源还原到其原始位置，并向图表添加一条与探针/激励源相同的参考线。轨迹可能与图表中的左 y 轴或右 y 轴相关联，并且探针/激励源将添加到最靠近其所在那一侧的轴上。无论将探针/激励源放在何处，任何新的迹线总是添加在现有迹线的下方。



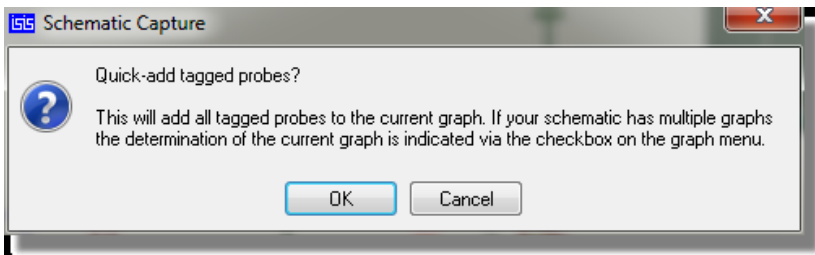
将探针/激励源添加到图表的第二种方法和第三种方法都是使用图表菜单上的“添加跟踪”命令。

此命令将探针添加到当前图表（当有多个图表时，当前图表是图表菜单上当前选择的图表）。

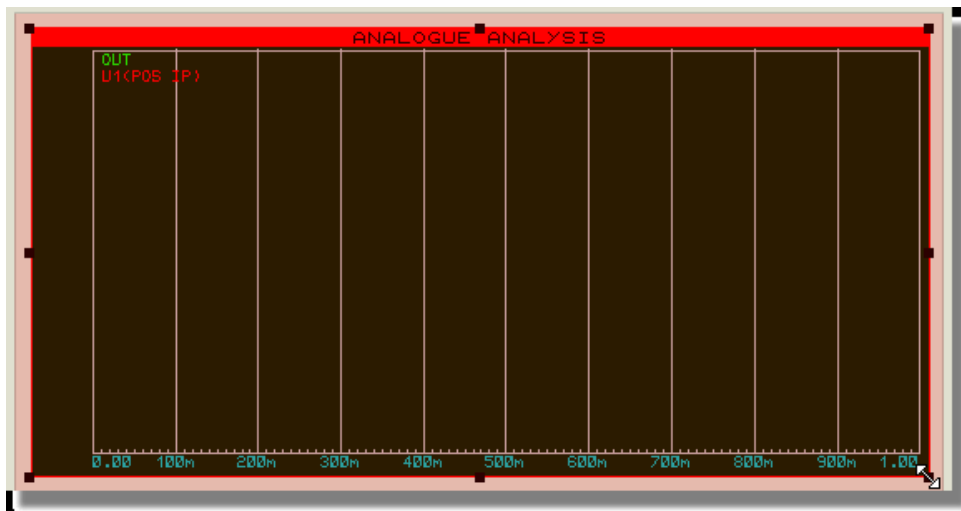
如果在没有任何标记探针或激励源的情况下调用“添加跟踪”命令，则会显示“添加瞬态跟踪”对话框形式，并且可以从本设计包含的所有探针的列表选择一个探针（包括其他原理图图纸上的探针）。



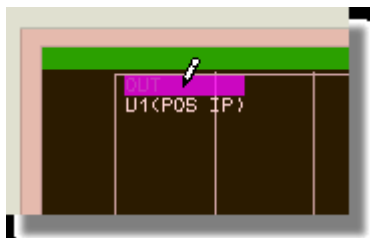
如果存在标记的探针/激励源，则调用“添加跟踪”命令会提示您将标记的探针快速添加到当前图表；选择“取消”选项将调用“添加瞬态跟踪”对话框，如前所述。选择“确定”选项，将所有标记的探针/激励源按字母顺序添加到当前图表。



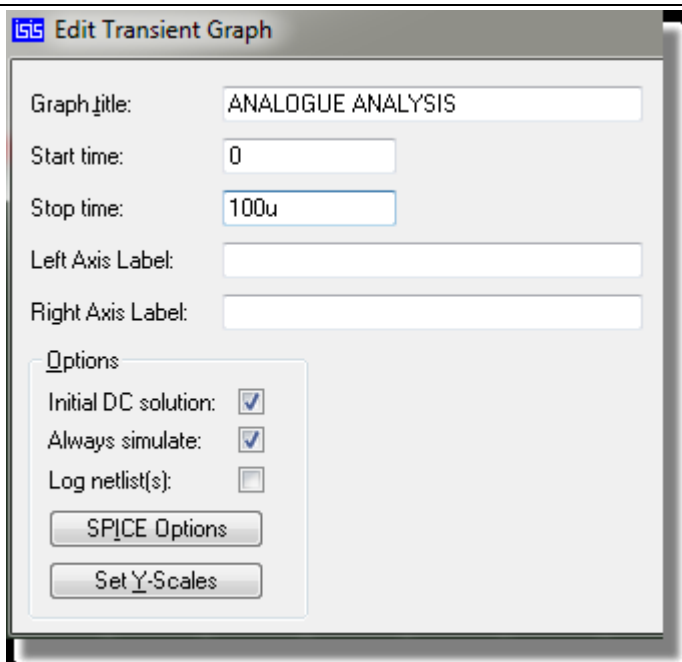
如何快速将探针和激励源添加到图表中？分别标记探针和激励源，或者更快地在整个电路上拖动标记框-快速添加功能将忽略探针和激励源以外的所有已标记对象。从图表菜单中选择“添加跟踪”选项，然后对提示回答“是”。轨迹将显示在图表上（因为只有一个图表，并且它是最后使用的图表，因此被视为当前图表）。此刻，迹线由一个标号（在轴的左侧）和一个空的数据区域（图的主体）组成。如果迹线没有出现在图表上，则 ISIS 可能无法将其画入。通过选中图表并拖动一个角来调整图形的大小以使其足够大。



碰巧的是，我们的轨迹（按字母顺序排列）已经以合理的顺序出现了。但是，我们可以改组有关的迹线。因此，请确保未标记图表，然后在你想要移动或编辑的跟踪标号上单击鼠标左键。跟踪标号突出显示表明它已被选中。现在，您可以使用鼠标左键向上或向下拖动或编辑轨迹（通过在不移动鼠标的情况下单击鼠标左键），并使用右键单击删除轨迹（即从图形中删除轨迹）。要取消选中所有迹线，请在图表上的任意位置单击鼠标左键，但不要在迹线标签上方单击（这将标记或删除迹线）。



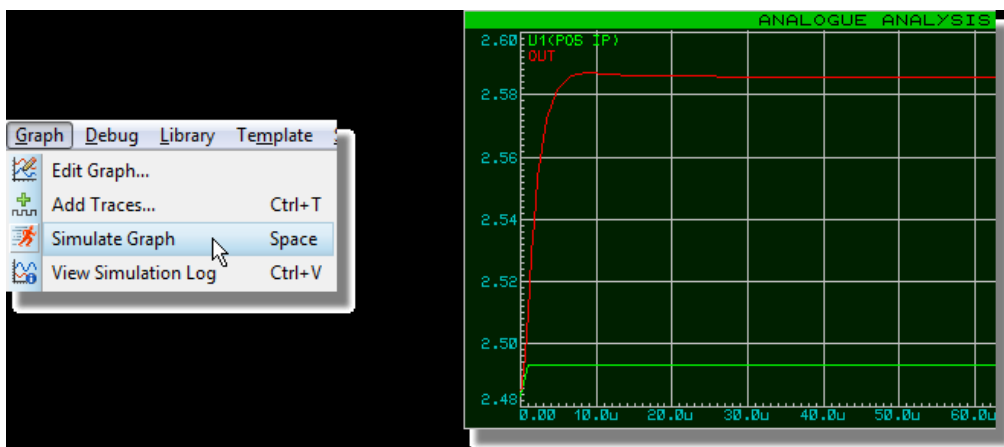
在开始仿真之前，需要完成最后一项设置—设置仿真运行时间。ISIS 将根据图形 x 刻度上的结束时间来仿真电路，对于新图表，默认为一秒钟。为了达到我们的目的，我们希望输入方波具有相当高的音频频率，例如大约 10kHz。这总共需要 100ms。选中图表并在其上单击以调出其“编辑瞬态图表”对话框。此选项窗口可让您为图表添加标题，指定其仿真开始和停止时间（它们对应于 x 轴的最左和右边的值），选中左和右轴（这些不会显示在数字图表上）并指定仿真运行的常规属性。我们需要更改的是从 1.00 到 100u 的停止时间（您可以直接输入 100u-ISIS 会将其转换为 100E-6），然后选择“确定”。



现在可以进行仿真设计了。可以加载我们的设计版本（“打开示例”->“教程”->“模拟仿真教程”（第2部分）），以避免在实际仿真和后续章节中出现任何问题。或者，您继续使用自己输入的电路，等出现问题时再加载默认文件。

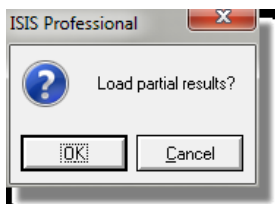
仿真

要仿真电路，您需要做的就是调用仿真命令（或使用其键盘快捷键：空格键）。使用仿真命令可以对电路进行仿真，并使用仿真结果更新当前图表（在图表菜单上选中的图表）。

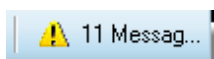


现在就做。状态栏提示了仿真过程已完成的程度。仿真完成后，将使用新数据重新绘制图表。对于当前版本的 ISIS 和仿真器内核，将忽略图表的开始时间-仿真始终从零点开始，一直运行到停止

时间或直到仿真器达到静止状态为止。您可以通过按 ESC 键中止仿真。



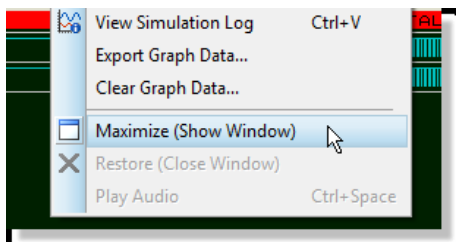
为最后执行的仿真保留仿真日志。您可以通过单击原理图框架左下方的仿真顾问来查看此日志。除非有警告或报错，否则仿真日志很少去阅读。在报错或有警告时，您可以在其中找到具体出了什么问题的详细信息。但是，在某些情况下，仿真日志提供了一些有用的信息，这些信息很难从图表跟踪中获得。



至此，第一次仿真就完成了。查看图表上的迹线，很难看到任何细节。为了检查电路是否按预期工作，我们需要进行一些测量。

测量

位于原理图电路旁的图表是最小化的状态。要进行时序测量，我们首先必须最大化该图表。因此，右键单击图表，然后从出现的菜单栏中选择最大化选项。



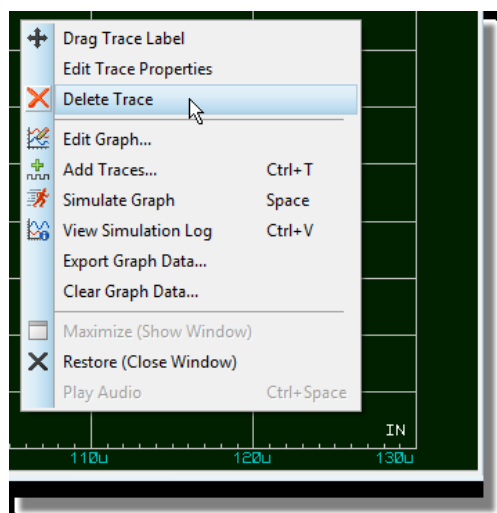
图表将出现在一个新的选项卡中，且屏幕顶部的图标和菜单栏将包括一些适用于图表的功能。



在窗口的左侧是一个区域，其中显示了跟踪标号，而在其右侧是迹线。放置光标后，数据和增量值将显示在图表的底部。由于这是一个新图表，并且我们尚未进行任何测量，因此图表上没有可见的光标，并且状态栏仅显示标题消息。

迹线采用颜色编码，以匹配其各自的标号。OUT 和 U1 (POS IP) 迹线聚集在显示屏的顶部，而 IN 迹线位于底部。要更详细地查看这些迹线，我们需要将 IN 迹线与其他两条迹线分开。这可以通过使用鼠标左键将跟踪标号拖动到屏幕的右侧来实现。这将导致出现右 y 轴，该轴与左 y 轴分别缩放。现在，IN 轨迹看起来要大得多了，但这是因为 ISIS 为右轴选择了比左轴更精细的缩放比例。

为了弄清楚该图，也许最好完全删除 IN 迹线，因为 U1（POS IP）同样有用。在 IN 标签上单击鼠标右键，然后选择删除迹线。现在，图表恢复为左侧的单个 y 轴。



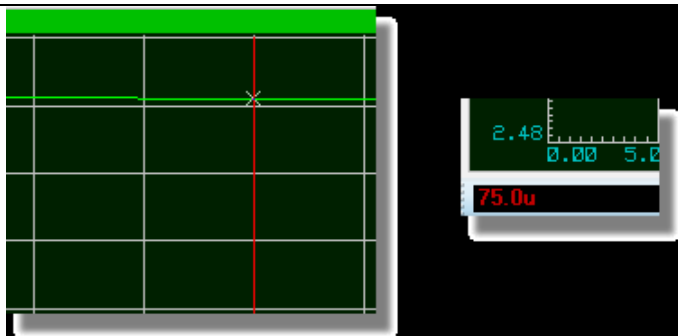
我们将测量两个量：

- ◆ 电路的电压增益。
- ◆ 输出的近似下降时间。

这些测量是使用游标进行的。

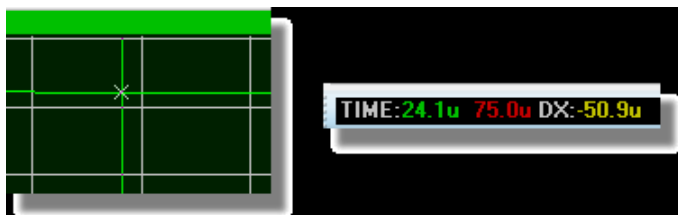
每个图表都有两个光标，称为参考和主光标。参考光标显示为红色，主光标显示为绿色。一旦放置光标将一直被“锁定”在轨迹上，在光标被锁定的轨迹上用‘X’表示。x 轴和 y 轴上的坐标标号都将沿‘X’的位置移动，以方便准确地读取轴。如果使用键盘移动，光标将逐步移动 x 轴的一小格。

让我们开始放置参考光标。访问参考光标和主光标使用相同的键/操作。唯一不同的是有没有按下键盘上的 CTRL 键；参考光标（两者中使用较少）始终可以通过按下键盘上的 CTRL 键来访问。要放置光标，您只要将鼠标指向您想要将光标锁定到的跟踪数据（而不是跟踪标号-其用于其他目的），然后单击左键。如果按下 CTRL 键，则将放置（或移动）参考光标；如果 CTRL 键没有按下，则将放置（或移动）主光标。按住鼠标按钮（以及“参考”光标的 CTRL 键）的同时，您可以左右拖动光标。因此，按住 CTRL 键，将鼠标指针移到图形的右侧，两条迹线上方，然后按鼠标左键。出现红色参考光标。将光标（仍然按住 CTRL 键的同时向下）拖动到 x 轴上大约 70u 或 80u。状态栏上的标题将被删除，将显示光标时间（左侧，红色）和光标电压以及相关迹线的标号（右侧）。这是我们想要的 U1（POS IP）跟踪。

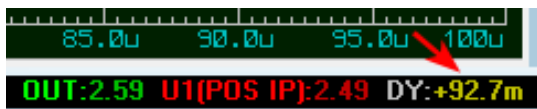


您可以使用左右光标键在 x 轴上移动光标，也可以使用上下光标键将光标锁定到上一个或下一个轨迹。左键和右键分别将光标移动到 x 轴的左边界或右边界。按住 Ctrl 键的同时，按下键盘上的向左和向右箭头键，使参考光标沿时间轴上的小格移动。

现在，将主光标放在 20u 到 30u 之间的 OUT 轨迹上。该过程与上面的参考光标完全相同，不同之处在于您无需按住 CTRL 键。现在，主光标的的时间和电压（绿色）已添加到状态栏中。



还显示两个光标位置之间的时间差和电压差。电压差应小于 100mV。输入脉冲为 10mV 高，因此放大器的电压增益为 10。请注意，该值是正值，因为主光标位于参考光标上方-在增量读数中，该值为主减去参考。



我们还可以通过将光标定位在输出脉冲下降沿的任一侧通过相对时间值来测量下降时间。这可以通过使用鼠标拖动或使用光标键（不要忘记参考光标的 CTRL 键）来完成。主光标应位于下降沿的右侧，而参考光标应位于下降沿开始处的拐角处。您会发现下降沿不到 10ms。

电流探针

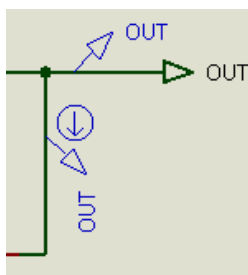
既然我们已经完成了测量，就可以返回电路了，只需点击窗口右侧交叉点来关闭最大化图即可。



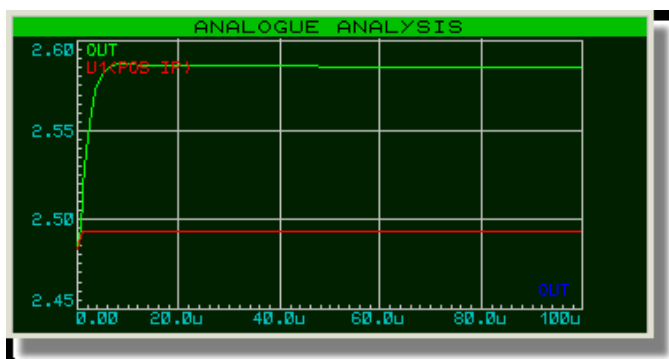
现在，我们将使用电流探头通过测量流入 R4 的电流来检查反馈路径周围的电流。

电流探头的使用方式与电压探头类似，但有一个重要的区别。电流探针需要具有与其相关的方向。电流探针的工作原理是有效地断开电线，并将其插入到间隙中，因此它们需要知道要走的路。只需通过放置它们的方式即可完成。在默认方向（向右倾斜）上，电流探头会测量水平导线中从左到右的电流。要测量垂直导线中的电流，需要将探头旋转 90° 或 270° 。将探头放置在错误的角度是一个错误，将在执行仿真时报告该错误。如有疑问，请查看符号中的箭头，它指向电流的方向。

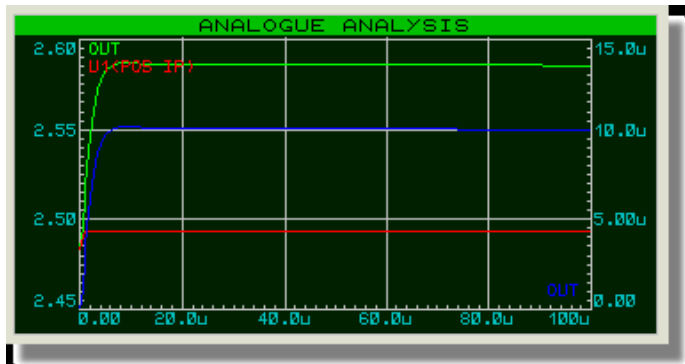
通过单击“电流探针”图标选择一个电流探针。在顺时针旋转图标上向左单击，使箭头指向下方。然后将探头放在 R4 右侧和 U1 引脚 6 之间的垂直线上。



通过标记并拖动到最小化图的右侧，将探针添加到图的右侧。对于电流探头，右侧是一个不错的选择，因为它们通常在比例尺上与电压探头相差几个数量级，因此需要单独的轴来详细显示它们。



目前，没有为电流探针绘制轨迹。按空格键以重新模拟图形，并显示轨迹。



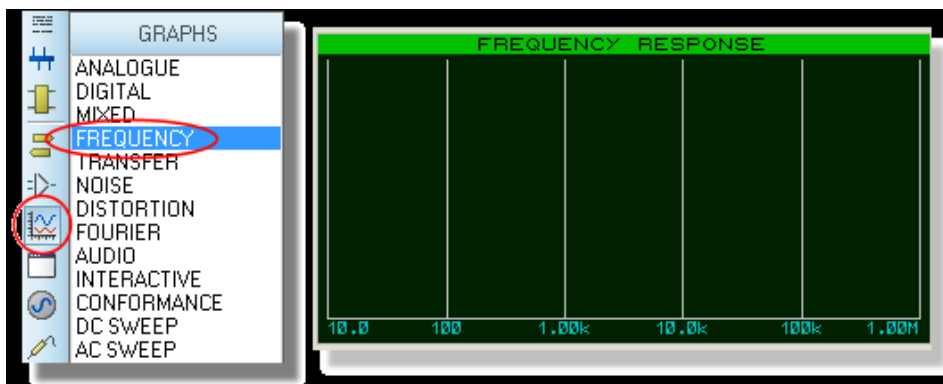
即使从最小化图中，我们也可以看到，反馈环路中的电流紧随输出端的波形，就像您对运算放大器的期望一样。在走线的高和低部分，电流分别在 10mA 和 0 之间变化。您也可以最大化图表以

更仔细地检查轨迹。

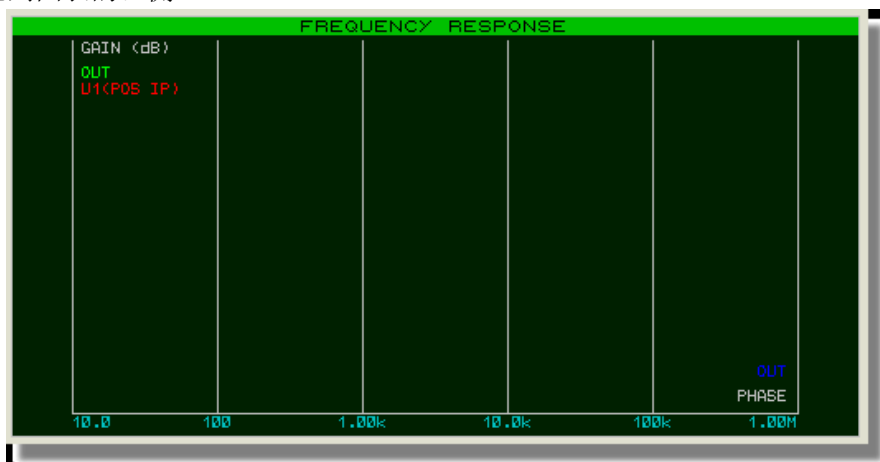
频率分析

除瞬态分析外，在模拟电路仿真中还有其他几种分析类型可用。它们与图表，探针和激励源的使用方式几乎相同，但是在这个主题上都不同。我们将考虑的下一分析类型是频率分析。在频率分析中，x 轴变为频率（对数刻度），并且探测点的幅度和相位都可以显示在 y 轴上。

要执行频率分析，需要频率图表。左击图表图标，在对象选择器中重新显示图形类型的列表，然后单击频率图表类型。然后像之前一样用鼠标左键拖动一个框在原理图上放置一个图表。无需删除现有的瞬态图，但是您可能希望这样做以创建更多的空间（右键单击并从生成的菜单栏中删除对象）。



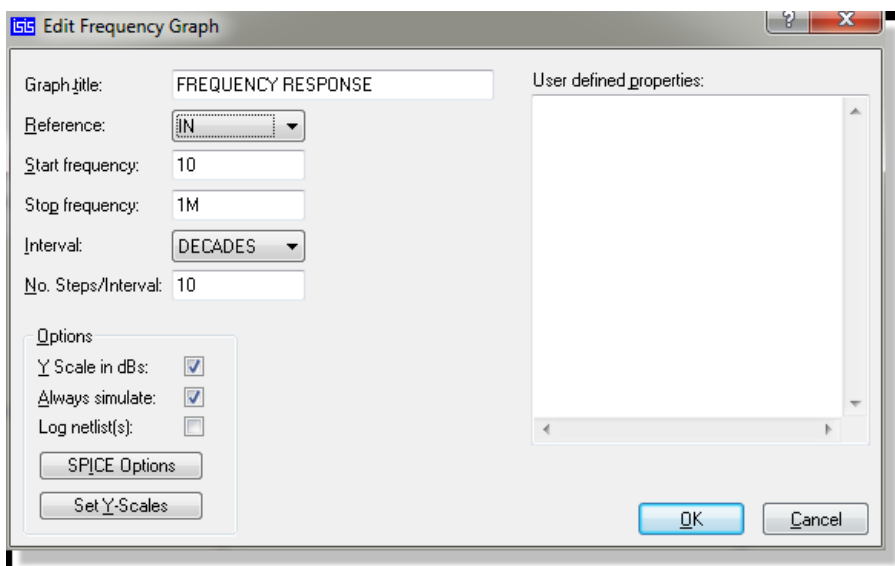
现在添加探针。我们将同时添加电压探头 OUT 和 U1 (POS IP)。在频率图表中，两个 y 轴（左和右）具有特殊含义。左 y 轴用于显示探测信号的幅度，右 y 轴用于显示相位。为了同时看到两者，我们必须将探针分别添加到图表的两侧。标记 OUT 探针并将其拖到图的左侧，然后再次将探针拖到右侧。每条迹线都有单独的颜色，但是它们具有相同的标号。现在，标记 U1 (POS IP) 探针并将其仅拖到图表的左侧。



幅度和相位值都必须相对于某个参考量指定。在 ISIS 中，通过指定参考激励源来完成。参考激励源在 0° 处始终具有 0dB（1 伏）的输出。任何现有的激励源都可以指定为参考激励源。频率分析中将忽略电路中的所有其他激励源。要将 IN 激励源指定为电路中的参考，只需将其标记并拖动到图表上，就好像您将其添加为探针一样。由于它是一个激励源，所以 ISIS 假定您将其添加为参考激励源，并在状态行上打印一条消息以确认这一点。确保已完成此操作，否则仿真将无法正常工作。

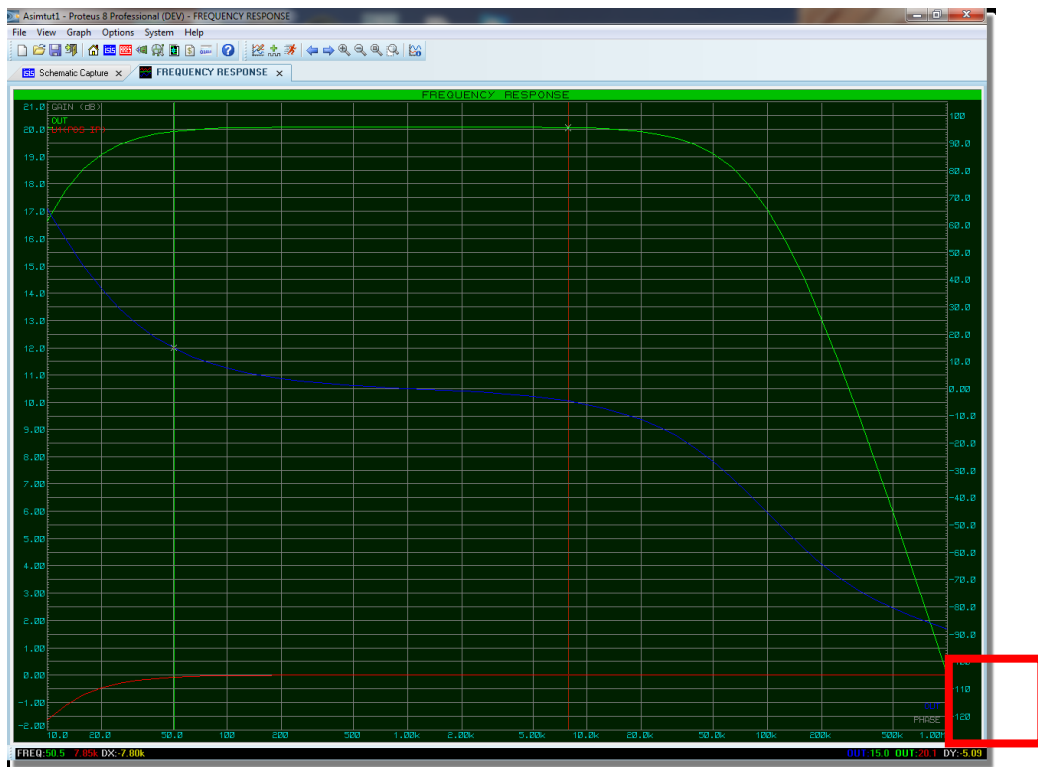


无需编辑图表属性，因为默认选择的频率范围对我们而言是合适的。但是，如果这样做（在图表上单击鼠标右键，然后选择“编辑属性”），您将看到“编辑频率图表”对话框的形式与瞬态情况略有不同。无需标记轴，因为它们的用途是固定的，并且有一个复选框可以显示幅度图（以 dB 或标准单位）。最好将此选项设置为 dB，因为否则显示的绝对值将不是电路中的实际值。



现在，通过右键单击图表并从菜单栏中选择“仿真”选项来开始仿真。完成后，像之前一样在新选项卡中打开图表来最大化图表（右键单击菜单栏）。

首先考虑 OUT 幅度轨迹，我们可以看到通带增益刚好超过 20dB（如预期的那样），可用频率范围约为 50Hz 至 20kHz。游标的工作方式与之前完全相同-您可能希望使用游标来验证上述效果。



OUT 相位迹线显示了在响应极限处的预期相位失真，单位增益对应的频率，相位刚好在曲线图的右侧下降到-90。如果检查 U1（POS IP）幅度轨迹，则可以清楚地看到输入偏置电路的高通滤波器效果。请注意，x 轴的刻度是对数，要从该轴读取值，最好使用光标。

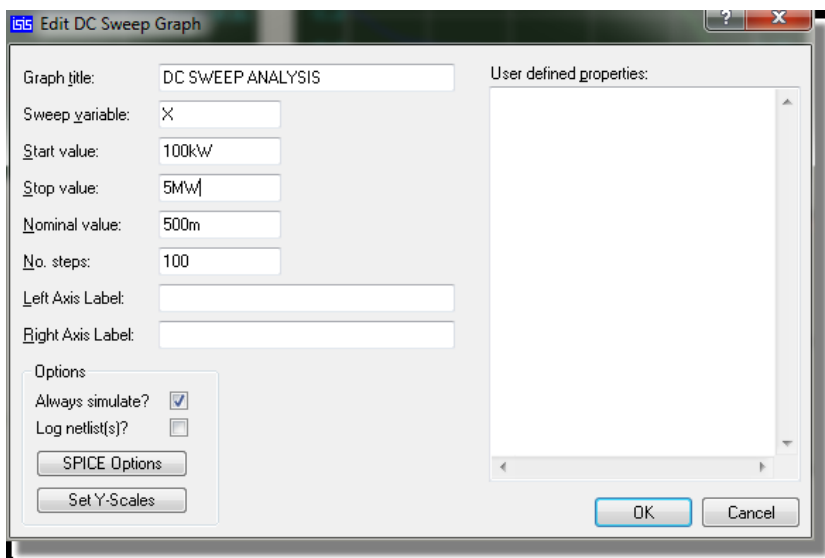
扫描变量分析

ISIS 可以查看更改某些电路参数后对电路的影响。有两种分析类型可以使您执行此操作-DC 扫描和 AC 扫描。直流扫描图表显示一系列针对扫描变量的工作点值，而交流扫描图表显示一系列单点频率分析值，其幅度和相位形式类似于频率图。

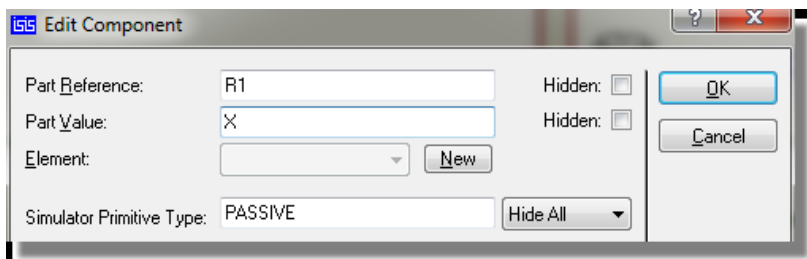
由于这两种分析形式相似，因此我们仅讨论 DC 扫描。输入偏置电阻 R1 和 R2 受流入 U1 的小电流的影响。要了解改变这两个电阻器的值如何影响偏置点，可使用直流扫描。

首先，将直流扫描图表放置在原理图未使用的空间上。然后标记 U1（POS IP）探针并将其拖到图表的左侧。我们需要设置扫描值，编辑图表（右键单击-编辑属性）。“编辑 DC 扫描图表”对话框包含设置扫描变量名称，其开始值和结束值以及扫描中执行的步数的选项。我们要在 100k 至 5M 的范围内扫描电阻值，因此将 Start 字段设置为 100k，将 Stop 字段设置为 5M。单击确定以

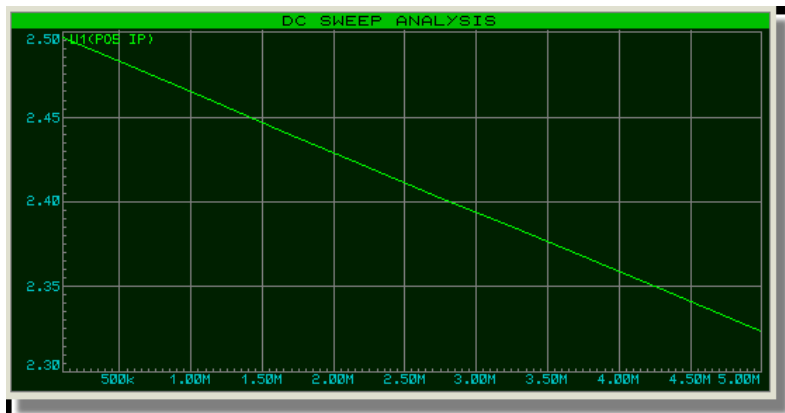
接受更改。



当然，需要改变电阻器 R1 和 R2 使其扫过，而不是已经固定的值。因此请在 R1 上单击鼠标左键以对其进行编辑，然后将“值”选项从 470k 更改为 X。请注意，图表对话框形式中的扫描变量也保留在 X 处。单击“确定”，然后在 R2 上重复编辑过程将其值设置为 X。



现在，您可以右键单击图表并选择“仿真”选项进行图表仿真。然后，通过最大化图表，您可以看到随着偏置链电阻的增加，偏置水平也随之降低。到 5MW 时，它的变化很大。



当然，改变这些电阻也会对频率响应产生影响。我们可以在 50Hz 的频率下进行 AC 扫描分析，以查看对低频的影响。

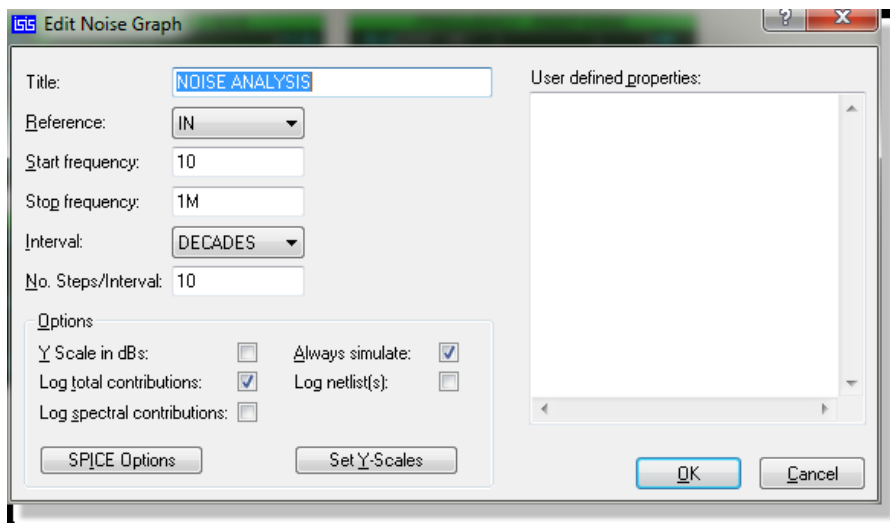
噪声分析

最后一种可用的分析形式是噪声分析。在这种分析形式中，模拟器将考虑每个组件会产生的热噪声量。然后，将电路中每个探测点贡献的所有这些噪声求和（已平方）。将结果与噪声带宽作图。

噪声分析如下重要特点：

- ◆ 仿真时间与电路中电压探头（和激励源）的数量成正比，因为要考虑到每个探头。
- ◆ 电流探头在噪声分析中没有意义，因此被忽略。
- ◆ 仿真日志文件中提供了大量的信息。
- ◆ PROSPICE 计算输入和输出噪声。因此必须定义参考输入-通过将激励源拖到图表上来完成，就像使用参考频率一样。然后，输入噪声图表会显示每个探测到的输出点在输入端的等效噪声。

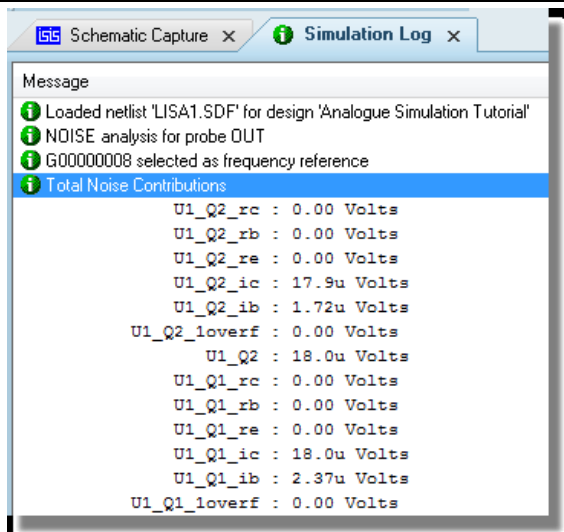
为了对电路进行噪声分析，我们必须首先将 R1 和 R2 恢复到 470k Ω 。然后选择“噪声”图类型，然后在原理图的未使用区域上放置一个新的图表。实际上，我们感兴趣的只是输出噪声，因此标记 OUT 电压探头并将其拖到图表上。和以前一样，仿真的默认值可以满足我们的需要，但是您需要将参考输入设置为激励源 IN 输入。“编辑噪声图表”对话框具有用于显示结果（以 dB 为单位）的复选框。如果使用此选项，请注意 0dB 会被认为是 1 伏 r.m.s。单击取消关闭对话框。



像之前一样图表仿真。当图表最大化时，您可以看到这种分析形式产生的值通常非常小（在本例中为 pV），就像您从此类噪声分析中所期望的那样。但是，如何跟踪电路中的噪声源呢？答案在于仿真日志。现在我们来查看仿真日志，单击原理图框架底部的仿真顾问。



使用向下箭头图标往下移，从“...的总噪声贡献”那一行文字开始，您可以看到打印输出列表。



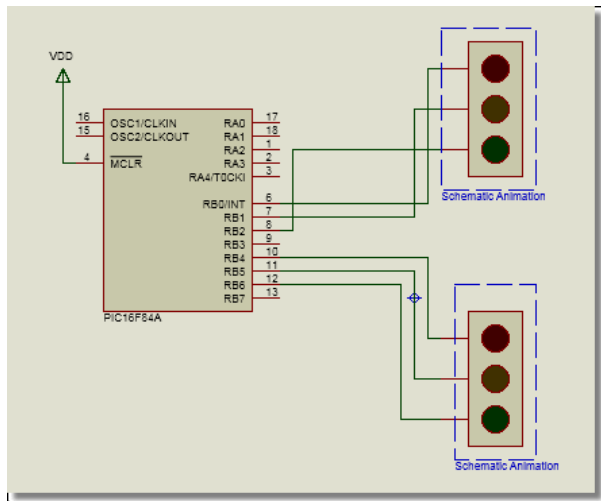
这一列数据列出了每个产生噪声的电路元件的噪声贡献（在整个频率范围内）。实际上，大多数元素都在运算放大器内部，并以 U1 作为前缀。如果在“编辑噪声图表”对话框中选择“对数频谱贡献”选项，那么您将获得更多的对数数据，显示每个分量在每个频率点处的贡献。

2.4 新建工程

介绍

本教程的目的是向您展示从头至尾的原理，如何使用 Proteus VSM 进行交互式仿真。原理图和固件都保持尽可能简单，但是所涵盖的技术将可扩展到任何规模的项目。

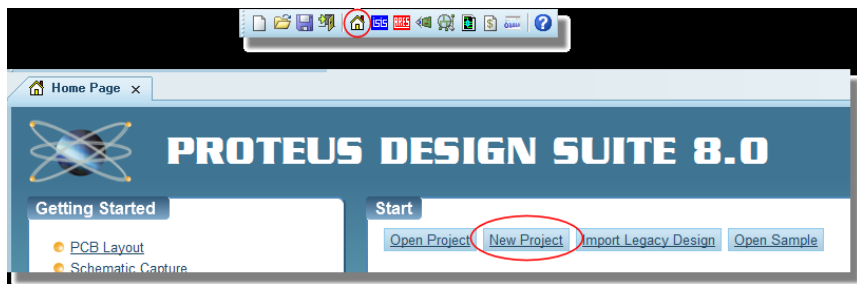
我们将用于仿真的电路是连接到 PIC16F84 单片机的一对交通灯，如下所示：



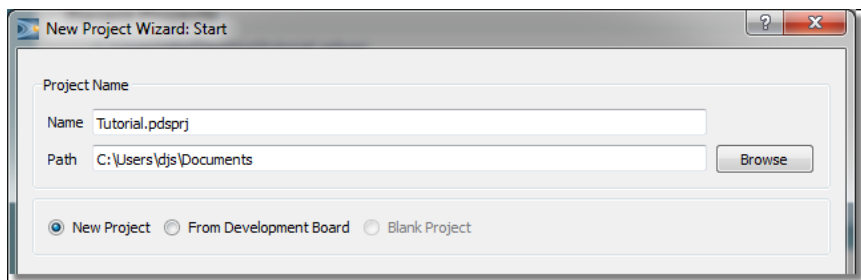
②注意：一旦购买 Proteus 均免费提供 PIC16F84，因此即使您没有任何 Proteus VSM 产品的许可证，也可以按照本教程进行操作。

新建工程向导

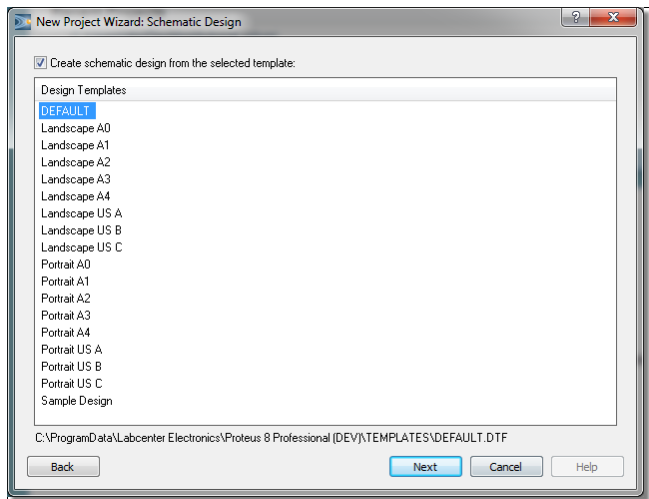
入门 Proteus 8 项目的最简单的方法是使用主页上的“新建工程向导”。首次打开 Proteus 时会显示主页，或者可以从应用程序顶部的模块工具栏中启动主页。



向导的第一页允许您为工程指定名称和保存路径。



我们需要一个原理图设计，因此请选中下一页顶部的框，然后选择合适的尺寸（默认设置为可以）。

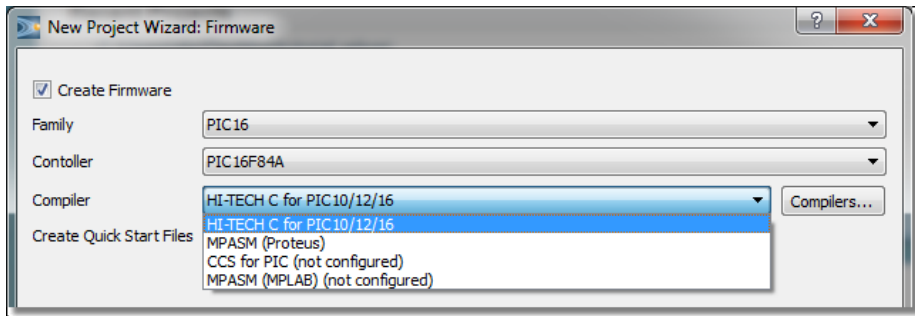


我们不需要此项目的 PCB 布局，因此，如果您看到此页面，则可以跳过它。如果您没有 PCB 产品的许可证，则此页面不会出现在向导中。

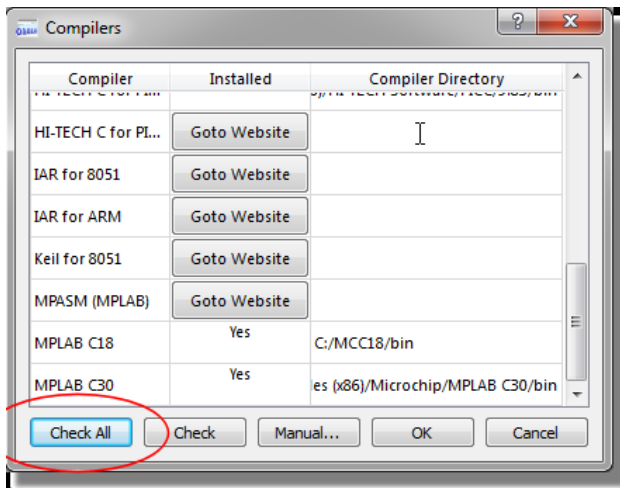
在向导的固件页面上，我们需要进行一些配置。首先选择创建固件复选框，然后指定系列和微控制器。

为简单起见，我们将编写汇编程序，并使用 Proteus 软件安装的 MPASM 汇编程序。因此，我们需要做的就是从编译器下拉列表中选择预配置的 MPASM 汇编器。

这会将支持的编译器的范围缩小到适用于指定控制器的编译器。如果编译器没有安装在计算机上，在组合框中的输入将带有“未配置”后缀。



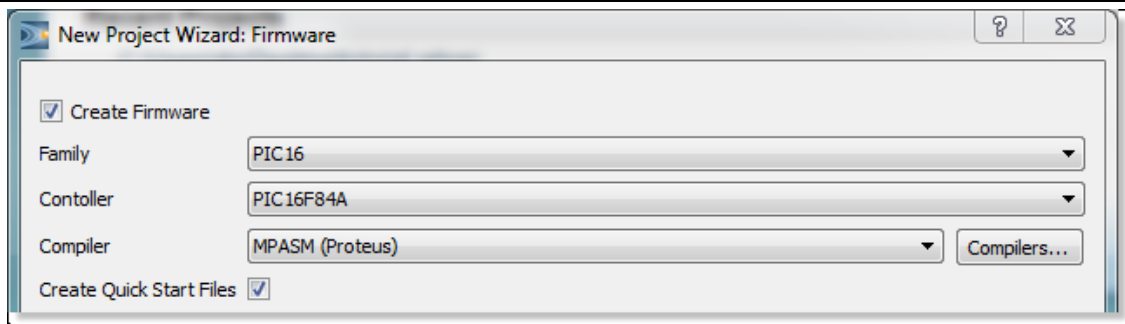
如果要使用未安装的编译器，则可以单击右侧的编译器按钮。结果对话框将使您可以直接从 Labcenter 的网站下载，也可以带您进入编译器供应商网站上的下载页面。无论哪种情况，您都能安装编译器，然后单击对话框底部的“全部检查”按钮以启用其 Proteus 配置。



扩展阅读：

[受支持的编译器](#)

我们的最后一个选择是创建一些框架固件文件。这非常有用，因为它还会为我们设置链接描述文件，因此请选择此项。

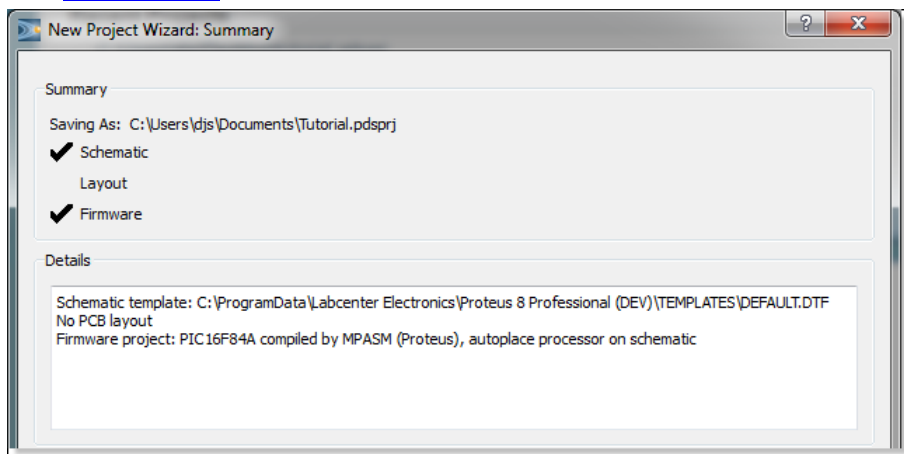


教程项目工程的配置

如果我们正在使用 Arduino，那么我们也可以检查“Create Peripherals”的选项。这样就可以将 Labcenter 外设驱动程序用于 Arduino 扩展板和分线板，从而使这些设备的开发变得更加简单。

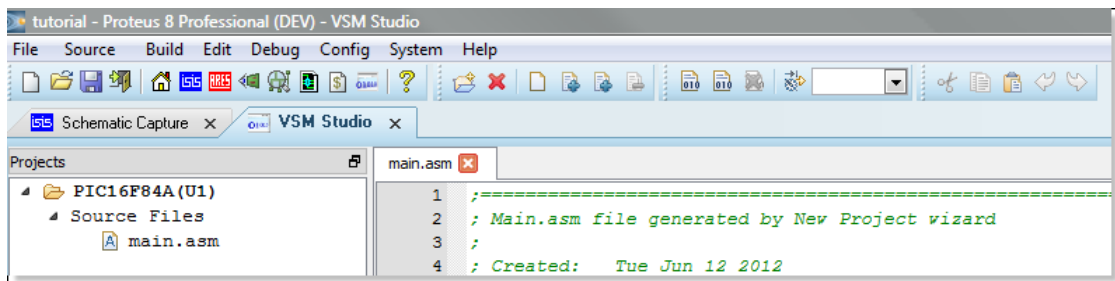
扩展阅读：

[使用 Arduino](#)



新建工程

当您退出向导时，Proteus 将创建一个新项目，乍一看将类似于以下内容。



到这时 Proteus 已经处理了很多事情。Proteus 已经创建了两个模块，一个模块用于原理图设计，第二个模块用于您的固件开发。这些模块以选项卡模式显示，您可以通过单击应用程序顶部的选项卡（或通过 CTRL +TAB 快捷键）在它们之间进行切换。

❗注意：您也可以通过将两个选项卡拖到另一台监视器或桌面的空闲区域来拆分屏幕。

VSM Studio 模块已为您创建了一个固件项目，并在其中填充了框架源文件，并配置了与 Proteus VSM 一起使用的编译选项。同时，在 ISIS（原理图模块）中，已放置所选微处理器并为其供电。

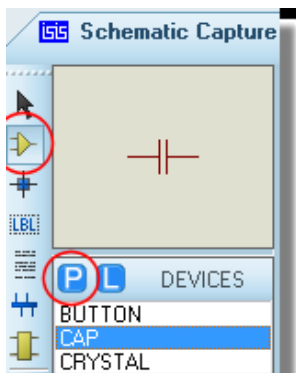
现在，我们需要完成原理图设计并编写固件代码来驱动仿真。

绘制电路图

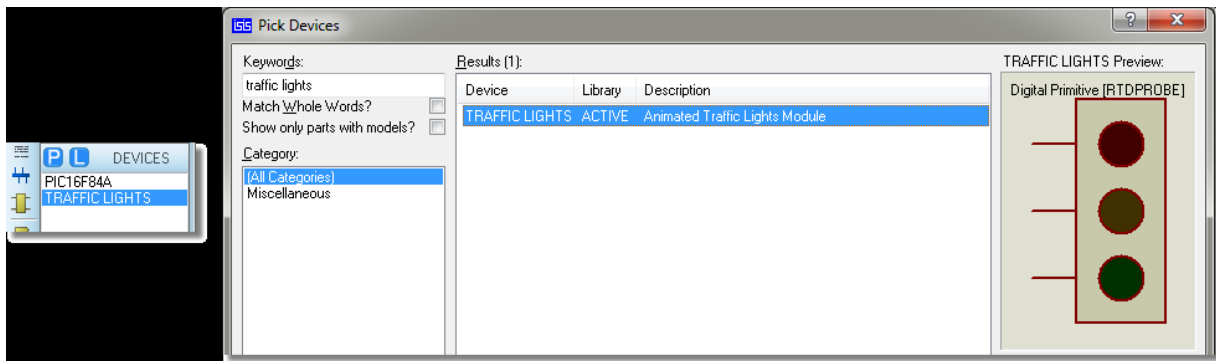
❗注意：如果您使用 [Arduino](#) 做开发，电路将自动绘制，您只需导入扩展板或分线板。

我们从在原理图上放置和连接两组交通灯开始。由于原理图教程涵盖了原理图放置的许多细节，因此我们将快速浏览本节。

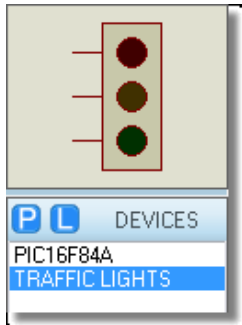
单击原理图设计选项卡以进入原理图模块，然后选择组件模式，然后使用对象选择器上方的“P”按钮启动库浏览器。



在关键字列表中输入“traffic lights”，然后在结果列表中双击该对象以将零件选择到设计中。然后，您应该看到它出现在对象选择器中。



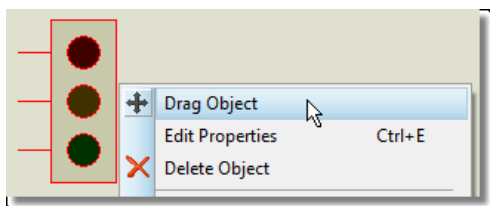
关闭库浏览器，然后左击对象选择器中的交通信号灯。您的选择将突出显示，并且设备预览将出现在“概述”窗口中。



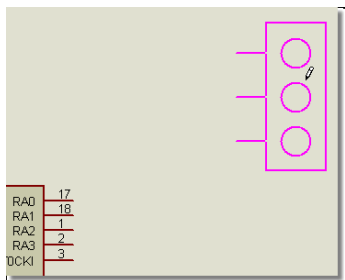
现在，在“编辑窗口”上单击鼠标左键以进入放置模式，将鼠标移至所需位置，然后再次单击鼠标左键以将零件放置在原理图上。重复该过程以放置第二组交通信号灯。

运动与方向

如果交通信号灯位于处理器的右侧，我们的接线将简单得多。要移动组件，请将鼠标指向该组件，右键单击（该组件将突出显示），然后从出现的菜单栏中选择“拖动组件”。

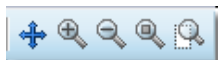


然后，您可以携带组件移动鼠标并单击左键将其放置在所需位置。



缩放

为了方便连接原理图电路，放大到特定区域很有用。最简单的方法是按住 **SHIFT** 键并在要放大的区域周围拖动一个框。或者，您可以使用鼠标中键或 **F6** 和 **F7** 键盘快捷键在当前鼠标位置周围进行放大和缩小。您可以随时使用 **F8** 快捷键恢复为全屏视图。最后，应用程序顶部的菜单图标提供了这些功能的按键。

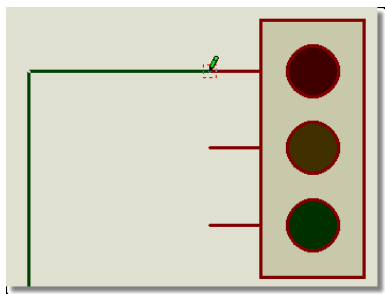


接线

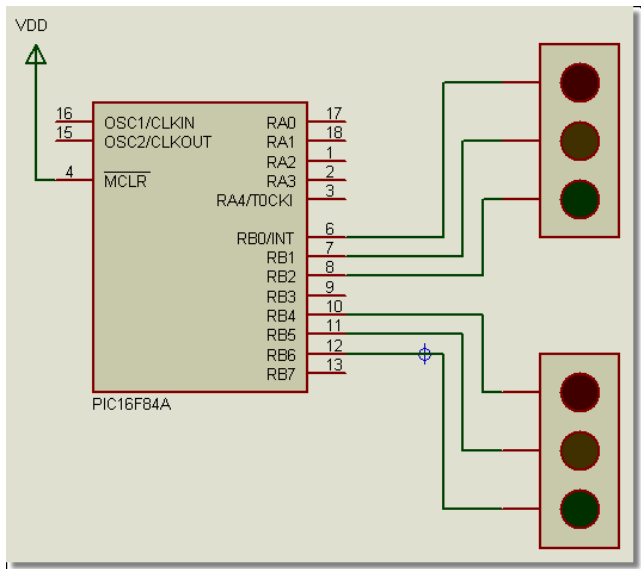
放大到 PIC 微控制器附近，直到所有引脚都可见，然后将鼠标指针放在引脚 6（RB0 / INT）的末端。鼠标光标将变为绿色的笔，表明您在针尖上，可以开始放置导线。



左键单击鼠标以启动电线，然后将鼠标移至连接到其中一组交通信号灯的红灯的引脚。当您到达交通信号灯上的引脚末端时，鼠标光标将再次更改颜色，您可以单击鼠标左键以确定放置导线。



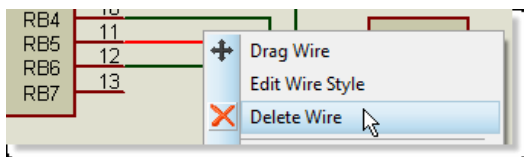
遵循以上步骤，重复此过程来连接两组交通信号灯。



❗注意：如果需要对电线放置进行更多控制，则可以在放置电线时随时单击以更改方向。

如果在导线放置过程中出错，请使用 ESCAPE 键中止导线。如果发生接线错误，可以通过右键

单击电线并从出现的菜单栏中选择“删除”来删除导线。



关于接线过程，有几点值得注意：

- ◆ 您可以以任何模式下进行连接-ISIS 足够聪明，可以确定您在做什么。
- ◆ 您可以在布线过程中单击鼠标左键以提交现有布线并在该点更改方向。
- ◆ 如果在放置导线时将鼠标光标放在编辑窗口上缓缓移动，ISIS 将自动平移屏幕。这意味着您可以放大到最适合放置电线的水平，并且只要知道目标组件的大致位置，就可以轻推屏幕直到其可见。或者，您可以使用鼠标中键或 F6 和 F7 键在放置电线时放大和缩小。

现在，我们已经有了一个简单的原理图，可以通过编写固件来驱动。

编程

单击 VSM Studio 选项卡，将应用程序切换到固件 IDE。就本教程而言，我们将使用以下代码，该代码将使 PIC 能够控制交通信号灯。您可以直接键入，也可以将以下内容复制粘贴到 VSM Studio 中的源文件中。

```

=====
;
=====

; DEFINITIONS

;
=====

#include p16f84a.inc           ; Include register definition file

;
=====

; VARIABLES

;
=====

cblock 0x10    ; Temporary storage

    state

    11,12

endc
    
```

```
=====
;
=====
; RESET and INTERRUPT VECTORS
=====
;
=====
```

```
; Reset Vector
```

```
RST code 0x0
```

```
goto setports
```

```
=====
;
=====
; CODE SEGMENT
=====
;
=====
```

```
PGM code
```

```
setports
```

```
clrw
```

```
movwf PORTA
```

```
movwf PORTB
```

```
bsf STATUS,RP0
```

```
clrw
```

```
movwf TRISB
```

```
bcf STATUS,RP0
```

```
initialise
```

```
clrw
```

```
movwf PORTB
```

```
loop
```

```
call getmask
```

```
movwf PORTB
```

```
incf state,W
```

```
andlw 0x04
```

```
movwf state
```

```
call wait
```

```
goto loop
```

; Function to return bitmask for output port for current state.

; Top nibble controls one set of lights and bottom nibble the other.

```
getmask
```

```
movf state, W
```

```
addwf PCL,F
```

```
retlw 0x41 ; Green and Red.
```

```
retlw 0x23 ; Amber and Red/Green.
```

```
retlw 0x14 ; Red and Green.
```

```
retlw 0x32 ; Red/Amber and Green.
```

; Simple delay routine.

```
wait
```

```
movlw 0xFF
```

```
movwf l1
```

```
w1    decfsz l1
```

```
goto w1
```

```
return
```

```
wait2 clrf l2
```

```
w2    decfsz l2
```

```
goto w2
```

```
return
```

```
;=====
```

END

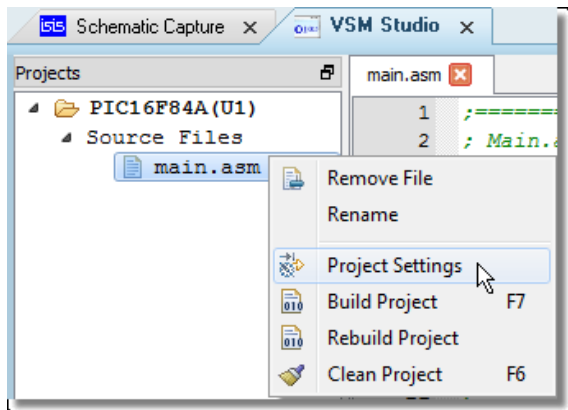
扩展阅读：

[固件开发](#)

实际上，该程序中存在有意为之的错误，但是我们稍后会再讨论。

汇编

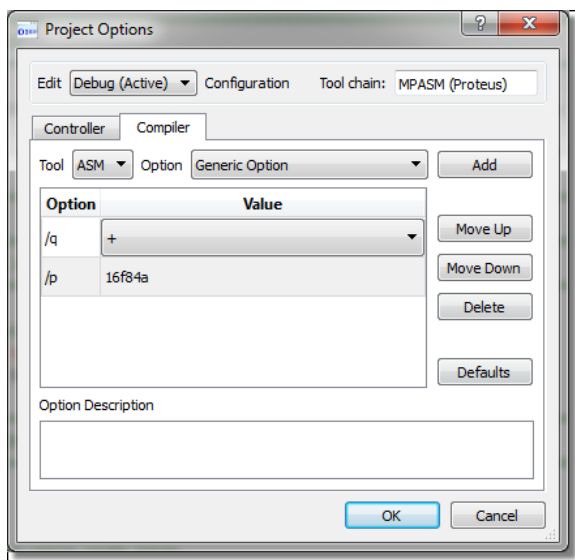
如果需要，可以在右键单击程序名称后（或通过构建菜单）从菜单栏访问工程设置。



然后切换到编译器选项，即可访问指定工具链可用的所有参数。请注意，已经为我们配置了汇编程序和链接程序选项，并且-对于这个简单的项目-默认设置就足够了。

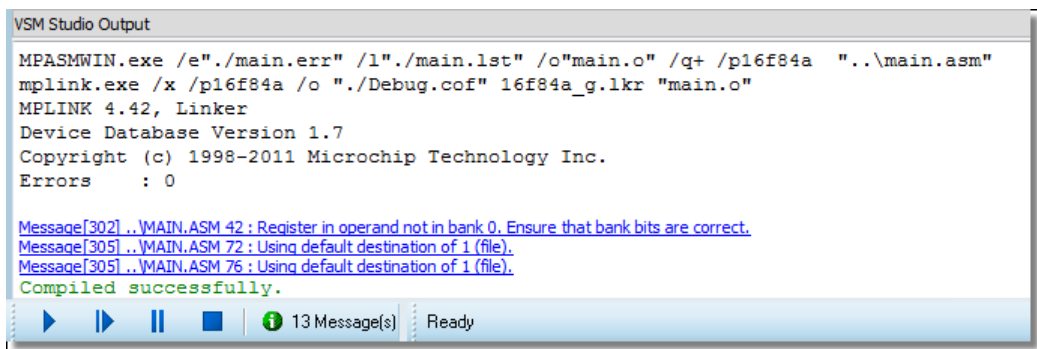
扩展阅读：

[编译器和编译器检测](#)



由于不需要进一步的配置，因此我们可以继续构建程序。从“构建”菜单中选择“构建工程”选项，或使用 F7 快捷键开始编译。汇编器的输出将被写入 IDE 底部的输出窗口，完成后，您应该

可以看到“编译成功”消息。



```

VSM Studio Output

MPASMWIN.exe /e"./main.err" /l"./main.lst" /o"main.o" /q+ /p16f84a  "..\main.asm"
mplink.exe /x /p16f84a /o ".\Debug.cof" 16f84a_g.lkr "main.o"
MPLINK 4.42, Linker
Device Database Version 1.7
Copyright (c) 1998-2011 Microchip Technology Inc.
Errors      : 0

Message[302] ..\MAIN.ASM 42: Register in operand not in bank 0. Ensure that bank bits are correct.
Message[305] ..\MAIN.ASM 72: Using default destination of 1 (file).
Message[305] ..\MAIN.ASM 76: Using default destination of 1 (file).
Compiled successfully.
    
```

仿真与调试

现在，我们已经编译了固件并进行了测试。要开始仿真，只需按应用程序左下方的播放按钮。



这会将应用程序切换到原理图模块，并且开始运行仿真。状态栏上将显示经过的时间和 CPU 消耗的百分比，而原理图将根据当前正在执行的固件来指示电路的状态。

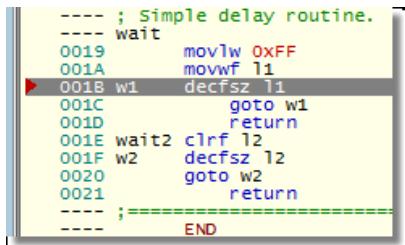
在我们的示例中，您将看到交通信号灯之一为绿色，而另一个为红色。不幸的是，由于我们的固件错误，它们没有改变状态。是时候看一下源代码并进行一些调试了。

设置断点

首先单击应用程序底部的动画控制面板上的“暂停”按钮。这将暂停仿真，并且界面将切换回 VSM Studio IDE。



在 IDE 的编辑窗口区域中，您应该在执行程序中加上当前指令的行覆盖。



```

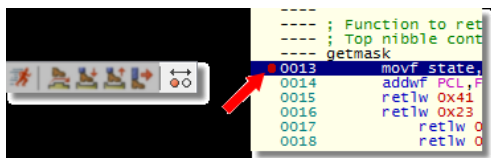
----- ; Simple delay routine.
----- wait
0019      movlw 0xFF
001A      movwf 11
001B w1   decfsz 11
001C      goto w1
001D      return
001E wait2 clrf 12
001F w2   decfsz 12
0020      goto w2
0021      return
-----
-----      END
    
```

查看该程序，可以看到它在一个重复的循环中不断循环后返回自身。因此，最好在此循环中调用的 `getmask` 函数的开始处设置一个断点。

```

----
---- ; Function to return bitmask for output port for current state.
---- ; Top nibble controls one set of lights and bottom nibble the other.
---- getmask
0013      movf state, W
0014      addwf PCL,F
0015      retlw 0x41 ; Green and Red.
0016      retlw 0x23 ; Amber and Red/Green.
0017      retlw 0x14 ; Red and Green.
0018      retlw 0x32 ; Red/Amber and Green.
    
```

您可以通过单击该行和断点图标来执行此操作。或者，双击源代码行将自动设置断点。

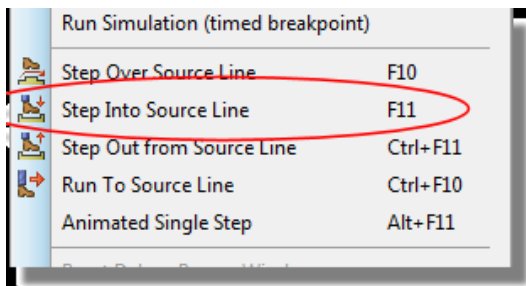


设置断点后，使用播放按钮运行仿真。它很快就会再次暂停，这一次是在断点上执行源代码行。

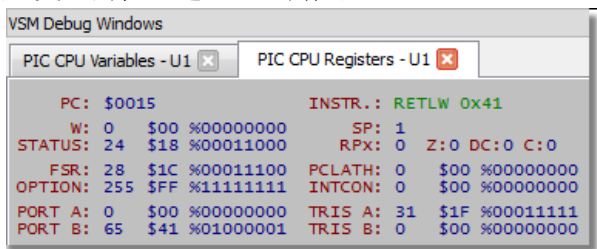
```

---- ; Function to return
---- ; Top nibble controls
---- getmask
0013      movf state, W
0014      addwf PCL,F
0015      retlw 0x41 ; G
0016      retlw 0x23 ; A
0017      retlw 0x14 ; R
0018      retlw 0x32 ; R
    
```

调试菜单上列出了一系列调试命令，这些调试命令已复制到调试工具栏上，但我们最常使用的命令是 F11，单步执行程序。

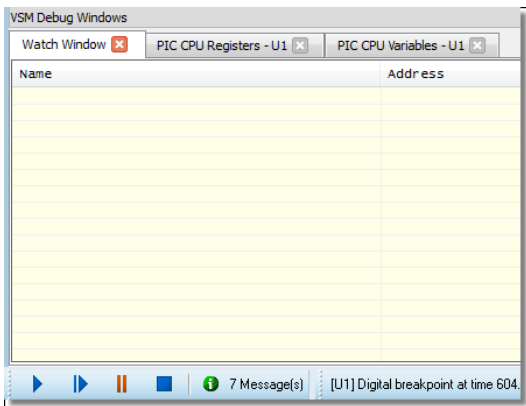


这样做了之后，您应该会看到 call 指令的执行，将我们留在 getmask 指令的入口点。在我们进一步操作之前，有必要打开更多的调试窗口，以便我们可以验证正确的操作。从调试菜单中，选择寄存器窗口，该窗口将显示在 VSM Studio 底部的选项卡中。如果执行单步调试 (F11)，现在应该看到状态变量的值已进入 W 寄存器。

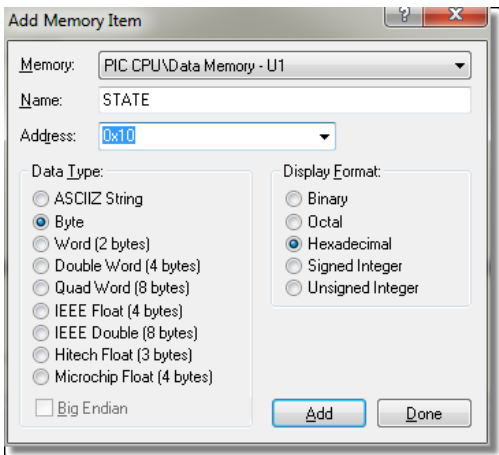


再次前进，我们将看到我们“降落”在了期望的位置，以便在查询表中添加零偏移量。进一步的步骤使我们从 `getmask` 例程中退出，并带有代表绿色和红色的有效遮罩-正是我们在原理图交通灯上看到的。

如果在观察寄存器显示的同时继续执行单步调试（F11），可以看到我们将正确的掩码写入端口（`PORTB` 等于掩码），然后正确地增加了状态（`W` 的值增加了）。下一条指令使状态递增到 3 以上时将状态置零，我们需要更仔细地查看状态变量以验证这一点。最简单的方法是打开监视窗口，然后监视状态变量的地址。您可以从“调试”菜单打开监视窗口，该窗口将显示在 `VSM Studio` 的底部。

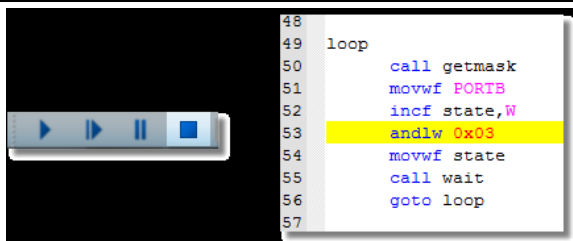


现在，右键单击监视窗口，然后选择按地址添加项目。输入变量名称（状态），地址为 `0x10`（如我们在固件中指定的那样），然后单击添加，然后单击完成。

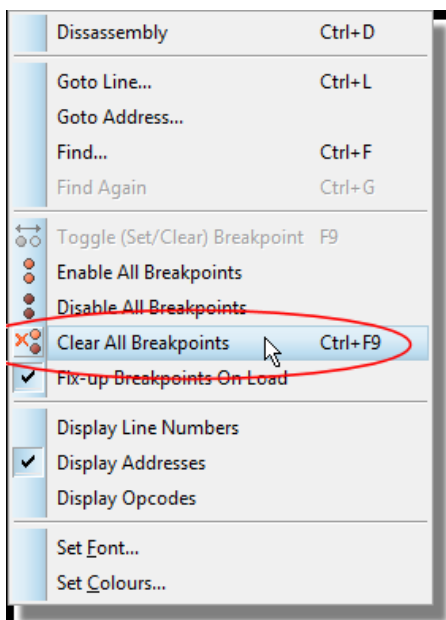


现在，单步执行代码，超出状态变量的设置范围，以检查代码是否正确递增。我们可以立即看到这没有发生。经过仔细研究，问题是与 4 而不是 3 进行与运算引起的。我们想要的状态为 0、1、2、3，而与 4 进行与运算得到的那些值均为 0。解决方案是将问题指令更改为与状态 3 而不是 4。

因此，我们必须首先停止调试会话（单击动画控制面板上的“停止”按钮）。



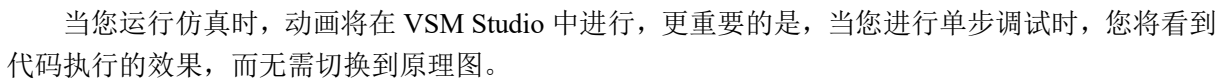
更改问题代码行后，我们需要重新编译（F7），然后再次运行仿真。我们将很快达到断点，并可以重复我们的单步练习来验证我们的更改。或者，右键单击 VSM Studio 中的编辑窗口，然后选择“清除所有断点”命令以允许仿真自由运行。仅从原理图就应该清楚，现在一切正常！



活动弹窗

在逐步浏览的源代码和正在测试的原理图设计之间切换十分不方便。幸运的是，您可以使用活动弹窗将原理图的一部分带入调试 IDE。

在仿真停止的情况下，切换至逻辑示意图捕获模块，然后选择“活动弹窗”图标。现在，在原理图上感兴趣的区域周围拖出框-在我们的示例中，这将是两组交通信号灯。



2.5 导入旧工程

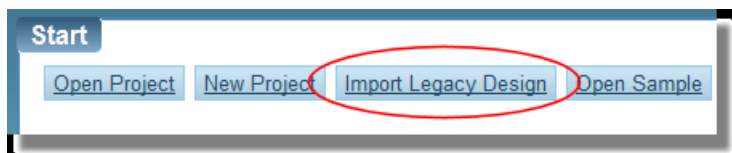
介绍

本简短教程的目的是向您展示如何将 Proteus VSM 旧项目引入 Proteus8。它不作为在仿真或调试过程中使用 Proteus 8 的功能性教程。

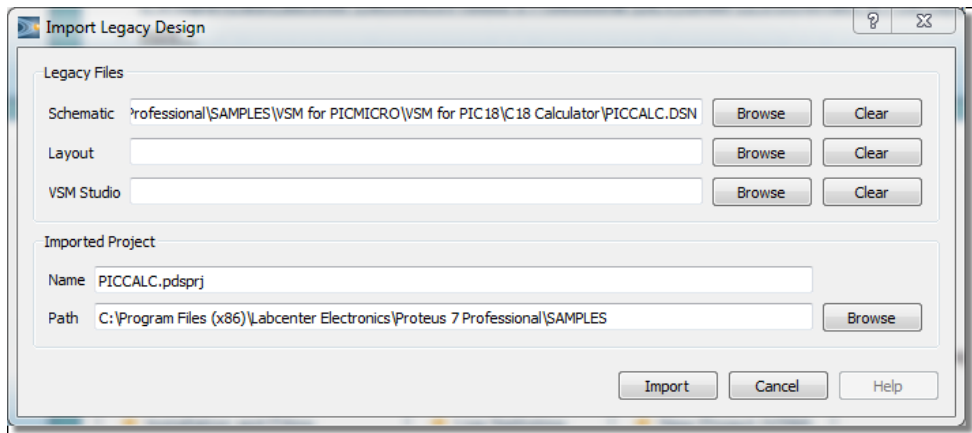
我们将使用旧的（Proteus 7）PIC 计算器设计作为示例，但本教程中的步骤将适用于任何旧版 Proteus VSM 项目。

导入原理图

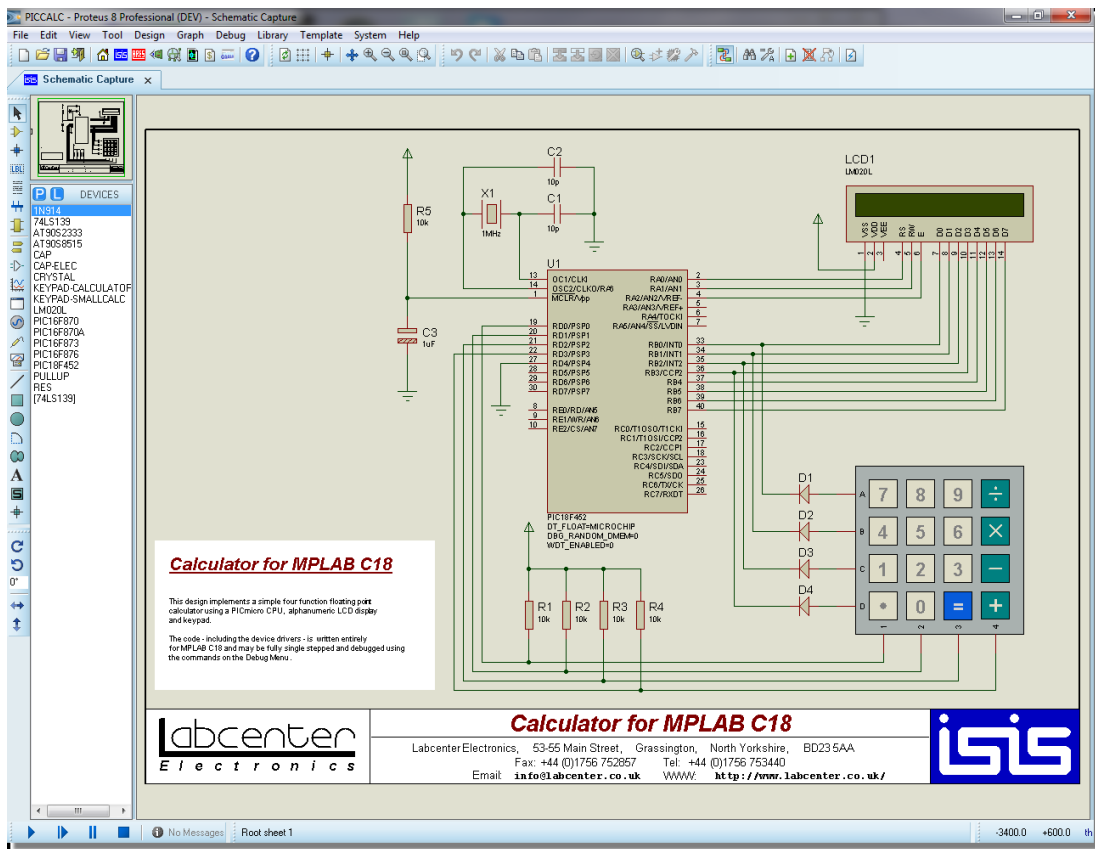
首先要做的是将现有的原理图设计文件（*.dsn）放入 Proteus8。为此，请转到 Proteus 的主页，单击 Import Legacy Design 按钮。



找到要导入的原理图，并为新的 Proteus 8 项目指定名称和位置。



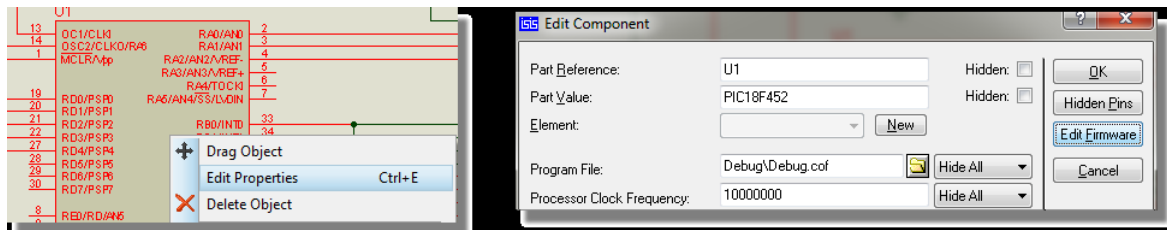
单击导入按钮后，您应该在 ISIS 应用程序选项卡中看到您的 VSM 项目已打开。



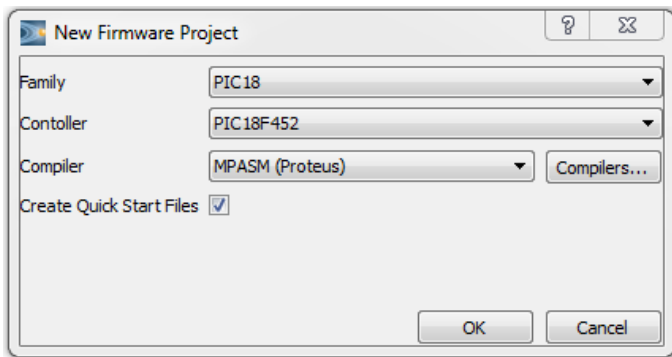
为了最好地仿真和调试该设计，我们现在需要创建一个关联的 VSM Studio 项目。

创建 VSM Studio 项目

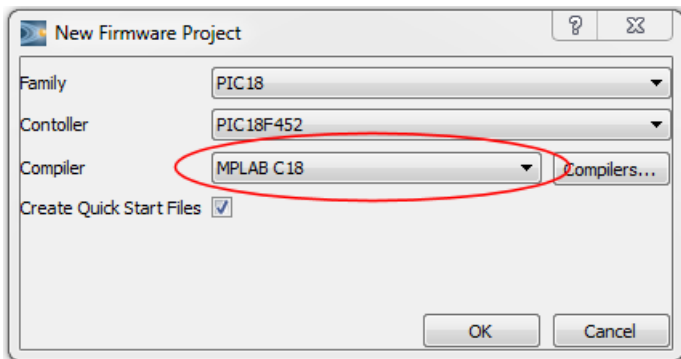
我们需要将原理图上的微控制器绑定到我们的新固件项目。为此，请编辑微处理器部件，然后在出现的对话框中单击“编辑固件”按钮。



由于我们没有当前项目或固件，因此将获得一个空白的 VSM Studio 选项卡，并且将出现一个新的固件工程向导来帮助我们设置项目。



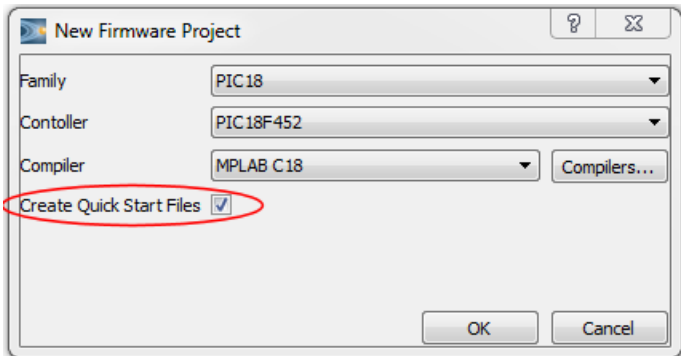
处理器系列和处理器型号将根据原理图上的微控制器部分自动选择。我们需要根据已安装的内容以及先前用于编译程序的内容来更改编译器字段。在本例中，我们使用了 MPLAB C18 编译器，因此我们选择该编译器。



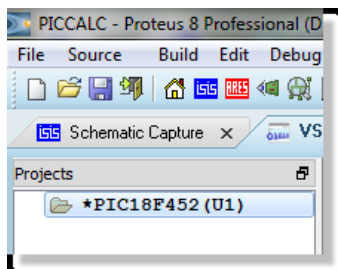
扩展阅读：

[受支持的编译器](#)

最后，向导底部的复选框用于创建框架模板源文件，以帮助您开始编写固件。从头开始新项目时，这非常有用，但是由于我们正在导入现有的源代码，因此我们不需要它，可以取消选择该选项。

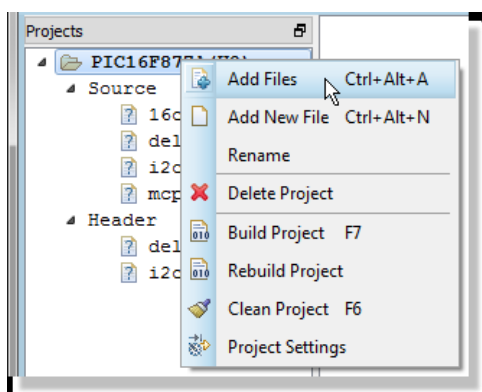


退出对话框后，您现在应该在 VSM Studio 的项目树区域中看到一个带有处理器名称的项目。我们的下一步是将源代码导入到该项目中。

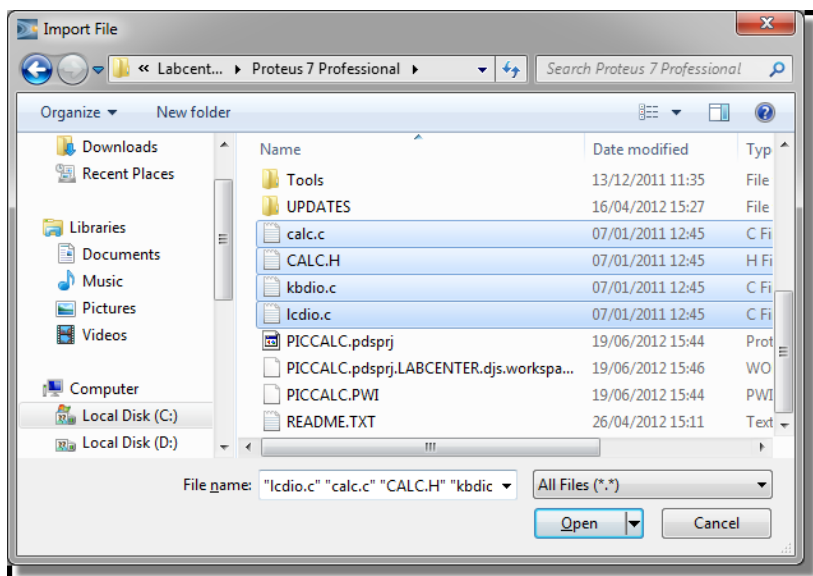


导入源文件

右键单击项目名称，然后从出现的菜单栏中选择“添加文件”命令（或从“源”菜单中启动）。



这将打开一个标准的 Windows 文件对话框形式，您可以从中导航并选择旧的源文件。



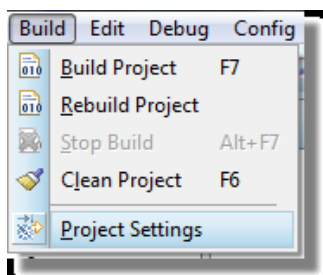
②注意：在此对话框中单击文件时，按住 CTRL 键，可以同时选择多个文件进行导入。

根据需要重复，直到所有源文件都已导入。完成后，所有目前需要的工作都在 Proteus 8 中。如果需要更改，双击源文件将在“编辑窗口”中将其打开。

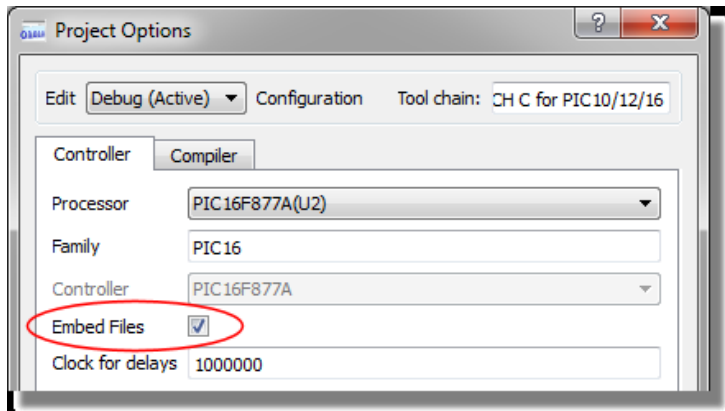
工程设置

为了成功地编译项目，我们需要具有与之前使用的选项相同的选项集。在大多数情况下，这只是 VSM Studio 自动为您配置的默认选项集。但是，通常需要手动将其他包含目录或其他项目的特定选项添加到项目设置中。

您可以通过右键单击项目树或从 VSM Studio 中的“构建”菜单启动工程设置。

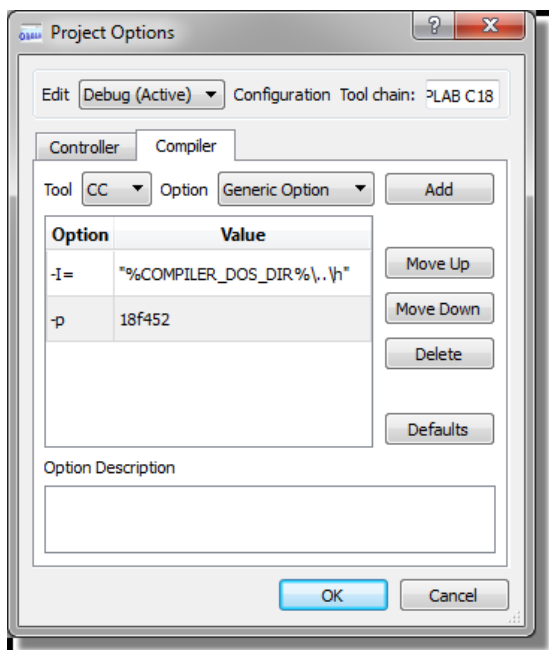


微控制器选项卡上的“嵌入式文件”复选框值得进一步讨论。



选择此选项后，所有源文件都存储在 Proteus PDSRJR 主文件中。临时会话目录用于编译和仿真。这具有 Proteus 项目可移植且自包含的优点。如果关闭此复选框，则所有源文件都将与 Proteus 主文件存放在同一目录中。然后，您有责任组织和维护项目中各个文件的相对/绝对路径。

然后切换到“编译器”选项卡，可以在所选工具链中为编译器，汇编器，工具链和 lib 工具设置选项。



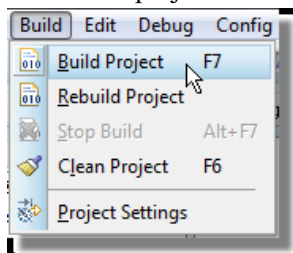
在我们的示例中，默认配置很匹配此工程，因此不需要更改。

扩展阅读：

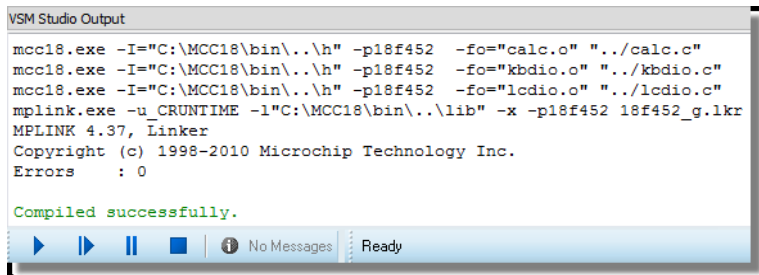
[工程设置](#)

编译并运行

我们需要做的最后一件事是构建源代码并运行仿真。您可以随时使用 F7 快捷键或通过“构建”菜单上的 **build project** 命令来编译工程。



在编译过程中，来自工具链的消息将显示在 VSM Studio IDE 底部的“输出”窗口中，如果没有问题，则显示“已成功编译”消息。



```
VSM Studio Output

mcc18.exe -I="C:\MCC18\bin\..\h" -p18f452 -fo="calc.o" "../calc.c"
mcc18.exe -I="C:\MCC18\bin\..\h" -p18f452 -fo="kbdio.o" "../kbdio.c"
mcc18.exe -I="C:\MCC18\bin\..\h" -p18f452 -fo="lcdio.o" "../lcdio.c"
mplink.exe -u_CRUNTIME -l"C:\MCC18\bin\..\lib" -x -p18f452 18f452_g.lkr
MPLINK 4.37, Linker
Copyright (c) 1998-2010 Microchip Technology Inc.
Errors      : 0

Compiled successfully.
```

在 VSM Studio 中成功编译后，会自动将固件发送到原理图上的微控制器部分，以进行仿真。为了在原理图上运行程序，我们需要做的就是按下动画控制面板上的播放按钮。



❗ 注意: 调试和测试不在本教程的讨论范围之内,但是在[新建工程](#)中进行了简要介绍,在[Proteus VSM 使用指南](#)中进行了完整介绍。

第三章 交互式仿真

3.1 基本技能

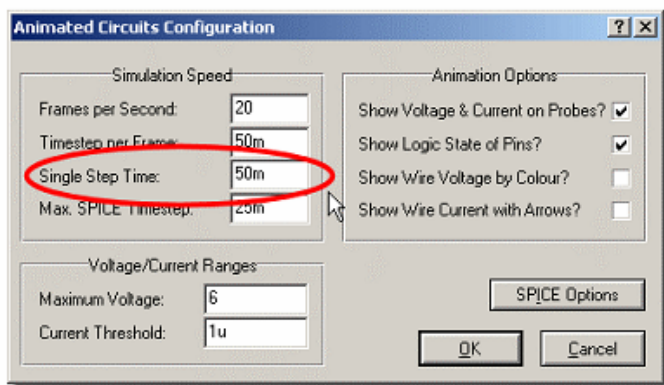
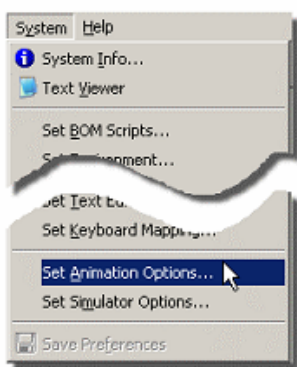
仿真控制按钮



交互式仿真通过上图中的一组按钮控制，它们位于编辑窗口的右下方。如果控制按钮没有显示，可以从 GRAPH 菜单中 CIRCUIT ANIMATION OPTION 进行设置。按钮包含四个：

1. PLAY：用来开始仿真。
2. STEP：用来步进仿真。步进时间由 Animated Circuit Configuration 对话框设置。
3. PAUSE：用来暂停仿真。（也可以通过 PC 键盘上的 PAUSE 按键暂停）
4. STOP：用来停止实时仿真。（也可以通过 PC 键盘上 SHIFT-BREAK 停止）

使用者可以设置仿真步进时间，通过相关设置可以更好的观察电路变化的细节。



注意一点，单步按钮并不是指程序的单步执行，而仅表示一段时间内电路变化。

在仿真过程当中，在状态栏中显示电路仿真时间及平均 CPU 负荷。如果 CPU 能力不足以进行实时仿真，CPU 负荷会显示 100%。如果没有错误，电路会按照设置的帧率自动进行仿真工作。

指示及执行

除了常规的电子元件，交互式仿真还需使用到动态器件。这些动态器件包含一组图形状态，两种方式：指示及执行。INDICATORS（指示）显示电路参数变化时的图形状态，而 ACTUATORS（执行）允许用户决定状态并修改一些电路的参数。

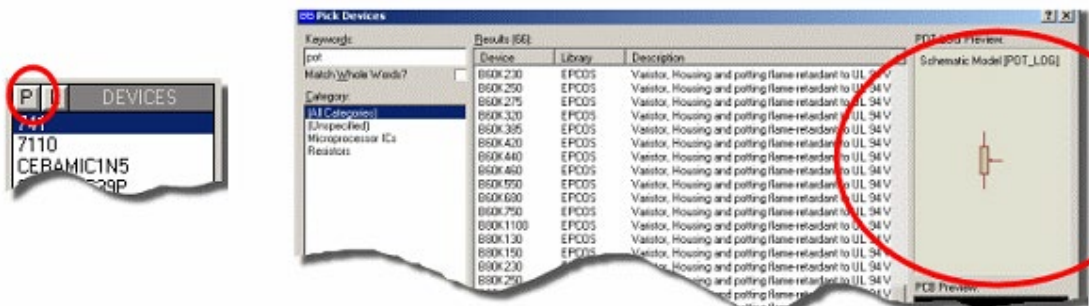
ACTUATORS 可以通过一个小红圈控制，通过鼠标点击来控制，也可以直接通过鼠标滚轮来确

定位置是否合适。

建立一个交互式仿真

在 ISIS 中绘制一个原理图，整个过程如下：

1. 在元件库中选择需要的元器件。



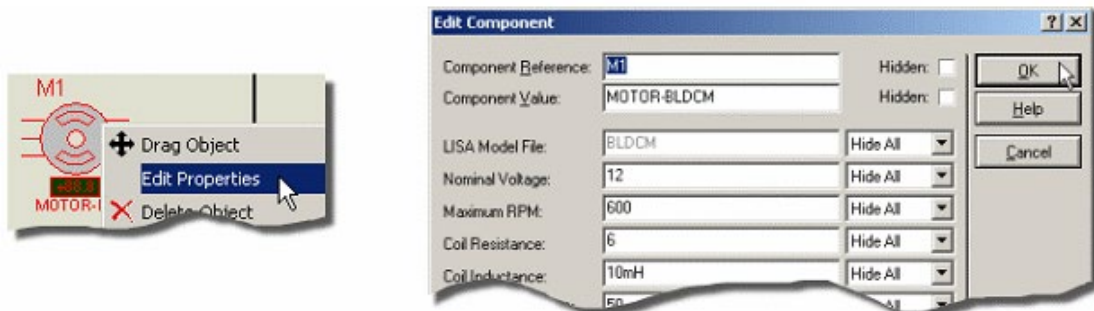
通过库浏览器选择有仿真功能的元器件

2. 在对象选择窗口中左键选中元件，在编辑窗口中放置元件。



在 ISIS 中放置元件

3. 选中元件，左键单击调出属性窗口，编辑属性。

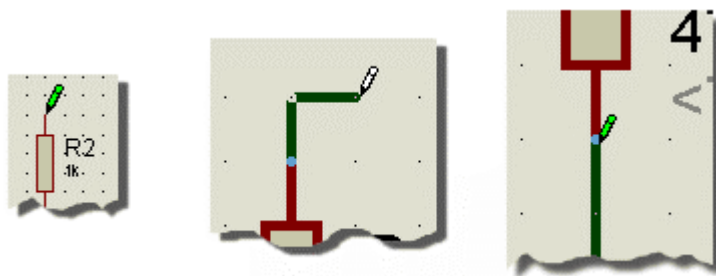


仿真前在 ISIS 中编辑元件属性

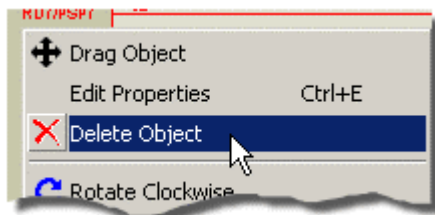
4. 处理器源代码调试：自带汇编器编译的代码可以在 PROTEUS VSM 的 SOURCE 菜单下调出源码，其他可以通过装载第三方 IDE 生成的调试代码（COF，ELF，HEX 等）进行源码调试



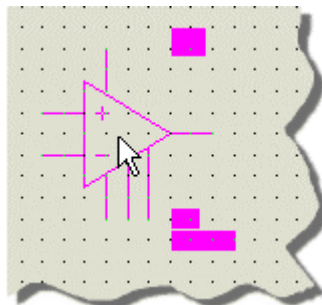
5. 原理图连线



6. 删除：右键单击元件，从下拉菜单中选择删除元件。



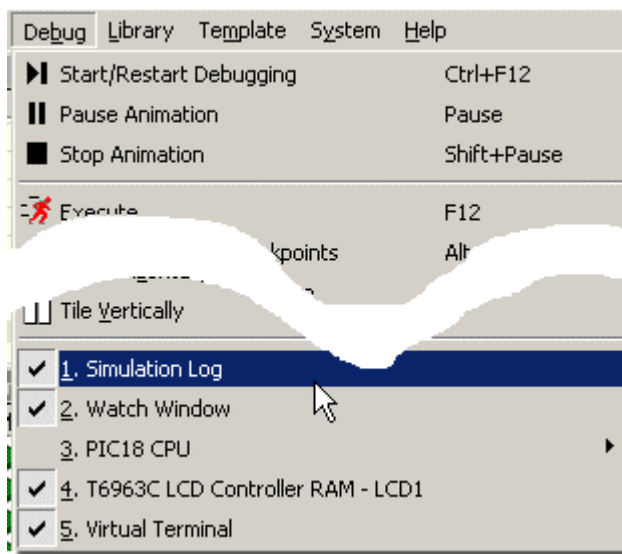
7. 移动：选中元件，按住左键移动。



8. 点击暂停键，仿真暂停，仿真时间归零。



9. 在使用虚拟仪器、处理器 VSM 模型时，在仿真时 DEBUG 菜单下会多出若干命令，使用者可以添加自己关心的窗口进行观测。



3.2 测量

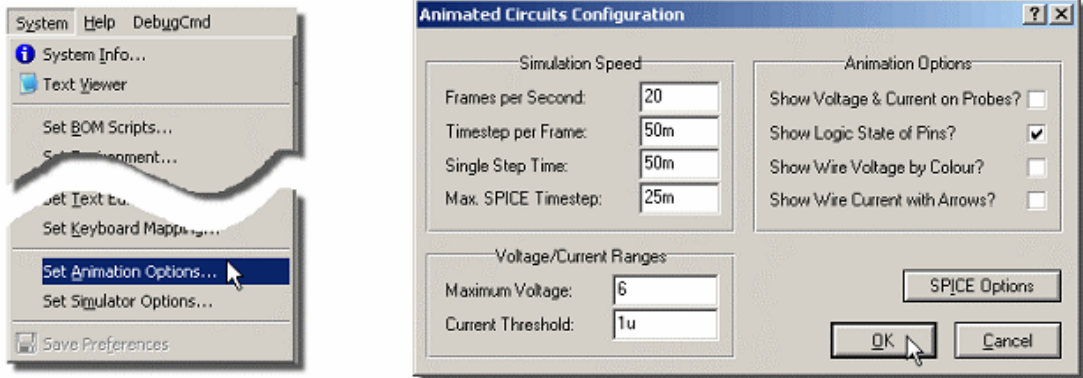
概况

在交互式仿真中可以运用下列测量方法：

1. 仿真效果
2. 指示操作参数信息
3. 电压/电流探针
4. 虚拟仪器

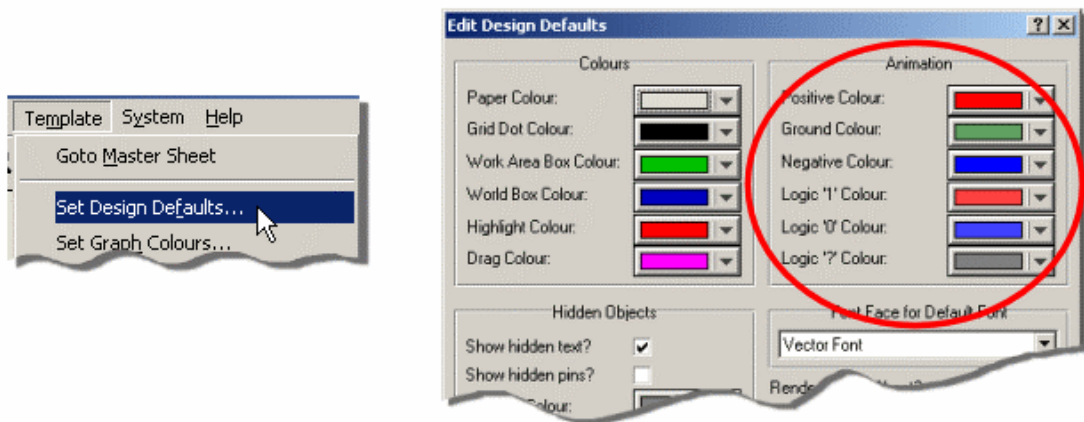
动画效果

就像电路中使用的动态器件一样，动画效果也可以帮助使用者学习电路的动作。这些选项可以通过 SYSTEM 菜单下的 SET ANIMATION OPTION 来进行设置。

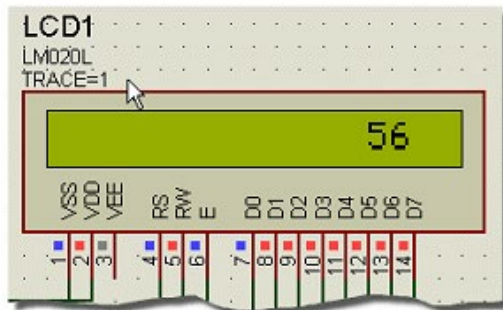
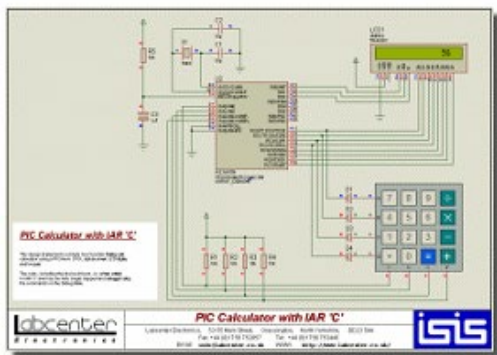


引脚逻辑状态

引脚状态通过有颜色的方块确定，方块蓝色表示逻辑 0，红色表示逻辑 1，灰色表示浮空状态。颜色方案也可以通过 TEMPLATE 菜单下的 SET DESIGN DEFAULTS 命令设置。

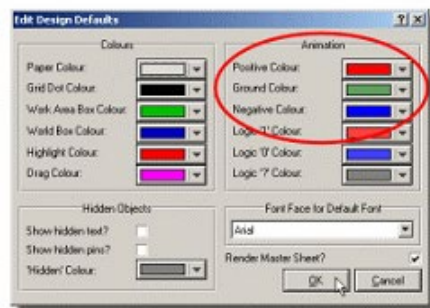
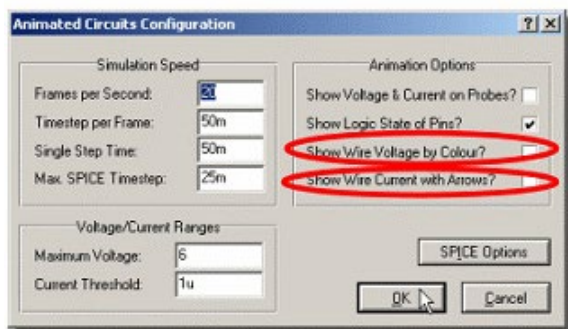


引脚状态显示在进行程序调试时非常有用，当使用断点、单步等各种调试手段时，使用者可以通过其观测各个口线的逻辑状态。



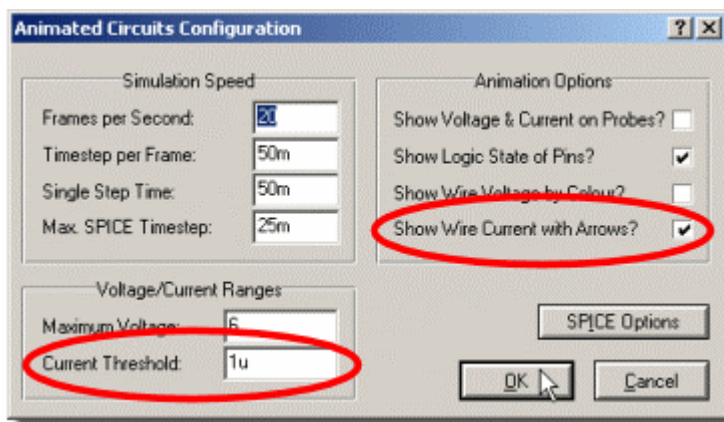
通过颜色显示线路电压

默认设置是：-6V 蓝色、0V 绿色、6V 红色。默认设置可以通过 SET ANIMATION 、SET DESIGN DEFAULTS 对话框进行相应更改。



显示电流方向

箭头显示实际电流流向，默认电流最小值是 1UA，可以通过 SET ANIMATION OPTION 对话框进行更改。



电压颜色和电流方向这些功能在电路分析上非常有用，可以帮助使用者动态的分析电路状态变化。

指示和显示参数信息

当仿真暂停时，使用者可以捕捉原理图上任一器件的电气参数信息。一般的元件都包含结点电压、引脚逻辑状态等信息，有些器件还有参考电压、功率信息。

a. 示参数信息设置过程：

1. 仿真暂停，在 Animation Control Panel 上进行设置。
2. 在 ISIS 上选择 MULTIMETER ICON（万用表按钮）。
3. 左键点击元件显示信息。

b. 直流工作点

仿真暂停时，使用者可以观测直流工作点，这在分析耗散功率上是非常有益的；这个值不是平均值，它是瞬时电流乘以瞬时电压。

c. 模拟还是数字？

PROTEUS VSM 对电路网络表进行分析时，如果是数字信号，显示逻辑状态，如果是模拟信号，显示节点电压值。

电压电流探针

在 ISIS 中的电压探针主要应用于高级图形仿真，也可以和电流探针一起在交互式仿真中实时测量电压电流值。

探针相比虚拟电流电压表有两大优势：

可以在电路中随便移动位置，无需另外连线。

电压电流表在原理图中占用空间太大。

探针被放在纯数字网络上时，将显示逻辑状态而不是节点电压。

探针放置方法

选择探针类型——电压探针或电流探针。

在原理图中需要的探测点放置探针。

*电流探针是有方向，放置时应注意箭头方向和实际电流方向平行。

虚拟仪器

点击 ISIS 中万用表工具，对象选择窗口中会以列表的形式列出 PROTEUS 含有的虚拟仪器，详细的使用方法请查阅虚拟仪器使用章节。

3.3 动态步进控制

概况

两个参数决定交互式仿真是否实时，ANIMATION FRAME RATE（仿真速率）决定屏幕每秒中刷新的次数，ANIMATION TIMESTEP 决定每一帧的仿真量。在实时仿真中，这两个值呈倒数关系。

仿真帧率

通常 20 帧/S 的采样速率已经能够提供一个平滑的动态效果，无须做任何的改变。除非当 CPU

的占有率溢出时，可以考虑降低采样帧率。

帧步进时间

这个量值可以改变仿真的快慢，它是仿真帧率的倒数。

当然，这个时间是一个期望值，是否能达到它还得看仿真使用的 PC 计算能力是否足够强。当 CPU 显示占有率达 100% 时，说明其能力不足，实际帧步长将减慢。

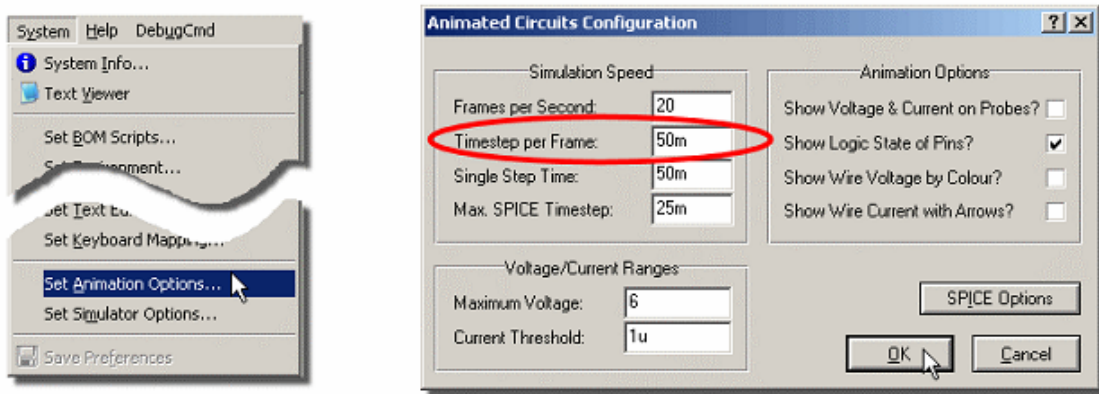
单步时间

这个量表示动画控制面板中步进的时间。

3.4 亮点和提示

电路时间尺度

交互式仿真视为实时行为，所以不必要用电路来产生时钟或频率信号，你只需调整 SYSTEM 菜单下 ANIMATED CIRCUITS COFIGURATION 对话框中 TIMESTEP PER FRAME 参数。



对于快速电路的仿真要注意以下几点：

1. 对于特定能力的 CPU，单位时间内仿真计算能力是一定的。PROTEUS VSM 设定了动态仿真帧率，降低仿真帧率带来的结果可能是不实时的，但会使整个仿真过程更加平滑。
2. 模拟元件模型仿真比数字仿真模型困难得多。一个高速 PC 可以实时仿真几 M 的数字事件，而模拟电路只能达到 10~20KHZ。

电压范围

使用有颜色的线表示电压高低通过 ANIMATED CIRCUITS CONFIGURATION 对话框进行设置。改变 Maximum Voltage value 可是设置显示电压范围。

接地

PROSPICE 对于一些动态电路无须定义地参考点，系统可以自动识别（通常被认为是电池的中点或者在双电源电路的中点）。但如果需要标示，可以从 ISIS 终端中选择地。

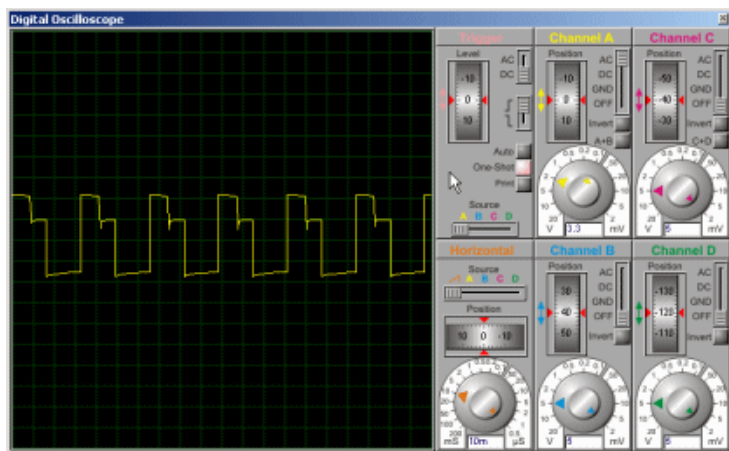
高阻抗点

对于任何未接地的元器件终端，为了保证 SPICE 仿真收敛性，会插入高阻值的电阻。这样，一些未连接的电路或有一部分未连的电路也可仿真（虽然有时得到的结果是不正确的）。

第四章 虚拟仪器

4.1 虚拟示波器

总述



VSM 示波器以 ProSPICE 版本为标准，模拟了基本的双通道（V7 以前）或四通道（V7 以后）单元并且有以下特性：

- i、双通道/四通道，X—Y 模式
- ii、增益范围从 20V/每格到 2mV/每格
- iii、时基范围从 200ms/每格到 0.5us/每格
- iv、可以锁定任何通道的自动触发电压
- v、AC 或 DC 耦合输入
- vi、A + B 和 C + D 频道模式
- vii、每个通道的反转按钮
- viii、鼠标放大和缩小
- ix、光标测量
- x、一键式放大和缩小功能
- xi、打印
- xii、每个通道的颜色变化

4.2 使用示波器

显示模拟波形

- ◆ 通过选择 ISIS 中的“仪表”图标，然后从“对象选择器”中选择示波器来选择 OSCILLOSCOPE 对象。



- ◆ 将示波器放置在原理图上，并将其输入连接到要记录的信号上。
- ◆ 在仿真控制面板上按下“开始”按钮，进行交互式仿真。此时，出现示波器窗口。
- ◆ 将时基刻度盘设置为适合该电路的值。您将需要考虑电路中存在的波形频率，并将它们的倒数转换为周期时间。
- ◆ 如果显示信号带有直流偏量，请根据需要在任一或每个输入通道上选择 AC 模式。您也可以通过选择 OFF 模式来关闭未使用的通道。
- ◆ 调整通道 A~通道 D 部分中的电压和位置刻度盘，以使波形具有合适的大小和位置。如果波形由大直流电压之上的小交流信号组成，则您可能需要在测试点和示波器之间连接一个电容器，因为电容能隔直通交。
- ◆ 使用触发部分的源滑块确定要触发的通道。
- ◆ 使用触发部分中的滑块选择所需的波形部分。如果滑块拨向上方，锁定为上升沿触发；如果拨向下方，则锁定为下降升沿触发。

使用模式

示波器可以在以下模式下运行：

- ◆ 自动-在此模式下，自动按钮 LED 点亮。
- ◆ 一键式-自动按钮 LED 熄灭。在此模式下，如果正在进行捕获，则一键式 LED 指示灯将点亮，并在完成捕获后熄灭。
- ◆ X-Y 模式-源滑块位于选定 X 通道中对应的水平位置。

触发

VSM 示波器提供了一种自动触发机制，可使其将时基与输入波形同步。

- ◆ 触发部分中的源滑块可以选择使用哪个输入通道进行触发。
- ◆ AC / DC 滑块用于选择独立或通道链接的触发偏置量。
- ◆ 电平拨盘用于设置触发偏移值。
- ◆ 边沿选择滑块用于选择波形的上升沿或下降沿。

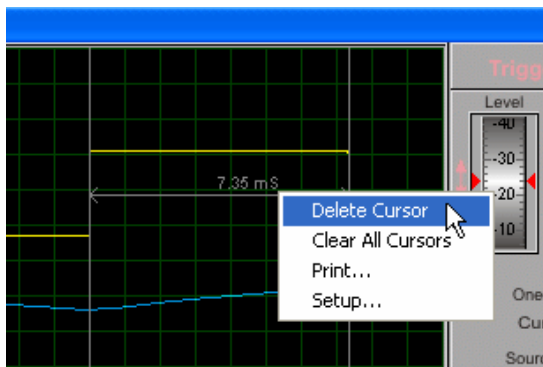
输入耦合

每一通道既可采用直接耦合方式，也可通过仿真电容采用交流耦合方式。其中，交流耦合方式的测量适用于带有较高直流偏压的交流小信号。

输入也可以通过选择 GND 滑块位置暂时接地，进行测量之前它们将与刻度对齐。可以通过将滑块移到 OFF 位置来关闭输入。可以使用 A + B 或 C + D 按钮对两个输入求和。使用“反转”按钮可以反转每个通道。

测量

可以使用光标进行电压、电压差、触发点之间的时间差以及任意两个点之间的电流的测量。要使用此功能，请点击“光标”按钮。要将绝对光标设置在屏幕上，只需将鼠标移至适当的位置，然后按鼠标左键。要同时放置两条光标求差，按住左键（确定了第一条光标的位置）的同时移动鼠标，释放左键的位置出现第二条光标。光标可以通过鼠标移动位置（鼠标悬停在光标上的任一点），也可以使用菜单栏（“删除光标”或“清除所有光标”）删除光标。



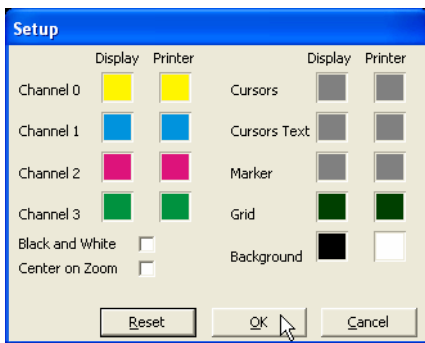
从示波器上删除光标

打印

在示波器上右键单击，使用菜单栏上的“打印”选项来打印屏幕上显示的当前波形。

色彩

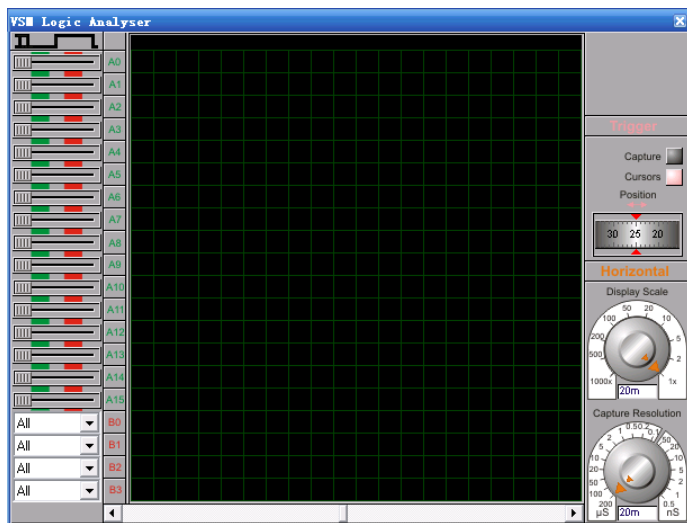
可以在“颜色设置”对话框中更改每个通道的颜色，选择菜单栏“颜色设置”来打开该通道。



右键单击示波器并选择设置来更改颜色

4.3 逻辑分析仪

概况



逻辑分析仪连续地将输入的数字信息记录到一个大的捕获缓冲区。这是一个采样的过程，因此有可调节的分辨率来定义可以被采样的最短脉冲。触发部件监测输入的数据并导致数据捕捉行动在触发条件已经生效后停止一定的时间；捕捉由 arming 按钮开始。最终显示的内容是在触发时间前后的捕捉器所捕捉到的。因为捕捉缓冲器非常大（可存放 10000 个采样数据），因此提供了放大/缩小和全局的显示。最后，可移动的测量标尺可以精确地测量脉宽。

VSM 逻辑分析仪提供的特性有：

- ◆ 8*1 位 Traces 和 2*8 位 Bus Traces
- ◆ 10000*24 位捕获缓冲器
- ◆ 捕获分辨率从 0.5ns/采样点~200us/采样点，相应的捕捉时间是为 5ms~2ms.

- ◆ 显示的缩放范围为 1 个采样点/格~1000 个采样点/格
- ◆ 位输入信号的逻辑电平和/或边沿与总线值进行“与”操作后触发逻辑分析仪
- ◆ 触发位置在捕获缓冲器的 0%,25%,50%,75%和 100%处
- ◆ 提供两个坐标用于精确测量时间

缩放和全局显示

因为捕捉缓冲器可以捕捉到 10000 个采样点，但是显示屏仅能显示 250 个像素宽，因此需要在捕捉缓冲器进行缩放和全局操作。ZOOM 拨盘决定每格采样点的数量，同时滚动条可以实现左右移动。

注意在 ZOOM 拨盘设置下的每格以 s 为单位显示的是当前时间，而不是拨盘设置的实际值。每格的实际时间为缩放（ZOOM）设置值与分辨率（Resolution）设置值的乘积。

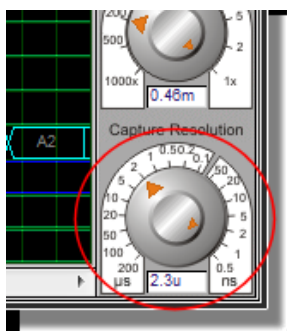
使用逻辑分析仪

捕获和显示数字数据

- ◆ 通过选择 ISIS 中的仪表图标，然后从对象选择器中选择逻辑分析器，来选择 LOGIC ANALYZER 对象。



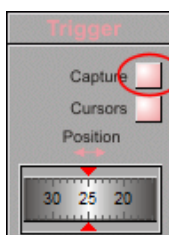
- ◆ 按仿真控制盘的 PLAY 按钮启动交互式仿真。逻辑分析仪界面将会出现。
- ◆ 根据实际需要在分辨率刻度盘设置合适的分辨率。分辨率表示了能记录的最小的脉冲宽度。分辨率越高，捕获数据的时间间隔就越短。



- ◆ 在仪器的左边找到复选框并设置以满足要求的触发条件。例如，如果当连接到通道 1 的信号为高且连接到通道 3 的信号是上升沿信号时，想要驱动仪器，则需要设置第一位为高，第三位为“low-High”。



- ◆ 确定要在触发条件发生之前还是之后查看数据，然后将“位置”拨盘设置为所需的触发位置。
- ◆ 准备就绪后，单击“捕获”按钮为仪器设防。捕获按钮会亮粉红色的灯。逻辑分析仪现在将等待触发事件并连续捕获输入数据，同时监视触发条件的输入。这时候“捕获”按钮将亮绿灯。然后，数据捕获将继续进行，直至触发位置之后的捕捉缓冲器满为止，此时“捕获”按钮将熄灭，并且捕获的数据将出现在显示屏中。



测量

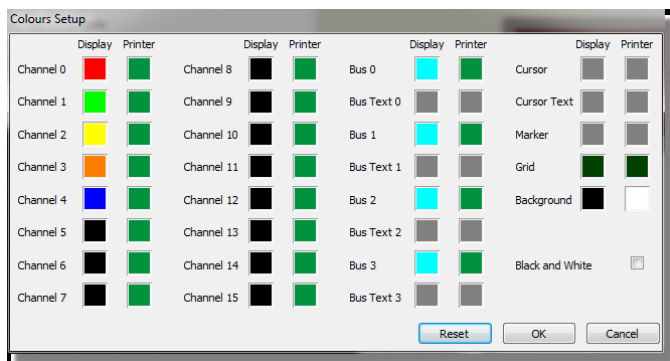
仪器提供了两个可调光标用于精确地测量。点亮“光标”按钮即可使用此功能。要将绝对光标设置在屏幕上，要将绝对光标设置在屏幕上，只需将鼠标移至适当的位置，然后按鼠标左键。要同时放置两条光标求差，按住左键（确定了第一条光标的位置）的同时移动鼠标，释放左键的位置出现第二条光标。光标可以通过鼠标移动位置（鼠标悬停在光标上的任一点），也可以使用菜单栏（“删除光标”或“清除所有光标”）删除光标。

打印

在示波器上右键单击，使用菜单栏上的“打印...”选项来打印屏幕上显示的当前波形。

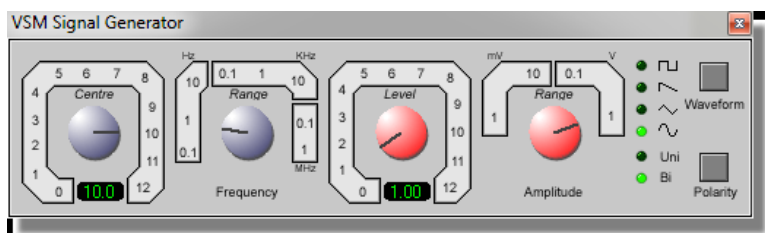
色彩

可以在“颜色设置”对话框中更改每个通道的颜色，选择菜单栏“颜色设置...”来打开该通道。



4.4 虚拟信号发生器

总述



VSM 信号发生器模拟了一个简单的音频函数发生器，它具有以下特点：

- ◆ 可输出方波，锯齿波，三角波和正弦波。
- ◆ 信号输出分 8 个波段，范围是 0~12MHZ
- ◆ 信号输出幅值分 4 个波段，范围是 0~12V
- ◆ 具有调频输入和调幅输入功能

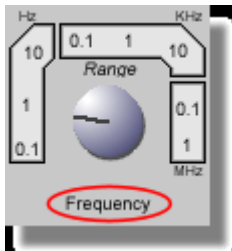
使用信号发生器

设置一个简单的音频信号源

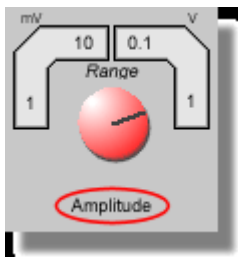
- ◆ 在 ISIS 中选中仪表按钮并在对象选择器中选中 SIGNAL GENERATOR。放置在原理图部分上，然后引其输出端至电路。通常情况下（例如：电路需要的输入源为一个平衡的输入源的情况）需要信号发生器的“-”终端与接地终端相连接。利用接地终端很容易做到这一点。在不使用幅值与频率的调制输入时，可以不连接 AM、FM 这两个输入端口。



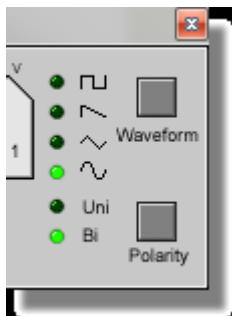
- ◆ 按仿真控制盘的 PLAY 按钮启动交互式仿真。信号发生器界面将会出现。
- ◆ 设置频率拨盘，以满足电路设计的需要。当 Center 指针等于 1 时，表示信号的频率为 1。频率单位以指针对应的标识为准。



- ◆ 设置幅值拨盘，以满足电路设计的需要。当 Level 指针等于 1 时，表示信号的幅度为 1。幅度单位以指针对应的标识为准。幅值为输出电平的高峰值。



- ◆ 单击 Waveform 按钮，代表相应波形类型的 LED 灯将会点亮，输出合适的信号。



使用 AM、FM 调制输入

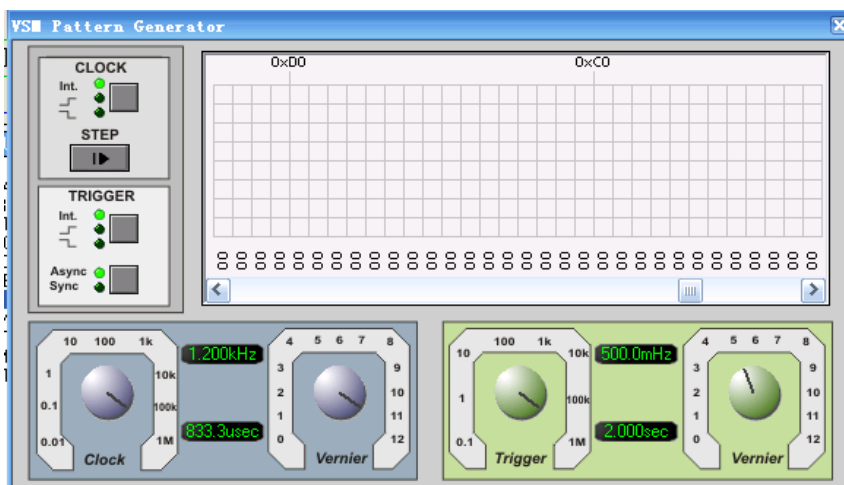
信号发生器支持在幅度调制和频率调制的后的输出。AM 和 FM 有以下特性：

- ◆ 调制输入的增益由频率和幅度的 Range 拨盘分别按照 HZ/V 和 V/V 进行设置
- ◆ 调制输入的电压范围是-12V~+12V.
- ◆ 调制输入有无限大的输入阻抗
- ◆ 调制输入电压值加上相应的游标值，然后与频率 Range 设置值相乘，决定了瞬时输出频率对应的幅度。

例如，如果频率的 Range 设为 1KHZ，频率的 Range 拨盘值设为 2，则 2V 的调频信号的输出频率为 4KHZ.

4.5 模式发生器

总述



VSM 模式发生器在数字方面等价于模拟信号发生器。在 Proteus 专业版仿真软件中都以此为模版发布。

VSM 模式发生器支持高达 1K 字节的模式信号，具有以下特性：

- ◆ 既可以在基于图表的仿真中使用，又可以在交互式仿真中使用
- ◆ 支持内部时钟和外部时钟模式以及触发模式
- ◆ 十六进制和十进制栅格显示格式
- ◆ 直接输入具体值获得更为精确的值
- ◆ 加载和保存模式脚本文件
- ◆ 可以手动配置信号的周期
- ◆ 可以单步运行发生器
- ◆ 实时显示工具观测在栅格中的位置
- ◆ 可通过外部控制保持模式当前状态
- ◆ 栅格块编辑命令便于设置图案

使用模式发生器

在交互式仿真下使用

- ◆ 在 ISIS 中选中仪表按钮并在对象选择器中选中 Pattern Generatoor



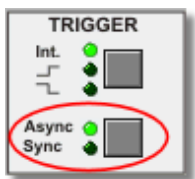
- ◆ 按仿真控制盘的 PAUSE 按钮初始化交互式仿真。信号发生器窗口将会出现。
- ◆ 在栅格编辑窗口点击小方格切换逻辑状态，此即为输出信号的模式
- ◆ 确定使用内部时钟还是外部时钟，通过 CLOCK 按钮可以进行设置。



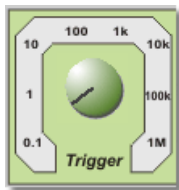
- ◆ 如果选择内部时钟，则可以通过 CLOCK 刻度盘来进行调整，以获得期望的时钟频率。



- ◆ 确定触发是内部触发还是外部触发，然后使用 TRIGGER 按钮设置相应的模式。如果使用了外部触发，则需要考虑是时钟同步触发还是时钟异步触发。



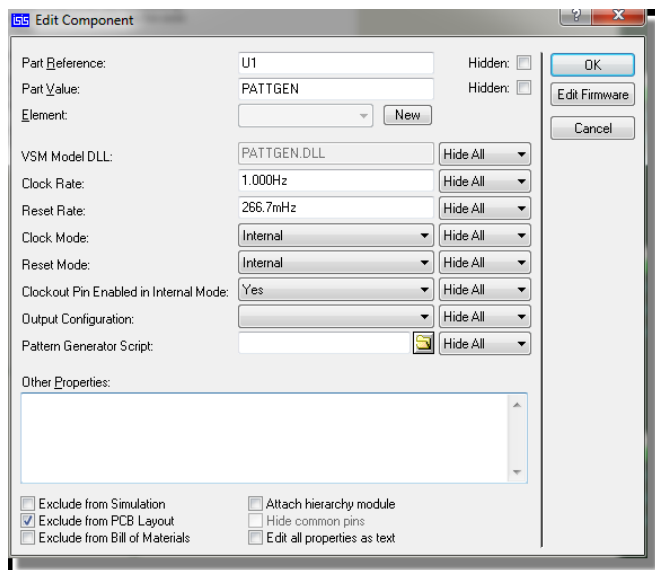
- ◆ 如果使用了内部触发，可以通过调整触发刻度盘来获得期望的触发频率。



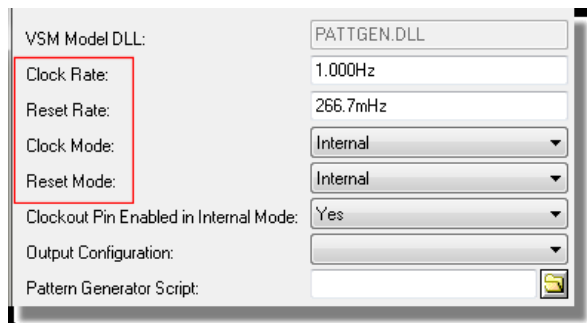
- ◆ 按仿真控制盘的 PLAY 按钮启动交互式仿真，输出模式信号。
- ◆ 如果期望等到单时钟周期的模式信号，须在按仿真控制盘的 SUSPEND 按钮，后使用 STEP 按钮使栅格向左移动。

在图表仿真下使用

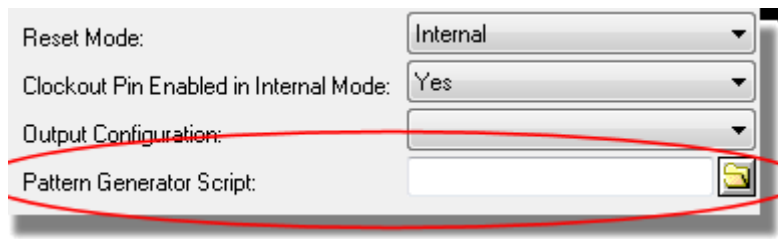
- ◆ 按通常的方法创建原理图
- ◆ 在原理图感兴趣的节点插入探针，然后把探针加进到图表。
- ◆ 对准模式发生器先右键后左键调出器件属性对话框



◆ 设置触发和时钟参数



◆ 在模式发生器脚本区域加载期望的模式文件



◆ 退出模式发生器的属性编辑窗口，按空格键运行图表仿真

模式发生器的引脚说明

- ◆ **Data Output Pins:** 模式发生器可以以总线、引脚的方式输出模式信号
- ◆ **Clockout Pin:** 当使用内部时钟时，可以通过该引脚观测内部时钟脉冲。此性能可以在器件属

性设置窗口中修改。该性能默认是关闭的，因为它会减弱系统的性能，尤其是在高频的系统中更为明显。

- ◆ **Cascade Pin:** 当模式信号的第一位正在受驱动时该引脚被驱动为高电平，并且一直保持高电平直到驱动下一位信号驱动（一个时钟周期以后）。这表明在开始仿真时的第一个时钟周期它呈现高电平，在接下来重新传输中的第一个时钟周期它又是高电平。
- ◆ **Trigger Pin:** 此引脚为输入引脚，用于接收外部触发脉冲。触发模式在下面将会详细讲述。
- ◆ **Clock-In Pin:** 此引脚为输入引脚，用于接收外部时钟信号，时钟信号的模式在下面将会详细讲述。
- ◆ **Hold Pin:** 当该引脚被驱动为高电平时，模式发生器将保持在暂停点，直至该引脚被释放。对于内部时钟或内部触发，时钟将从暂停点重新开始。例如，对于 1HZ 的内部时钟，如果发生器在 3.6s 处暂停，在 5.2S 处重新开始，则下一个下降时钟边沿将发生在 5.6S 处。
- ◆ **Output Enable Pin:** 使用高电平驱动该引脚。如果该引脚不为高，虽然模式发生器仍然工作在指定的模式，但并不驱动引脚输出模式信号。

时钟模式

内部时钟

内部时钟是负沿脉冲。即每周期以低—高一低的形式。

内部时钟既可以在仿真仿真之前使用元器件属性对话框指定，也可以在仿真暂停期间使用时钟模式按钮进行指定。

Clockout 引脚使能后可以观测内部时钟信号。该性能默认是关闭的，因为它会减弱系统的性能，尤其是在高频的系统中更为明显。但是可以在器件属性设置窗口中启动此性能。

外部时钟

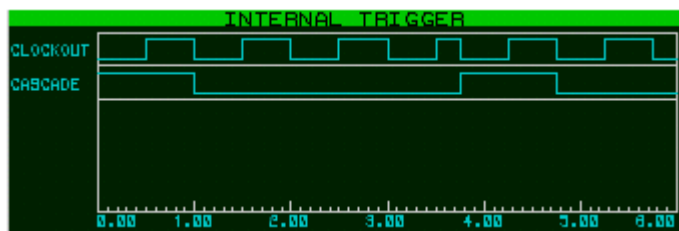
模式发生器有 2 种外部时钟模式——负脉冲(LOW-HIGH-LOW)和正沿脉冲(HIGH-LOW-HIGH)将外部时钟脉冲连接到时钟输入引脚，并选择其中一种时钟模式

与内部时钟一样，外部时钟既可以在仿真仿真之前使用元器件属性对话框指定，也可以在仿真暂停期间使用时钟模式按钮进行指定。

触发模式

内部触发器

模式发生器的内部触发模式是按照指定的时间间隔触发的。如果时钟是内部时钟，则时钟脉冲在这一触发点复位。如图

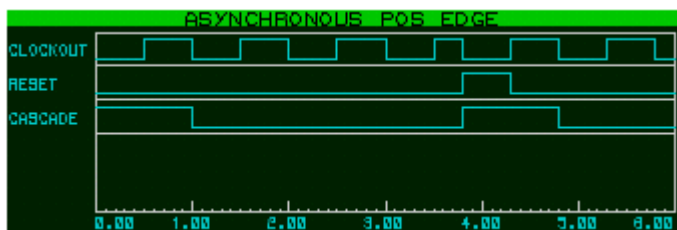


例如，设定内部时钟脉冲是 1Hz，并且设定内部触发时刻是 3.75s。则 CASCADE 引脚在模式第一位时为高，而在其他时间为低。

❗注意：在触发时间，内部时钟被异步复位。模式的首位被驱动至输出引脚（图中 CASCADE 被拉高）。

外部异步正脉冲触发器

触发器由触发引脚的正边沿转换信号触发。当触发发生时，触发器立即动作，在下一个时钟边沿（即位时钟 1/2 处，与复位时间相同）发生由低到高转换。



例如，设定内部时钟脉冲是 1Hz，并且设定触发引脚在时刻 3.75s 处被拉高，则时钟立即在触发脚正边沿复位，模式的第一位驱动到引脚。

外部同步时钟正脉冲触发器

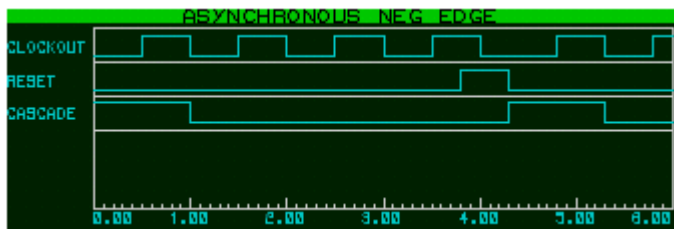
触发器由触发引脚的正边沿转换信号触发。触发立即使模式的第一位驱动到引脚。



例如，设定内部时钟脉冲是 1Hz。注意：时钟不受触发器的影响。在时钟下降沿时触发器立即动作，同时模式的第一位驱动到引脚。

外部异步负脉冲触发器

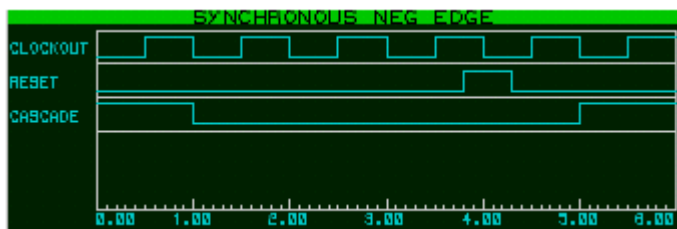
触发器由触发引脚的负边沿转换信号触发。当触发发生时，触发器立即动作，且模式第一位在输出引脚输出。



例如，设定内部时钟为 1HZ。从图中可以看到时钟在触发脉冲的负边沿复位，同时模式的第一位在此时被驱动。

外部同步负脉冲触发器

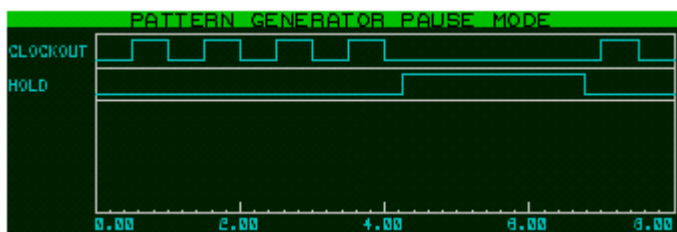
触发器由触发引脚的负边沿转换信号触发。触发被锁定，并与下一个时钟的下降沿同步动作。



例如，设定内部时钟为 1HZ。注意，触发发生在触发脉冲的下降沿，模式直到时钟脉冲时钟脉冲的下降沿，与触发动作并发复位。

外部保持

想要在一段时间内保持状态，可以在该期间使保持引脚为高电平进入保持模式。如使用的是内部时钟，当在释放保持引脚的同时，模式发生器将同时重新启动。也就是说，在半个时钟周期里保持引脚应该为高电平。然后，当释放保持引脚时，下一位将要在以后的每半个时钟周期时驱动输出引脚。



当保持引脚为高时，内部时钟被暂停。当释放保持引脚时，时钟将在相对于一个时钟周期的暂停点重新启动。

附加功能

加载和保存模式脚本

在栅格上单击右键，从菜单中选出相关的选项。这一方法对要在多个设计中使用特殊的模式非常有用。

脚本文件是纯文本，每个字节由逗号分隔。每个字节代表栅格上的一列，以分号起始的行被记作注释行，并且被解析器忽略。在默认状况下，字节格式为十六进制。当用户创建脚本时，输入值可以是十进制、二进制或十六进制。

为刻度盘设置指定值

用户可以通过双击合适的刻度值，指定位和触发频率的精确值。双击后，将出现浮动的编辑框。在默认状况下，输入值被认作频率，同时，也可以通过为输入值加上合适的后缀（如 sec、ms 等）来指定输入值的类型。此外，如果希望触发为时钟的精确乘倍乘，可以附加期望的倍乘后缀“bits”（如 5bits）。

按 Enter 键，或 Escape 键或者单击模式发生器件窗口的任何其他部位，用于确定输入。

设定特定值到模式的栅格

左键在显示当前一栏值的文本上单击，可以指定栅格上该栏中任何一个值。出现的一个浮动的编辑框，用户可以在框中输入期望值。格式可以是十进制（如 135）、十六进制（如 0xA7）或二进制（如 0b01011101）。按 Enter 或 Escape 键，或单击模式发生器的任何其他部位，用于确认输入。为了方便，用户可以在想要编辑的栏，使用组合键 Ctrl+1 设置一栏，或者使用组合键 Ctrl+Shift+1 清除一栏。

手动指定模式的周期长度

在期望模式结束的栏的栅格中，单击左键指定期在同样的区域，单击右键取消指定周期。

单步执行模式

当仿真时间周期等与内部时钟或外部时钟指定的位时钟相等时，Step 按钮可以用于进一步仿真。仿真将一直进行到下个时钟周期结束并中止。

切换栅格显示方式

栅格显示格式可以在十六进制和十进制中转换。此操作可以在栅格处点击右键，然后从菜单中选取设置的参数项或者通过快捷键 CTRL+X（十六进制的条件下）和 CTRL+D（十进制的条件下）进行转换。

指定输出

调出模式发生器的属性编辑窗口。通过设置该窗口下方的属性可以指定模式信号在总线和引脚输出还是只以引脚或总线的方式输出。

显示工具

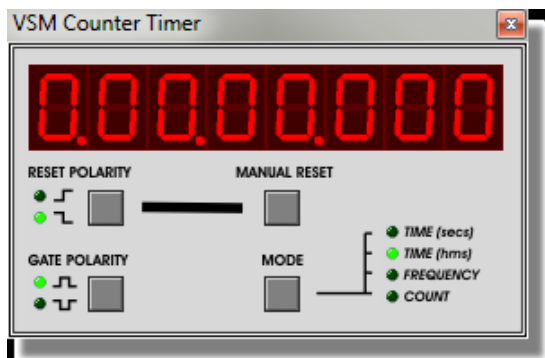
用户可以设定随光标移动显示当前行、当前列信息的工具。通过右击出现的文本，或组合键 CTRL+Q 控制。注意：在块设置或块清除时该工具不能使用。

块编辑

用户可以通过块设置（Block Set）和块清除（Block Clear）命令快速设置栅格等到期望的模式。可以在右键出现的文本或组合键（CTRL+S 是 Block set 命令的组合键，CTRL+C 是 Block clear 命令的组合键）进行操作。注意：块编辑命令在 Tooltip Display 模式下不能使用。

4.6 定时/计数器

概述



VSM 计数/定时器是一个多用途的数字仪器。它可以用来测量时间间隔、信号频率和脉冲数。计数/定时器有以下特点：

- ◆ 定时模式（显示秒），分辨率为 1us

- ◆ 定时模式（显示小时、分、秒），分辨率为 1m
- ◆ 频率计模式，分辨率为 1HZ
- ◆ 计数模式—最大计数值到 99,999,999

时间、频率和脉冲值既可以在虚拟界面显示也可以在其弹出窗口界面显示。在仿真期间，用户可以定时/计数器单击左键或在 Debug 菜单调出。

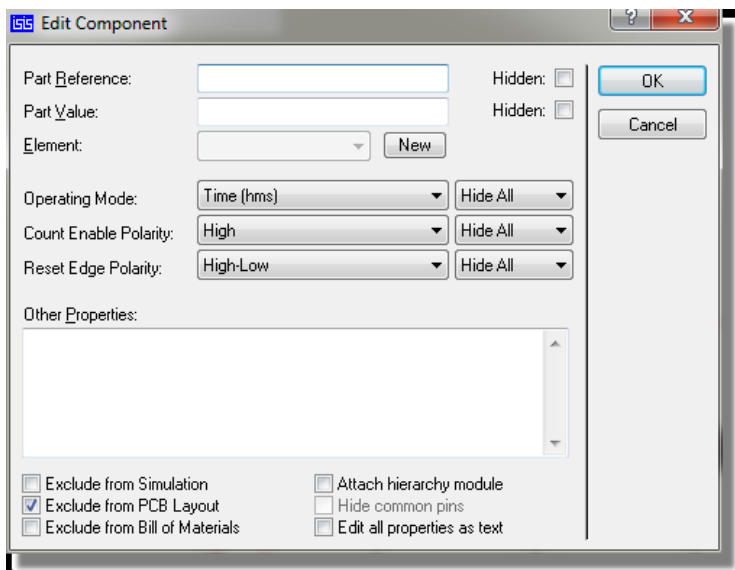
使用定时模式

测量时间间隔

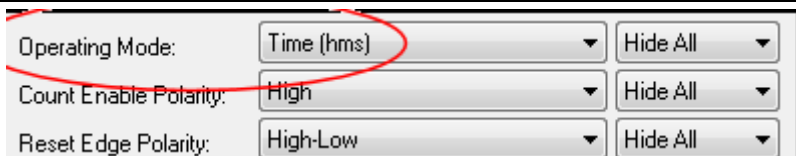
- ◆ 选中万用表按钮，在对象选择器中选中 COUNTER TIMER 并把它放入原理图编辑窗口。



- ◆ 如果需要时钟使能信号则可以将使能控制信号连接到 CE 引脚。如果不需要时钟使能信号，可以将该引脚悬空。
- ◆ 如果需要定时器复位，可以将复位信号连接到 RST 引脚。如果不需要复位信号，可以将该引脚悬空。
- ◆ 将光标放在计数/定时器上，按组合键 CTRL+E 调出属性编辑窗口。



- ◆ 选择要显示的时间模式、CE 和 RST 引脚的逻辑极性。



◆ 启动交互式仿真。

- ② 注意：计数器计时器弹出窗口提供了一个手动重置按钮，它将在模拟运行期间的任何时间。这在嵌入式系统工作中特别有用，
- ② 注意：RST 引脚边沿触发方式而非电平触发方式。如果使定时器保持为零，可以同时使用 CE 和 RST 引脚。计数/定时器提供了手动按键复位的功能。该操作可以在仿真期间的任何时刻操作将计时器重置为零。在嵌入式系统工作中，您可以使用计数器计时器来确定执行程序特定部分所花费的时间。

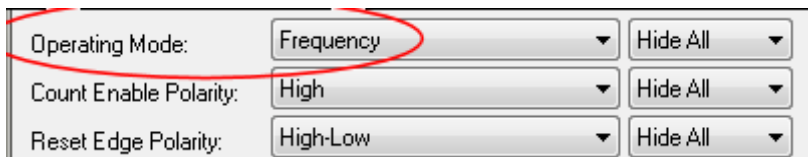
使用频率模式

测量数字信号的频率

- ◆ 在对象选择器中选 COUNTER TIMER 并把它放入原理图编辑窗口。



- ◆ 将待测信号连接到 CLK 引脚。在频率模式下，不使用 CE 和 RST 引脚
- ◆ 将光标放在计数/定时器上，然后按组合键 CTRL+E 调出属性编辑窗口。



- ◆ 选中频率模式（Frequency Mode）关闭属性编辑窗口并启动交互式仿真。

- ② 注意：在仿真每一秒中频率计计算信号上升的数量，因此要求输入信号稳定且在一秒内有效。同时，如果仿真不是在实时的速率下进行，频率计数器将在相对较长的时间内输出频率值。
- ② 注意：计数器是个纯数字器件。在测量低电平模拟信号时，需要将待测信号通过 ADC 和逻辑开关后才输入计数器的 CLK 引脚。同时，因为模拟仿真比数字仿真仿真速率慢 1000 倍，所以计数器不是真正适合测量频率高于 10KHZ 的模拟振荡器电路。在这种情况下，可以利用虚拟示波器（或图表）测量信号的周期。

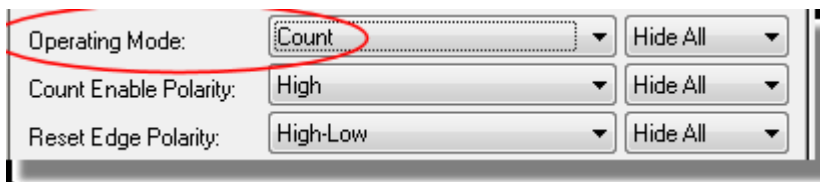
使用计数模式

测量数字脉冲

- ◆ 选中万用表按钮，在对象选择器中选中 COUNTER TIMER 并把它放入原理图编辑窗口，将待测信号连接到 CLK 引脚。



- ◆ 如果需要时钟使能信号则可以将使能控制信号连接到 CE 引脚。如果不需要时钟使能信号，可以将该引脚悬空。
- ◆ 如果需要定时器复位，可以将复位信号连接到 RST 引脚。如果不需要复位信号，可以将该引脚悬空。
- ◆ 将光标放在计数/定时器上，然后按组合键 CTRL+E 调出属性编辑窗口。
- ◆ 选择 Counter 模式、CE 和 RST 引脚的逻辑极性。



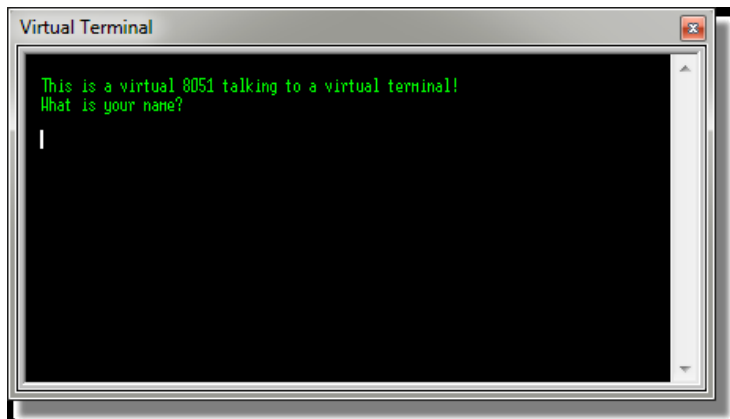
- ◆ 启动交互式仿真。



- ② 注意：当 CE 有效时，在 CLK 引脚的上升沿开始计数
- ② 注意：RST 引脚是边沿触发非电平方式触发。如果使定时器保持为零，可以同时使用 CE 和 RST 引脚。
- ② 注意：计数/定时器提供了在手动复位的功能。该操作可以在仿真期间的任何时刻操作。

4.7 虚拟终端

概述



VSM 虚拟终端允许用户通过 PC 的键盘和屏幕与仿真微处理器系统收发 RS232 异步串行数据。在显示用户编写程序产生的调试/跟踪信息时非常有用。

虚拟终端有以下特性：

- ◆ 全双工 以 ASCII 码方式显示接收的串行数据，同时以 ASCII 码方式传输键盘信号。
- ◆ 简单的两线传输。RXD 用于接收数据，TXD 用于传送数据。
- ◆ 简单的两线握手信号：RTS 表示发送准备好，CTS 表示清除发送数据。
- ◆ 波特率范围：300~57600
- ◆ 7 或 8 位的数据位
- ◆ 包含奇校验、偶校验和无校验
- ◆ 0、1 或 2 的停止位
- ◆ 除了硬件握手外，还提供了 XON/XOFF 软件握手
- ◆ 可对 RS/TS 和 RTS/CTS 的逻辑极性进行转换

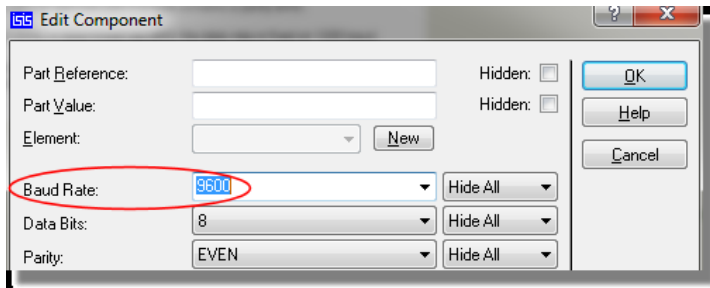
使用虚拟终端

- ◆ 选中虚拟仪器按钮，在对象选择器中选 Virtual terminal 并把它放入原理图编辑窗口。

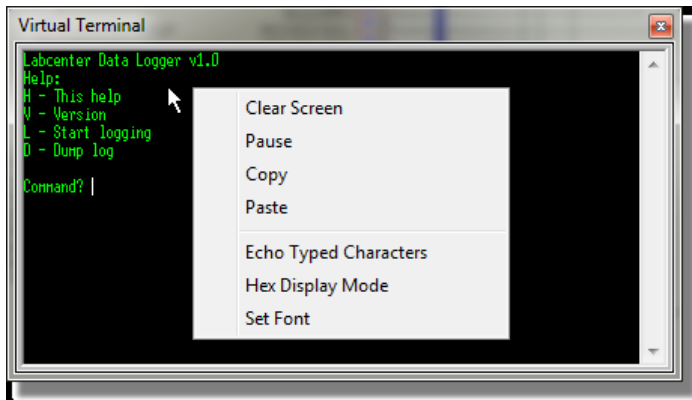


- ◆ 将 RX 和 TX 引脚分别接到测试电路的传送线和接收线。其中，RX 是输入，TX 是发送。

- ◆ 如果待测系统使用硬件握手，则须将 RTS 和 CTS 引脚连接到数据流控制线上。RTS 是输出，发送信号，表明虚拟终端准备接受数据。CTS 是输入，此引脚在虚拟终端进行传送之前必须为高。



- ◆ 调出终端的属性窗口，设置合适的波特率、字长、校验位、流控制和数据传输停止位。
- ◆ 启动交互式仿真。终端立即显示接收到的数据；当传送数据时，在光标置于虚拟终端窗口的前提下，使用键盘输入数据。



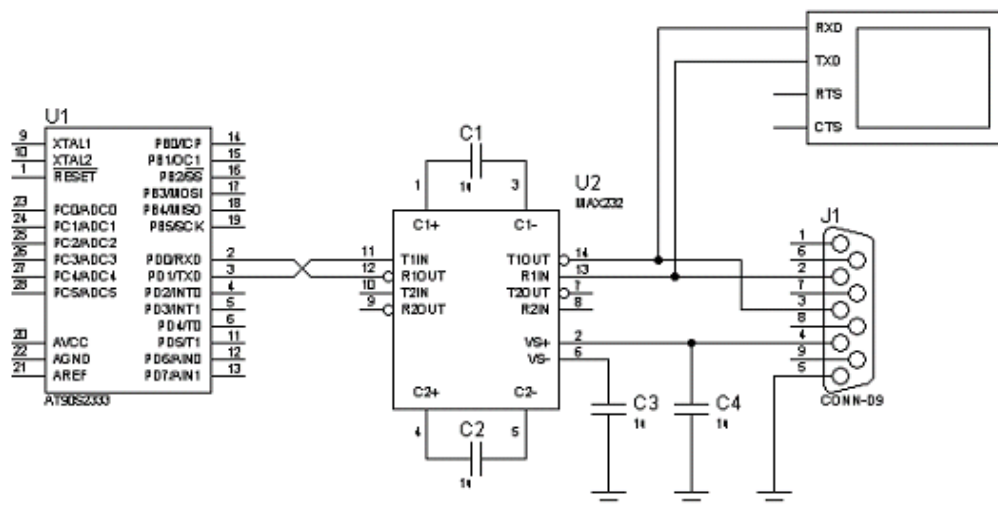
- ◆ 在仿真的过程中，在虚拟终端窗口单击右键，出现一系列可操作菜单。该菜单可以实现显示、复制和粘贴等操作。

- ② 注意：虚拟终端支持 ASCII 控制代码 CR (0X08)，BS (0X80) 和 BEL (0X07)。其它代码包括 LF (0X0A) 被忽略。
- ② 注意：虚拟终端是纯数字模型因此在其引脚处不需要任何特别的电平要求。也就是说可以将其直接连到 CPU 或 UART 而不用通过 RS232 驱动器件例如具有逻辑电平转换功能的 MAX232。
- ② 注意：RX 和 TX 引脚默认为高电平。因此空闲状态为高电平，起始位是低电平，停止位是高电平。数据位中，逻辑高代表“1”，逻辑低代表“0”。这与多数微控制器 UART 定义兼容，并且与诸如 6850 和 8250 的定义也兼容。当不是上述情况时（例如，因为将虚拟终端连接到 RS232 驱动器件的输出端）需要将 RTS/CTS 极性反向。
- ② 注意：RTS 和 CTS 默认为高电平。如果想将两引脚连到反向控制线，则需要反向 RTS/CTS 的极性。
- ② 注意：默认情况下，虚拟终端不显示用户键入的字符；也就是说，主系统将驱动输出终端显示用户键入的字符。如果想显示用户键入的字符，从快捷菜单中选择 Echo Typed Characters 选项。
- ② 注意：用户可以用 TEXT 属性预先定义传输的数据。例如，定义好 TEXT=“Hello World”后，在

电路启动时就传输数据，Hello World 就会被传送到电路中，除非 CTS 未激活。

MAX232 模型

Proteus VSM 包含了 MAX232 这个 RS232 驱动器的模型，因此虚拟终端可以跟目标 CPU 进行连接。如下图所示：

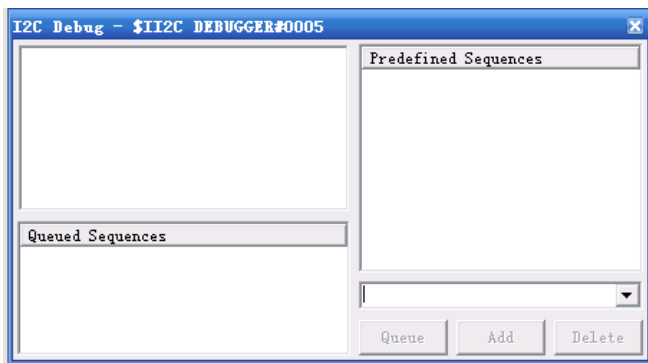


 **警告：**然而要注意 MAX232 含有逻辑反相器，因此虚拟终端的 RX/TX polarity 须改为 **inverted**（反相）才能正确仿真。

也要注意，MAX232 也只是一个纯数字的器件，并没有模拟它内部电压转换器的工作。仿真这一特性可能带来巨大的仿真性能损耗。将任何模拟器件（电阻、电容、示波器等）连接到 TXOUT/RX 引脚也会造成同样的损耗。

4.8 I2C 调试器

概述



I2C 协议调试器提供了监视 I2C 接口、与 I2C 接口交互的功能。该调试器有两个目的：一是用户查看沿 I2C 总线发送的数据，二是以主器件或从器件向总线发送数据。在编写 I2C 程序时既可以作为调试工具又可以作为开发和测试的辅助手段

原理图部件

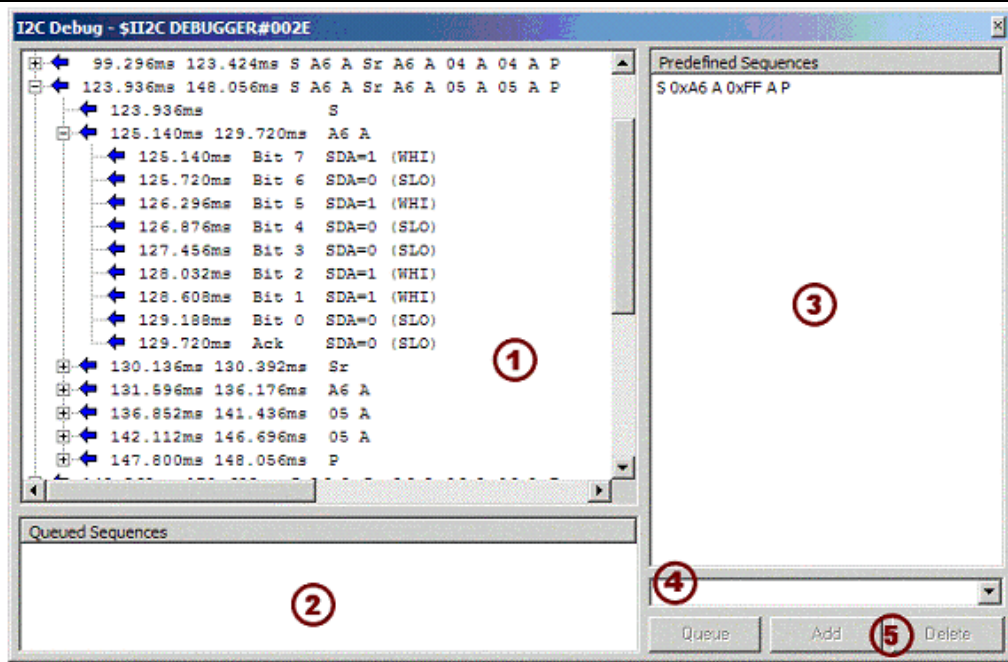
I2C 调试器在 ISIS 中器件界面非常简单，如下图：



- ◆ SCL 引脚：双向引脚，用于连接 I2C 总线的时钟线。
- ◆ SDA 引脚：双向引脚，用于连接 I2C 总线的数据线。
- ◆ TRIG 引脚：输入引脚。用于触发一系列连续的储存数据到输出队列

I2C 调试窗口

当仿真暂停或运行时，I2C 调试器界面将会出现（如果界面没自动弹出，可以通过 Debug 菜单调出）。界面可以分为几个区域。它不仅可以监视 I2C 总线上的数据还可以以主机或从机的身份仿真总线。



警告：当仿真有很多 I2C 总线活动的电路时，I2C 调试器会导致很大的仿真消耗。

①输入数据的显示

主窗口区域显示在总线上所有活动记录。从上图可以知道，用户可以查看数据每一位的传输。

②注意：当以字节或更长数据为单位观察时，序列起止时间被记录。当以位为单位观察时，引脚的逻辑状态和位标号记录在时间旁。

③注意：当箭头上显示不可见时表明传输是双向的。如果有非法的序列，箭头上会出现问号标志。

②预定义序列表

预定义序列表在对话框的右上角。该表显示了在 SCL 引脚控制下的 SDA 引脚传送的预编制的一系列数据。用户可以在序列输入窗增加预编数据（利用 ADD 按钮完成向序列表添加数据）。

④注意：当仿真正在进行时，双击事先定义的序列会把该序列添加到缓冲器/队列表后用于总线上的传输。注意传送的时间决定于总线的特性，例如此时的总线是主机还是从机等。

③缓冲/队列的序列表

缓冲序列表位于窗口的左下角。用于显示在下一个有效时机 I2C 将要传送的一系列数据。用户可以在序列输入窗增加预编数据。（利用 Queue 按钮完成向序列表添加数据序列）。

④序列输入窗口

该窗口（位于界面的右下角）允许用户输入序列，既可以放在预编程的序列窗或立即放在输出队列以便在下一个有效的时机传送到 SDA 引脚。支持以下语法规则：

语法	描述
----	----

'0x'	表示十六进制值的前缀，例如 0xFE
'\$'	表示十六进制值的前缀，例如 \$FE
'h'	表示十六进制值的后缀，例如 FEh
'%'	表示二进制值的前缀，例如 %1101b
'b'	表示二进制值的后缀，例如 1101b
'd'	表示十进制值的后缀，例如 47d

注意十六进制和二进制必须有限定的前缀或后缀。十进制的后缀是可忽略的。以上字符必须要用单引号或双引号括住。

⑤ 控制按钮

控制按钮用于激励 I2C 总线。该调试器既可以作为主机又可以从机，此特性提供了一种强大的测试 I2C 执行情况的功能。

ADD 按钮

向预定义序列表输送在 Sequence Entry Box 定义的序列，为以后的操作使用。

Delete 按钮

删除在预定义序列表中的序列。

Queue 按钮

从 Sequence Entry Box 或 Pre-defined Sequence List 中排列已被选项。被排列的选项在会出现在 Buffered/Queued 案 Sequences List.

使用 I2C 调试器

发送数据

将 SDA 和 SCL 引脚连接到电路的相应引脚。

在按钮的上方的终端窗口输入要传输的数据。数据可以是单字，逗号或空格隔离的一系列字节，或用双引或单引号括起来的文本字符串。一个字可以是具体的数字或是具体的控制信号。

一旦完成了传输序列的输入，则可以直接排列数据准备发送或者将数据放到预定义序列表以便后面使用。点击 queue 按钮即可以立即将传输序列放到缓冲序列表中；点击 add 按钮即可以将传输序列放在预定义序列表里。

要想从预定义序列表中删除一个输入的数据，可以先在预定义序列表中选中要删除的数据然后按 delete。

接收数据

将 SDA 和 SCL 引脚连接到电路的相应引脚。在 I2C 总线上的数据将以十六进制和特定的格式显示在调试窗口的左上方。

模型特性

ADDRESS1	地址 1。如果使用此终端来仿真一个从元器件，则这一属性用于指定从元器件地址的第一个地址字节。主机使用最低有效位作为系统进行读操作或写操作的标志位，而在寻址时，这一位被忽略。如果属性设置框为默认值，或为空，此终端将不被认为是从元器件。
ADDRESS2	如果使用此终端来仿真一个从元器件并且期望 10 位的地址，则本属性用于指定从元器件的第二个地址字节。如果属性设置框为空，则假定地址为 7 位。
STOPONEMPTY	指示表明当输出缓冲器为空和一个字节需发送时，仿真是否应该暂停。
CLOCKFREQ	当处于主元器件时，用于控制 SCL 的时钟频率
SEQUENCE_FILE	用于指定存储预先定义的输出序列的文本文件名 如果此属性框为空，序列将被当做器件属性的一部分而保存
AUTOLOAD	
TIMEPREC	定义十进制的数量以显示时间
WRAPLENGTH	定义在新线创建之前在单一线上允许的项目的数量。

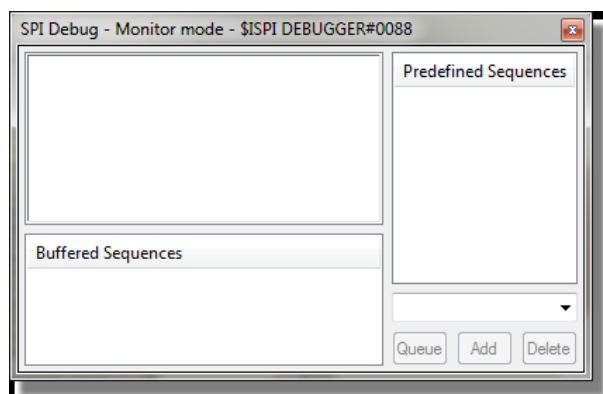
语法参考

以下内容详细介绍了调试器用到语法

语法	描述
S	表示开始条件
Sr	表示重新开始条件
P	表示停止条件
N	表示负应答条件
A	表示应答条件
L	表明仲裁已经丢失，转入到主机模式
*	表示接收到局部的数据
?	表示检测到无效的逻辑电平

4.9 SPI 调试器

简述



SPI 调试器允许用户监测 SPI 接口并与 SPI 接口互动。该调试器不仅允许用户观测 SPI 传送的数据并且允许用户直接向总线发送数据。该终端可以工作在以下三种模式之一：

- ◆ 从模式：以从机的身份运行
- ◆ 主模式：以主机的身份运行
- ◆ 监测模式：只报告当前 SPI 总线传送的内容

原理图部件

原理图部件如下图所示：



管脚说明：

SCK 引脚

该引脚与 SPI 总线的时钟线相连。它是双向引脚，当作为从机使用时是输入，当作为主机使用时是输出

DIN 引脚

数据接收引脚。接收来自 SPI 总线上的数据

DOUT 引脚

数据输出引脚。向 SPI 总线发送数据

SS 引脚

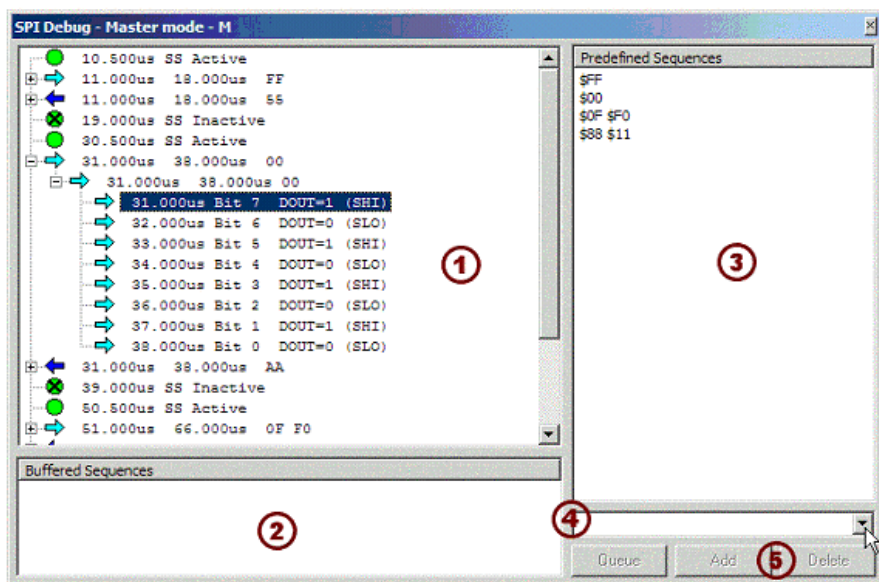
选择从设备引脚，在从机模式时，该引脚必须有效以激活调试器。在主模式时，引脚激活的同时正在传送数据

TRIG 引脚

该引脚为输入引脚。用于将调试器存储的下一个序列输入到 SPI 输出序列

SPI 调试窗口

当仿真处于暂停状态或全速运行时，调试窗口可用（如果该窗口不是自动弹出来，用户可以从 Debug 菜单中调出来）。该界面分为几大部分，不仅允许用户 SPI 的当前情况并且以主机或从机的模式仿真总线。



① 输入数据的显示

主窗口（在左上角）显示了总线所有的运行情况。从上图可以看到，用户可以仔细观察单个位传输。

② 注意：当以字节或更大单位查看时，序列传送的开始和结束时间都将给予记录。当以位单位查看时，引脚的逻辑电平和位指示器将在界面同时间一起显示

③ 注意：在上述界面中不可见时，箭头以双向的方式表示传输或接收数据（上述例子是接收的状态）。如果检测到一个无效的序列，箭头和旁边的问号一同出现。将此特点和以位观测数据的功能相结合，则在调试潜在的问题时将会发挥极大的作用。

② 定义序列表

在右上角的是预定义序列表。它显示了一列可在总线上进行传输的预编程数据序列。用户可以利用在 Sequence Entry Box 中添加事先定义的序列，方法如下：（用 Add 按钮将序列递交到序列表）

②注意：当仿真正在进行时，在一预定义序列上双击左键（或先将预定义序列高亮选中后点击 Queue 控制按钮）将会把这一预定义序列添加到 Buffered/Queued Sequence 中，用于在总线上的传送。注意，无论是主机还是从机传输的时间取决于总线的活动并且很有可能不是瞬时的。

③ 缓冲/序列表

缓冲/序列表位于窗口的左下方。它显示了一队将在下一个有效的时机被 SPI 调试器传送的序列。用户可以在 Sequence Entry Box 中增加一缓冲序列。用户用 Queue 按钮将序列递交到该序列表。

④ 序列输入框

该输入框（位于窗口的右下方附近）允许用户在可编程序列表中放置序列或立即将序列加到输出序列中，准备在下一有效时机发送到 DOUT 引脚。支持的语法如下表：

语法	说明
‘Ox’	前缀，表示十六进制的值（例如，OxFE）
‘\$’	前缀，表示十六进制的值（例如，\$FE）
‘h’	后缀，表示十六进制的值（例如，FEh）
‘%’	前缀，表示二进制的值（例如，%1101）
‘b’	后缀，表示二进制的值（例如，1101b）
‘d’	后缀，表示十进制的值（例如，47d）

注意十六进制和二进制的数值必须有一前缀或后缀，而十进制可以省略。字符串必须以双引号或单引号括住。

②注意：利用 Control 按钮去决定一个已给定的序列是加在预定义序列表还是加在缓冲/序列表。

⑤ 控制按钮

控制按钮允许用户为 SPI 总线提供激励。协议分析仪完全既可以做主机又可以做从机，因此为检测 SPI 执行情况提供了很好的方式。

Add 按钮

使用此按钮将在序列输入框定义的序列递交到预定义序列表，以便以后使用。

Delete 按钮

使用此按钮删除预定义序列表的一个序列。

Queue 按钮

使用此按钮对直接在序列输入框或预定义序列表选中的项目进行排列。所排列的项目将会出现在缓冲/序列表中。

模型的特性

SPI 仿真器有很多个性化的特性.用户可以根据特定的需求进行相应的设置.所有特性都可以通过属性对话框进行设置(右键选中后点击左键弹出属性对话框)。具体个性属性说明如下表：

特性	说明
WORDLENGTH	字长，指定每一个传输数据的位数，可以指定从 1~16 任意一个数据
BITORDER	位顺序，指定每一个传输数据的位顺序，既可以先传输 SSB，也可以先传输 LSB
SAMPLEEDGE	采样边沿指定 DIN 引脚采样的边沿。当 SCK 从空闲到激活、或从活跃到空闲时进行采样。
IDLESTATE	SCK 空闲状态，指定 SCK 何时为空闲状态，或当 SCK 为高电平，或当低电平时为空闲。
STOPONEMPTY	输出缓冲器为空时停止。指定当输出缓冲器为空时是否停止仿真
SEQUENCE_FILE	序列文件。允许用户给用来保存预定义输出的序列的文件命名。如果该属性设为空，则序列将作为器件属性的一部分被保存。
MODE	主机或从机的模式选择控制
CLOCKFREQ	在主机模式下，用于控制 SCK 上的时钟频率
AUTOLOAD	控制是否在仿真进行前立即将预定义序列放置到序列表中
WRAPLENGTH	指定在建立一新线之前一个单线上允许的字的数量。
TIMEPREC	某一时间显示十进制数值的数量
DATAINVERT	指示 DOUT 和 DIN 的数据是否被反向
LOOPBACK	当被使能时，任何接收到数据将会被排列以便发送

使用 SPI 调试器

用 SPI 调试器接收数据

观测 SPI 总线上的数据要求将终端的 SCK 和 DIN 引脚连接电路中相应的引脚,然后设置终端模型的字长、位顺序、SCK 空闲状态和采样边缘的属性等。SPI 总线上的数据将以十六进制的格式显示在终端窗口的左上角。如果一个字长的接收数据位传输完毕（例如在接收整个字的数据之前 SS 无效），则显示显示的不在是十六进制而是星形符号。如果 DIN 上的接收数据位有因冲突不稳定，则显示的不在是十六进制而是问号。

使用 SPI 调试器传输数据

使用 SPI 发送数据，用户首先需要将 SCK 和 DIN 引脚连接到电路的相应引脚上，然后设置终端模型的字长、位顺序、空闲状态和采样边沿。

在终端窗口的右下角输入需要传输的数据，就在按钮的上方。（见上面的序列输入框）

输入的数据可以为单字、以逗号或空格键间隔开的一串字，也可以为用单引号或双引号括住的一串文本字符。数值可用十进制、十六进制或二进制表示。十六进制以一个“0x”或“\$”为前缀，或者以一个“h”为后缀表示。二进制以 '%' 为前缀或以 'b' 为后缀表示。

十进制不需要前缀或后缀表示，但有时可以一个“d”为后缀。

一旦用户已经输入要传输的序列后，可以直接发送数据序列，也可以将数据序列保存在预定义序列表中以便以后使用。点击 **queue** 按钮可将传输序列立即保存在缓冲序列表中。点击 **add** 按钮可将传输序列保存在预定义序列表中。

如果需要将预定义序列表中的序列复制到缓冲序列表，首先确定序列输入框为空白，然后选择需要预定义的序列，最后点击 **queue** 键。

如果需从预定义序列中移除数据，先选择需要移除的序列，然后点击 **delete** 键。

4.10 电压表和电流表

Protues VSM 提供了 AC 和 DC 的电压表、电流表。它们可以像其它元器件一样在在电路图连线。仿真开始后，它们通过自带的终端或以易读的数字格式显示电压或电流值。

仪表的 FSD 可以现实 3 位有效数字，最多可显示 2 位小数。显示电压范围可通过元器件编辑窗口的 Display Range 属性进行设置。

电压表模型含内阻属性，默认值是 100M，属性值可以通过属性编辑窗口进行设置。当内置电阻为空时，加载模型内阻选项无效。

交流电压表和交流电流表以用户可定义的时间常数显示有效值。

 **注意：**功率表仅适用于正弦波形。

第五章 微处理器仿真

5.1 介绍

概述

Proteus VSM 支持基于微控制器的设计的仿真和调试。为了正确调试固件，仿真需要了解高级源代码行、地址信息、局部变量和各种其他信息，这些信息不包含在用于对物理设备进行编程的原始 HEX 文件中。

幸运的是，所有现代编译器都会产生一个调试输出文件，其中包含此信息，可以将其加载到 Proteus 中进行调试。有两种方法可以设置 Proteus VSM 项目进行调试：

- ◆ 使用 VSM Studio IDE，能自动检测和配置已安装的编译器，以产生与 Proteus 兼容的调试输出，并将固件自动发送到 Proteus VSM 模拟器。
- ◆ 通过在 ISIS 中进行手动操作。在这种情况下，您必须手动设置工具链以生成合适的调试文件，然后必须在仿真之前将调试文件应用于 ISIS 中的原理图部分。

在文档的[源代码调试](#)部分和下一节中分别讨论了这两种方法。

5.2 VSM Studio

介绍

总览

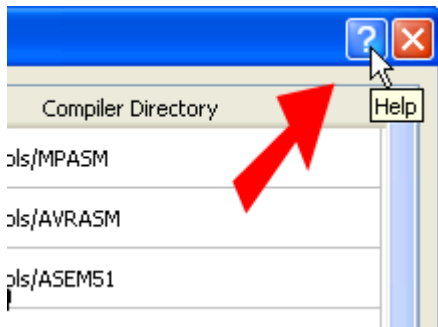
VSM Studio 是 Proteus 8 附带的集成开发环境 (IDE)，旨在与 Proteus VSM 仿真软件无缝地协同工作。除了您期望从开发环境中获得的所有功能之外，我们还包括：

- ◆ 自动检测和配置数量不断增长的编译器，以便它们生成与 Proteus VSM 仿真最兼容的输出格式。只需按一下按钮，此操作便能生效，因此您可以直接在项目上进行工作，然后在 VSM Studio 内部进行编译，而无需学习和更改编译器参数列表。
- ◆ 与原理图模块直接通讯。成功编译源代码会自动将固件应用于原理图上的微控制器组件。VSM Studio 中的调试命令（断点，单步等）可以同步推进和暂停原理图仿真。
- ◆ 活动弹窗技术允许您在原理图上指定感兴趣的区域，并使它们在仿真期间在 VSM Studio IDE 的窗口中显示。这使您可以在 IDE 内进行调试的同时查看（例如 LCD 显示屏）和与原理图进行交互（例如按钮，开关）。
- ◆ 独特的外设库和与 Arduino 一起使用的高级驱动程序功能。

关于本文档

该帮助文件包含一个简单的演练教程以及有关使用 VSM Studio IDE 的一些常规用法信息。

通过对话表单顶部的问号按钮或 F1 键盘快捷键，可以为 VSM Studio 中的所有对话表单提供更具体的帮助。



② 注意：[Proteus VSM 主教程](#)涵盖了有关 Proteus VSM 和高级调试技术的更多详细讨论，而 ISIS 教程则涵盖了有关仿真和为 PCB 布局创建原理图的信息。我们强烈建议您除了本文档之外，还花一些时间来复习这些文档中适用于您的工作的内容。

指南

概述

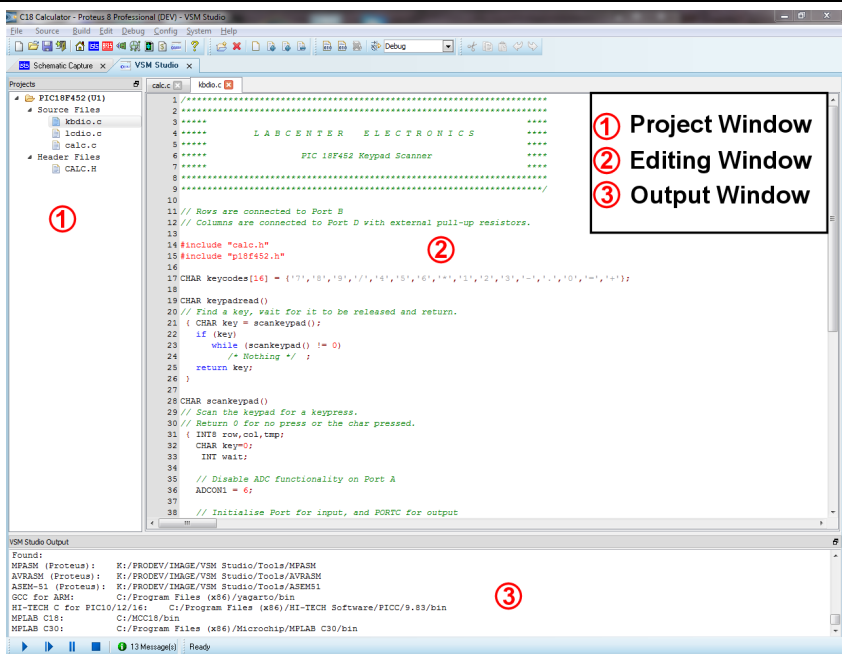
对于大多数以前在 IDE 中编写过代码的用户，VSM Studio 应该看起来很熟悉。它是 Proteus 8 中的顶级应用程序模块，可以随时从应用程序模块工具栏打开。



此模块可用于固件开发和固件调试。因此，根据操作模式，有两个布局配置文件。

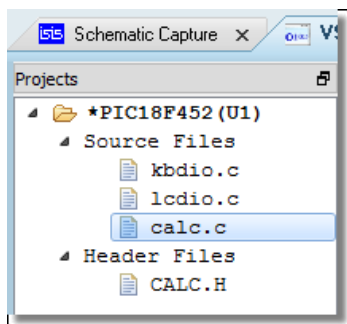
1、编辑版面

编辑布局具有三个主要区域：



项目树

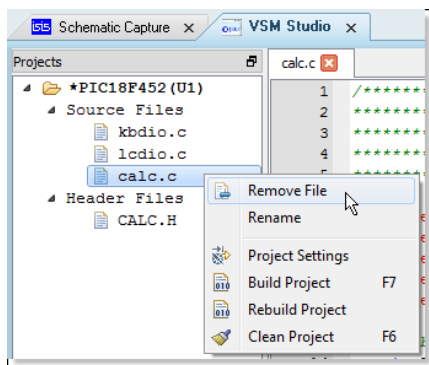
项目树包含当前项目中文件的排序列表。这可以包括源文件，头文件，库文件和包含文件。



已打开工程的项目树视图

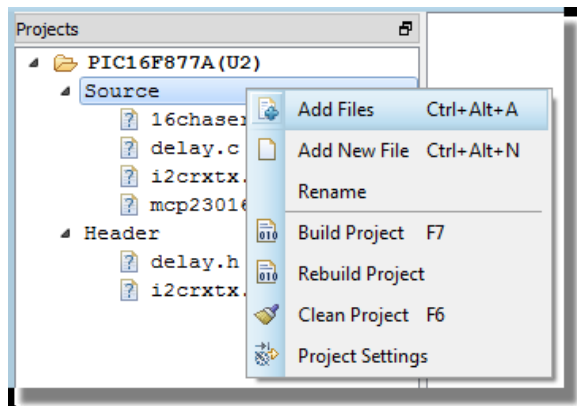
双击项目树中的文件将在“编辑窗口”中打开该文件。

右键单击文件将显示一个菜单栏，该菜单允许您从项目中删除该文件。



文件上的菜单栏

右键单击类别（例如源文件）会显示一个不同的菜单栏，该菜单可让您将新文件或现有文件添加到项目的该类别下。

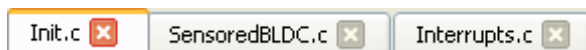


源文件类别上的菜单栏

❗注意：您也可以直接从“源”菜单中添加和删除项目中的文件。

编辑窗口

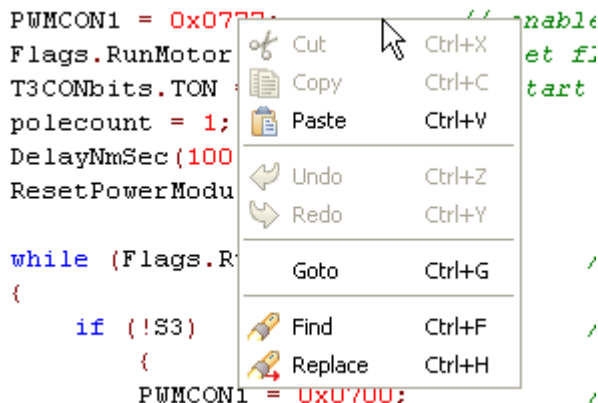
在编辑窗口中，将打开所有源文件以供查看和编辑。如果一次打开多个文件，则每个文件都将成为编辑窗口中的选项卡。



编辑窗口中已打开文件的多个选项卡

您可以通过单击要查看的选项卡在选项卡之间切换。从“编辑窗口”中关闭文件（通过选项卡右侧的叉号）将从“编辑窗口”中删除文件，而不是从“项目树”中删除文件。

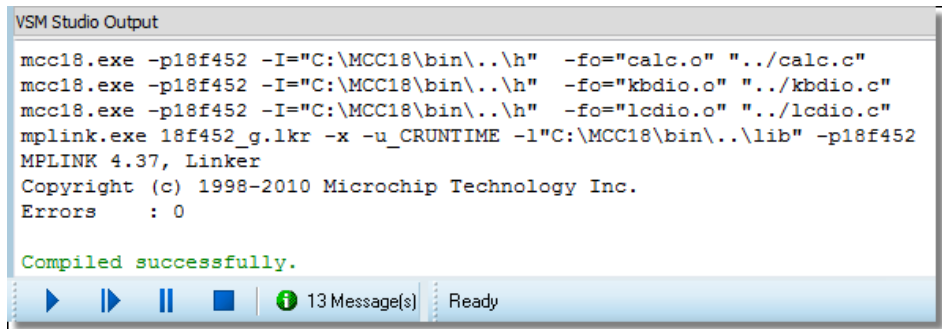
在编写或编辑固件时，您可直接在右键单击菜单栏中找到最常用的编辑命令。这些命令也可以从“编辑”菜单中获得。



编辑窗口菜单栏

输出窗口

输出窗口将显示项目编译的结果以及其他信息和提示。



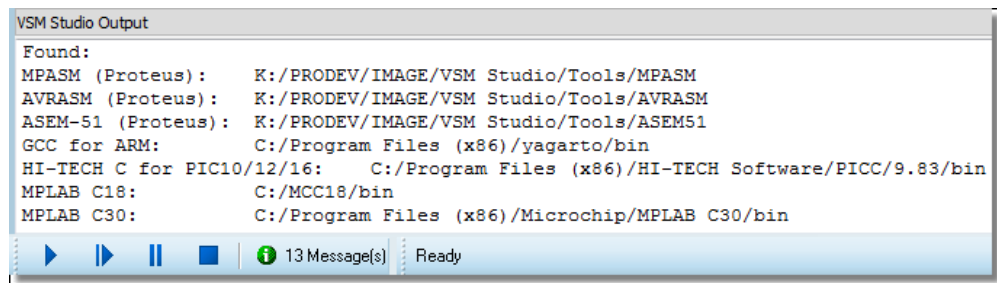
```
VSM Studio Output

mcc18.exe -p18f452 -I="C:\MCC18\bin\..\h" -fo="calc.o" "../calc.c"
mcc18.exe -p18f452 -I="C:\MCC18\bin\..\h" -fo="kbdio.o" "../kbdio.c"
mcc18.exe -p18f452 -I="C:\MCC18\bin\..\h" -fo="lcdio.o" "../lcdio.c"
mplink.exe 18f452_g.lkr -x -u_CRUNTIME -l"C:\MCC18\bin\..\lib" -p18f452
MPLINK 4.37, Linker
Copyright (c) 1998-2010 Microchip Technology Inc.
Errors      : 0

Compiled successfully.
```

编译后的输出窗口

当您检查已安装的编译器时，它还将显示检测和配置状态



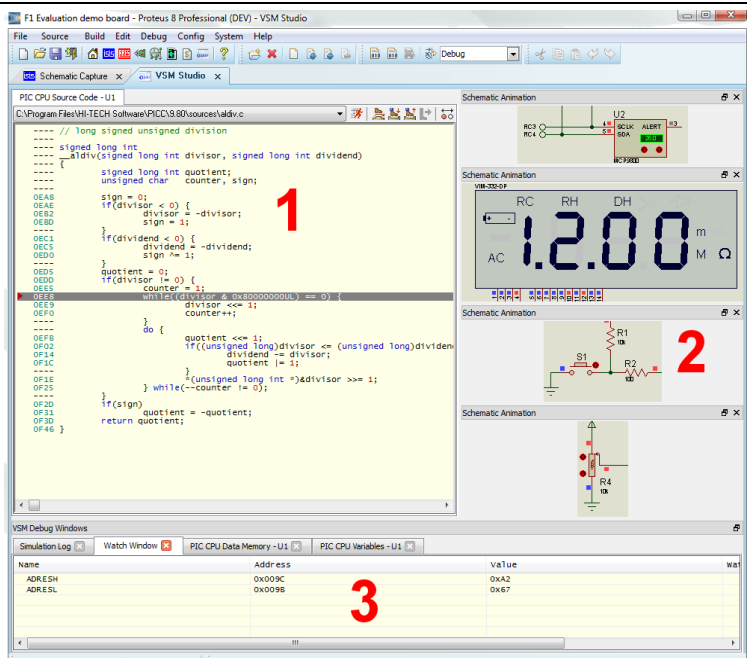
```
VSM Studio Output

Found:
MPASM (Proteus):      K:/PRODEV/IMAGE/VSM Studio/Tools/MPASM
AVRASM (Proteus):     K:/PRODEV/IMAGE/VSM Studio/Tools/AVRASM
ASEM-51 (Proteus):    K:/PRODEV/IMAGE/VSM Studio/Tools/ASEM51
GCC for ARM:          C:/Program Files (x86)/yagarto/bin
HI-TECH C for PIC10/12/16: C:/Program Files (x86)/HI-TECH Software/PICC/9.83/bin
MPLAB C18:             C:/MCC18/bin
MPLAB C30:            C:/Program Files (x86)/Microchip/MPLAB C30/bin
```

扩展阅读：

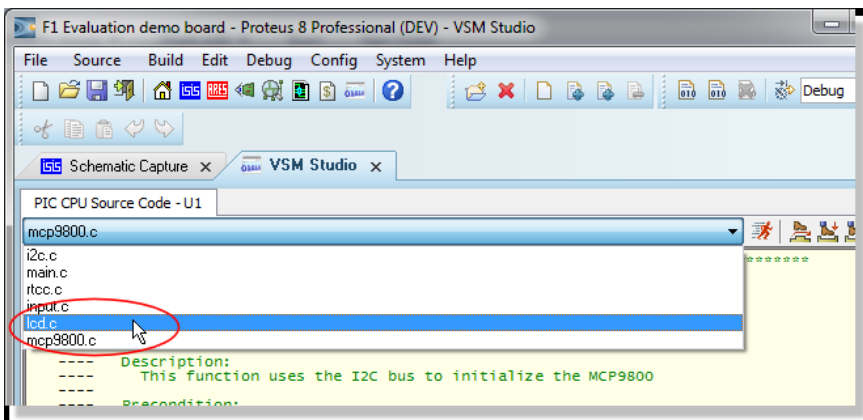
[使用指南-编程](#)

2、调试布局



源代码显示

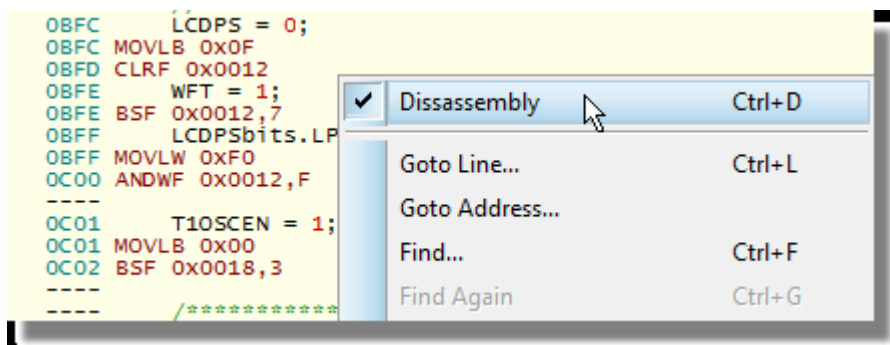
手动暂停仿真或达到断点时，将显示源代码。在这里，您可以：
浏览源文件或从顶部的组合框中选择其他源文件



从调试功能按键或通过键盘快捷键命令进行单步调试或设置断点。



在汇编器和高级代码之间更改调试级别。

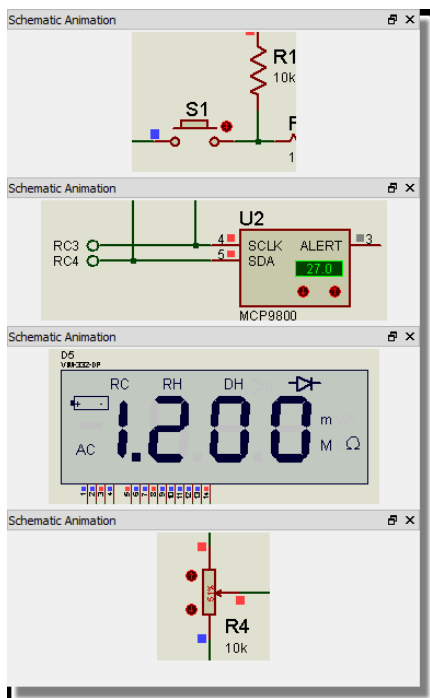


扩展阅读：

[仿真与调试](#)

活动弹窗面板

如果您在原理图上定义的任何活动弹窗，则默认情况下，它们将在仿真运行时停靠在右侧显示。



活动弹窗既可视又互动。您可以单击按钮或在活动弹窗口中轻击开关，其方式与完整原理图完全相同。

扩展阅读：

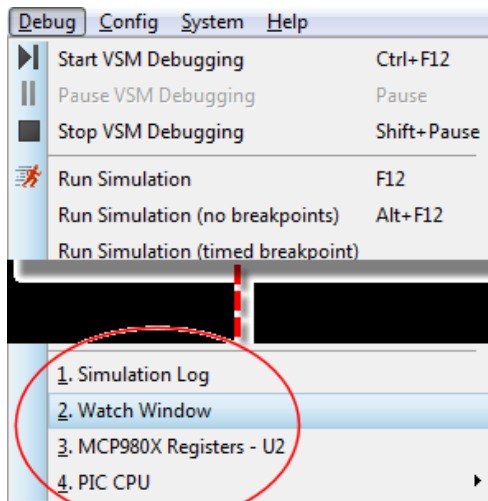
[活动弹窗](#)

❗注意：当您定义一个活动弹窗时，您的主要兴趣是调试固件而不是硬件。因此，您会发现在启动仿真时，“VSM Studio”选项卡将自动出现在前台。如果原理图上没有定义活动弹窗，则在启动仿

真时，“原理图”选项卡将成为前台。

调试窗口

Proteus 包含大量的内存转储窗口，监视窗口，寄存器和堆栈显示，所有这些都可以从 VSM Studio 中的“调试”菜单启动。



当您打开调试窗口时，它会停靠在 VSM Studio 框架的底部。自由运行的仿真期间，除监视窗口外的所有调试窗口均处于非活动状态，但在仿真暂停或单步调试时将显示状态。

工具栏

VSM Studio 中的工具栏提供了对许多常用命令的快速访问，其中大多数将很容易为用户所识别。



VSM Studio 工具栏

有关每个图标的说明，请参见[图标参考表](#)。

有关将 VSM Studio 与 Proteus VSM 一起使用的更多信息，请参见[调试](#)部分。

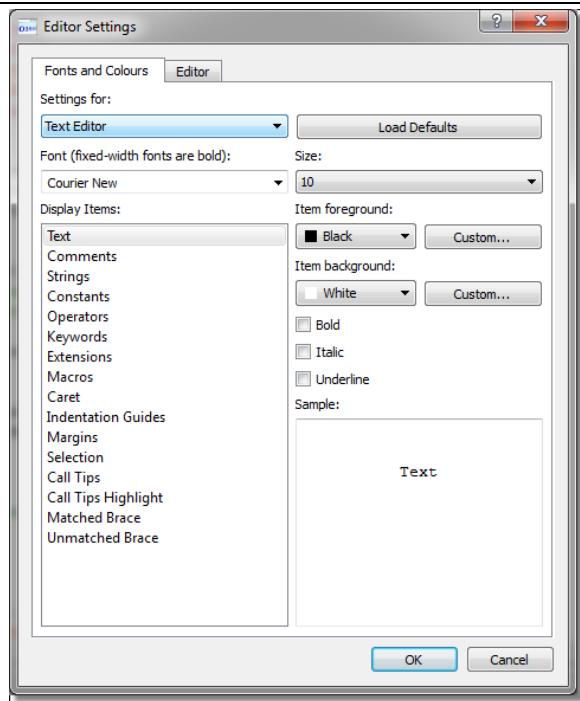
窗口定位和样式

编辑和调试布局中的各个窗口都可以使用标准技术来调整大小和位置：

- ◆ 从窗口的灰色标题栏中拖放可以重新定位。
- ◆ 双击灰色标题栏或双击矩形图标将使窗口浮动。
- ◆ 窗口浮动时，双击灰色标题栏将使该窗口停靠在其先前的位置。或者，您可以将浮动窗口拖动到新的停靠位置。
- ◆ 如上所述，可以通过其标题栏右侧的叉或右键单击工具栏来完全关闭输出窗口。
- ◆ 可以通过拖放窗口的边缘来调整“项目树”和“输出窗口”的大小。

可以通过系统菜单上的编辑器配置命令来修改各个窗口的样式和颜色

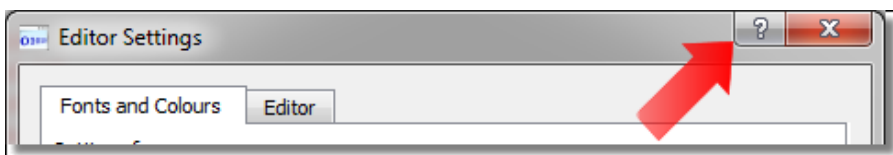
首先选择“字体和颜色”选项卡，然后选择要配置的窗口。



选择配置窗口

- ◆ 接下来，使用对话框右侧的样式命令选择要更改和调整的显示项目。
- ◆ 对其他显示项目重复以上操作。
- ◆ 单击确定按钮退出对话框以提交更改。

编辑窗口包含其他几个样式选项，可用于配置诸如代码折叠，缩进和行高亮之类的内容。与所有对话框形式一样，可以通过选择编辑器选项卡，然后单击右上角的问号图标来找到有关这些选项的特定信息。



为编辑器配置启动相关帮助

受支持的编译器

前言

VSM Studio 中最重要的功能之一是自动检测和配置受支持的编译器，以确保它们与 Proteus VSM 仿真软件一起正常工作。这意味着对于典型项目，用户只需知道已安装受支持的编译器即可。

支持的编译器

以下是受支持的编译器的最新列表-请参阅以下部分以获取有关扫描和下载的说明。虽然目前市场上编译器的子集相对较小，但我们专注于为每个受支持的处理器系列提供至少一个免费工具链的支持，然后包括其他一些流行的编译器。

我们计划随着时间的推移扩展受支持的编译器列表，并欢迎提出建议（support@labcenter.com）。为了增加对编译器的支持，它必须能够产生标准的调试格式，并且我们必须能够使用 Proteus 测试产品以确保兼容性。另外，对给定编译器的支持将取决于它的流行程度和用户的需求。

IAR

- IAR for 8051
- IAR for ARM

MPLAB

- MPLAB C18
- MPLAB C30
- MPASM
- MPLAB XC8
- MPLAB XC16

CCS

- CCS for PIC

CodeComposer

- CodeComposer for MSP430
- CodeComposer for PICCOLO

GCC

- GCC for ARM
- WinAVR
- SDCC for 8051
- GCC for MSP430

Hi-Tech

- Hi-Tech for PIC10/12/16
- Hi-Tech for PIC18
- Hi-Tech for dsPIC

ASEM-51

- ASEM-51 for 8051 (assembler only)

AVRASM

- AVRASM for AVR (assembler only)
- BASCOM AVR

Keil

- Keil uVision4 for 8051
- Keil for ARM

编译器检测

首次在新安装的 Proteus 上打开 VSM Studio 模块时，它将自动扫描您的计算机以检测支持的编译器的现有安装。您可以通过从系统菜单中选择编译器配置命令来查看此扫描的结果。

- ◆ 使用“全部检查”按钮执行全面重新扫描并找到新的安装。
- ◆ 如果您在列表中选择了一个编译器，则可以使用“检查”按钮尝试找到该指定的编译器。
- ◆ 如果在另一个驱动器上安装了编译器，则可以通过选择编译器，然后使用“手动”按钮来手动告诉 VSM Studio 它在哪里。或者，单击有关编译器的编译器目录字段，然后手动键入文件路径。

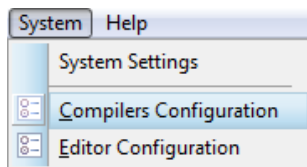
ASEM-51 (Proteus)	Yes	D:/Image/tools/ASEM51
CCS for PIC	No	
CodeComposer for MSP430	No	C:\Program Files\
GCC for ARM	No	Program Files
HI-TECH C for PIC10/12/16	No	
HI-TECH C for PIC18	Yes	am Files/Hi-TECH Software/PICC-18/PRO/9.63/BIN

手动输入编译器的位置

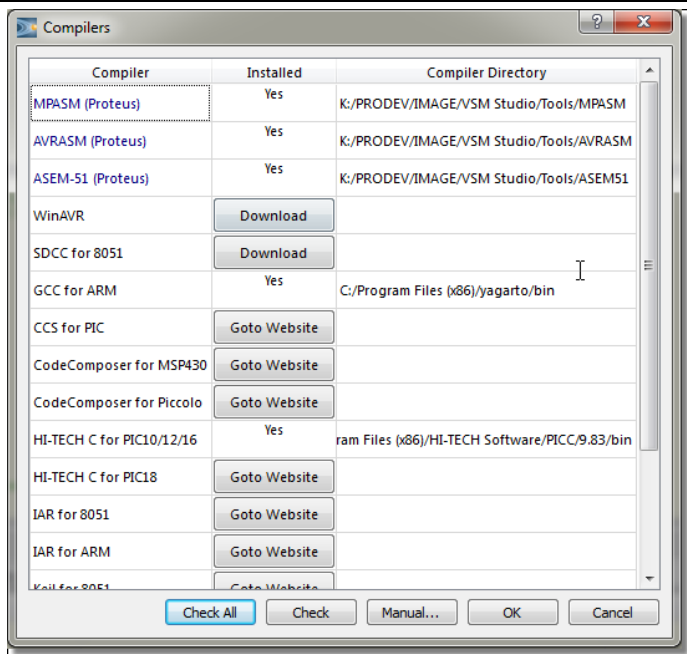
单击确定按钮退出对话框以提交更改。

安装新的编译器：

从 VSM Studio 的“系统”菜单中打开“编译器配置”对话框。

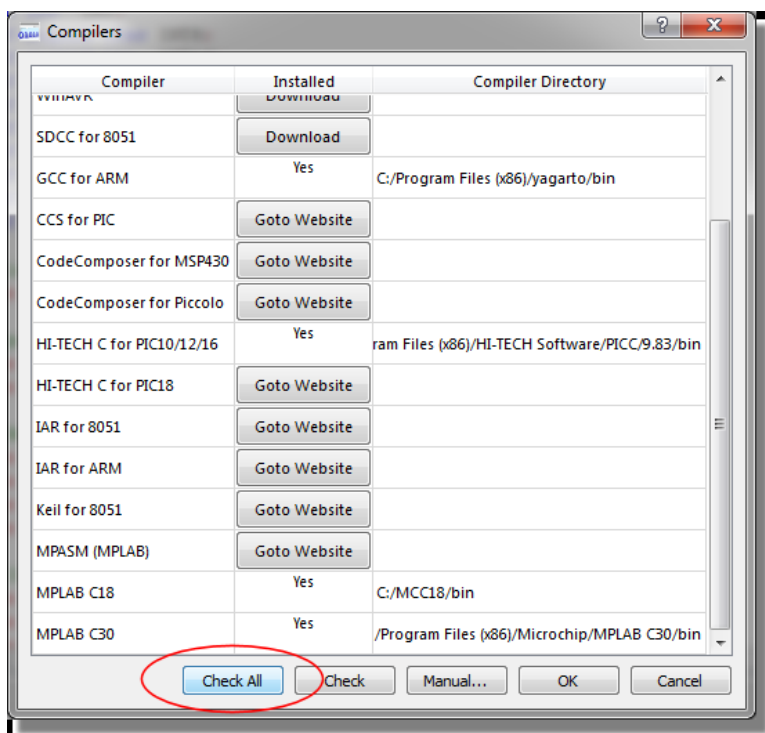


单击“下载”按钮（“开源”）或“网站”按钮（专有）以下载编译器。



以常用方式安装编译器。

使用编译器配置对话框中的“检查全部”按钮来查找并自动配置编译器设置。



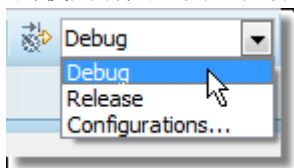
❗ 注意：任何扫描的结果将显示在“输出”窗口中。请注意，不支持某些编译器的较旧版本，因为它们要么无法测试，要么无法生成合适的输出格式。

使用 F1 键或在对话框上的问号上单击鼠标以获取更多详细信息。

编译器配置

一旦检测到编译器，它将自动配置为产生与 Proteus VSM 中的源代码级调试兼容的输出，并产生用于对物理芯片进行编程的二进制文件。这些分别称为“调试”和“Release”配置。

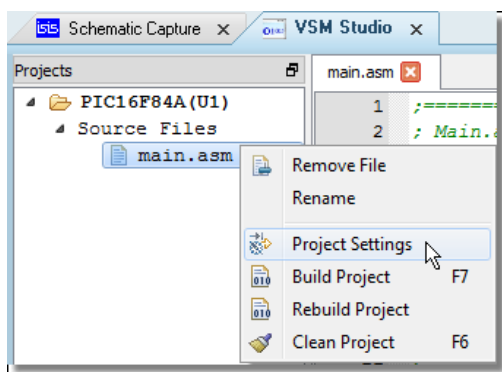
实际上，只有 Debug 配置与 Proteus 的使用有关，因此在打开或创建工程时始终是默认配置。



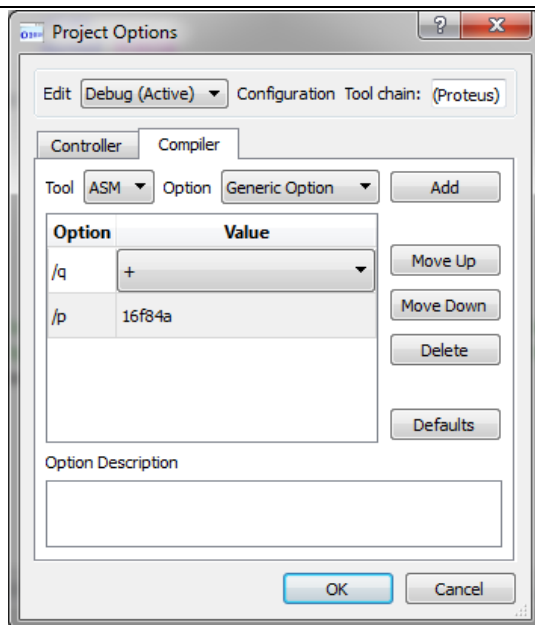
用于 Proteus 仿真的调试配置

对于许多项目，我们需要做的就是一次写入固件，就构建工程（工程菜单-构建工程命令）。但是，如果需要，可以从“工程设置”对话框中编辑或添加所有编译器/汇编器/工具链选项。

从 VSM Studio 内的“构建”菜单中选择“工程设置”（或右键单击项目树上的文件，从菜单栏中选择）。

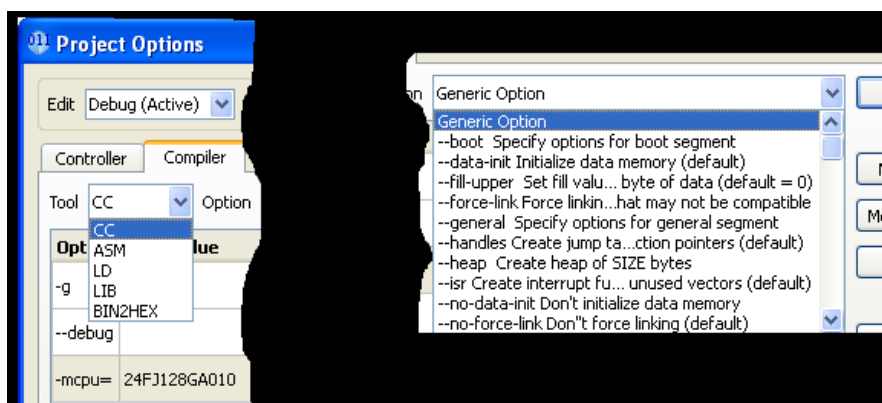


确保您正在编辑正确的配置，然后切换到对话框的“编译器”选项卡。



编辑调试配置的编译器选项

可以从此选项卡的左上角分别选择许多工具（取决于所使用的工具链），然后可以为每个工具添加许多选项。



编辑工具链中工具的配置选项

有关特定选项的更多信息，请参见特定编译器的参考手册。

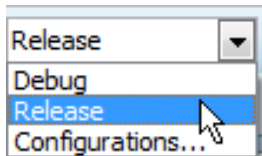
- ④ 注意：对话框上的默认按钮会将设置恢复为预配置。
- ④ 注意：选择后，您将无法更改给定工程的工具链。如果错误选择了一个工具链，则需要删除该工程（在 VSM Studio 项目树中右键单击该工程），然后创建一个新工程。

输出文件

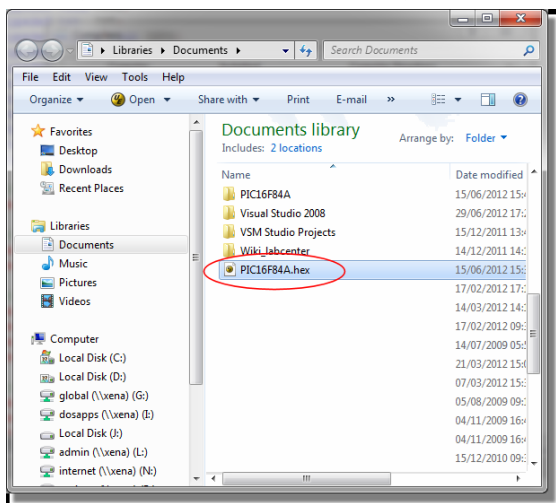
在“调试”配置（默认设置）中进行编译时，输出文件在很大程度上取决于编译器。例如，在 PIC 世界中，目标文件通常是 COF 文件，而对于 AVR 和 ARM，通常是 ELF / DWARF 文件。实际上，您无需担心任何这些，因为 VSM Studio 会自动配置编译器以生成最适合 Proteus 仿真的文件集。

由于 VSM Studio 是 Proteus 8 的一部分，因此输出文件也将自动传输到逻辑示意图捕获以进行仿真。简而言之，使用 Proteus 进行仿真不需要用户在此处做任何操作。

但是，如果您正在对物理设备进行编程，则需要进行一些调整。首先，您需要将配置从 Debug 更改为 Release。这将确保生成二进制 HEX 文件。



与 Debug 配置不同，Release 配置会将 HEX 文件拖放到 Proteus 8 项目文件 (*.pdsprj) 之外，使您可以将其用于对实际电路板进行编程。该文件与 Proteus 项目存放在同一目录中，但分开存放。

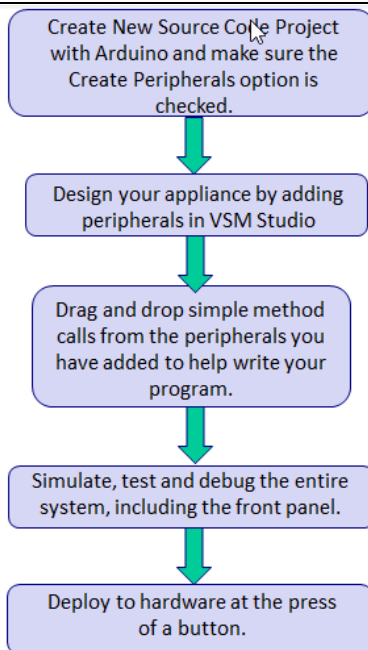


Arduino 开发

介绍

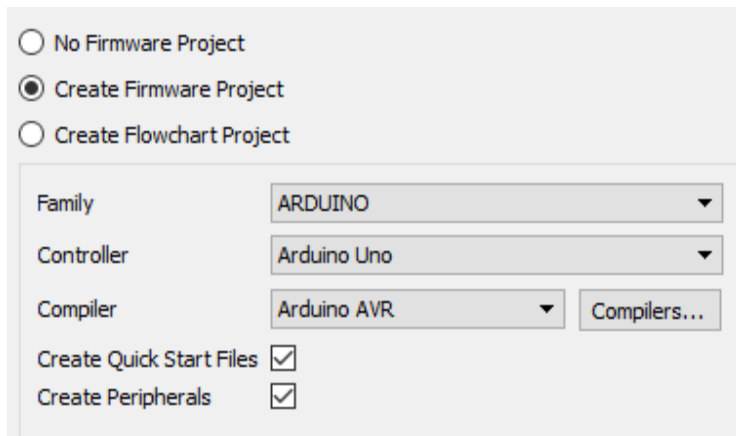
Arduino 是一种非常流行的电子环境，它由现成的电路板（外设和分线器）和易于驱动它们的综合软件库组成。Proteus 通过对所需 AVR 模型的支持来支持 Arduino 仿真，而我们庞大的嵌入式外围模型库也意味着我们也可以仿真大多数外设模块和分线板。

由于我们拥有了所有技术，因此我们花了很多时间使 Proteus 中的 Arduino 工作尽可能简单直观。这意味着与 Arduino 一起在 VSM Studio 中工作与在其他章节中描述的其他处理器对象大不相同。



新建工程

在“新建工程”中，当我们要选择“创建固件工程”选项时，默认选项能满足固件页面的要求。对象系列是 Arduino（不是 AVR），选择控制器为 Arduino Uno。其他重要选项中确保同时选择了“创建快速入门文件”和“创建外围设备”选项。



❗注意：“创建外围设备”选项在这里至关重要，因为它使您可以从所有已经准备好的外设和分线板中进行选择，并初始化所有与之配合使用的 Labcenter 外围设备驱动程序。如果未选中此框，则您仍可以使用 Arduino，但无法访问 VSM Studio 中的 Labcenter 外设驱动程序。

“新建工程向导”其余部分均选择默认设置就可以了。

完成后，您将在 VSM Studio 中获得一个源代码工程，其中包括了大量的预生成代码。

```

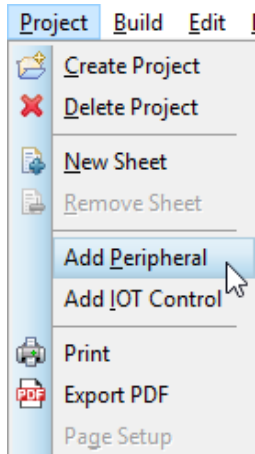
9  //---PERIPHERAL_BEGIN---
10 #pragma GCC push_options
11 #pragma GCC optimize ("Os")
12
13 #include <core.h> // Required by cpu
14 #include <cpu.h>
15 #include <TimerOne.h>
16
17 #pragma GCC pop_options
18
19 // Peripheral Constructors
20 CPU &cpu = Cpu;
21 TimerOne &timer1 = Timer1;
22
23 void setup () {
24 // Peripheral initializers
25
26
27 // Call user SETUP routine
28 user_setup();
29 }
30
31 void loop() {
32 // Poll the peripherals
33
34 // Call user LOOP routine
35 user_loop();
36 }
37 //---PERIPHERAL_END---
    
```

使用 MFC 或类似工具的人会发现这很熟悉，但关键是不要打扰，更改或移动生成的代码。它由 Labcenter 驱动程序使用，并在您添加外围设备时更新，因此需要由 Proteus 软件进行管理。您应该忽略本节代码，并像往常一样在 `setup()` 和 `loop()` 程序中编写代码。

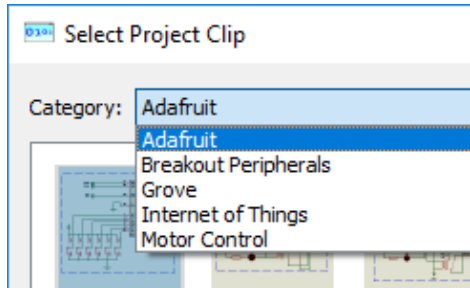
在原理图上，您将找到一个预先放置的 Arduino Uno。

设计阶段

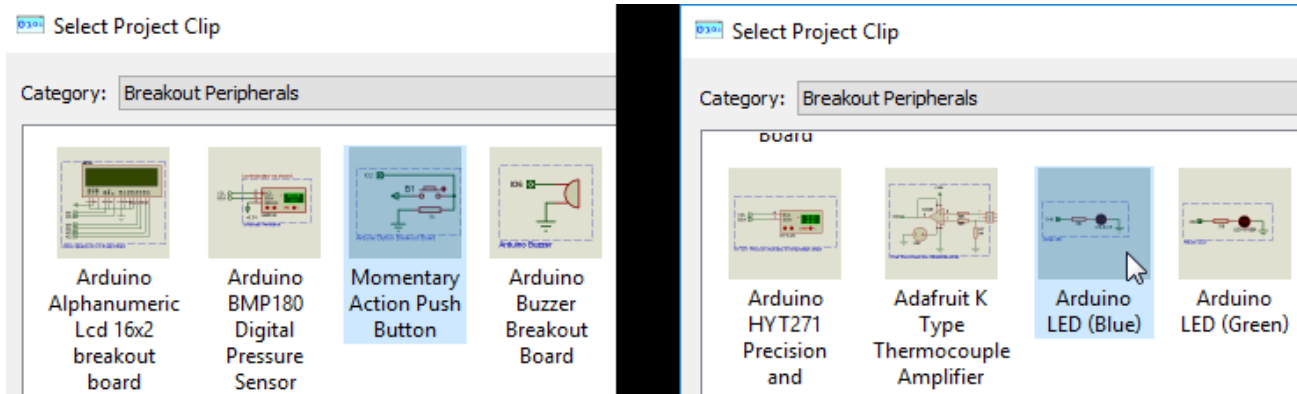
现在要做的第一件事是查询并添加您的外设。我们通过在“工程”菜单和项目树上右键单击的菜单栏中执行“添加外围设备”命令来简化此操作。



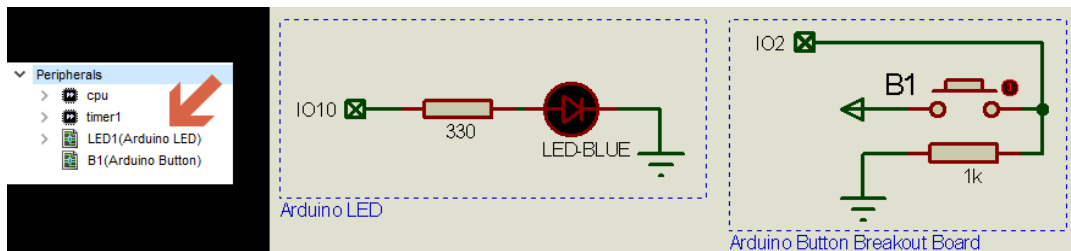
这为您提供了一个外围设备库，其中包含许多 Adafruit 外设，分线板，Grove 外围设备和电机控制模块。



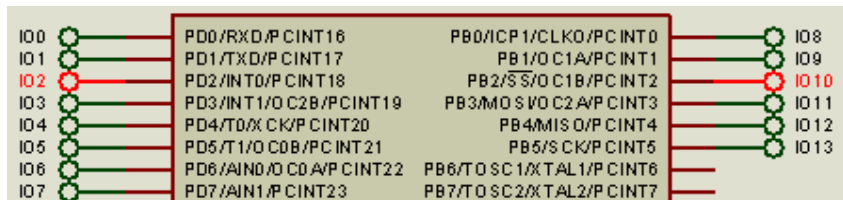
您可以通过选择它们并单击“添加”按钮或双击将它们添加到工程中来。为了进行简单的测试，我们将添加一个触发按键和一个来自 Breakout 外设类别的 LED。



这些将同时出现在 VSM Studio 的项目树中，并将自动为您放置在原理图上。

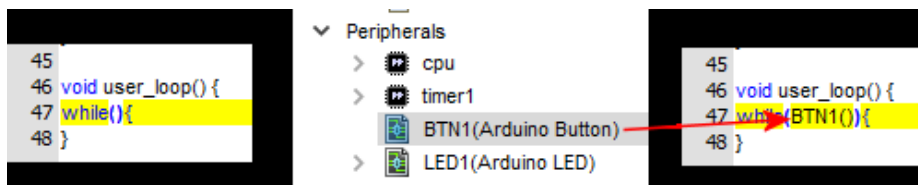


通过端子可在原理图上实现连通性，在此原理图上，按钮连接到 IO2，LED 连接到 IO10。

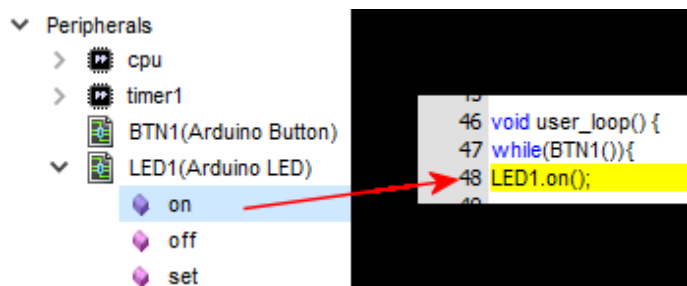


在固件中，我们可以看到许多控制 LED 的方法，但是按键没有多余的操作方法。我们可以将这些方法拖放到源代码上，以帮助我们控制外围设备。假设我们要在按键按下时打开 LED。我们开始

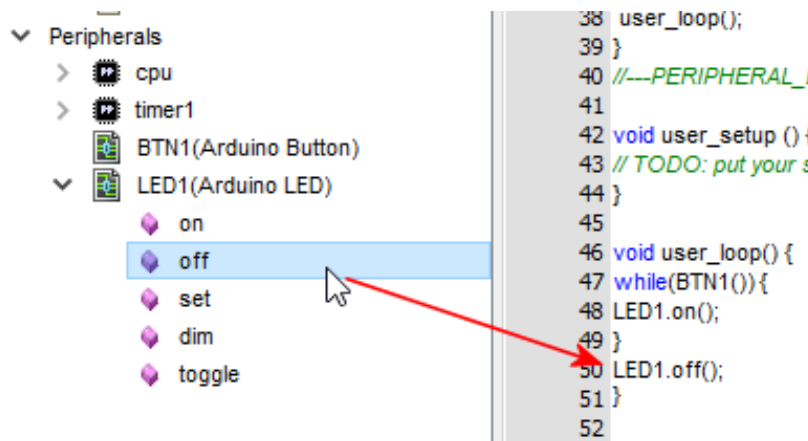
while 循环，然后只需将项目树中的按键函数拖动到源代码上即可。



现在我们要在 while() 函数中点亮 LED。按住 CTRL 键并拖放 LED 的 on 函数。请注意，插入位置跟随鼠标插入符移动，以便您可以将其放置在任意位置。



最后，我们需要在 while 循环之外关闭 LED。同样，我们可以将 off 函数拖放到程序中的正确位置。



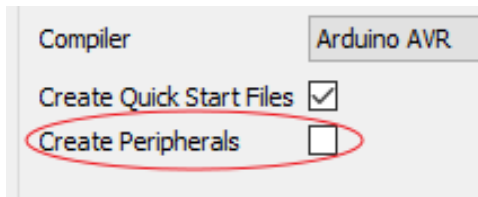
到此完成编写程序。显然，这是一个琐碎的示例，但无论是 LED，BLDC 电机还是 TFT 显示器，都适用相同的原理。

总结：

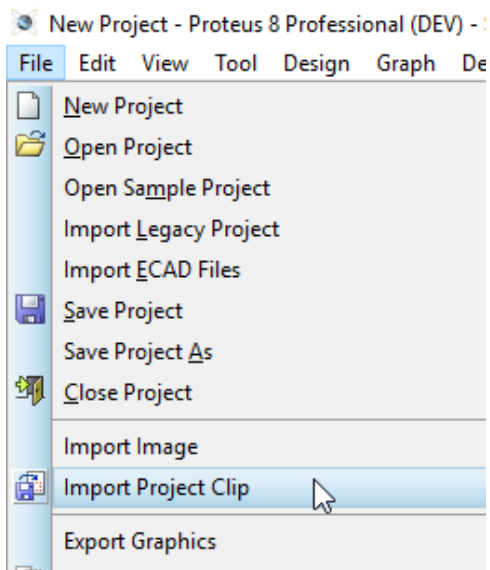
- ◆ 请勿在文件顶部更改或移动生成的代码。
- ◆ 如果正常拖放，则代码段将插入文件中鼠标插入符所在的任何位置。
- ◆ 如果在拖动时按住 CTRL 键，则鼠标插入符将随拖动而移动，并且在释放鼠标时，代码段将插入到插入符的位置。

在没有 Labcenter 驱动程序的情况下工作

您可能会决定不想使用 Labcenter 外围设备驱动程序，或者想要创建自己的外设板/在原理图中添加电子设备。在这种情况下，应在“新建工程向导”中取消选择“创建外围设备”。



您将得到一个空白草图（没有预生成的代码），并且必须从原理图上的“文件”菜单上的“导入工程 clip”命令中添加任何原理图剪辑。



执行此操作时，您将不会导入用于该外设的 Labcenter 驱动程序，而必须在 Arduino C++级别上编写程序。这可以提供额外的灵活性，但不太简单。

固件开发

前言

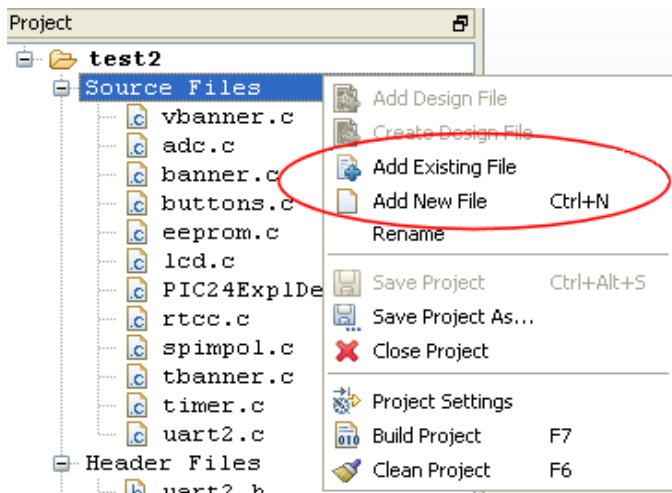
在 VSM Studio 中编写或编辑代码与在任何其他 IDE 中进行编写或编辑都差不多。创建或导入工程后，即可开始认真工作。本小节详细介绍如何在 IDE 中执行一些常见任务。

添加/删除/重命名文件

向工程添加文件

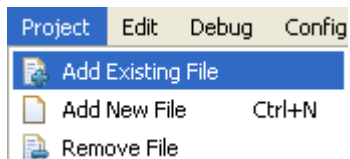
方式一：

右键单击工程类别，然后从菜单栏中选择“件添加现有文”或“添加新文件”命令。请注意，您必须右键单击类别（例如源文件），而不是文件本身。



方式二：

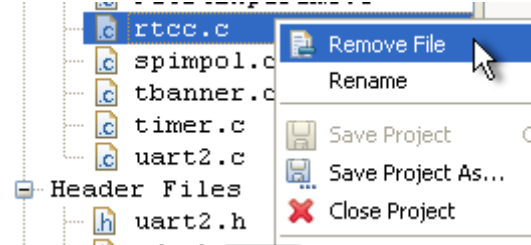
直接从“工程”菜单中选择“添加现有文件”或“添加新文件”命令。



从项目中删除文件

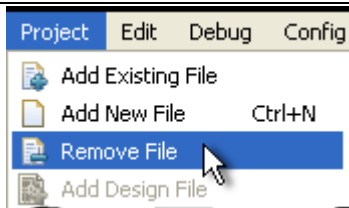
方式一：

右键单击项目树中的文件，然后从出现的菜单栏中选择“删除文件”命令。



方式二：

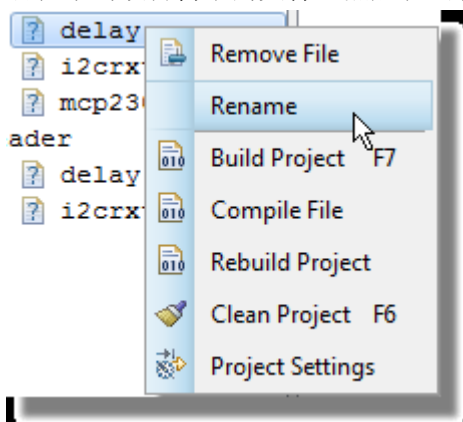
在工程菜单上选择删除文件命令。这将删除当前在“项目树”中选择的文件。



❗注意：在这两种情况下，文件都将从工程中删除，但不会从磁盘中删除。

重命名文件

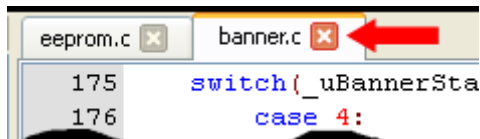
右键单击项目树中的文件，然后从出现的菜单栏中选择重命名选项。



❗注意：重命名文件将在工程和磁盘上都重命名该文件。

打开和关闭文件进行编辑

- ◆ 要在“编辑窗口”中打开工程文件，只需在项目树中双击该文件即可。
- ◆ 要在“编辑窗口”中关闭文件，请点击选项卡右上方的叉。这将关闭文件，但不会将其从工程中删除。

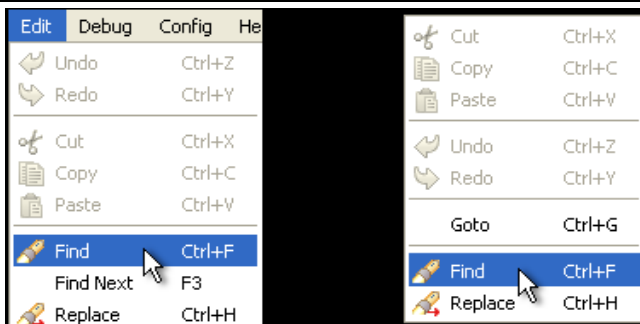


- ◆ 要在“编辑窗口”中打开的文件之间切换，请使用 CTRL + TAB 快捷键或单击编辑窗口顶部相应的选项卡。

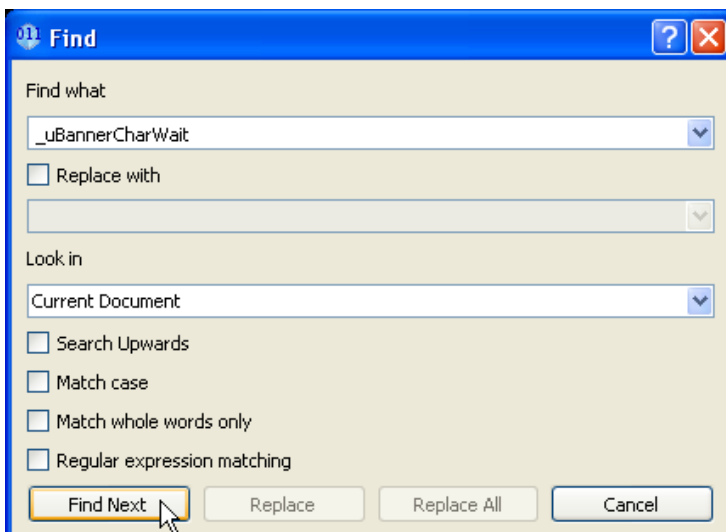
查找和替换

在项目中查找文本

- ◆ 通过右键单击“编辑窗口”或从“编辑菜单”中启动“查找”对话框。



- ◆ 输入您要查找的字符串，以及您要取代其的文本。



- ◆ 指定搜索范围（当前文档，所有打开的文档，当前工程）。
- ◆ 指定其他任何搜索选项。正则表达式语法基于 Perl 正则表达式语言，但应视为高级选项。
- ② 注意：在文件中搜索特定搜索词实例时，可以使用 F3 快捷键代替“查找下一个”按钮。

编辑器颜色和样式

通过从“配置”菜单中选择“编辑器配置”命令，可以更改 IDE 的外观。此对话框的“字体和颜色”选项卡将允许您更改 IDE 中各个窗口的语法突出显示选项，而“编辑器”选项卡为“编辑窗口”提供一些特定的配置选项。

使用对话框上的相关帮助可获得更具体的帮助。

编译器设置和编译

编译您的工程

- ◆ 确保通过工具栏上的组合框进行配置（调试配置以与 Proteus 配合使用）。
- ◆ 单击工具箱上配置组合框左侧的构建按钮，或从“工程”菜单中选择“构建工程”。
- ◆ 编译器信息将显示在“输出”窗口中，并以“已成功编译”消息结尾。


```

Output
pic30-gcc.exe -g --debug -mcpu=24FJ128GA010 -c -o
pic30-gcc.exe -g --debug -mcpu=24FJ128GA010 -c -o
pic30-gcc.exe -g --debug -mcpu=24FJ128GA010 -c -o
pic30-gcc.exe -g --debug -mcpu=24FJ128GA010 -c -o
pic30-gcc.exe -o "Debug.cof" "vbanner.obj" "adc.obj"
Compiled successfully.
Click the ISIS icon to launch the design file in IS
    
```

查找并修复编译器错误

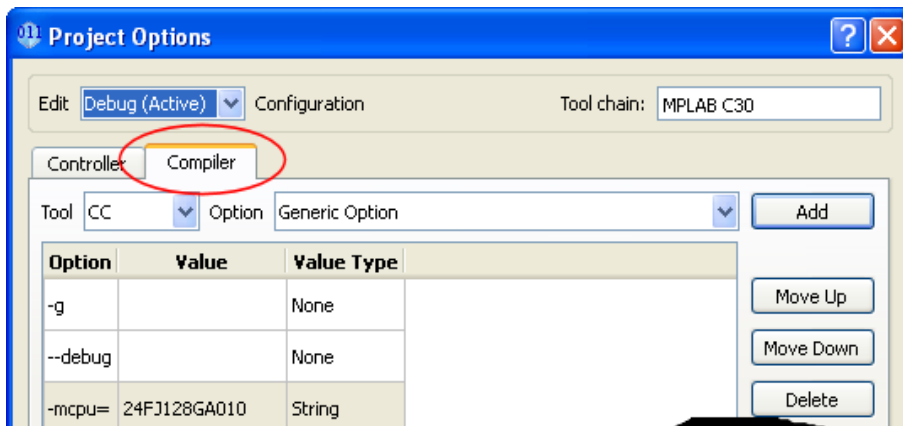
- ◆ 在“输出”窗口中找到错误消息。
- ◆ 单击消息以在“编辑”窗口中找到该行。

```

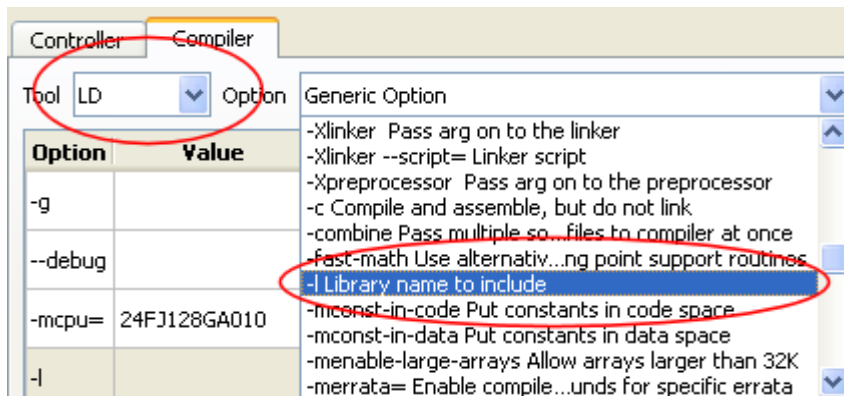
Output
pic30-gcc.exe -g --debug -mcpu=24FJ128GA010 -c
../banner.c: In function 'BannerProcessEvent':
../banner.c:188: error: syntax error before '!' token
../banner.c:195: error: break statement not within loop or switch
../banner.c:197: error: case label not within a switch statement
../banner.c:205: error: break statement not within loop or switch
    
```

查找并修复工具链错误

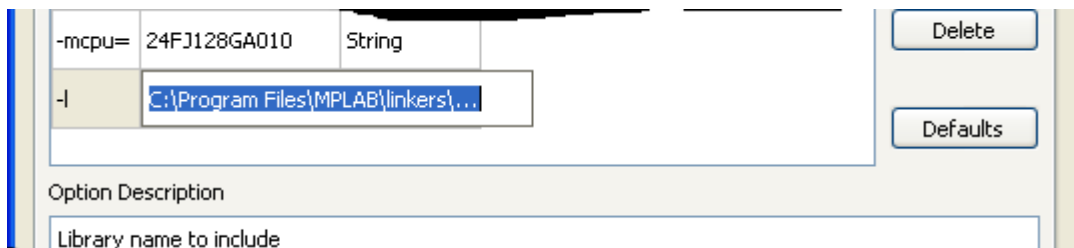
该错误可能是工具链错误，或者要求您编辑工程设置以引入库或所包含的文件。为此，首先启动工程设置对话框，然后选择“编译器”选项卡。



接下来，选择您需要为其指定其他选项的工具（CC 是编译器，ASM 是汇编器，LD 是工具链等），然后从右侧选择所需的选项。关于选项的更多信息将在编译器本身的用户手册中提供，尽管最常见的是将“-L”添加到库搜索路径的目录。



添加选项并输入值（以上示例的目录路径）。



最后，退出对话框并重新编译工程。

使用 Proteus VSM

VSM Studio 中的所有工程都应具有与其关联的 ISIS 示意图，以便您可以从固件开发无缝过渡到测试和调试阶段。

- ◆ 如果原理图与工程关联，则只需单击编辑器顶部工具栏上的 ISIS 按钮以启动 ISIS 并为处理器编程以进行仿真。
- ◆ 如果尚未将原理图与项目关联，则可以通过从“工程”菜单中选择“添加设计文件”选项来实现。一旦添加到项目中，请如上所述启动 ISIS 进行仿真。
- ◆ 如果尚未设计原理图以伴随工程一起启动 ISIS，请先开发原理图，然后按如上所述将其添加到 VSM Studio 项目中。

有关测试和调试的更多信息，请参见下一节。

测试和调试

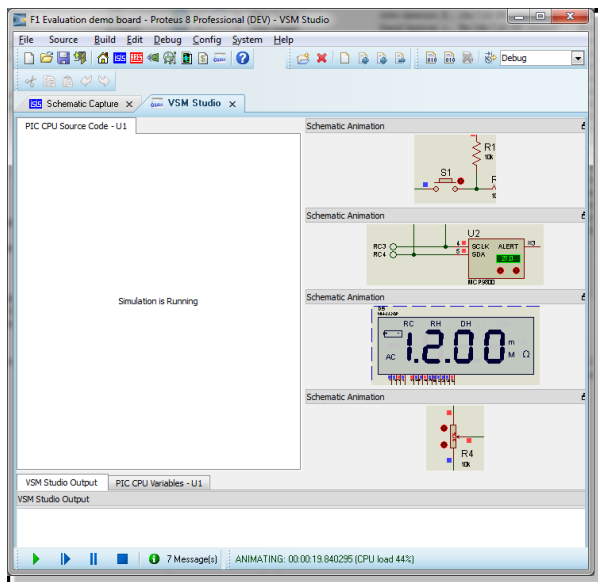
概述

Proteus 包含大量调试工具，可帮助您在硬件和固件中进行故障查找。我们强烈建议您阅读 [Proteus VSM 教程](#)，其中单独介绍了这些技术的完整演练。在本节中，我们将提供有关 VSM Studio IDE 中常见调试技术的一些简要说明。

运行仿真时，VSM Studio 中的窗口布局将从编辑布局更改为调试布局。这使您可以启动和查看各种调试窗口，查看并与活动弹窗进行交互，以及设置断点和单步执行代码。它还有禁止编辑代码的功能-如果要更改固件，必须首先停止仿真。

断点

当仿真自由运行时，源窗口和大多数调试窗口都将被禁用。这是因为指令通常以无法肉眼查看的速度执行。



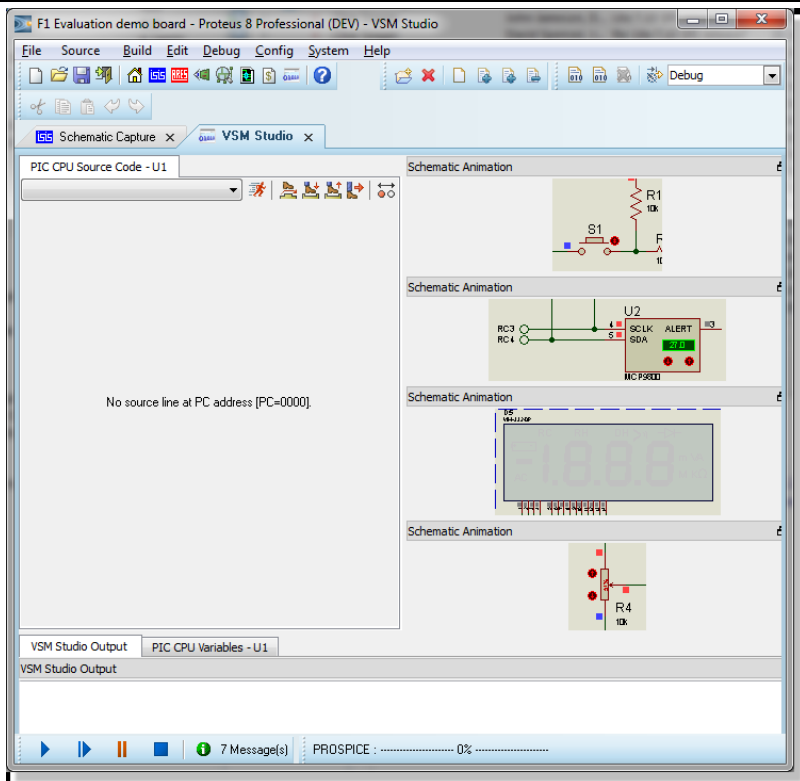
要查看源代码，您需要暂停模拟。您可以随时通过按下动画控制面板中的暂停按钮来执行此操作。



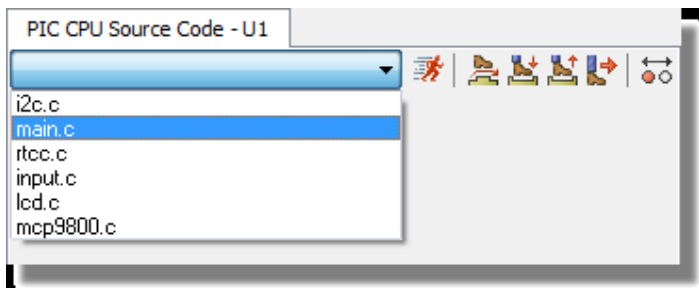
这实际上是一个断点，但在用户直接控制下-当您按下暂停按钮时，您通常不知道正在执行什么源代码。在源代码中指定要暂停的位置的功能更加有用。

设置断点

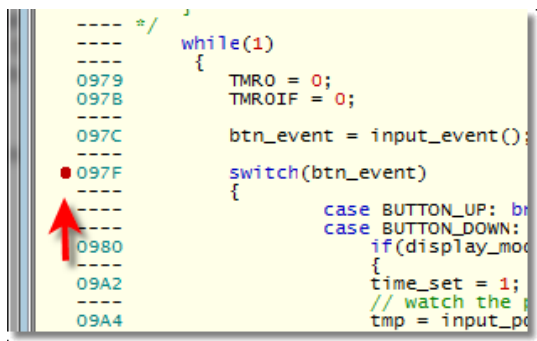
使用暂停按钮开始仿真。这将在零点暂停（仿真“已启动”，但未执行任何代码）。



在 VSM Studio 中，从源窗口顶部的组合框中选择相关的源文件。



找到您想要在其上设置断点的源代码行，双击源代码行，或者单击源代码行，然后使用断点图标设置断点。或者左键单击该行，再右键单击该行，然后从出现的菜单栏中选择“切换断点”选项，即可放置断点。



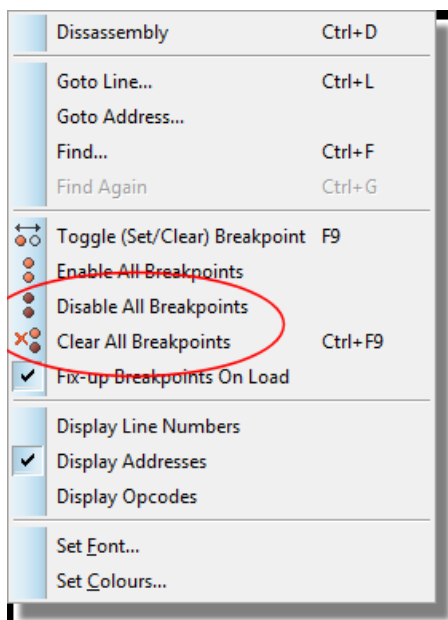
现在已经设置了断点，请按播放按钮以运行仿真。仿真将在执行设置了断点的代码行之前自动停止。



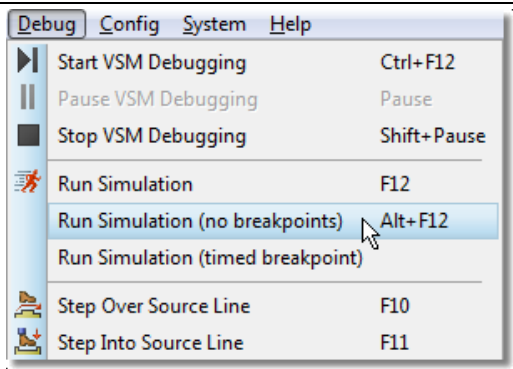
控制断点

您可以通过多种方式有选择地或全局禁用和清除断点。

如果在暂停仿真时在源代码窗口上单击鼠标右键，则可以禁用所有断点或清除所有断点。



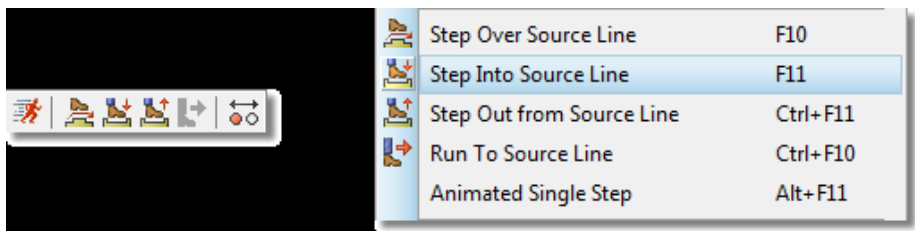
您可以从调试菜单中使用“无断点运行”命令启动仿真。



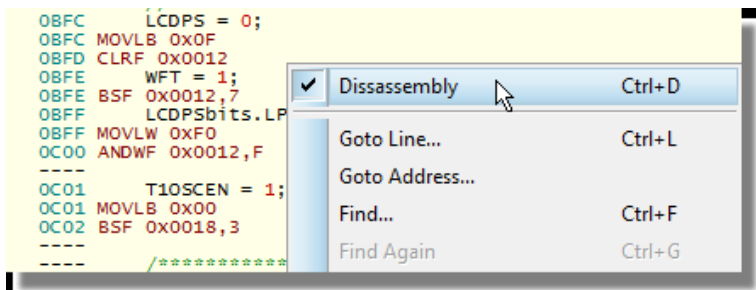
断点分别在三种状态（设置/禁用/清除）中循环，因此上述用于设置断点的所有操作都可以用于禁用和/或清除断点。

单步调试

单步执行代码的工作原理与其他调试器一样，具有三个主要命令（进入，执行和退出）。只要通过源窗口上的图标、调试菜单上的命令或键盘快捷键暂停仿真，就可以操作这些命令。



如果您的程序是使用 C 语言之类的高级语言编写的，则可以选择单步执行高级指令或汇编级。这会根据源窗口上显示的内容自动发生。要查看反汇编（以及单步汇编程序说明），请右键单击源窗口，然后从出现的菜单栏中选择反汇编选项。



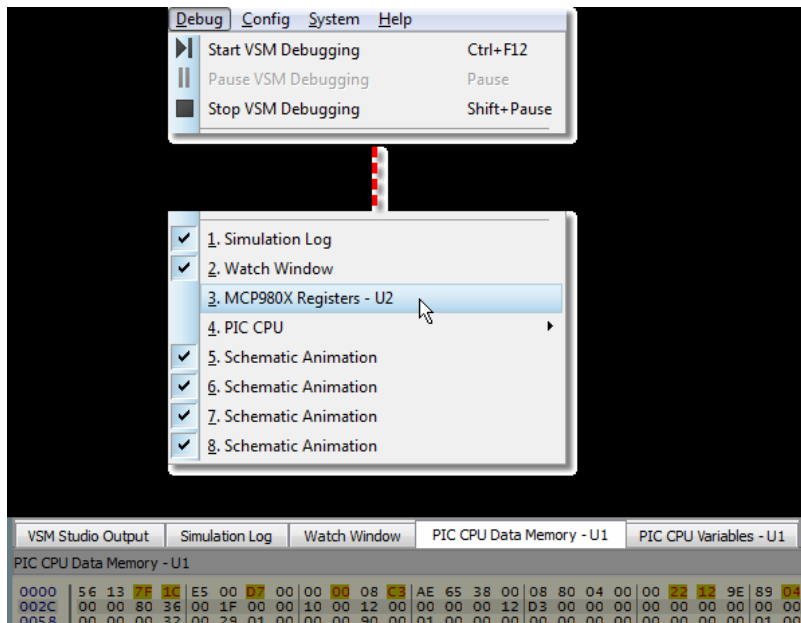
相同的操作将使源代码返回到高级语言（进而执行单步调试）。

❗注意：暂停仿真时，只能通过按暂停按钮或在预定义的断点处执行单步代码。

调试窗口

Proteus 包含大量调试窗口，可用于帮助跟踪和查找错误。这些包括内存转储，堆栈和寄存器窗口，变量窗口，模拟顾问程序和监视窗口。

所有这些窗口都可以从“调试”菜单中打开，并将在 VSM Studio 底部显示为停靠的选项卡。



模拟运行时，除监视窗口外的所有调试窗口均被禁用。当模拟暂停或您是单步执行代码时，所有调试窗口将变为活动状态。

在 Proteus VSM 教程中可以找到使用监视窗口和其他调试窗口的实用示例。

活动弹窗

活动弹出窗口允许您在调试代码时选择要查看或与之交互的原理图区域。反过来，这避免了调试期间在示意图选项卡和 VSM Studio 选项卡之间不断切换，从而实质上将相关硬件带入了软件调试环境。

在 VSM Studio 新项目教程中可以找到定义活动弹出窗口的简单示例，而在 Proteus VSM 主教程中可以找到更详细的介绍。

笔记：

- ◆ 您只能在仿真停止后在原理图上定义活动弹出窗口。
- ◆ 与其他调试窗口一样，可以使用窗口右上角的叉号在 VSM Studio 中关闭活动弹出窗口。关闭后，可以从“调试”菜单中重新打开它们。
- ◆ 当您在原理图上定义了活动弹出窗口并在 VSM Studio 中打开时，Proteus 将在启动模拟时自动切换到 VSM Studio 选项卡。假定您的会话将主要在调试环境中进行。
- ◆ 当原理图上没有定义活动弹出窗口时，或者如果您在 VSM Studio 中关闭了它们的窗口，则 Proteus 将在启动仿真时自动切换到原理图选项卡。
- ◆ 无论哪种情况，在暂停模拟以显示源代码时，Proteus 都会自动切换到 VSM Studio 选项卡。

图标参考

以下是 VSM Studio 应用程序模块中使用的图标的列表，以及用法的简短说明。

图标	说明
	新建项目：启动“新建工程向导”以创建新的 VSM Studio 工程。
	保存工程/文件：将当前工程/文件保存到磁盘。
	将工程另存为：使用其他名称保存当前工程。
	打开工程：从磁盘打开现有工程。
	关闭工程/文件：关闭当前打开的工程/文件。
	打开文件：打开现有文件而不将其添加到当前工程。
	将现有文件添加到工程：将磁盘上的现有文件添加到当前工程。
	向刚吃过添加新文件：向当前工程添加（空白）新文件。
	从工程中删除文件：从当前工程中删除文件。
	将新设计文件添加到工程：将现有原理图文件与当前工程关联。
	工程设置：启动工程设置对话框。
	生成/重新生成工程：编译并生成当前工程。
	清理工程：清理并从当前工程中删除所有临时文件。
	撤消：撤消在编辑器中执行的最后一个操作。
	重做：重做在编辑器中执行的最后一个动作。
	剪切：剪切当前文档中的选定文本。
	复制：将所选文本复制到剪贴板。
	粘贴：将剪贴板的内容粘贴到当前文档中。
	查找：搜索文本字符串或表达式。
	替换：使用指定的替代文本字符串替换搜索字符串的所有匹配项。
	运行仿真：在 ISIS 窗口中运行仿真。
	单步执行：单步执行一个函数/一段代码。
	进入：进入函数/代码部分。
	跳出：跳出函数/代码部分。
	跑：跑到断点。
	切换断点：打开或关闭断点。
	编译器/编辑器设置：启动编译器/编辑器设置对话框。
	帮助：启动应用程序帮助文件。

5.3 基于 Proteus VSM 源代码调试

概述

PROTEUS VSM 支持基于汇编程序和编译器的源代码调试。调试下载器的设置包含在 LOADERS.DLL 中并且此类工具的数量正在以很快的速度增长。在我方网站“3rd Party Compilers”可以查询到最新信息。

如果已经使用了支持的汇编器和编译器，则 PROTEUS VSM 会为项目中每一个源文件生产一个源代码窗口。当仿真暂停时，这些窗口会出现在 DEBUG 菜单上。

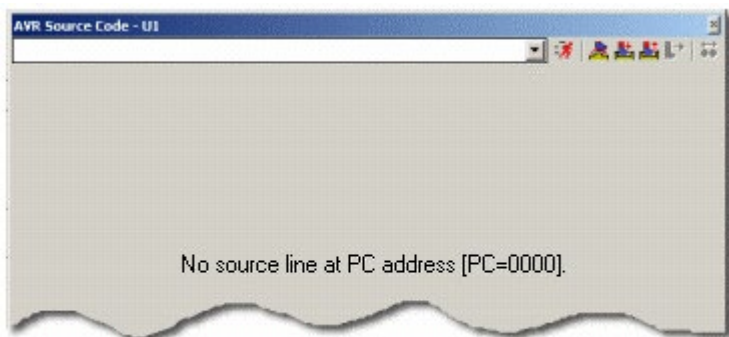
源代码窗口

源代码窗口是 Proteus 中用来仿真电路系统的最基本的工具。运行程序时允许客户单步仿真程序或同时运行整个嵌入式系统。程序可以通过两种方式加载到源代码窗口。一是通过编译器输出的仿真文件，二是通过由源代码控制系统生成的 SDI 文件。当使用编译器时，仿真文件应该通过如下所示的程序属性窗口载入到原理图部件。

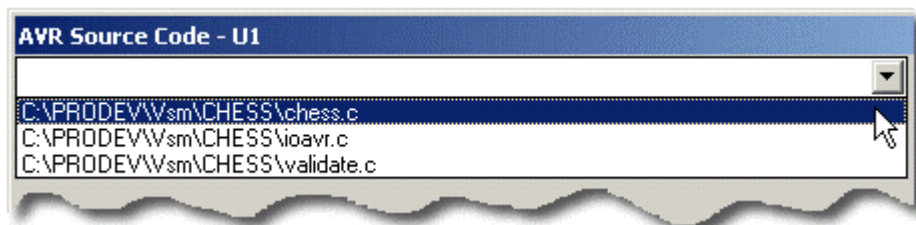


大家通常会犯一个错误就是载入到原理图部件的是 HEX 文件然后企图采用源代码仿真的方式。这样是不太正确的。为了能进行源代码仿真，仿真系统还得需要更多的有关地址、变量等信息。只有当用汇编程序编写或编译工具不能生成任何可支持的仿真格式时才使用 HEX 格式文件（这种情况的可能性很小）

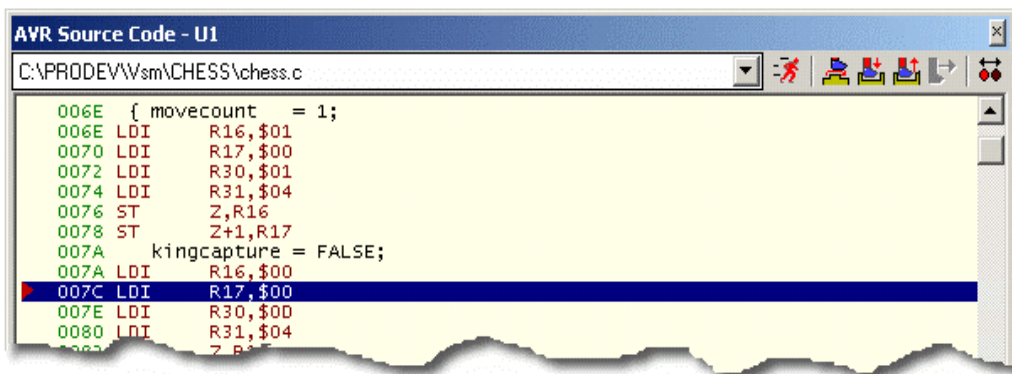
在进程控制按钮中按 PAUSE 启动仿真，在默认的情况下源代码窗口就会出现，界面如下图所示。



界面中所示信息的原因是仿真时间为 0，此时微处理器的程序计数器并不是指向程序可执行行。用户当然可以从源程序窗口顶端的下拉框选择想仿真的程序然后按需要进行单步仿真或设置端点，如下图。



如果使用高级语言编写的程序，则可以使用高级语言或机器码进行单步调试。如果想调出机器码，可以使用组合键 CTRL+D 或者在源代码窗口上单击右键后选择 Disassembly 即可。



在源代码窗口单击右键后选择合适的使能/禁止选项，用户便可以观测行号、代码和地址。

支持的仿真文件格式

1. ELF/DWARF

此格式是 Proteus 推荐的仿真文件格式。该格式一般为大多数模型编译器所支持，在可预见的未来它将流行于大多数编译器。在 Proteus 使用 ELF 文件，只要在微处理器的属性对话框指定该文件即可。

- 目前 ELF/DWARF 只支持 AVR 和 ARM7 模型，但不久将会支持 PIC 和其它系列的芯片。
- ELF（二进制文件）和 DWARF（调试文件）必须保存在同一个目录下——当在编译器选择 ELF/DWARF 作为仿真文件格式时，默认情况下 ELF、DWARF 保存在同一个目录。

2. COFF

此格式虽然为 Proteus 支持的格式，但在 ELF/DWARF 为有效格式的情况下还是应以 ELF/DWARF 为优先选项。这就意味着一个 PIC 编译器可能将 COF 中的浮点数描述成 IEEE 浮点数或 PIC 的浮点数并且不提供清晰的信息说明格式的种类。如果需要更多有关 COF 格式的信息请查阅相关网站。如果想使用 COF 格式，则需在编译器指定仿真格式然后在微处理器的属性对话框的 program property 中使用 COF 格式。

3. UBROF

UBROF 是 IAR 编译器一种可选的仿真格式。如果想使用 UBROF8 格式（非最新版本），则需在编译器指定仿真格式然后在微处理器的属性对话框的 program property 中使用 UBROF 格式。

用户还可以在 IAR Embedded Workbench 中直接驱动 Proteus，这方面的信息可以在 Using Proteus as a Virtual ICE 章节找到。

4. OMF51

OMF51 是由 Keil 编译器生成的仿真格式。如果想使用 OMF51 格式，则需在编译器指定仿真格式然后在微处理器的属性对话框的 program property 中使用 OMF51 格式文件。

更新版本的 Keil 编译器同样可以生成 ELF/DWARF 格式文件，因此建议使用 ELF/DWARF 格式。

用户同样可以在 Keil 中直接驱动 Proteus，这方面的信息可以在 Using Proteus as a Virtual ICE 章节找到。

5. COD

COD 格式由 Bytecraft 生成并广泛使用于 PIC 系列中。如果提供的信息不齐全并且有其它格式可以采用时，建议不使用 COD 格式。特别地，COD 符号仿真数据格式的限制是指 VSM 仿真支持局限于机器码仿真和只能观察指定的寄存器位置并且不支持变量显示。

如果想使用 COD 格式，则需在编译器指定仿真格式然后在微处理器的属性对话框的 program property 中使用 COD 格式文件。

6. BAS

目前 Proteus 针对 Crownhill Proton Development Suite 为客户提供一种解决方案。编译完程序后在 PIC 微处理器的属性编辑框的 program property 中指定 BAS 文件。

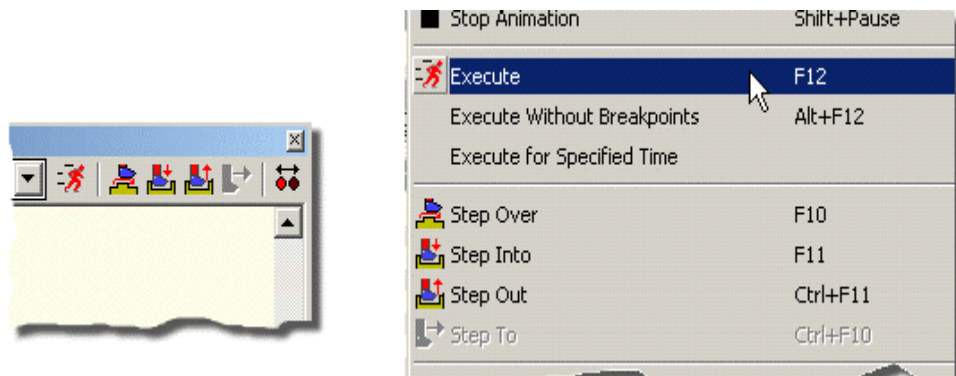
7. SDI

SDI 格式只在用汇编程序编写的程序能使用并且通过 ISIS 中的源代码控制系统进行组合生成。如果使用 SDI 格式，则需在 Code Generation Tools 对话框设置相应选项从而指定 Debug Extaraction tool（在安装软件时将会正常创建）然后在微处理器的属性对话框的 program property 中使用 SDI 格式文件。

注意：在微处理器的 Program property 选项中不要输入源程序文件的名字——Proteus VSM 不仿真“C”或“ASM”文件；CPU 从上述的仿真文件中载入和执行二进制机器码。

单步执行

Proteus 提供一系列用于单步执行的选项。全部选项可以在代码窗口本身选取或在 Debug 菜单操作。



- Step Over—单行执行程序。当遇到子程序时，以子程序为一行执行（即整个程序被 执行）
- Step Into—执行一条源代码指令。如果没有激活了的源代码窗口，将执行一条机器码指令。
- Step Out—执行到当前的子程序返回
- Step To—执行到程序运行到光标所处行。该功能只在有激活的源代码窗口的情况下才有效。

注意：除了 Step To 命令外，其它单步执行命令都可以在没有有效的源代码窗口的情况下使用。Proteus 仍然可以仿真由 Proteus 不支持的下载工具生成的代码。当然这样做还是存在不少困难的。

使用断点

对于研究软件的问题或设计中软硬交互性，断点是研究这些特性的一种非常有用的方法。一般情况下，在发生问题的子程序开端设置一个断点，启动仿真，然后与设计进行交互式仿真一直到程序运行到断点处。在断点处，仿真将被挂起。之后，利用单步执行观测寄存器值，存储单元和其它电路中涉及到的条件。打开 Show Logic State of Pins 对于调试会更有帮助。

在有效的源代码窗口里，可以按 F9 或右键菜单的选项对断点进行设置或清除。只能在有结果代码的程序行设置断点。



如果源程序被更改，Proteus VSM 将会尽力重新导入基于子程序地址的断点。如果源代码更改的地方过多则导入过程将会发生错误，但是一般情况下程序仍能正常工作，用户不必考虑导入过程中发生的错误。

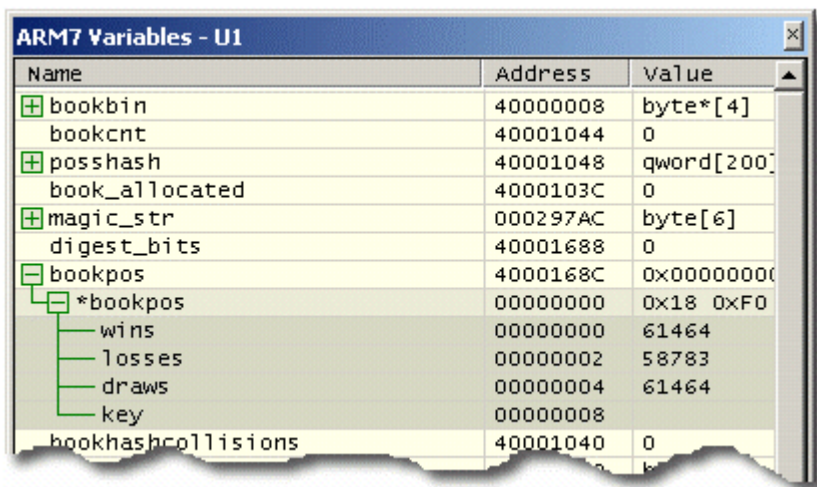
变量窗口

由 Proteus VSM 提供的大部分下载器可以读取程序变量的位置和源程序行的地址。Proteus 显示变量窗口和源代码窗口。

变量窗口是一个非常强大的仿真帮助工具。它提供了一系列可自定义的特性以便能更容易获取和明白所显示的信息。

1. 混合类型和指针的扩展

变量窗口将会很好处理混合类型（结构，数组，枚举）和指针。它以扩展了的树型方式显示类型的内容或者指针的相关参考值。



Name	Address	Value
bookbin	40000008	byte*[4]
bookcnt	40001044	0
posshash	40001048	qword[200]
book_allocated	4000103C	0
magic_str	000297AC	byte[6]
digest_bits	40001688	0
bookpos	4000168C	0x00000000
*bookpos	00000000	0x18 0xF0
wins	00000000	61464
losses	00000002	58783
draws	00000004	61464
key	00000008	
bookhashcollisions	40001040	0

载入变量窗口的信息由编译器生成的 Object Module File 提供。有可能的话推荐使用编译器生成的 ELF/DWARF 格式，在 ISIS 中的原理图部件指定使用 ELF 格式。在没有 ELF/SWARF 的情况下，可以使用 COFF 格式并且该格式应该提供完全的变量支持。

2. 更改信息和旧的数据值

当某变量值改变并且仿真暂停时，对应变量的值在变量窗口中以高亮显示。同样可以通过在变量窗口中点击右键选中 Show Previous Values 来查看旧的数据值。

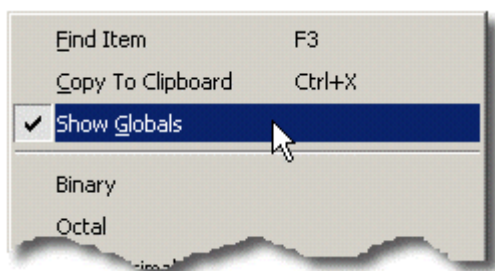
如果一混合类型（例如一结构），它的任何元素发生变化时，只是对应的元素以高亮的方式显示。

3. 拖放到观察窗口

虽然在程序运行时变量窗口是隐藏的，但用户可以向观察窗口拖放变量。观察窗口中的变量在仿真的过程中一直是可见的。因为观察窗口的可见性，使得可以针对变量设立观测条件，并当条件匹配时暂停程序。

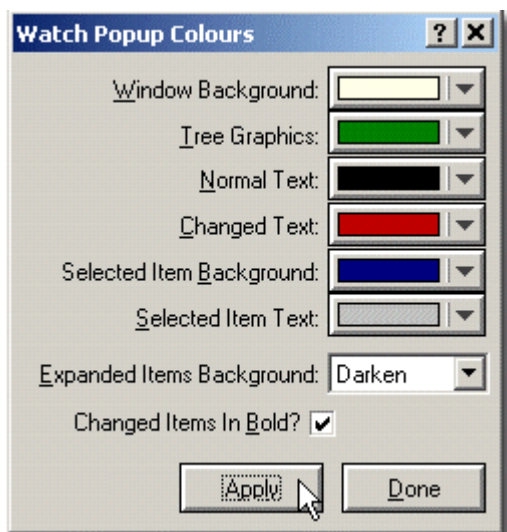
4. 隐藏和显示全局变量

在大型设计里可能有很多全局变量。在变量窗口点击右键从中选择 Show Globals 选项，可以隐藏全部局部变量，以便更加容易观测局部变量。



5. 颜色设置

在变量窗口点击右键，选择 Set Colours 选项后出现一对话框允许用户为变量窗口全部项目设置颜色。



多个 CPU 仿真

Proteus VSM 可以仿真含两个或两个以上 CPU 的电路系统。每一个 CPU 各自生成包括代码和变

量的弹出式窗口。这些窗口在 Debug 菜单的子菜单整合在一起。

当单步执行时，光标所在的源代码窗口对应的 CPU 被认为是主处理器，其它 CPU 将自由运行。当主 CPU 已经完成一条命令时，将中止从 CPU 的执行。如果将光标从源代码窗口退出，则最后光标所在的源代码窗口的处理器为主 CPU；单击其他任意处理器的源代码窗口，则改变主 CPU。

5.4 源代码控制系统

概述

源代码控制系统有两大主要功能：

- 程序源代码寄存于 ISIS 中。这一功能使用户可以直接在 ISIS 编辑环境中直接编辑源代码，而无须手动切换应用环境。
- 在 ISIS 中定义了源代码编译为目标代码的规则。一旦程序启动，并执行仿真时这些规则将被执行，因此目标代码被更新。

注意并不一定需要利用源代码控制系统仿真微处理器。事实上，如果用户定义的汇编程序或编译器自带的 IDE，可直接在其中编译。生成了可执行程序后切换到 Proteus 即可。

创建源代码文件

- 将源代码程序添加到设计中
- 在 Source 菜单中选择 Add/Remove Source Files
- 为源程序文件选择 Code Generation Tool。如果第一次使用新的汇编程序或编译器，需要使用菜单项 Source—Code Generation Tool。
- 点击 NEW 按钮，用文件选择器为源程序文件选择或输入文件名。如果期望的文件名不存在，用户可以在文本框中键入期望的文件名。
- 在 Flags 框输入处理源文件所需的标记。当每次使用某一特殊工具并且在该工具已登记的前提下，可以在 Flags 框输入标记。
- 点击 OK 将源文件加载到设计中。

请记住编辑微处理机的属性并指定目标代码文件名（通常是一 HEX，S19 或带调试信息的数据文件）。ISIS 不能自动加载目标文件，因为原理图中有可能有多于一个微处理器而导致 ISIS 无法判断某一目标文件的所属取向。

在源代码上工作

1. 编辑源代码程序：

使用组合键 ALT+S

在 source 菜单中选择相应源文件的序号

2. 返回 ISIS，编译目标代码并运行仿真：

在源文件窗口中按组合键 Alt+Table 返回 ISIS 编辑环境。

按 F12 执行程序，或按组合键 Ctrl+F12 开始调试。

任何一种方法，ISIS 会指示文字编辑器保存其文件，检查源代码和目标代码文件的采样时间并唤醒适当的代码生成工具编译目标代码。

3. 重新编译目标代码：

选择 Source 菜单 Build All 命令重新编译全部目标代码

ISIS 将运行需要的代码生成工具来编译目标代码，命令行输出将在窗口显示。这样可以方便地检查编译中的警告和错误。

安装第三方目标代码生成工具

许多共享汇编软件和编译器可以从系统 CD 安装到 Proteus Tools 目录下，并且会被自动作为 Proteus 的代码生成工具。然而，如果想使用其他生成工具，则需要使用 Source 菜单的 Define Code Generation Tools 命令。

注册新的代码生成工具

在 Source 菜单中选择 Define Code Generation Tools 命令

点击 NEW 选项，利用文件选择器为新的代码生成工具导入可以执行文件。用户也可以注册批处理文件作为代码生成工具。

为源文件和目标代码文件输入扩展名。扩展名决定文件的类型，因为对于特定的源文件，ISIS 将会通过文件类型确定是否执行这一工具。如果选择了 Always Build，则该工具一直处于激活状态，可以省去目标代码的扩展名。

在 Command line 指定路径。%1 代表源代码文件，%2 代表目标代码文件。也可以使用 %\$ 代表 Proteus 目录路径，%~ 代表 DSN 文件放置目录。

如果想要使用 Proteus VSM 源代码调试功能，用户需要一个汇编器或编译器的 Debug Data Extractor (DDX)。调试数据提取程序是一个小的命令程序，它通过由汇编程序或编译器产生的列表文件提取目标代码/源代码行的交叉参考信息。

如果用户有一个 DDX 程序，为由代码生成工具生成的列表文件或符号调试文件键入扩展名，然后单击 Browse 选择 DDX 程序的路径和文件名。如果用户的编译器可以输出 COF, ELF/DWARF 等格式文件，则没有必要使用 DDX。

使用 MAKE 程序

在有些情况下，由 ISIS 执行的简单的编译规则不能满足用户的要求——特别是设计项目中包含多个源代码文件和目标代码文件。在这种情况下，用户需要使用外部 MAKE 程序，可按以下步骤建立项目：

安装 MAKE 程序（通常提供汇编程序/编译器环境）作为代码生成工具。设置源文件名的扩展名为 MAK，将 Always Build 选项选中。对于典型的 MAKE 程序命令行设置为 -p%1。

使用 Source—Add/Remove Source Files 命令添加 make 文件（例如。MYPROJECET.MAK）到设计项目中。

添加源代码文件，但选择代码生成工具为 <NONE>

每次项目的编译，ISIS 将会运行外部 MAKE 程序，并将项目的 make 文件作为参数。该参数将确定哪一代码生成工具将被运行。一个好的 MAKE 程序将会提供很大的灵活性。

使用第三方源代码编辑器

Proteus VSM 提供了一个简明的源代码文本编辑器 SRCEDIT。使用该编辑器可以编辑源代码文件。SRCEDIT 本质上是 NOTEPAD 的改进版本。它可以打开多个源文件，并且响应 DDE 的要求，保存任一修改后的缓存。

如果用户有更加先进的文本编辑器，例如 UltraEdit，便可以在 ISIS 中使用这一编辑器。注意 IDE 类型的环境有可能不响应 DDE 命令，因此以这种方式整合并不是那么合适。

建立第三方源代码编辑器

选择 Source—Setup External Text Editor 菜单项

点击 Browse 按钮，使用文件选择器定位文本编辑器的可执行文件。

ISIS 利用 DDE 协议指示文本编辑器打开并保存文件。参考文本编辑器的说明文档向供应商索取命令语法的详细资料。如果服务名称不确定的，不妨以诸如 ULTRAEDIT 的产品名一试。

使用第三方 IDE

大部分专业的编译器和汇编器都有自己的集成开发环境或称 IDE。例如，IAR's Embedded Workbench, Keil's uVision, Microchip's MP-LAB and Atmel's AVR studio。如果用户使用其中一个 IDE 开发程序代码，可以很容易地在 IDE 中进行编辑，生成可执行文件（HEX, ELF/DWARF, COF 文件）后切换到 Proteus VSM，然后进行仿真。

PROTEUS VSM 提供两总方式支持外部 IDE 运行：

- 将 Proteus 作为外部调试器

当 Proteus 作为外部调试器时，Proteus 中可用的下载器支持由编译器生成的带调试信息的目标文件格式。下载器提取的不仅是高级语言程序行的地址而且在允许的条件下是程序变量的

位置。

通常的仿真格式是 COD(在 PIC 系列中使用), UBROF (被全部 IAR 编译器使用), ELF/DWARF (普通格式), COFF (普通格式) 和 OMF (在 8051 系列中使用)。我们同样提供下载器支持其它格式, 例如支持由 Crownhills Proton Development Suite 生成的列表文件。

点击仿真进程控制盘的 PLAY 按钮开始实时仿真或者按 STEP 按钮运行到第一行命令行。在后一种情况, 代码窗口将会出现, 用户可以开始单步运行代码。

- 将 Proteus 作为虚拟 In-circuit 仿真器

到目前为止, 虚拟的 ICE 只支持 Keil、IAR、MPLAB。虚拟 ICE 是个发展很快的领域, 用户应该经常访问公司网站专题“Supported Compilers”以获得最新的信息和说明资料。安装 Proteus 实现与这些工具一起运行的说明同样可以在这网站中找到。

调试工具

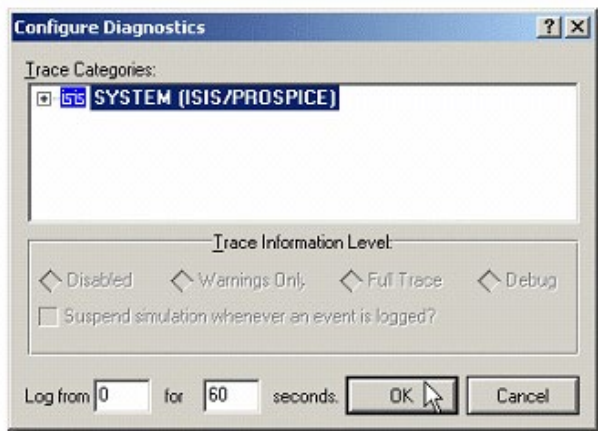
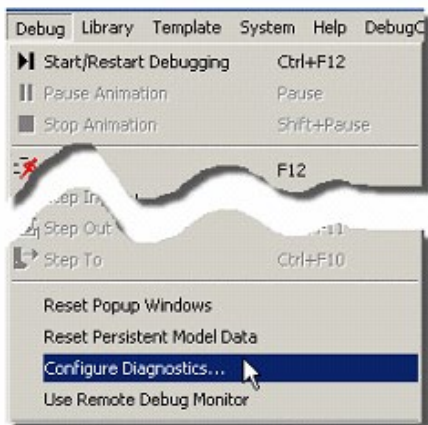
Proteus VSM 包含了扩展的调试工具和跟踪模式。它们在查找错误和验证系统运转方面很有作用。该机制使得全部仿真行为或某一指定的仿真行为在仿真过程中被记录下来并显示在仿真指示器上。

作为一个系统级的仿真器, Proteus VSM 的调试模式不仅仅可以调试微处理器还可以调试合适的外围设备 (LCD 显示, I2C 寄存器, 温度控制仪器等), 这些跟踪模式可以通过调试对话框使能。

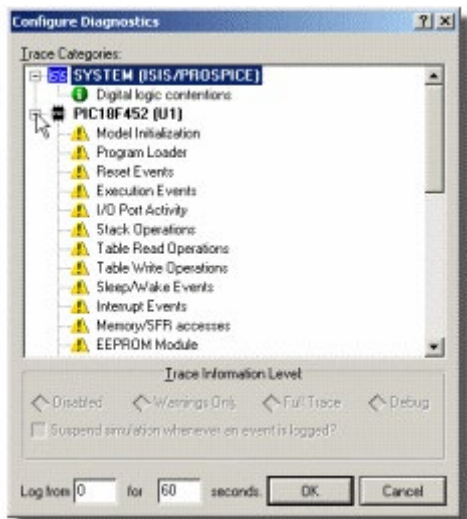
设置调试参数

当调试功能使能时设置仿真

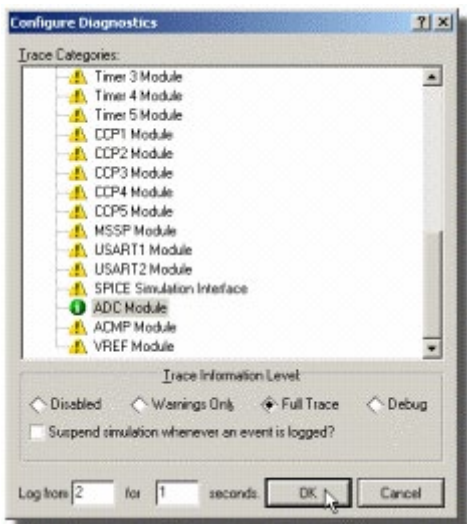
从 Debug 菜单中运行 Configure Diagnostics 对话框



在对话框里展开选项, 从中找到欲使能的调试项。



点击左键选定感兴趣的项目，然后在 Trace Information Level 下选中 Full Trace。用户可以设置'Arm'的时间和运行时间以控制调试功能激活的时间间隔。



重复选择感兴趣的其它选项，然后退出对话框

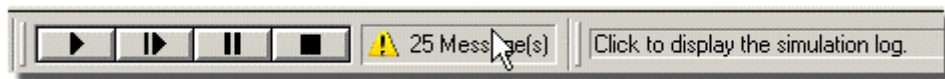
仿真时调试功能将在 Arm 时刻被激活，运行指定的周期，所有调试结果将显示在仿真指示器上。

注意：使能了的跟踪调试增加了仿真的负荷。但是，因为它们是一般用来确定模棱两可的问题，达到仿真的目的，因此仿真非实时的情况不再是问题。

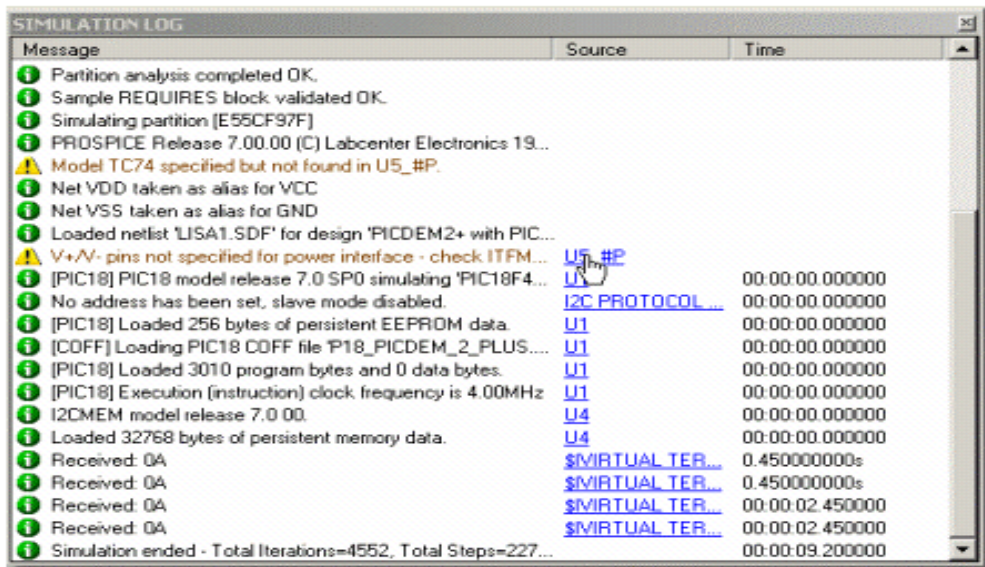
仿真指示器

仿真指示器存放了在仿真期间产生的全部错误、警告和调试信息。该指示器在 ISIS 中的位置处于仿真进程控制盘附近的状况栏。指示器的状态显示栏在仿真过程中不断更新，不仅

显示记录信息（包括错误、警告和调试信息）而且显示了此类信息的种类。用户可以在仿真过程的任何时刻或者在完成仿真后左键点击状态栏指示器所在处调出仿真指示器（可以显示全部信息）。



在由一个物理元器件产生的全部信息（信息量非常丰富，包含除了系统信息外的其它信息）的右边有一个与这些信息相关的栏目。这项功能可以显示产生信息的原理图器件。在 Source 列中单击左键将最小化仿真指示器并且可以放大/缩小、标记相应的元器件。

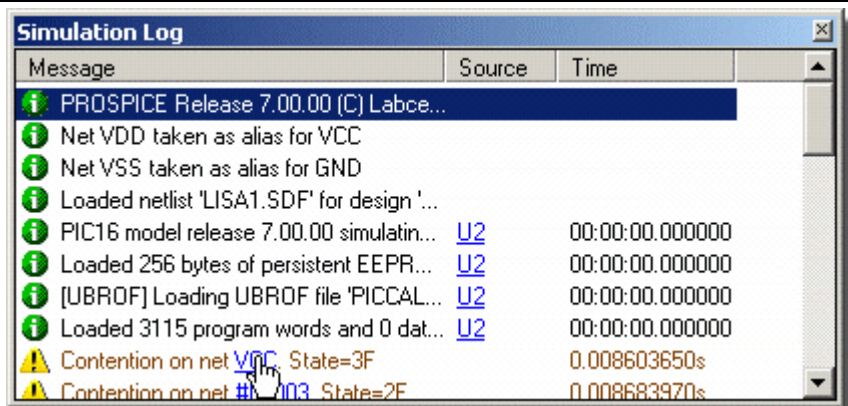


该功能特别有用，尤其是在复杂的设计中。利用它可以检查一器件周围的硬件设计以便解决问题。

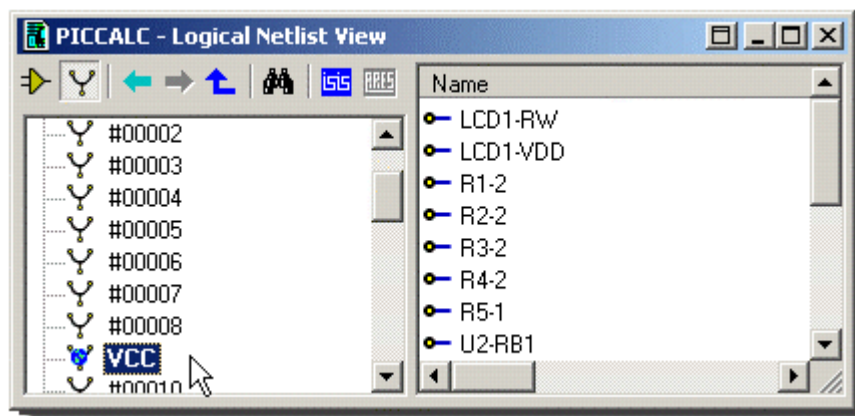
关于仿真错误信息方面最让人感到挫折的事情之一就是—一些问题（例如网点分歧，SPICE 单复数等）与网点相关而不是与某一具体的元器件相关，因此在原理图中将冲突电路隔离出来是非常困难。仿真指示器包含了在信息上的超链接，点击链接允许客户主导整个设计。这样就简化了仿真的任务。

从仿真指示器中操作一个网点

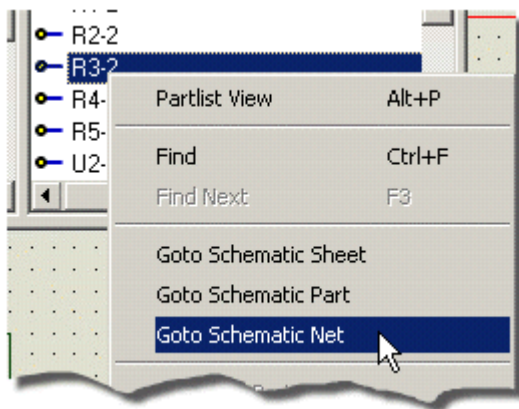
在仿真指示器中点击感兴趣的信息所包含的“NET” 链接。



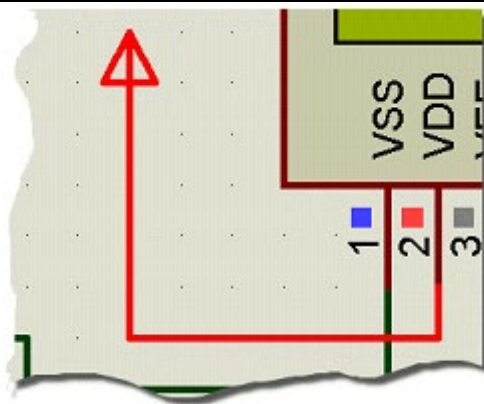
点击“NET”链接后，仿真指示器将会最小化回到状态栏并调出 ISIS 中 Design Explorer。Design Explorer 在窗口的左边显示一系列网点同时在窗口的右边显示了与冲突网点相关的连接点（引脚）。



在其中之一引脚单击右键后从右键菜单中选择 Goto Schematic Net.

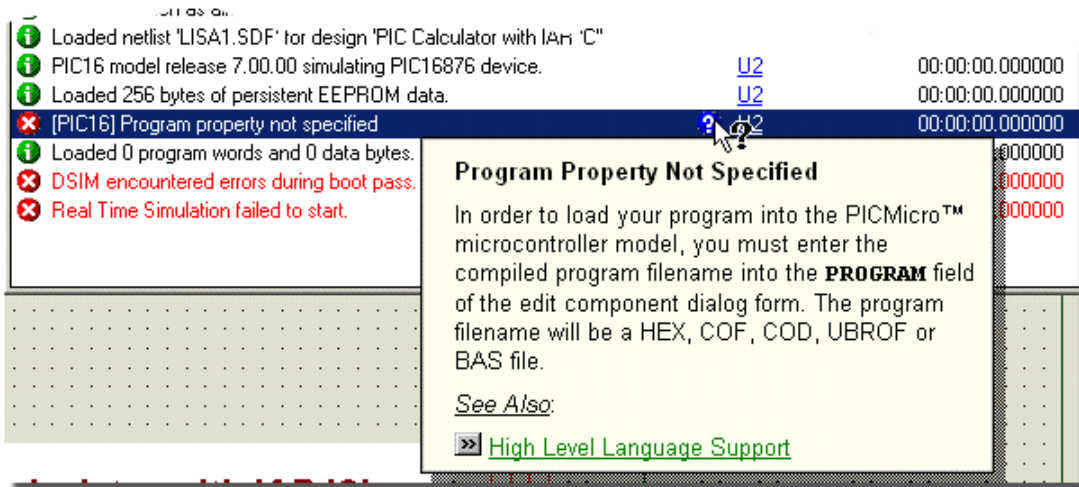


出错的网点将在原理图中以高亮方式显示

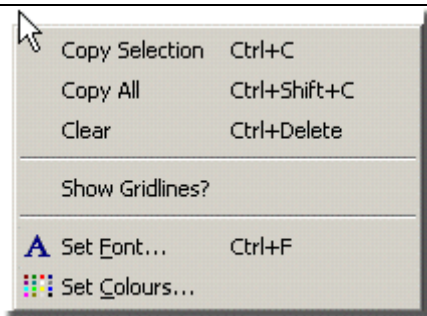


注意：该检测方法并不是绝对可行。当原理图部件通过等效电路（用户利用 ISIS 已有器件构成的与某一元器件等效的电路）进行仿真并且出错的网点在该等效电路里时，这种仿真方法就不可行了。然而，虽然存在着一些不能检测的情况，但是上述的技术在大多数正常条件下是可以正常起作用的并且是一个强大的分析工具。

一般的错误信息或者设计缺陷的错误信息包含了帮助文件。这些帮助文件详细介绍了解决问题的方法。这就不仅清晰显示了发生错误的地方而且提供了解决问题的相关建议。用户可以为某一错误信息调出相应的帮助文件。方法是在信息的右边左键点击“help”按钮。如下图所示：



在仿真指示器点击右键，产生带几个选项的菜单栏。利用这些菜单栏可以实现拷贝已有数据到粘贴板或设置窗口的颜色/网格、外观。



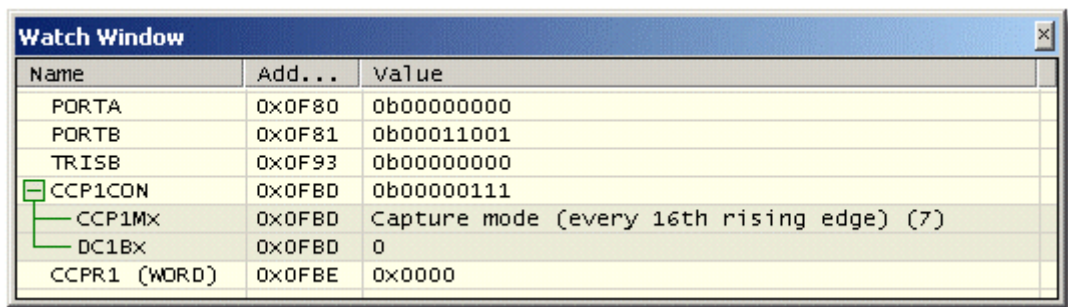
5.5 观测窗口

给观测窗口增加项目

微处理器模型包含的变量，存储器和寄存器窗口只能在仿真暂停时显示，但是观测窗口则可以实时更新显示值。它同时可以给独立存储单元指定名称，这比从存储器窗口查找要容易得多。

在观测窗口中添加项目的步骤如下：

1. 按组合键 CTRL+F12 开始仿真或系统处于仿真状态时，按 pause 按钮，可暂停仿真。
 2. 单击 Debug 菜单中的窗口序号，显示包含被观测的项目的存储器窗口和观测窗口。
 3. 使用鼠标左键定义存储单元的位置。所选定单元以反色显示。
 4. 从存储器窗口中拖动选择的项目到观测窗口。
 5. 用户同样可以使用鼠标右键菜单的“Add Item by Name ”或“ Add Item by Address”。
- 对于微处理器的 SFR 来说，观测窗口特别有用。因为该窗口可以监测个别的位变量（PIC、AVR 和 ARM 模型）。



（观测窗口监测 PIC18 的寄存器 CCP1CON）

在观测窗口中修改项目：

1. 已经将项目添加到观测窗口中，用户可以选择其中一个项目，然后进行以下编辑。

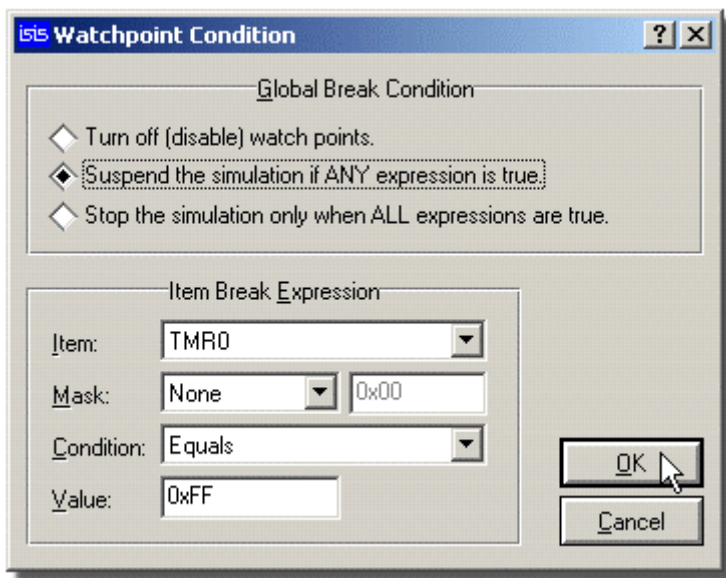
2. 按组合键 CTRL+R 或 F2 键重命名项目
3. 利用右键菜单的选项改变数据量。系统提供了几种数据类型（例如，16 或 32 位字，或字符串）。第二个和后续的字节假定是服从项目地址的，因此，选择多字节或字串显示方式时，只须从存储器窗口拖出第一字节即可。
4. 改变数据的格式，如二进制、八进制、十进制或十六进制。

观测点条件

当项目的值出现特定情形时，观测窗口可以延缓仿真。当特定的项目值发生改变时，观测窗口可延缓仿真。当然，用户也可以定义更加复杂的情形。

设定观测点条件：

1. 按组合键 CTRL+F12 开始仿真，如果系统已经处于仿真状态，按暂停键，可暂停仿真。
2. 单击 Debug 菜单中的窗口序号，显示观测窗口。
3. 增加观测点
4. 选择观测点并右击，选择右键快捷菜单中的 Watchpoint Condition
5. 在 Global Break Condition 选项区进行设置。这一设置决定了当任一项目表达式为真时，或所有项目表达式为真时，系统是否延缓仿真
6. 对 Item Break Expressions 选项区进行设置。一个 Item Break Expressions 选项由 Item、Mask、Condition、Value 构成。



增加观测情形会给仿真器增加一定的负荷，这是因为 ProSPICE 在每次到达观测点时刻时必须重启观测情形。然而，当寻找模棱两可的设计漏洞时，该特性还是非常有用的。一般来说，系统多

消耗一点时间到达测试点并不影响系统的正常运行。

5.6 断点触发

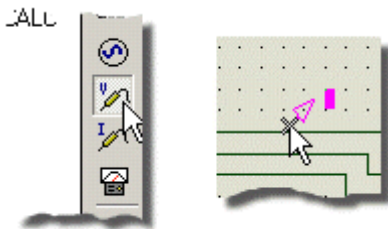
许多元器件都具有当一特定电路情形发生变化时触发仿真延缓的功能。与单步仿真结合使用时，这一功能非常有用，因为电路可以直至某一特定情形出现时，才可以正常仿真，然后使用单步执行，就可以看到电路下一步将会发生什么动作。

建立硬件断点

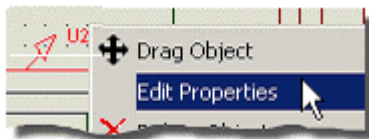
用户利用硬件断点可以在匹配硬件断点的条件下暂停仿真。当调试设计，尤其是异步仿真电路并且需要分析电路所受的影响时，该特性非常有用。

建立硬件断点的步骤如下：

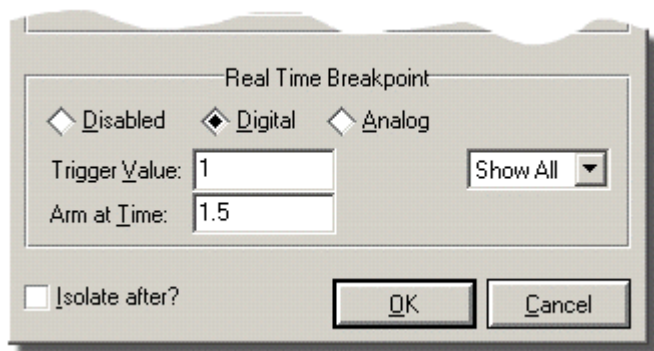
1. 在欲触发断点的导线（总线）上放置电压探针



2. 在探针处点击右键，然后从菜单中选择 Edit Properties



3. 根据探针所在的网点，在对话框的底部选择 Digital 或者 Analog 并指定触发值。对于数字网点和单导线，1 或 0 对应的是逻辑高或逻辑低；对于模拟网点，将会是一具体的值。用户同样可以指定 arm 时间，使得断点在指定的一段时间后有效。



4. 点击 OK 退出对话框，然后按 PLAY（或组合键 CTRL+F12）运行仿真

电压断点触发（RTVBREAK）

电压断点触发元器件有单引脚和双引脚两重形式。当单引脚对地电压值，或两个引脚间的电压值比指定的值大时，触发断点。用户可以将一个元器件连接到任意控制电压源（AVCS）触发断点。

一旦触发电压超过指定值，元器件不会再次触发，直至电压回落到触发门限以下，并再次到达指定值。

电流断点触发（RTIBREAK）

电流断点触发元器件有两个引脚。当流过其电流超出指定值时，触发断点。

一旦触发电流超过指定值，元器件不会再次触发，直至电流回落触发门限以下，并再次到达指定值。

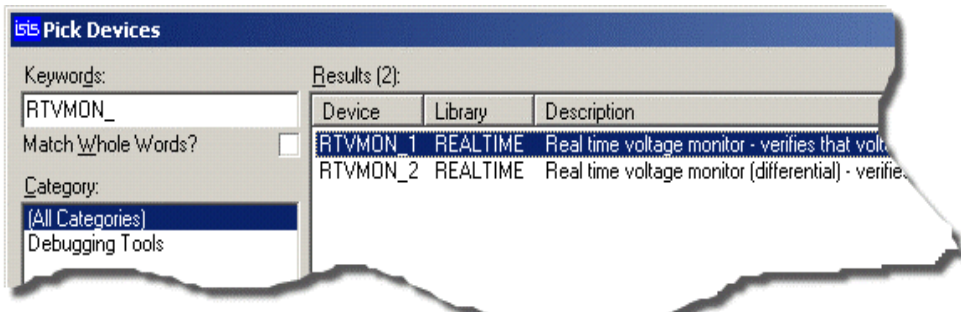
数字断点触发（RTDBREAK）

数字断点触发元器件的引脚有多种形式。用户可以根据需要进行选择。当输入引脚的二进制值等于 Trigger Value 时，触发断点。例如，指定 RTDBREAK_8 的值为 0x80，那么当 D7 为高电平，D0~D6 为低电平时，元器件进行触发。

一旦触发触发发生，元器件不会再次触发直至输入引脚出现不同的值。

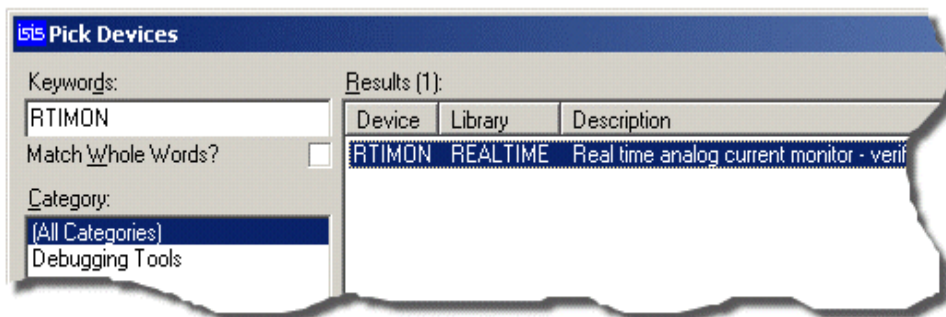
电压监测器（RTVMON）

该元器件有单引脚和双引脚两种形式。当输入电压在指定的最小值或最大值的范围外时，触发断点、警告或错误条件。将这些元器件整合到仿真模型里，则当模型里的电压超出指定的极限时，将会警告最终用户。



电流监测器（RTIMON）

该仪器有两个引脚。当流过仪器的电流在指定的最小值或最大值的范围外时，触发断点、警告或错误条件。将这些元器件整合到仿真模型里，则当模型里的电流超出指定的极限值时，将会警告用户。



Selecting the Current Monitor from the Library Browser.

第六章 图表仿真

6.1 介绍

交互式仿真有很多优势，但在很多场合需要捕捉图表来进行细节分析。基于图表的仿真是可以做很多的图形分析：比如小信号交流分析，噪声分析及扫描参数分析等。

*PROTEUS 用户只有在购买 ASF 模块后才能做基于图表的仿真。

6.2 建立一个基于图表的仿真

概况

基于图表的仿真过程建立有 5 个主要阶段。

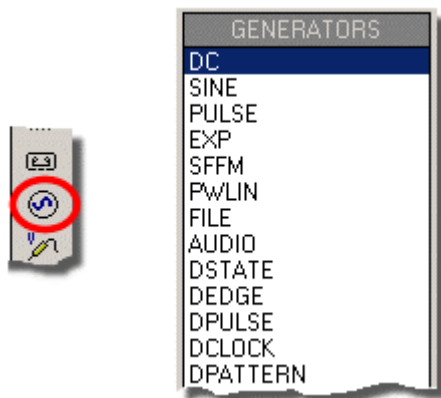
- 绘制仿真原理图。
- 在监测点放置探针。
- 放置需要的仿真分析图表，比如：用频率图表显示频率分析。
- 将信号发生器或检测探针添加到图表当中。
- 设置仿真参数（比如运行时间），进行仿真。

绘制电路

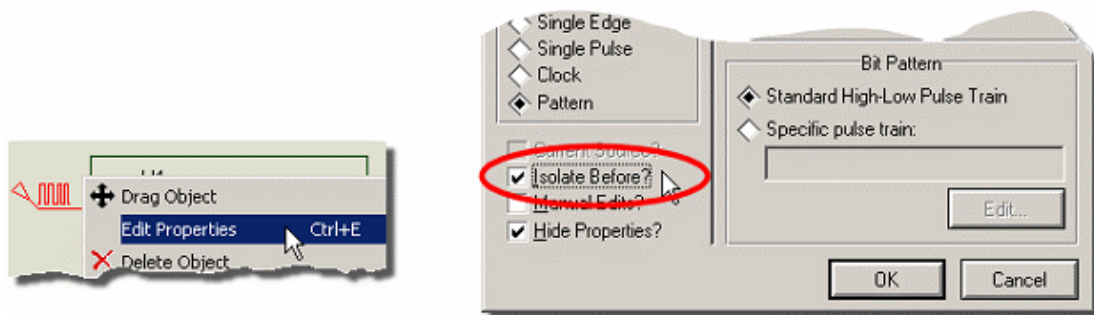
在 ISIS 中输入需要仿真的电路；电路图的绘制技巧在 ISIS 手册中有详细的介绍。

放置探针和发生器

仿真过程的第二阶段是在监测点放置信号发生器及探针。探针、信号发生器和其他元件，终端的放置方法是一样的。选择合适的对象按钮，选择信号发生器、探针类型，将其放置到原理图中需要的位置，可以直接放置到已经存在的连线上，也可以放置好后再连线。

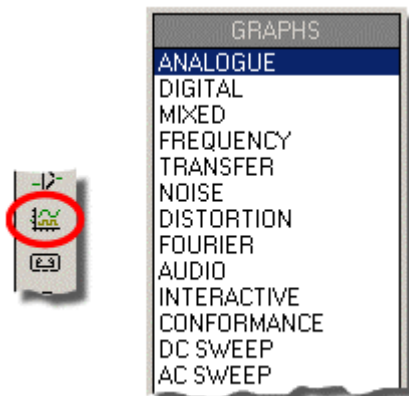


这个阶段中，使用者可以选择将信号发生器隔离出仿真范围。



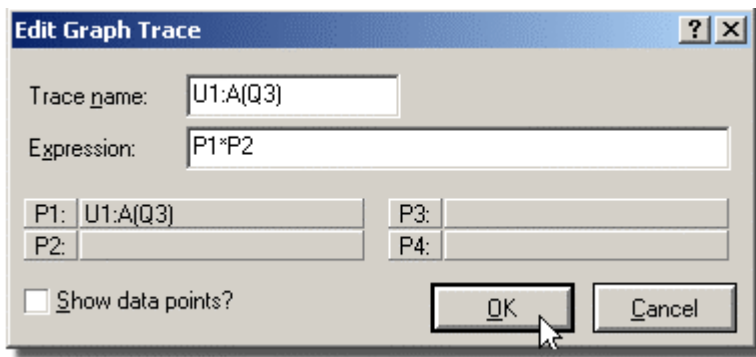
放置图表

仿真的第三个阶段放置分析图表。图表类型包括模拟、数字、转移、频率、扫描分析等等。选择图形按钮，选中需要分析的图表，用拖曳的方法放置在原理图中合适的位置。



在图表中添加轨迹

在原理图放置多个图表后，必须指定每个图表对应的探针/信号发生器。每一个图表也可以显示多条轨迹，这些轨迹数据来源一般是单个的信号发生器或者探针，但是 ISIS 提供一条轨迹显示多个探针，这些探针通过数学表达式的方式混合。举个例子，一个监测点既有电压探针也有电流探针，这个检测点对应的轨迹就会是功率曲线。



曲线显示对象的添加有两种方式：

- 在原理图中选中探针或激励源拖入到图表当中（在同一个图表当中可以添加多个探针或激励源）。
- 在 EDIT GRAPH TRACE 对话框中选中探针，需要多个探针时添加运算表达式。

仿真过程

基于图表的仿真是命令驱动的。这意味着整个过程是通过信号发生器，探针及图表构成的系统，设定测量的参数，得到图形，验证结果。其中，任何仿真参数都是通过 GRAPH 存在的属性定义（比如仿真开始及停止时间等），也可以自己手动添加其他的属性（比如对于一个数字仿真，你可以在仿真器系统当中添加一个‘RANDOMISE TIME DELAYS’属性）。

在仿真开始时系统做了以下工作：

- 产生网络表——网络表提供一个元件列表，引脚之间连接的清单，及元件所使用的仿真模型等。
- 分区仿真——ISIS 对网络表进行分析，将其中的探针分成不同的类，当仿真进行时，结果也保存在不同的分立文件当中。
- 结果处理——最后，ISIS 通过这些分立文件在图表中产生不同的曲线。将图表最大化进行测量分析。

如果在以上中的任何一步有错误，仿真日志会留下详细的记载。有一些错误是致命的，有一些是警告。致命的错误报告会直接弹出仿真日志窗口，不产生曲线；警告不会影响到仿真曲线的产生。

大多数错误产生源于电路图绘制，也有一些是选择元件模型错误。

6.3 图表对象

概况

图表也是设计中的一部分。图表的目的是用来显示仿真结果。做什么样分析就用什么样的图表。仿真图形能否正常显示主要决定于激励源及探针是否正确的添加到图表当中。

当前图表

所有图表命令 GRAPH 菜单下。在这个菜单下包含一个图表清单，当前图表前有一个标志加以区分。

任何在 GRAPH 菜单下的命令在图表命令盘框中都有对应。



放置图表

1. 在模式选择工具栏中选择图表按钮。
2. 从对象选择器中选择合适的图表类型。
3. 在编辑窗口中左键拖曳放置图表。

编辑图表

所有的图表可以移动，重新定义大小，编辑图表属性等。

图表属性是通过 EDIT 按钮或 CTRL+ 'E' 进入对话框中进行编辑。

在图表中添加轨迹

每个图表都可以显示多条轨迹。每一条轨迹数据显示都是和信号发生器及探针相联系。然而，对于模拟及混合图表类型，一条轨迹最多可以表示 1~4 个探针，这些探针通过数学表达式合成。

每一条轨迹都有一个指定作用的标号，这个标号在 Y 轴旁边。有些图表只有一个 Y 轴，所以没有必要指定某条轨迹对应特定的 Y 轴。在默认设置中，轨迹名是和探针名是一致的（多个探针，取第一个探针名）-这些可以在轨迹编辑中修改。

轨迹可以通过三种方式定义：

- 在图表中放置单个激励源或者探针。
- 选取一部分探针，使用 ADD TRACE 命令快速添加。
- 使用 ADD TRACE COMMAND 对话框添加。

前两种方式可以快速添加激励源及探针，每一次添加都生成独立的曲线；第三种方式可以对探针、激励源灵活操作，特别是曲线需要几个不同监测量合成的场合。

向图表中拖放探针或激励源：

1. 选中需要添加的激励源或者探针。
2. 按住左键，将探针/激励源移动到图表中，松开即添加完成。

在图表中添加一条新的曲线，曲线代表一个独立的探针/激励源。注意一点，图表中添加新的曲线时，原有曲线可能会有所变化。

有些图表有两条 Y 轴，这需要在图表的左右两边都放置探针/激励源。此外，对于数模混合图表，添加激励源/探针，系统会自动生成各自类型的曲线（一般在混合图表中的第一个探针/激励源通常会产生一个模拟曲线）。

在一个图表中快速添加激励源/探针：

保证你想添加激励源/探针的对象是当前图表。当前图表在 GRAPH 菜单下有选中标志。

选中要添加的探针或激励源。

点击 GRAPH MENU 菜单下的 Add Trace command 命令。如果有探针/激励源选中，系统会弹出“Quick add tagged probes?” 提示。

选择 Yes 按钮将选中的探针或激励源添加到当前图表中。

这种方式添加的每一个探针/激励源都会产生一条独立的曲线，这条曲线显示了所代表的探针/激励源数据。如果存在两个 Y 轴，曲线默认设置是左边的 Y 轴。

添加曲线命令对话框

如果使用者未采用快速添加模式，点击 Add Trace 命令会弹出 Add Trace 设置对话框（不同的图表类型有对话框会有一些小差别）。这个对话框允许您选择新建曲线名字，曲线的类型（模拟、数字等），Y 轴的位置（左或右），多达 4 个的探针，及探针表达式等。

使用 Add Trace command 命令在一个图表中添加一条曲线：

1. 在 GRAPH 菜单下选择 Add Trace 命令，如果原理图中有选中的探针，系统会弹出快速添加的提示，选择 NO，弹出 Add transient Trace 对话框。
2. 通过 TRACE TYPE 按钮选择图表允许范围内的合适曲线类型，添加到图表中。
3. 选中所添加曲线相适合的 Y 轴位置。只有在图表允许两条 Y 轴时有效。
4. 通过 P1 到 P4 的对话框选择探针，这些对话框有显示探针名的下拉菜单。选中探针，探针名会显示在探针选择对话框及名字对话框中。
5. 如果曲线表达式无效，那么只有探针 P1 有效—仿真后，系统会产生一条代表 P1 的曲线。

6. 重复步骤 3、4，添加曲线需要的所有探针或者激励源。
7. 在曲线表达式对话框输入表达式。添加的探针名由 P1、P2、P3、P4 表示，通过表达式完成探针叠加。
8. 如果曲线类型并不支持曲线表达式，表达式对话框无效。
9. 点击 OK 确定，在图表中添加曲线完成。

图表轨迹编辑

图表中一条轨迹可以通过编辑改变轨迹名称及表达式。

选中，编辑及取消选择轨迹曲线：

1. 保证要被编辑的曲线所在图表未被选中。
2. 右键单击曲线名选中曲线，曲线名呈高亮状态。
3. 在曲线名上点击鼠标左键，弹出 EDIT GRAPH TRACE 对话框。
4. 编辑曲线名或表达式。对于曲线不支持的表达式更改无效。
5. 选择 OK 完成改变。
6. 在图表中曲线名外位置点击右键释放曲线的选中。

改变轨迹的顺序或颜色

按住鼠标左键可以拖动标号来改变曲线在图表中的位置：

对于数字信号，移动它们仅仅改变了电平变化的顺序。

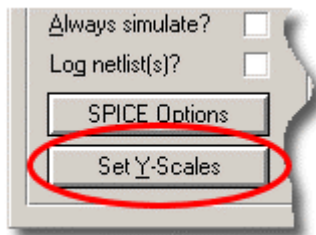
对于模拟信号，可以将曲线从左拖到右，也可以改变曲线颜色分配等。

手工 Y 轴设置

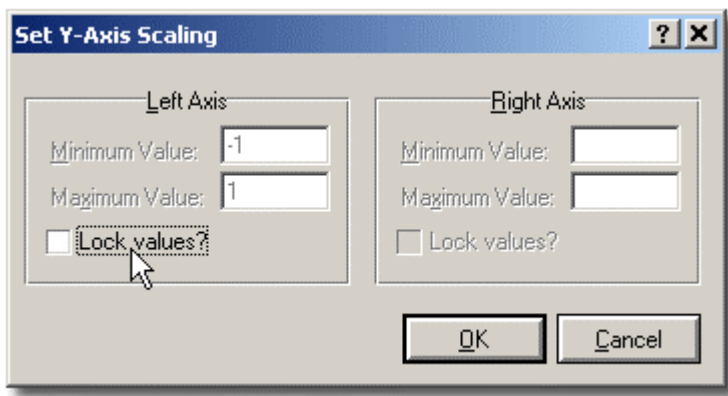
一个图表中 Y 轴的默认设置是提供曲线最大可视的 Y 值。在一些场合，当你需要在原理图中比较多个图表时，这种默认设置就需改变。ISIS 允许你设置并锁定左右 Y 轴的最大坐标值。

手动改变 Y 轴范围：

1. 放置图表，添加曲线，仿真图表。
2. 选中图表，进入 编辑窗口，在右下方即为 Y 轴范围更改选项。



3. 点击选项进入 Y 轴设置对话框



4. 选中 LOCK VALUES 复选框，输入合适的最大值及最小值。

注意：当这些选项呈灰色时表示它们在这个图表中是不可用的。比如有些图表只有左 Y 轴，这样右边 Y 轴不可用的。

5. 关闭对话框重新仿真。

6.4 仿真过程

命令驱动仿真

在设计 PROTEUS 时，一个主要的目标是将软件设计得更为人性化。以前的一些仿真平台都是按照标准输入一些数据，得到仿真结果。

这方面来说，PROTUES 有其独到之处，绘制原理图，放置图表以后，每一次更新图表只需按空格键重新仿真，这种方式称为命令驱动仿真。ISIS 中也可以同时放置多个图表，进行多个实验。

对于给定图表，重要的一步是对放置合适的探针。ISIS 要利用这些信息来计算确定设计中的哪一部分需要仿真，无需手动分割，这对以后的 PCB 整板设计也是非常有益的。

执行仿真

一旦图表放置好，探针及激励源指定好以后。在图表中使用  命令或直接通过空格键执行仿真。ISIS 会自动分析设计，确定哪一部分需要仿真，更新图表，显示新的数据。

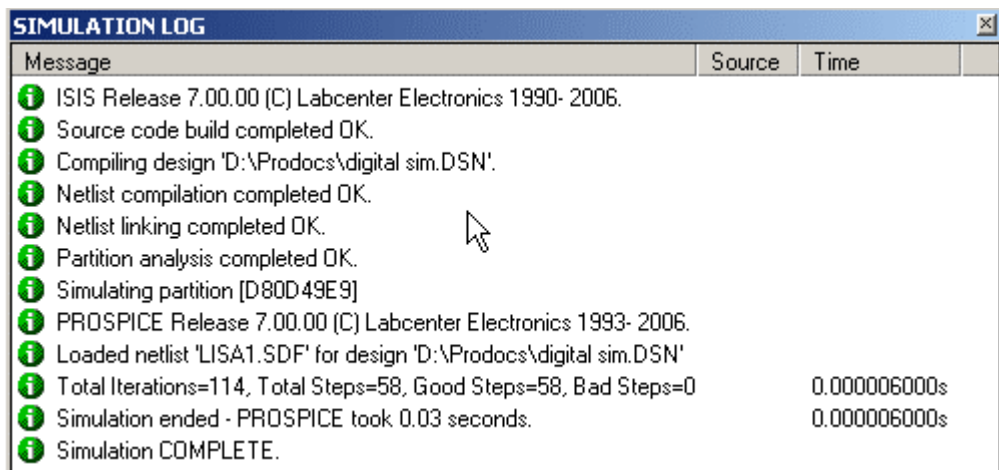
通过这个仿真过程，产生一个仿真日志。仿真正确或只有警告时，日志不会弹出，只有存在致命错误时，仿真日志会弹出，图表也不进行更新。

更新图表观测仿真日志：

保证要更新的图表是当前图表，将图表最大化，选择 **SIMULATE** 命令

仿真结束时，如果有错误发生，会有提示显示这部分结果，选择 **YES**，仿真数据，如果选择 **NO**，系统会弹出仿真日志。

没有错误发生或者有错误发生选择 **YES**，仿真日志可以通过 **VIEW LOG** 菜单下调出。



仿真开始后发生的事件

当使用者刷新图表，使用 **GRAPH** 菜单下的 **SIMULATE** 命令或者 **SPACE** 键。在这个过程中包含了以下内容：

- 网络表编辑——网络表提供一个元件列表，引脚之间连接的清单，及元件所使用的仿真模型等。网络表在进行引脚连线时非常有用。
- 网络表链接——有一些元器件是通过子网络表或模型文件建模，这些模型文件的保存路径可以通过 **SET PATHS COMMAND** 对话框中 **MODULE PATH** 设置。**PROTUES** 提供很多模型文件，使用者也可以创建自己的模型文件。一个模型文件看成一个分立的子电路设计，它的父电路就是那些需要建模的元器件。
- 每一次利用这种方式建模时候，在模型文件替换网络表中原始器件时，模型和原来器件的输入输出连接管脚应保持一致。
- 在网络表链接过程的最后，网络表中剩下的每一个元器件都有一个 **PRIMITIVE** 属性，这种模式可以直接由 **PROSPICE** 仿真。
- 分区仿真——**ISIS** 观测那些放置探针的点，曲线产生就由信号注入点和这些地方决定。一个信号激励视为一个输入，仿真过程中每一点析结果都会保存于一个独立的文件当中。
- 做更深入的分析，需要注意一些仿真前提：比如：一个分区电路 **A** 有一个输出到分区电路 **B** 的输入，这时，**A** 必须先进行仿真。
- 仿真器调用——**ISIS** 在对每一个分区电路进行仿真计算时，都需调用 **PROSPICE**。
- 每一次仿真器调用导致一个新的分区文件，这个过程信息同样添加到仿真日志当中，仿真日志中保留仿真过程中的每个动作。

- 结果处理——最后，ISIS 通过分区文件在图表当中建立不同的曲线，图表重新刷新，对曲线进行测量分析。

第七章 分析类型

7.1 介绍

Proteus 中有 13 种基于图表的分析方法，每一种都展示了 PROSPICE 支持的各种电路分析的结果：

模拟信号分析	与示波器非常相似，模拟信号分析图用于绘制电压和/或电流随时间变化的图形。另外，包含几个探针值的表达式的图形也可以绘制，例如：电流乘以电压的表达式可以绘制即时功率的图形。此类分析通常称之为瞬态分析。
数字信号分析	与逻辑分析仪非常相似，用于绘制随时间变化的逻辑电平的图形。描记线可以通过单一的数据位或一条总线的二进制值来表示。 数字图表使用事件驱动的仿真方法来计算。
混合模式分析	在同一个图表中包含模拟和数字信号。
频率分析	绘制小信号电压或电流随频率变化的图形，也即是波特图，可以显示幅度和相位的变化。另外，通过使用表达式，可以绘制输入和输出阻抗图。 注意：图中的值是相对于指定输入激励源的增益值。
DC 扫描分析	绘制稳态工作点的电压或电流随扫描变量变化的图形。与模拟分析类似，可以绘制由多个探针值构成的表示式的图形。
AC 扫描分析	绘制一簇对应于扫描变量每一个值的频率图。
转移特性分析	通过扫描一个或两个输入激励源来绘制特性曲线或曲线簇，并绘制稳态的电压和电流值。第一个激励源变量值为 X 轴，第二个激励源变量的每一个变化值将生成一条曲线。
噪声分析	输入或输出噪声电压随频率的变化图。也产生单个噪声贡献的列表。
失真分析	绘制 2 次谐波和 3 次谐波的频谱图。也可以绘制互调失真图。
傅立叶分析	显示瞬态分析的谐波成分。类似于频谱分析仪。
音频分析	执行瞬态分析，然后通过 PC 机的声卡播放结果。也可以把电路的输出生成 Windows 的 WAV 文件保存。
交互式分析	执行交互式仿真，在图表中显示结果。本分析方法可以让你结合交互式仿真和基于图表的仿真的优点。
一致性分析	执行数字仿真，然后把结果与先前保存的一系列结果相比较。特别用于制作软件测试工具，来对基于微控制器的应用进行测试。

此外，所有类型的分析都是从计算工作点开始的，也即是，在时刻 0 的所有节点的初始电压值、支路电流和态变数。关于工作点的帮助信息可以从 ISIS 中得到。

7.2 模拟信号瞬态分析

概述

本类图表显示的图形与你在示波器上看到的一样。X 轴显示时间，Y 轴显示电压值或电流值。我们经常称此类分析为瞬态分析，因为它是发生在时域的。

瞬态分析也许是最经常用到的分析类型。所有你能够从真实示波器中测量到的数据都可以从模拟图表中测量得到。它可以用来检查电路是否正常工作，可以快速测量增益值，可以直观地看到已经失真的信号，可以测量流过电源或元器件的电流值等等。

计算方法

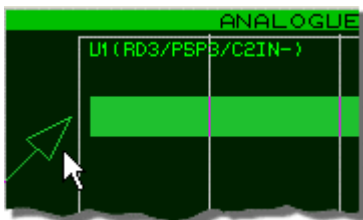
从学术上来说，本类分析与非线性节点分析有关，它使用了所有 SPICE 仿真器使用的计算方法。考虑到具体一个时刻，电路中的所有元器件都可以由电流源和/或电阻代替。这种安排可以通过根据欧姆定律和基尔霍夫定律建立的联立方程来描述，并通过高斯消除法来求解。每一次求得的解，包括电流源和电阻值，根据元件模型内部定义的规则进行调整，这个过程会一直重复，直到得到一个稳定的结果。

对于一个包含前导时间的仿真分析，有两个独立的阶段。先是计算电路的工作点——即在仿真一开始时电路的电压值。然后考虑前导时间对电路的影响，并对每一步进行重新计算（像前面说的迭代收敛）。每一步时间增量对于计算的稳定性是至关重要的，PROSPICE 将在用户定义的门限内自动进行调整。对于快速变化的电路，如高速开关线驱动器相对于平滑变化的电路，需要较小的时间步长，也更费事。使用时间步控制和迭代解都会增加大量的计算，使瞬态分析相当地慢。

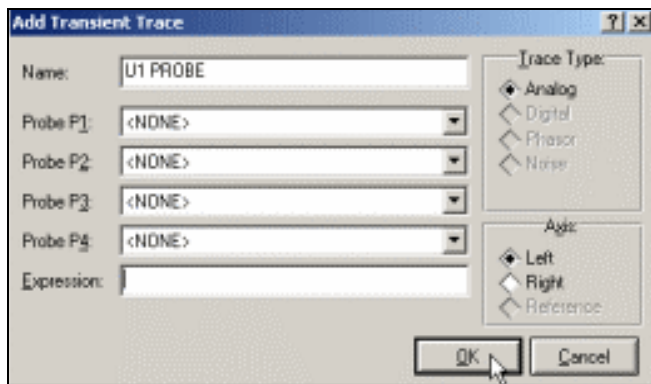
因为算法包含了迭代运算，所以结果有可能不会收敛。这种情况最经常发生在初始时间点，这意味着仿真不能在特定的时刻为工作点建立一个稳定的或唯一的一组值。偶尔也会出现在电路行为变得严重不可测的特定时刻。PROSPICE 使用了一系列的技术去避免这类问题，但不可能完全避免。也即是说，基于 Berkeley 的 SPICE3F5，你可以尽可能地得到你想要的。

使用模拟图表

1. 模拟图表既可以只使用左边的 Y 轴，也可以使用左边和右边的 Y 轴。探针可以通过两种方式放置到特定的轴上：
2. 选取然后拖动探针到图形的对应边上。



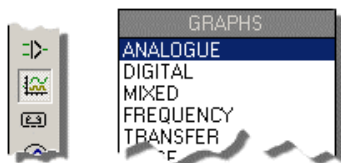
3. 通过“增加瞬态踪迹 (Add Transient Trace)”对话框。



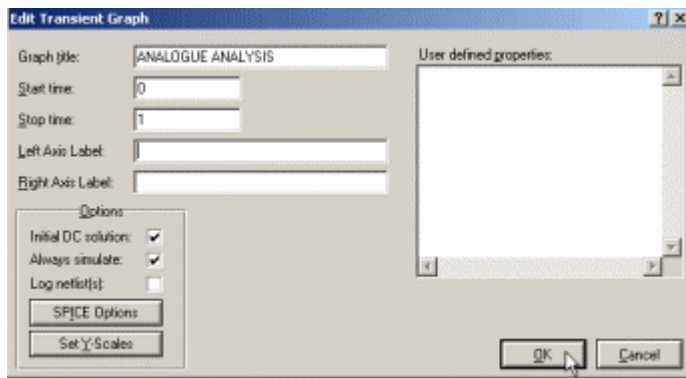
通常使用左右两边的 Y 轴去分离不同单位的踪迹。例如：如果要同时显示电压和电流踪迹，通常可以使用左边的 Y 轴显示电压，而右边的 Y 轴显示电流。也可以把数字探针放到模拟图表中，但混合图表更适合于此类应用。

使用瞬态分析图表分析模拟电路的步骤：

1. 为电路增加需要的激励源，驱动电路的输入端。
2. 在电路中放置探针，可以在电路中的各个节点放置，也可以在输出端放置。
3. 绘制模拟图表。



4. 把探针（如果需要，包括激励源）放入到图表中。
5. 编辑图表（用鼠标指向图表，然后按 CTRL+E 键，设置仿真停止时间、标签和需要的控制属性）。



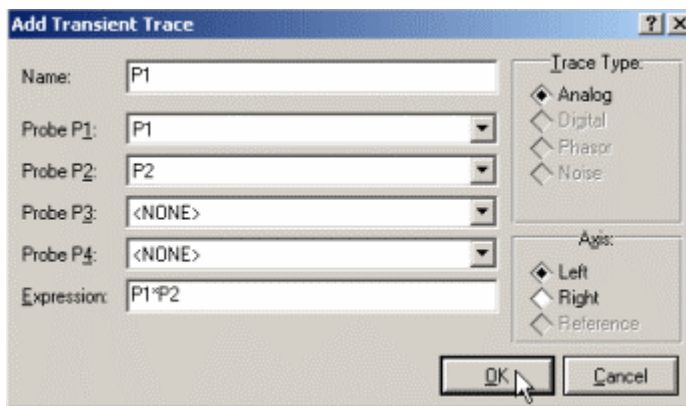
6. 开始仿真，可以从 Graph 菜单选择 Simulate 命令或使用空格键。

定义模拟轨迹表达式

通常，我们只是把电压和电流探针加入到图表中，来观察电路的工作情况。但是不要忘了，我们也可以建立基于探针值的数学表达式来进行模拟瞬态分析。例如：我们在电路的输出端放置了电压和电流探针，把它们相乘，就可以得到这一点的输出功率图。

绘制输出功率图的步骤：

1. 在电路的输出端放置电压和电流探针。
2. 打开“增加瞬态踪迹（Add Transient Trace）”，可以通过快捷键“CTRL+A”打开。
3. P1 赋值为电压探针。
4. P2 赋值为电流探针。
5. 把表达式改为 $P1 * P2$ 。
6. 单击“OK”按钮。
7. 单击空格键开始仿真。



7.3 数字信号瞬态分析

概述

数字图表正如逻辑分析仪一样，X 轴显示时间，Y 轴显示竖直信号堆，可以是单一信号，也可以是总线信号的二进制值。

计算方法

数字瞬态分析使用事件驱动仿真技术进行计算。这与模拟瞬态分析不同之处在于，只有当电路元器件的状态发生改变时才进行计算。另外，只有离散的逻辑电平才会考虑，因此，这使得我们可以在更高级别对元器件的功能进行描述。例如：对于一个计数器，我们是把它作为一个寄存器值来考虑的，在每个时钟周期对它的值加 1，而不是把它作为几百个晶体管来考虑。这使得对同一个电路，事件驱动的仿真比模拟仿真的速度快几个数量级。

引导入口

引导入口的作用是在正常仿真之前定义电路所有网络的初始状态。

通过以下几点定义引导入口：

所有连接到 VCC 和/或 VDD 网络的输入引脚被认为是高电平的。

所有连接到 GND 和/或 VSS 网络的输入引脚被认为是低电平的。

所有连接到与激励源相连的网络的输入引脚的初始态与激励源的 INIT 属性定义的一致。

所有剩下的引脚被初始化为不固定的电平。

所有模型都需要输入引脚电平进行检测，并依此设置输出引脚电平，当每个输出引脚设置完成，该引脚连接的网络的状态将会被重新计算。

当网络改变了状态，连接到该网络的模型将会被要求重新计算输出状态。这个过程会一直持续到出现新的稳定状态。

事件处理循环

紧跟着引导入口，DSIM 开始进行仿真。仿真是通过一个循环来实现的，这个循环重复执行以下两步：

- 当前时间的所有状态改变事件顺序读完并应用到相应的网络上，这个过程将产生一组新的网络状态。
- 当一个网络改变了状态，所有连接到这个网络的模型将重新进行仿真运算。当它们的输出改变了状态，将产生新的事件，并放置到事件队列中。

当然，不同的模型在不同的时刻将产生处理到期的事件。DSIM 内核因此必须为所有在每个循环周期尾产生的新事件进行排序。

另外值得注意的是，我们的原理图支持零延时的模型。在这种情况下，在同一个时间码产生的

事件将会依据产生它的方法分批处理。

停止条件

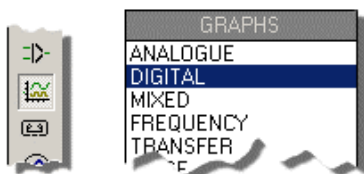
仿真将在遇到以下情形时停止：

- 设定的停止时间到了。
- 事件队列是空的。这意味着电路达到一个恒定的状态。
- 出现零延时的逻辑矛盾，导致当前时间停止前进。
- 系统错误，例如事件队列超出内存范围，一般不会出现这种情况，除非电路设计不稳定，很可能是由于在电路的某些地方产生了高频（例如 100MHz）振荡。

使用数字图表

使用瞬态分析图表分析数字电路的步骤：

1. 为电路增加需要的激励源，驱动电路的输入端。
2. 在电路中放置探针，可以在电路中的各个节点放置，也可以在输出端放置。
3. 绘制模拟图表。



4. 把探针（如果需要，包括激励源）放入到图表中。
5. 编辑图表，设置合适的仿真停止时间，和图表的标题、控制参数等。
6. 从菜单调用“仿真图表”命令，或者使用空格键进行仿真。

注意：

激励源要使用数字激励源，特别要注意的是脉冲（Pulse）激励源是模拟的。

在数字网络中只能使用电压探针，如果使用电流探针，将会迫使 PROSPICE 进行混合信号仿真分析，将会大大降低仿真速度。

数字数据的显示

一般的描绘线

所有 DSIM 仿真器的输出值使用以下六种数字状态：

状态类型	关键字	图形外观
强高	SHI	高电平青色
弱高	WHI	高电平蓝色
浮动	FLT	中间电平白色

冲突	CON	中间电平黄色
弱低	WLO	低电平蓝色
强低	SLO	低电平青色

因此可以通过描绘线的颜色和位置来确定描绘线在具体时刻的状态，或者通过放大图表，把鼠标指向需要查看的位置，从而读出状态。另外，在描绘线标号的右端将显示鼠标所指处的逻辑电平。

总线描绘线

总线描绘线（通过在总线上放置电压探针）在交叉线的中间，显示了总线状态的 16 进制值：



当总线的一位或者多位在中间状态时，总线描绘线也将处于中间状态。另外，当两边的交叉线太接近，没有空间显示十六进制值时，将会省略掉十六进制值，这时可以通过把鼠标指向该位置来显示。

7.4 混合模式瞬态分析

概述

混合图表可以在相同的时间轴（X 轴）上同时显示数字信号描绘线和模拟信号波形。混合模式分析用于同时包含数字和模拟模型的电路分析中，而且这是唯一可用的方式。

计算方法

混合模式瞬态分析既包含了 SPICE3F5 的非线性节点分析方法，也包含了 DSIM 的事件驱动仿真分析方法。详细的实现细节非常复杂，但对基本的算法，可以简单地概括一下：

- 在仿真之前，对每一个网络进行分析，以确定该网络连接的是数字模型还是模拟模型的器件。当某个网络使用模拟电压去驱动数字输入时，PROSPICE 将自动插入 ADC 接口对象，当数字输出驱动模拟器件时，它将自动插入 DAC 接口对象。
- 当几个数字输入由同一个网络驱动时，将建立多个 ADC 接口对象，从而每个输出都可以模拟不同的逻辑转换电平和负载特性。类似地，虽然很少几个数字输出会同时驱动同一个模拟器件，但遇到此类情况时，也会建立多个 DAC 接口对象。
- 建立工作点，将在下面详细描述建立过程。
- 然后进行标准的模拟信号分析，直到当 ADC 输出的电压超过数字信号的转换阈值，从而产生数字事件，此时模拟分析将被挂起，将进行数字仿真。
- 当数字仿真导致 DAC 接口对象输出的状态改变时，在开关时间内强制进行模拟仿真。实际

上，DAC 的输出模拟了转换时间（上升或者下降），并且在这期间将模拟几个时间点。

建立工作点

混合模式仿真工作点的建立特别复杂，因为模拟电路的状态将影响数字器件，反之亦然。基本上，PROSPICE 的处理过程如下：

- 初始条件（IC）值将应用到模拟和数字网络上。其它网络的开始电压假设为 0。
- 像标准的 SPICE 仿真一样，建立节点矩阵并进行运算。
- 在矩阵运算的每个重复过程中，数字电路根据 ADC 的变化不断重复进行计算。这些变化通过几个数字模型进行传递，数字电路不断重复计算直到达到稳定状态。
- DAC 接口对象将根据数字电路状态变化而重新调整输出，如果数字电路没有达到稳定状态，将会被忽略，因为它可能是一个瞬时的引用。
- 循环重复，直到找到一个稳定的状态，或者已经达到重复的限制。

使用混合图表

混合图表的使用与模拟图表的使用完全一样。除了还需要添加数字探针。添加第一个数字探针时，要使用“添加探针对话框”。然后，就可以通过拖拉的方式添加数字探针和模拟探针。数字探针拉到图表的数字部分，模拟探针拉到图表的模拟部分。

7.5 频率分析

概述

通过频率分析，可以检查电路在不同频率上的行为。本类分析只考虑单一频率的行为，而不像频谱分析一起考虑所有的频率。本类分析在效果上就像在输入端接上频率发生器，然后通过 AC 电压表去观察输出结果。可以检查信号幅度和相位。

频率分析产生电路的频率响应或者波特图。对于检查滤波器是不是像设计的那样工作或放大器在要求的频率范围内有没有正常工作时很有用。

频率分析图表也可以用来绘制电路在某个频率上的小信号输入和输出阻抗。

计算方法

频率分析先进行电路工作计算，然后把所有的有源器件用线性模型代替。有源器件的内部电容是在工作点上计算得到的，并假设在工作电压波动的时候不会变化。所有的激励源，除了频率参考激励源（见下面）之外，都使用它们的内阻代替，这将使电源线能有效地连接在一起。

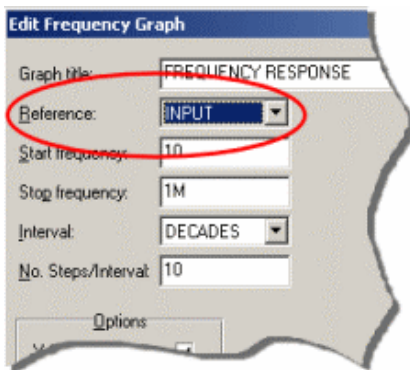
频率分析使用复数执行线性计算。关键频率将从开始频率到最后频率根据事件增量逐渐增强。频率分析的线性特性使得它比瞬态分析快很多，虽然它使用了复数进行计算。

必须注意的是，频率分析把电路假设为线性的，这意味着，在输入端加纯正弦信号，在所有的频率上，输出也必定是纯正弦信号。当然，没有任何实际的器件是纯线性的，但是有很多器件都非常接近线性，而适用于本类分析。也有些电路并非都是线性的，例如带施密特触发输入端的线中继器。对于非线性电路，“频率响应”没有实际的数学意义，所以在任何频率上进行仿真都不会产生有意义的结果。如果对此类电路的频域响应感兴趣，就应该使用傅立叶分析方法。

使用频率图表

为了计算输出的正弦信号的幅度，必须在输入端加入一个参考的正弦信号。**PROSPICE** 将会自动进行这个操作，但必须要知道电路的输入端是哪个。所以必须要对每个频率分析图表设置一个“参考激励源”，指示仿真器在哪里加入参考信号。这里有三种方法去进行设置：

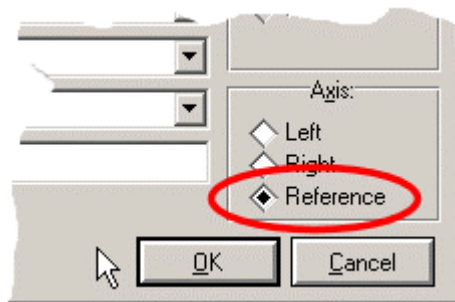
- a. 使用“编辑频率图表”对话框的“参考”字段去选择输入激励源。这个激励源可以是任何一个单脚的激励源，也可以是 ASIMMDLS 库中的 FREQREF 原型。



- b. 拖曳模拟激励源到频率分析图表中。



- c. 使用“Add Trace”对话框添加参考激励源。



FREQREF 仿真原型在定义一个新的模型时，经常用来驱动新模型的内部电路。如在麦克风模型中，当需要考虑模型其它部分的作用时，要包含一个明确指定的激励源作为参考。

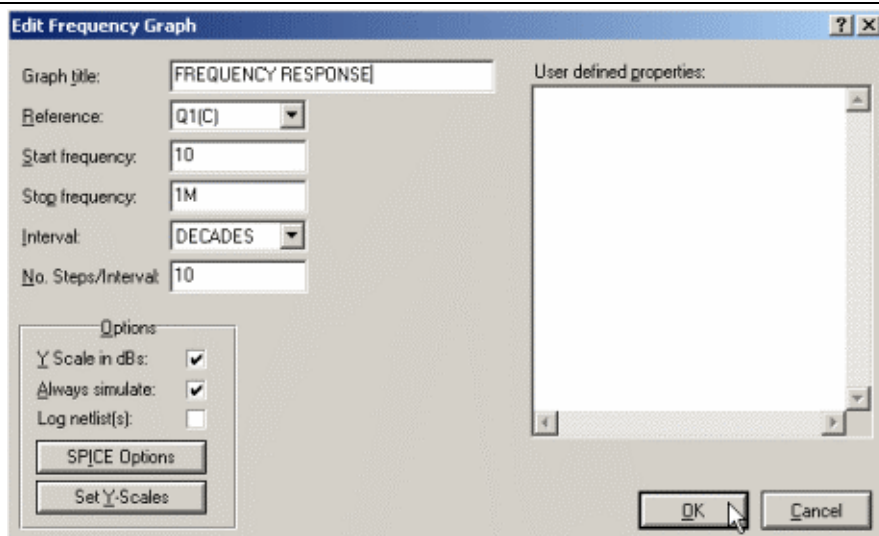
第二和第三种方法比较经常使用。拖拉一个激励源到频率分析图表与到瞬态分析图表有所不同，到瞬态分析图表是添加一个激励源探针，而到频率分析图表是把激励源作为电路的参考点。这个激励源不一定是正弦激励源，直流、脉冲和 PWLIN 激励源也一样能正常工作。如果想把激励源作为一个探针来添加，可以通过“Add Trace”对话框来进行。

参考激励源的幅值总是为 1 伏，相位总是为 0 度。参考激励源的内阻与第一处定义的激励源的一致。它是用于 dB 计算，即 $0\text{dB}=1\text{V}$ 。ISIS 会限制一个非常小的值（ -200dB ）去代替 $\log(0)$ 。

频率分析图表有左右两个 Y 轴。左边的 Y 轴是用于显示被探测信号的幅值，右边的 Y 轴是用于显示相位。如果拖拉探针到图表的左边，它将显示幅值，如果拖拉探针到图表的右边，将显示相位。X 轴用于显示参考激励源的频率。在频率分析图表中总是使用对数尺度，左边的 Y 轴可以显示 dBs 或正常的单位，右边的 Y 轴总是使用度为单位。

使用频率响应分析电路的步骤：

1. 在需要测试的节点放置探针。
2. 绘制频率分析图表。
3. 把探针添加到图表中。左边是幅值，右边是相位。对同一个探针可以同时分析幅值和相位。
4. 编辑图表（用鼠标指向图表，然后按 **CTRL+E** 键）设置仿真开始、停止频率、标签和需要的控制属性。
5. 按下空格键调用 PROSPICE 进行仿真。



例子 ZIN.DSN 和 ZOUT.DSN 显示了怎样使用表达式进行输入和输出阻抗分析。

7.6 DC 扫描分析

概述

通过 DC 扫描分析，可以看到电路参数的改变会怎样影响电路的工作。这是通过 PROSPICE 仿真器的属性表达式评估（Property Expression Evaluation）来实现。扫描图表分析定义了一个变量，可以在用户定义范围内以平均的步距进行扫描。扫描变量可以是电路的任何属性，例如电阻值、晶体管的放大系数，甚至是电路的温度。

DC 扫描曲线显示了探测点随扫描变量变化时稳态的电压或电流。它可以用来绘制电路的 DC 传输特性曲线，只要把激励源的值作为扫描变量，也可以绘制电路在某个工作点下，元器件参数变化对电路影响的曲线。

计算方法

在 DC 扫描中，PROSPICE 会在寻找电路的工作点，然后增加扫描变量进行计算中循环。PROSPICE 会重复计算步与步之间所有的变量值，所以扫描变量会经常被使用，基于扫描变量的变量也可以使用。所有的电路初始化参数都会在每次重新计算工作点时使用。

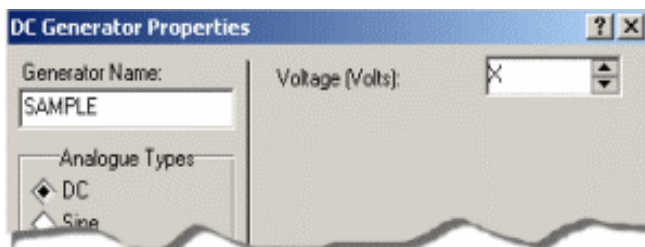
使用 DC 扫描分析图表

与模拟瞬态分析类似，可以使用两个 Y 轴进行分析。X 轴显示的是扫描变量值。可以使用追踪

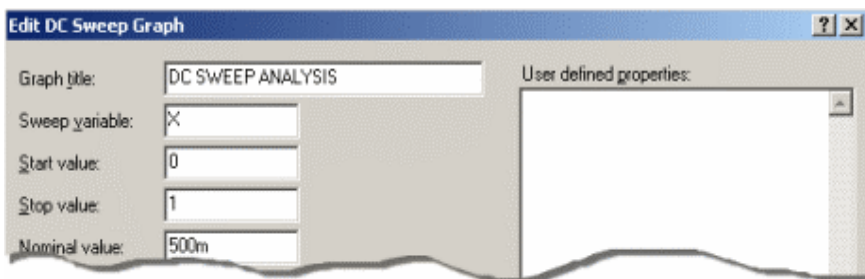
表达式 (Trace expressions)。当选择步数时，记住仿真的时间只是大概与步数成正比。

绘制电路的传输函数的步骤：

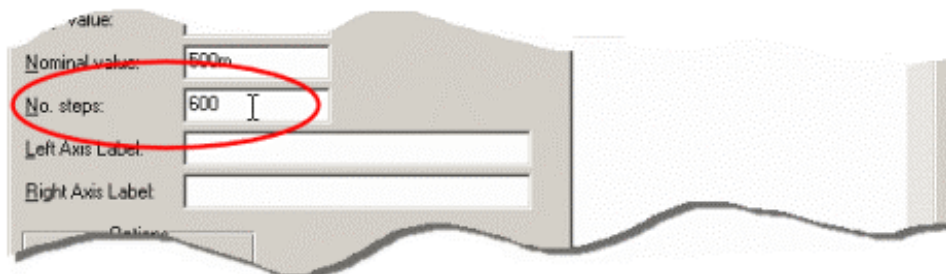
1. 添加 DC 激励源到电路的输入端，设置它的值为变量 X。



2. 放置一个或多个探针到电路的输出端。
3. 绘制 DC 扫描图表，添加探针到图表。
4. 编辑图表 (CTRL+'E')，设置输入扫描变量的开始和停止值，设置扫描变量为 X。



5. 按下空格键调用 PROSPICE。
6. 如果放大图表后，结果看上去是脱节的或有角的，请在“Edit DC Sweep Graph”对话框中增加步数。



绘制电路参数变化的步骤：

1. 绘制原理图，添加探针和激励源到电路中。
2. 编辑需要改变参数的元件，把它的值设置为都变量 X 的表达式，可以设置一个或多个元件。
3. 绘制 DC 扫描图表。添加探针到图表中。
4. 编辑图表，依照上一个例子设置扫描参数。
5. 按下空格键调用 PROSPICE。

提示：

Proteus 中所有的对话框都可以通过“Point and shoot”接口和对话框窗口右上角的问号来得到快速的帮助提示。

7.7 AC 扫描分析

概述

本类分析将为扫描变量的每一个不同值建立一簇频率响应曲线。

AC 扫描图表主要用于测试一个元器件变化对电路频率响应的影响。

计算方法

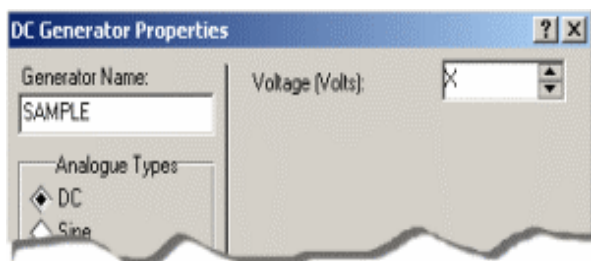
本分析方法与通常计算频率响应完全一样，对扫描变量的每个取值进行频率响应计算。因此也要满足线性电路的约束。

使用 AC 扫描图表

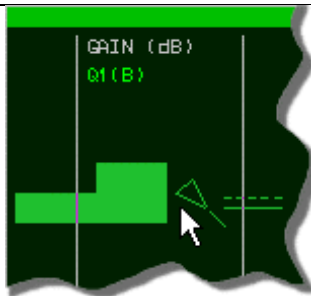
像频率分析一样，左边和右边的 Y 轴分别显示的是增益和相位。当然也要定义一个激励源作为参考点，用于计算增益。

电路参数变化对频率响应的影响：

1. 绘制原理图
2. 编辑要测试的元器件，设置属性值包含扫描变量 X，可以设置一个或多个元器件。



3. 绘制 AC 扫描图表，添加探针到图表中。
4. 添加激励源到电路输入端。
5. 拖拉电路输入端的激励源到图表中，作为参考激励源。



6. 编辑图表 (CTRL+E)，设置扫描参数，设置频率参数到需要测试的频率上。
7. 按下空格键，调用 PROSPICE 进行仿真。

7.8 DC 转移曲线分析

概述

本类图表特别用于测试半导体器件，产生特性曲线簇，当然也偶尔会用于其它用途。每条曲线都绘制了根据指定输入激励源变化时的一组电压或电流工作点。另外一个激励源值的变化用于产生一簇曲线。

计算方法

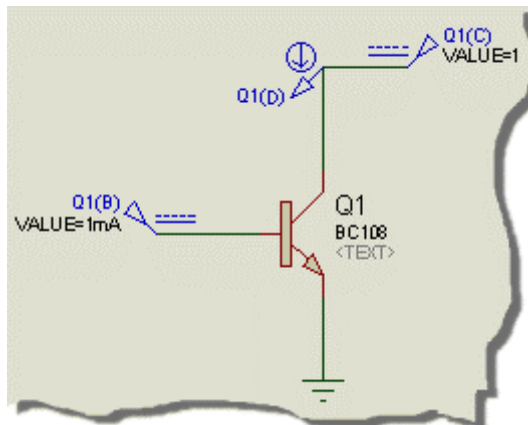
本类分析方法与 DC 扫描分析类似，但这里可以使用两个变量进行扫描。先找到激励源初始值时的工作点，然后第一个变量一步步在指定范围内增加，绘制一条曲线。在第一个变量的每一个扫描过程完成后，第二个扫描变量增加，用于绘制另一条曲线。

使用 AC 扫描图表

与模拟瞬态分析一样，可以使用左右两边的 Y 轴。X 轴显示的是第一个扫描变量，每一条曲线对应第二个变量的扫描值。在这里也可以使用变量表达式进行扫描。

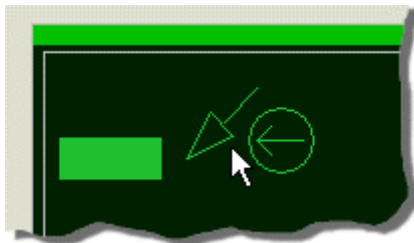
绘制晶体管转移特性曲线的步骤：

1. 绘制类似于下图的电路。



Q1(B)是电流源，Q1(C)是电压源，Q1(D)是需要测试的集电极电流。

2. 绘制转移特性图表，添加探针 Q1(D)到图表中。



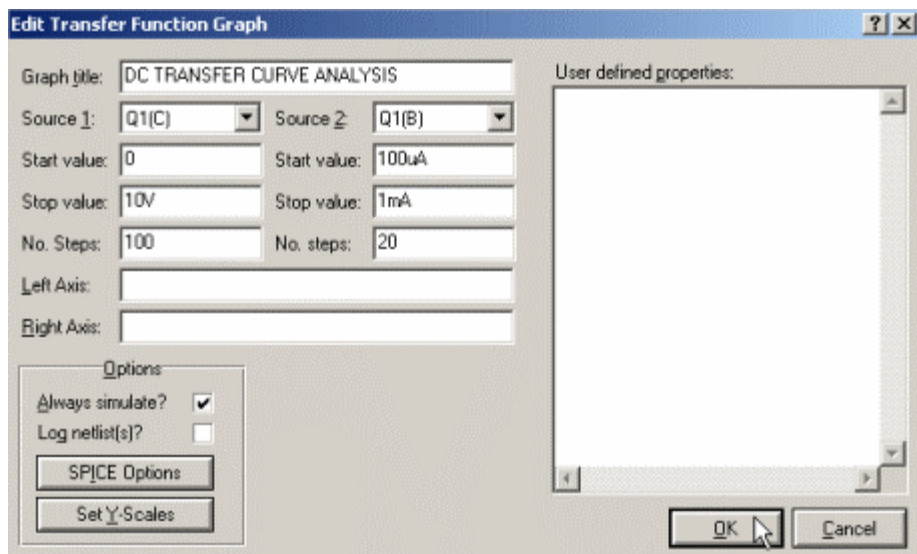
3. 编辑图表 (CTRL+E)。

4. 指定 Source 1 为 Q1(C)，Source 2 为 Q1(B)。

5. 设置电压和电流范围在合理的范围内，如 Q1(C)为 0 到 10V，Q1(B)为 100uA 到 1mA。

6. 调整 Q1(B)的步数，使用中间的间隔比较合理，应该注意，绘制出的曲线将比设定的步数多一条，因为一步有两个离散值。

7. 单击 OK，关闭对话框，然后进行仿真。



7.9 噪声分析

概述

SPICE 仿真器可以模拟电阻和半导体器件中产生的热噪声。电路中在某个频率范围内独立的噪声成分由电压探针收集到一起。噪声电压，归一化为噪声带宽的平方根，可以绘制相对于频率的图谱。噪声图表描绘线的单位总是 $V/\sqrt{Hz}$ 。

将计算两种类型的噪声——输出噪声和等效输入噪声。计算等效输入噪声将对噪声和输入信号或输入噪声进行比较。输入噪声代表了输入端的噪声水平，将在输出端产生噪声，考虑电路在特定频率的增益。

放置探针到左轴将显示输出噪声，而放置探针到右轴将显示等效输入噪声。

噪声分析可能会产生特别小的值（在纳伏在数量级），因此，可以选择用 dB 来显示结果。

警告：

正确噪声模型的电路调用 IC 宏模型是没有任何保证的，因为这些模型可能使用线性控制源仅仅是去模拟器件的基本行为。

计算方法

像通常一样，先计算电路的工作点，然后修改电路模型以正确地合计噪声分量。在噪声分析过程中（除了计算工作点时），除了输入参考激励源，其它所有的激励源都将被忽略。因为没有办法知道，在分析完成后，将查看哪个探针，PROSPICE 仿真器将为电路中的每个电压探针计算热噪声，包括与激励源和录音机有关的探针。不支持噪声电流，因此 PROSPICE 将忽略所有的电流探针。在噪声分析过程中，每个探针都将进行单独的仿真，因此仿真的时间大约与放置的电压探针数量成正比。

使用噪声分析图表

当使用噪声分析图表时，要记住外部电磁场引起的噪声是没有模型的，不能进行仿真。在很多情况下，输入端的噪声是最关键的，对系统的性能有决定性的影响。

如果使用录音机把电路分割成几块，必须很清楚一点，就是 PROSPICE 仿真器只考虑电流的分割。孤立的噪声图虽然也很有用，但必须去掉录音机，才能观察整个电路的等效噪声。

为了确定电路的噪声源，系统自动把独立的噪声电压被记录仿真日志中。轮流考虑每个探针，然后给出每个元件的噪声分量。这些分量通过计算得到，并以平方值的形式显示出来。这个过程在开始和结束频率上进行。如果电路中有大量的探针，将会得到大量数据。

进行噪声分析的步骤：

1. 绘制噪声分析图表，并进行编辑，设置频率范围和输入参考激励源。
2. 添加电压探针到电路的输出端或者其它节点，然后把它们拖进图表中。
3. 按下空格键去调用 PROSPICE 仿真器。
4. 如果要降低噪声电平，检查仿真日志 (CTRL+V)，确定噪声源。

7.10 失真分析

概述

失真分析确定了被测试电路产生的失真谐波的电平。这些失真谐波是单频率信号或互调信号的 2 次和 3 次谐波。

失真是由电路传输函数的非线性引起的，如果一个电路只是由线性器件（电阻、电容、电感、线性控制源），是不会产生任何的失真的。SPICE 失真分析图表可以对二极管、双极型晶体管，JFET 和 MOSFET 的模型的失真情况进行分析。

警告：

正确失真模型的电路调用 IC 宏模型是没有任何保证的，因为这些模型可能使用线性受控源仅仅是去模拟器件的基本行为。

类似的信息可以通过使用傅立叶分析得到，但失真分析也可以显示当基频变化时，失真是怎样变化的。

计算方法

本类分析是基于电路中器件的小信号模型的，因此，第一步要计算电路的工作点。每一个非线性器件的模型随着输入基频得到的分量，对相应的谐波产生复杂的失真。这些谐波在输出信号中的程度将决定图表中的值。这个过程会在指定的频率范围内不断重复。事实上，本类分析数学方法特别复杂，调用了泰勒级数的构造和处理过程，去表达器件的非线性。

注意，因为这里使用了复数，所以分析产生了每个谐波的幅度和相位信息。

对于单频率的谐波失真，图表中将绘制两条曲线，一条是 2 次谐波的，另一条是 3 次谐波。

对于互调失真，使用了两个输入频率，在基频 (F1) 和第二个频率 (F2) 站定一个比例参数。图表中将显示三条曲线 $F1+F2$ ， $F1-F2$ 和 $2F1-F2$ ，显示互调的结果。

选择比例 $F2/F1$ 时要小心，否则会出现奇怪现象。例如：当 $F2/F1$ 为 0.5 时， $F1-F2$ 的值是 $F2$ ，而 $F1-F2$ 的曲线图将变得没有任何意义，因为与第二个基频相同。通常，应该选择一个无理数，如 $F2/F1=49/100$ ，可能会更好点。

注意：

这里有一个约束条件： $F2/F1 < 1$ 。

使用失真分析图表

失真分析图表在左轴显示的是谐波的幅度，右边显示的是相位。测试频率(F1)绘制到 X 轴上。

对于谐波失真分析（单输入频率为 F1），会为每个探测点绘制两条曲线，分别是 2F1 和 3F1。

对于互调失真分析（两个输入频率（F1 和 F2）），会为每个探测点绘制三条曲线，分别是 F1+F2、F1-F2 和 2F1-F2。

在上面两种情形中，可以把鼠标指向相应的曲线，而把曲线分辨出来。当前所指的曲线的信息会显示在状态栏的右边。

7.11 傅立叶分析

概述

傅立叶分析是把时域数据转换为频域数据的过程，转换的结果类似于把示波器用频谱分析仪代替。在分析信号的谐波成分时非常有用，可以用来寻找特殊部分的失真，当然还有其它诸多用途。

分析方法

傅立叶分析先执行瞬态分析，然后对结果数据进行快速傅立叶变换。这个过程调用了在时域进行数据的离散采样的命令，即依据奈奎斯特准则进行。简单来说，就是可以观察到的最高频率是采样频率的一半。然而，可能会出现其它令人误解的结果，因为，容易把采样频率与输入信号的谐波频率混淆，谐波频率高于采样频率的一半。为了使这种可能最小化，可以对输入 FFT 的数据使用各种类型的窗口滤波。

使用傅立叶分析图表

在这个例子里，需要先绘制一个电路图，与用于瞬态分析时的一样，除了把瞬态分析图表用傅立叶分析图表代替。当然，也可以同时使用两种图表。

使用傅立叶分析分析信号的频率成分：

1. 像瞬态分析中一样绘制电路图。
2. 绘制傅立叶分析图表，拖曳探针到图表中。
3. 选择开始和停止时间、频率/分辨率的值以适合所分析的信号。如果可能，选择时间间隔(time interval) 和频率分辨率对应到所分析信号的基频。
4. 按下空格键调用 PROSPICE 进行仿真。

7.12 音频分析

概述

Proteus VSM 集成了一系列的特性让你可以“听”到你的电路的输出结果（当然你必须安装了声卡）。主要的部件是音频分析图表。它与模拟分析基本上是一样的，只不过在模拟分析以后，会从分析结果的时域数据中生成一个 Windows 的 WAV 文件，然后用声卡播放出来。

WAV 文件可以导出给其它应用程序使用。

分析方法

音频分析与模拟分析基本上是一样的，只不过在模拟分析以后，数据以标准的 PC 采样率（11025、22050 或 44100Hz）进行重新采样，然后调用标准的 Windows 函数写成 WAV 格式的数据，最后调用命令去播放 WAV 文件。

使用音频分析图表

在这个例子中，需要先绘制一个电路图，与用于瞬态分析时的一样，除了把瞬态分析图表用音频分析图表代替。当然，也可以同时使用两种图表。

进行音频分析的步骤：

1. 绘制电路图。
2. 绘制音频分析图表，拖拉电路输出端的探针到图表中
3. 选择开始、停止时间和循环时间，以产生合理的波形长度，并且符合真实仿真的最小值。通过分析 1 毫秒的数据，然后循环 1000 次，建立 1 秒时间的音频比完全分析 1 秒的数据要快很多。
4. 选择采样分辨率和速率，以适合信号的特性。建议使用 16 位的分辨率，否则你会产生一个大文件，导致磁盘空间不足。大多数的 PC 机的声卡并不会因为使用 44.1KHz 的采样率而声音得到改善。所以完全没必要使用 44.1KHz 的采样率。
5. 按下空格键调用 PROSPICE 进行仿真。
6. 按下 CTRL-SPACE 可以进行重放，而不进行重新仿真。
7. 可以导出音频数据到 WAV 文件中，如果需要，可以导入到其它设计图中（通过音频激励源）。

7.13 交互式分析

概述

交互式分析图表集合了交互式仿真和图表分析的优点。交互式分析在交互式仿真环境下进行，但它可以把结果记录，并像瞬态分析图表一样显示出来。

这种分析方法在检查某个控制操作时非常有用，可以把它想像成是存储示波器和逻辑分析仪的集合体。

计算方法

计算方法与混合模式瞬态分析的完全一样，只是仿真分析是在互动的模式下进行。因此，电路中开关、键盘和其它可动作器件的操作会影响结果。仿真会依照动画时间步长（Animation Timestep）定义的速度进行，而不是使用可以达到的最大速度。

必须清楚，此类仿真会捕捉到大量的数据。如果处理器使用真实的速度运行，每秒会产生数百万的事件，将这些事件显示在图表中，将占用几兆的存储量，特别是当你探测数据或地址总线的时候，将更加明显。如果 ISIS 载入的数据超过 20 或 30MB 时，可能很快就让系统崩溃。

如果在相对短的时间里，不可能捕捉到需要的数据，最好使用逻辑分析仪进行分析。

像正常的互动仿真一样，不支持电路的多部分仿真，所有的没有设置成播放模式的录音机对象将会自动从电路中移除。

使用交互式分析图表

通常你会使用互动分析图表去测试一个在进行互动仿真时，发现当进行某部分的控制操作时，发生了奇怪的事情。在下面这个例子里，你可以使用虚拟仪器去观察，但是在某些情形下，有必要捕捉结果，用图表显示出来，然后再慢慢研究它。

进行互动分析的步骤：

1. 添加探针到节点。
2. 绘制互动分析图表，把探针拖进图表中。
3. 编辑图表属性，选择合适的开始和停止时间。注意，PROSPICE 为了减少数据量将不会在开始时间之前捕捉数据。
4. 在原理图中设置所有的互动控制到合适的初始状态。
5. 准备进行任何的互动操作，然后按下空格键。你必须在第 4 步设置的时间内操作动态器件。如果很难做到，你既可以增加停止时间，也可以减少每帧的时间步长（Timestep Per Frame）的设置项。

7.14 数字一致性分析

概述

一致性分析把一组数字仿真结果与另外一组进行比较。这是对修改前后的两个电路的工作是否一致的快速检查方法，用来证明修改后的电路没有出现意想不到的副作用。这种分析方法特别适用于基于微控制器的电路，当固件程序改变后，需要对整个电路进行重新测试。

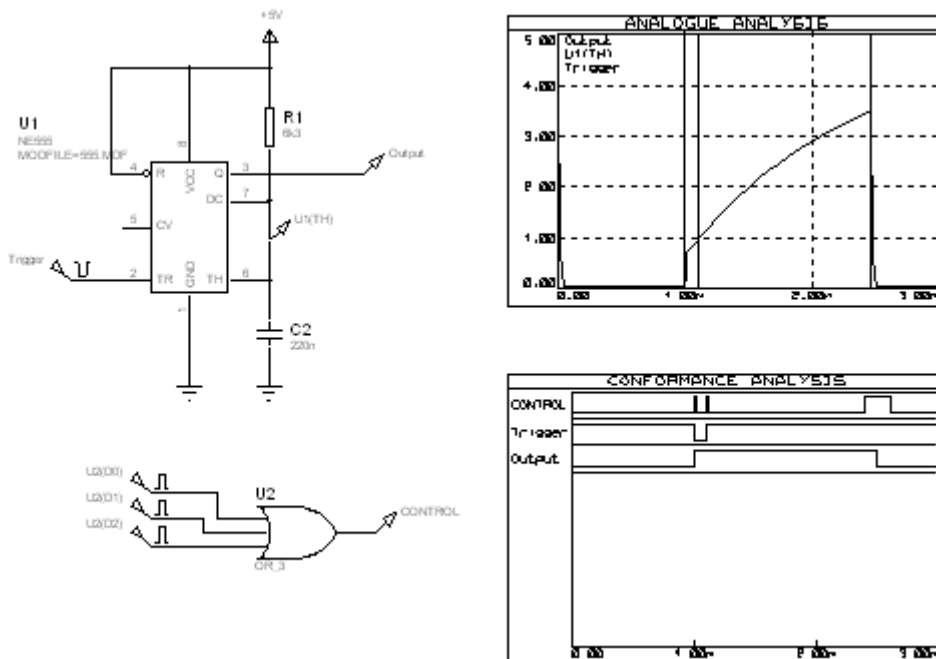
分析方法

除了能把两组结果数据存储到图表中，其它的计算方法与数字瞬态分析一致。这里把这两组数据称为测试数据和参考数据。

一致与不一致是通过在第一条描绘线的每个边沿对测试数据和参考数据进行比较得到的。这里把这条描绘线称为控制线，它以不同的颜色显示在图表中，以区别于其它描绘线。更加重要的是，测试数据和参考数据的边沿不必要一致。这意味着在结果数据中的绝对时间的改变并不是不一致的体现。这特别是在基于微处理器的电路中，任何代码的改变都会影响到事件的绝对时间。在这种情形下，控制线可以由代码来生成，从被测试程序的入口到出口。

使用数字一致性分析图表

通常，一致性分析用于测试策略的一部分，可用于测试仿真模型，但最常用于测试嵌入式系统。在下面这个简单的例子里，将详细描述使用一致性分析图表测试 555 单稳态触发器电路的操作过程。



被测试电路包含了 U1、R1 和 C2，上图显示了经典的 555 单稳态触发器的连接图。电路在 1ms 的时候被数字脉冲激励源触发，输出波形显示在上图的模拟瞬态分析图表中。为了测试电路是否正常工作，先考虑以下几个因素：

- 输出在触发脉冲前是低电平的。
- 输出在触发脉冲跳到低电平后，很快将跳到高电平。
- 输出在触发脉冲回到高电平后，仍然维持在高电平。
- 输出维持在高电平大约 1.5ms。

在经历上述时间后，输出回到低电平。

为了通过一致性分析图表达达到上述结果，需要在触发脉冲信号的每个边沿转换时，和输出信号的每个边沿对输出信号进行采样。可以通过为最后两国采样点选择间隔，而为单稳态触发器的延时时间设置一个容差。

可以通过一系列的数字脉冲激励源，把它们的输出通过或门连接在一起，而达到这个效果。每个脉冲激励源的脉冲宽度决定了每个意料中的事件的时间容差。例如，第三个脉冲激励源设置在 2.4ms 到 2.6ms 之间触发，将给输出脉冲提供 $\pm 100\mu s$ 的时间容差。

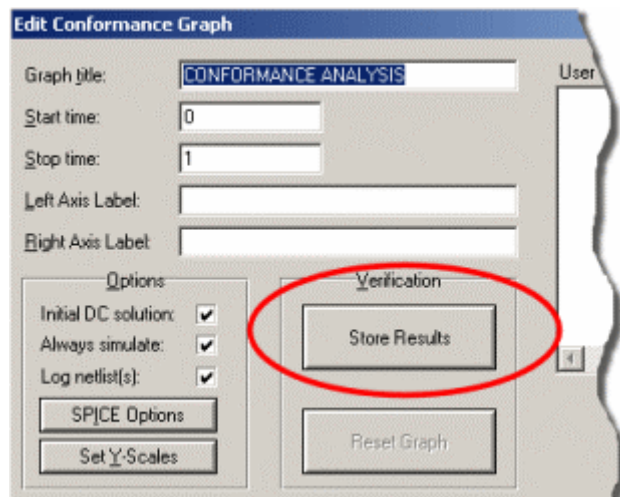
注意，可以使用数字时钟或模式激励源去产生控制信号；使用简单的时钟信号将满足大多数的应用，它会在有规律的时间间隔进行检验。

通常的操作过程总结如下：

进行一致性分析的步骤：

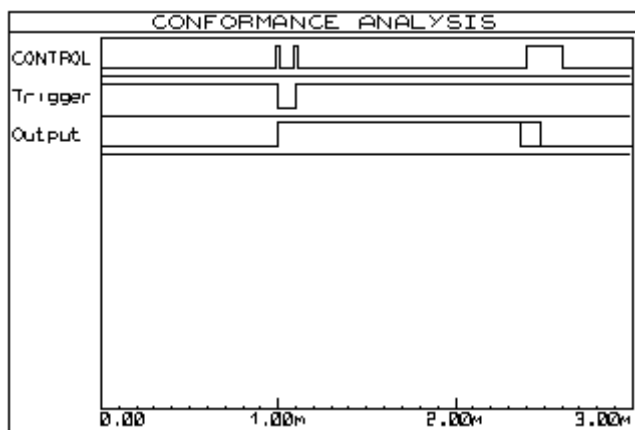
1. 确定想进行测试的部件和时间。
2. 设计一个测试原理图，可以产生想要检验的输出和控制信号。
3. 绘制一致性分析图表，像数字分析图表一样进行设置，并确保控制信号轨迹是在图表的最顶端。

4. 运行仿真，检验结果是不是像意想的那样，控制轨迹的转变时间是不是在想要的时间点上。
5. 编辑一致性分析图表，按下“保存结果”按钮，这将会把当前显示的结果变为参考结果数据。这将以暗淡的颜色显示，当重新进行仿真时，图表将对任何新的结果与它进行比较。



6. 保存设计，它将用来检验新数据。

假设因为元件的缺乏，R1 和 C2 的值必须要改变。如果把 C2 改为 100nf，R1 改为 15k，这个电路还能像以前一样工作吗？实验的结果如下图所示：



显然，答案是否定的，输出脉冲变短了，在仿真日志（simulation log）中可以找到以下信息：

Comparing Results...

Data mismatch at time 2.40m in trace 'Output'.

Data signatures were 'L' and 'H'.

Conformance analysis FAILED.

注意：图表的光标指向了第一个出现差异的时间 2.4ms

事实上，有三种方法可以调用一致性分析：

使用通常的方法重新仿真图表，例如：通过按下空格键或者使用“Graph”菜单中的“Simulate

Graph”命令。

使用“Graph”菜单中的“Conformance Analysis command”命令，这将对设计中的所有一致性图表重新仿真，然后报告任何的错误。

高级用户使用一使用命令行，输入：

ISIS <filename> /V

将像第二种方法一样进行整体一致性分析。如果有任何一个图表仿真失败，ISIS 将显示错误 level 1。这允许通过批处理文件自动进行多个测试。

第八章 激励源和探针

8.1 激励源

概述

激励源是一种用来产生信号的对象。有很多类型的激励源，每一种都产生不同种类的信号。

激励源	描述
DC（直流）	产生恒定的直流电压源。
Sine（正弦）	正弦波信号发生器，可以对幅度、频率和相位进行设置。
Pulse（脉冲）	模拟脉冲信号激励源，可以对幅度、周期和升降时间进行设置。
Exp（指数）	指数脉冲信号激励源，产生的波形与 RC 充放电电路相同。
SFFM（单频率调频）	单频率调频信号激励源，产生的波形与一个正弦信号对另一个正弦信号进行频率调制得到的波形一致。
Pwlin（分段线性）	分段线性激励源产生任意形状的脉冲或其它信号。
File（文件）	跟上面的激励源一样，但波形数据是从 ASCII 文件输入的。
Audio（音频）	使用 Windows 的 WAV 文件作为输入的波形信号。这在与音频分析图表一起使用时，将非常有用，可以听到电路的输出信号。
Dstate（稳态）	输出稳态逻辑电平。
Dedge（单边沿）	单个逻辑电平转换或边沿。
Dpulse（单脉冲）	单个数字时钟脉冲。
Dclock（数字时钟）	数字时钟信号。
Dpattern（图形）	逻辑电平的任意序列。

放置激励源

步骤：

1. 选择“Generator”图标，激励源类型列表将出现在对象选择器中。
2. 选择激励源类型，预览窗口将出现相应激励源的图形。
3. 使用旋转和镜像按钮调整激励源的放置方向。
4. 鼠标放到编辑窗口中，按下左键并拖拉激励源到合适的位置，然后释放鼠标左键。

提示:

可以直接把激励源的连接点放置到连线上，也可以先放在空的地方，以后再连线。

当激励源没有连接到任何网络时，它将得到一个默认的名称（?），以表示它没有进行标注。当它连接到一个网络（把它直接放置到连线上）时，它将以前所连接的网络名进行标注。如果该网络没有标注，它将以元件的参考名称和第一个连接到该网络的引脚名称的组合进行标注。激励源的名称会根据所连接的网络自动进行更新。也可以编辑激励源的属性，给激励源命名，在这种情况下，激励源的名称不会自动更新。

编辑激励源

所有的激励源都可以使用 ISIS 通用的编辑方法进行编辑，最简单的方法是单击右键（选择）然后再单击左键（编辑），或者用鼠标指向激励源然后按 CTRL+E。

激励源编辑对话框提供了一系列的通用属性和各激励源的特有属性。通用属性的解释如下：

属性	描述
Name（名称）	激励源的名称。 如果是手动输入的名称，ISIS 将不会对它进行自动更新。 如果希望激励源名称会自动更新，打开属性对话框，去掉手动输入的名称，单击 OK 确定。
Type（类型）	激励源的类型。 激励源的类型改变，对话框右边的属性选项也跟着改变。
Current Source（电流源）	除了数字激励源，其它激励源都可以作为电压源或电流源使用。勾选此选项将使目标激励源成为电流源。
Isolate Before（隔离之前的网络）	当激励源放置到连线的中间时，本控制选项用于控制是否把连线打断，分成两个网络，即只驱动激励源所指的方向。相反方向上不接激励源。
Manual Edits（手动编辑）	选中此选项后，激励源的属性以文本方式显示，这主要是为了方便软件向前兼容。然而，高级用户也可以使用属性表达式来设置属性，这种方法只有在手动编辑模式下是可行的。

直流激励源

直流激励源用于产生恒定的模拟直流电压或电流。它只有一个属性，用于设置输出值。

正弦激励源

正弦激励源用于产生固定频率的连续正弦波形，它有几个属性：

- 输出值（Output level）指定为峰值振幅（VA）和可选的偏移电压（VO）。幅度也可以有

效值（RMS）或峰峰值的形式表示。

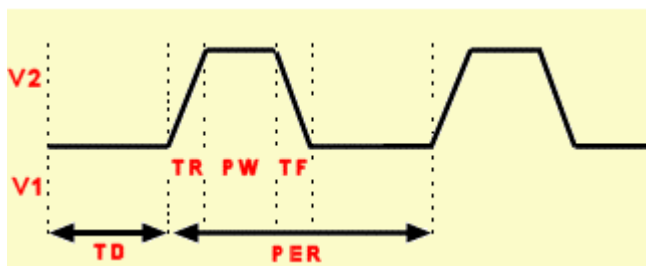
- 频率可以使用 Hz（FREQ），或周期（PER），或总的周期数来表示。
- 可以设定相位偏移，用度表示（PHASE），或者使用时间延迟（TD）。在后一种方式中，到达指定时间后才会输出正弦波（开始振动）。
- 可以通过阻尼因数（THETA）设置开始振动后的波形的指数式衰减。
- 正弦波的输出公式为：

$$V = VO + VAe^{-(t-TD)THETA} \sin(2\pi FREQ(t + TD)) \quad (t \geq TD)$$

$$V = VO \quad (t < TD)$$

脉冲激励源

脉冲激励源用于产生各种在模拟分析中使用到的周期性信号。如矩形波、锯齿波和三角波等，也可以产生单个短的脉冲信号。必须注意，上升时间和下降时间不能为 0，所以不可能产生一个真正的矩形波。因为在 PROSPICE 中，不允许有跳变。下图描述了脉冲激励源的工作情况：

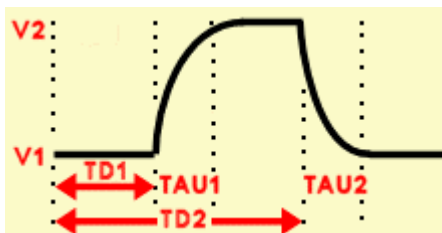


这里：

符号	描述
PER	波形的周期，如果没有指定，将使用 FREQ 的参数。
FREQ	波形的频率，在瞬态分析中默认为一个周期。
V1	输出的低电平值。
V2	输出的高电平值。
PW	每个周期中，输出为 V2 的时间。不包括 TR 和 TF。
TR	上升时间——每个周期中，输出电平从低到高所花的时间。默认为 1ns。
TF	下降时间——每个周期中，输出电平从高到低所花的时间。默认与 TR 相同。
TD	延迟时间。激励源输出电平开始为 V1，然后保持 TD 秒的时间。

指数激励源

指数激励源产生的波形与 RC 电路在充放电情况下产生的波形是一样的。参数描述如下图所示：



符号	描述
V1	输出的低电平值。
V2	输出的高电平值。
TD1	上升曲线的开始时间。
TAU1	上升曲线的时间常数。这个时间大约等于达到 0.63 满电压值时的时间。
TD2	下降曲线的开始时间。注意，TD2 是从 0 开始计算的。
TAU2	下降曲线的时间常数。

数学上，波形由以下三部分定义：

0 到 TD1 $V1$

TD1 到 TD2 $V1 + (V2 - V1)(1 - e^{\frac{-(t-TD1)}{TAU1}})$

TD2 到 TSTOP $V1 + (V2 - V1)(1 - e^{\frac{-(t-TD1)}{TAU1}}) + (V1 - V2)(1 - e^{\frac{-(t-TD2)}{TAU2}})$

单频率调频激励源

单频率调频激励源产生的波形是一个正弦波对另一个正弦波进行频率调制的结果波形。数学上，定义如下：

$$V = VO + VA \sin(2\pi FCt + MDI \sin(2\pi FSt))$$

参数解释如下：

符号	描述
VO	直流偏转电压。
VA	载波幅度。
FC	载波频率。
FS	信号频率。
MDI	调制系数。

分段线性激励源

分段线性激励源用于产生复杂的模拟信号，或者重现测量过的波形。输出波形使用时间或输出

幅度来描述。激励源的输出值在规定时间间隔内是线性的。

分段线性激励源对话框由曲线图表编辑器组成，在上面可以拖拉放置数据点。操作简单描述如下：

- 单击左键放置新的数据点。
- 使用左键拖拉数据点，可以移动数据点。
- 单击右键删除数据点。
- 须符合下面的限制：
 - 在时刻 0 必须有数据点，虽然它的 Y 值可以任意改变。也即是，只能对时刻 0 的数据点进行拖拉，而不能删除。
 - 如果试图创建垂直的边沿，图表编辑器将在两个数据点之间自动插入最小的上升/下降时间值。

如果有表列数据，使用手动编辑模式也许会更加容易。在这种情形下，每个数据点由 $V(t)$ 属性定义。请看下面的例子：

$V(0)=0$
 $V(2n)=0$
 $V(3n)=1$
 $V(5n)=1$
 $V(6n)=0$

如果数据量非常大，使用文件激励源将更方便。

文件激励源

文件激励源产生的信号是由保存在 ASCII 文件中的一系列时间点和数据值定义的模拟信号，用来驱动电路。因此它将与分段线性激励源非常类似，除了它使用外部文件定义数据点，而不是使用器件的属性来定义数据点。

它的对话框只有一个字段，定义了数据文件的名称。这些文件没有默认的扩展名，而且文件应该与设计文件保存在同一个目录下，否则要输入整个路径。

数据文件格式

ASCII 文件的格式定义为：每一行定义一个时间和电压对，之间用空格或 tab 键隔开，但不能用逗号。时间值必须是逐渐增大的，而且必须是简单的浮点数（不能使用后缀）。

例子：

下面的示例数据文件定义了三个周期的锯齿波，上升时间为 0.9ms，下降时间为 0.1ms，幅度为 1V。

0	0
9E-4	1
1E-3	0
1.9E-3	1
2E-3	0
2.9E-3	1

音频激励源

音频激励源使用 Windows 的 WAV 文件驱动电路。与音频分析图表一起，可以听到仿真电路处理声音信号的结果。

- 文件名具有默认的扩展名 WAV，文件应该与设计文件保存在同一个目录下，否则要输入整个路径。
- 幅度可以通过最大的幅度绝对值来设定，也可以通过峰峰值进行设置。
- 可以设置直流偏移电压。如果此电压为 0，则输出电压将在 0 周围震荡。
- 对于立体声 WAV 文件，可以选择播放的通道，或者把数据作为单声道的对待。

数字激励源

有五种子类型的数字激励源：

- 单边沿——从低到高或从高到低的单调信号。
- 单脉冲——在相反方向上的一对信号转换，共同形成一个正的或负的脉冲波形。既可以设置每个边沿的时间（开始时间和停止时间），也可以设置开始时间和脉冲宽度。
- 时钟——一个连续的脉冲间隔相等的脉冲队列。可以设置开始值、第一个脉冲边沿到达的时间、周期或者频率。周期定义的是整个周期的时间。
- 图案——这是最灵活的激励源，事实上可以产生所有其它类型的信号。图案激励源由以下参数定义：

符号	描述
Initial State 开始状态	在 0 时刻的电平值，在混合信号仿真过程中，用来寻找电路的工作点。
First Edge 第一个边沿	图案激励源真正开始的时间，输出在此时间之前将一直保持在开始状态。
Timing 时间	图案的每一步可以使用相同的时间（通过勾选 Equal mark/space timing），也可以为高低电平设置不同的时间。在这种情况下，脉冲宽度(Pulse width)定义逻辑“1”的时间值，而间隔时间(Space time)定义逻辑“0”的时间值。
Transitions 转换	输出类型可以是持续重复输出图案，直到仿真结束的。也可以是达到固定数量边沿转换后，自动停止的。
Bit Pattern 位图图案	默认的模式是简单的高低序列。另外，也可以设定图案字符串。图案字符串可以包含以下字符： 0, L—输出波形为强的低电平（注意使用大写的“L”）。 1, H—输出波形为强的高电平（注意使用大写的“H”）。 l—输出电平为弱的低电平（注意使用小写的“l”）。

	h——输出电平为弱的高电平（注意使用小写的“h”）。 F,f——输出电平为浮动电平。
--	--

脚本——激励源由数字 BASIC 脚本控制。在脚本文件中通过声明一个与激励源参考名相同的 PIN 变量，就可以访问该激励源。

8.2 探针

概述

探针用于记录所连接到的网络的状态。有两种类型的探针：

- 电压探针——可用于模拟仿真和数字仿真中，在模拟仿真中，它记录了真实的电压值，而在数字仿真中记录的是逻辑电平和强度。
- 电流探针——只能用于模拟仿真中，测量的方向由电流探针上的箭头表示。

警告：不能把电流探针放到数字仿真中，或者放置到总线上。

探针最常用于基于图表的仿真中，但也可用在交互式仿真中，用来显示工作点的数据和分割电路。

探针放置

步骤：

1. 选择电压探针或电流探针图标。在预览窗口将见到探针的图形。
2. 使用旋转和镜像按钮确定探针的方向。
3. 注意：电流探针的方向非常重要，系统是根据电流探针上的箭头来测量电流的。
4. 把鼠标移到编辑窗口，按下鼠标左键并保持不放，拖拉探针到合适的位置，然后释放鼠标左键，把探针放到连线上。

既可以直接把探针放置到连线上，也可以先放置探针，然后再连线。

当探针没有连到网络上时，它默认的名称是一个问号（?），表示它没有被标注。当探针连接到某一个网络时，它将标注成该网络的名称，如果该网络也没有标注，探针的名称将是最接近的元件的名称和连接到该网络的引脚名的组合。如果以此种方式命名，在更改网络的时候，探针名将会自动更新。也可以给探针手动命名，在这种情况下，探针的名称是固定的。

探针设置

通过编辑探针对话框，可以调整探针的两个参数：

负载电阻（Load Resistance）

电压探针可以在原理图中设置一个负载电阻，当探测点对地没有直流通路时，这项设置非常有用。

记录文件名 (Record Filename)

电压探针和电流探针都可以把数据记录到文件中，然后通过录音机激励源 (Tape Generator) 把记录的数据重新播放出来。这个特性可以让你把一个电路的输出波形记录到文件中，然后作为另一个电路的输入播放出来。

请参考录音机和分区章节以获得更多信息。

第九章 脚本化信号源

9.1 前言

PROSPICE 包含了支持可编程的脚本语言，该语言可以用于编写复杂的测试信号。我们命名该语言为“EasyHDL”，因为它比学习通用的硬件描述语言像 Verilog 或 VHDL 更容易些。不过，尽管其相对简单，但它却可以用来模拟及数字波形，并且可以用来产生复杂的测试向量，通过这种方式，在原理图上，一个脚本可以规定多个信号源对象的特性。

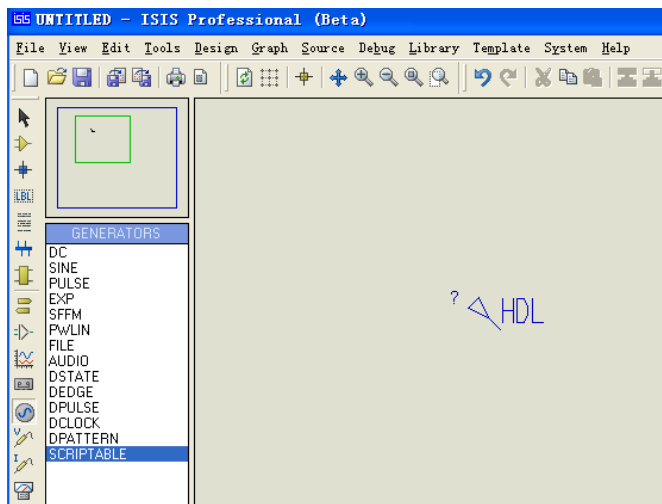
EasyHDL 是轻微的基于 BASIC 编程语言的，不过它添加了另外的专门用于规定测试向量的特性。本章的其余部分将向你展示如何创建 EasyHDL 脚本，及如何在电路原理图上连接它们到一个或多个脚本化信号源对象上。不过本手册并没有给你一个关于 BASIC 语言的详细解释，除非在以下情况下才解释 EasyHDL 提供的特殊功能，即为了成为一个有用的仿真模型和控制语言。对标准 BASIC 关键字的描述是简洁的且只限于解释语言的语法。

EasyHDL 脚本对一个特殊的脚本化信号源对象是局部的，或为了规定一个复杂的测试向量集，它们可以被放置到原理图上，然后同多个脚本化信号源建立联系。

9.2 创建、编辑及调试 EasyHDL 脚本

为了对一个脚本化信号源产生局部的 EasyHDL 脚本：

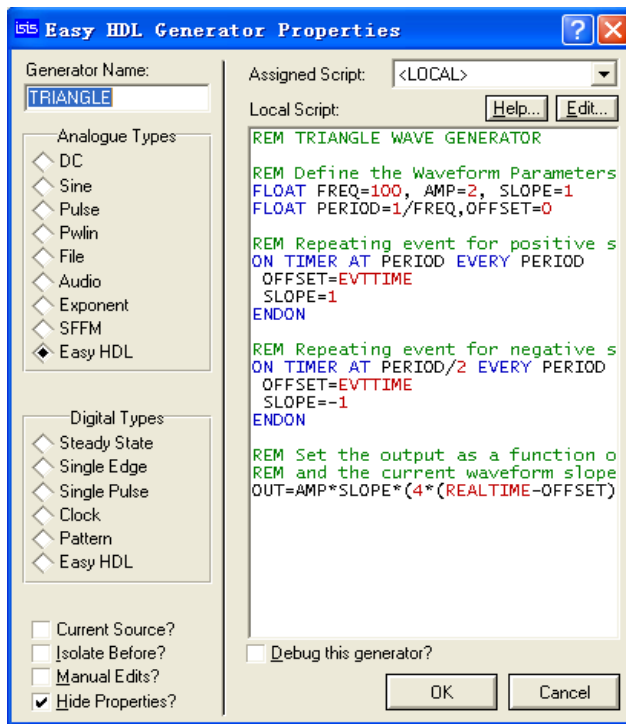
1. 选择一个脚本化信号源图标放置一个脚本化信号源，或单击鼠标右键编辑一个已经存在的脚本化信号源（具体方法见下图）。



放置一个脚本化信号源图示

2. 设置脚本化信号源类型为 Easy HDL – 依据你想创建的脚本化信号源类型，它能够是模拟或数字的。

3. 输入脚本到本地脚本编辑域中，可选的，你也可以点击“Edit”按钮在外部的文本编辑器中编辑脚本。你也可以通过菜单上的“Source/ Setup External Text Editor”命令，规定哪一个文本编辑器被使用（下图示出了其属性编辑界面）。



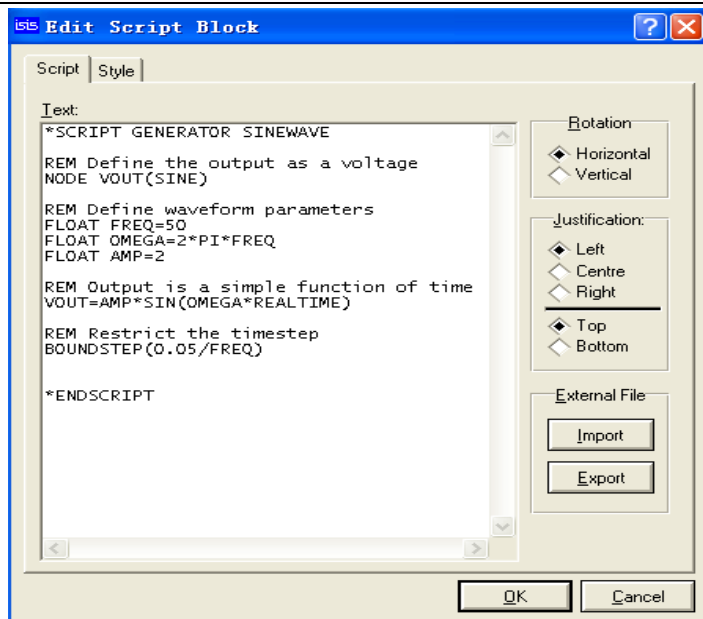
脚本化信号源属性编辑窗

输出引脚被自动指定名字为“OUT”，假如你在第 2 步规定为模拟信号源，那么“OUT”将是一个节点变量，假如你在第 2 步规定为数字信号源，那么“OUT”将是一个引脚变量。

为了在原理图上产生一个 EasyHDL 脚本：

1. 选取脚本图标然后在 ISIS 绘图域中点击鼠标左键放置一个脚本对象。则将弹出编辑脚本对话框。

其界面如下图所示



在 ISIS 中查看“SCRIPTS”的有关放置及编辑文本脚本的常用指令。

2. 脚本的第一行必须是（在“text”域中）：

***SCRIPT GENERATOR script_name**

关键词“SCRIPT”指示在脚本块中的文本将被包含在网表中，同时关键词“GENERATOR”告知 PROSPICE 有关脚本的类型（为了连接的目的），在“GENERATOR”关键词之后的名字是由你提供的确定脚本的名字。任何顺序的阿拉伯字符及下划线('_')都可以被使用。空格或任何其他字符都被认为是脚本名的结束。

3. 在脚本的最后一行必须是：

***ENDSCRIPT**

这通知 PROSPICE 中的网表装载机脚本的结束位置。假如在网表编辑中没有发现它 ISIS 将报告错误。在之后的任何行将被忽略。

4. 在开头的“*SCRIPT...”及结尾的“*ENDSCRIPT”中间输入你的脚本，你必须声明发生器为“NODE”变量或“PIN”变量。

5. 当你做完成时，关闭脚本对话框窗口。

为了连接原理图中基于脚本的信号发生器对象：

1. 选择一个脚本化信号源图标放置一个脚本化信号源，或单击鼠标右键编辑一个已经存在的脚本化信号源。

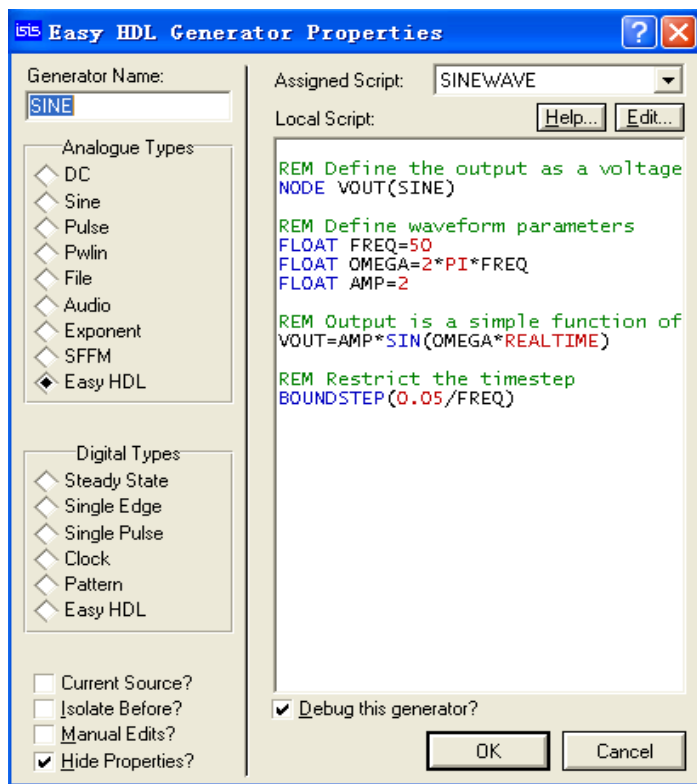
2. 设置脚本化信号源类型为 Easy HDL – 依据你想创建的脚本化信号源类型，它能够是模拟或数字的。

3. 使用指定的脚本选择器规定将要驱动信号发生器的脚本。发生器的名字必须同在控制脚本中的“NODE”或“PIN”的变量相匹配。

在仿真过程中调试发生器脚本：

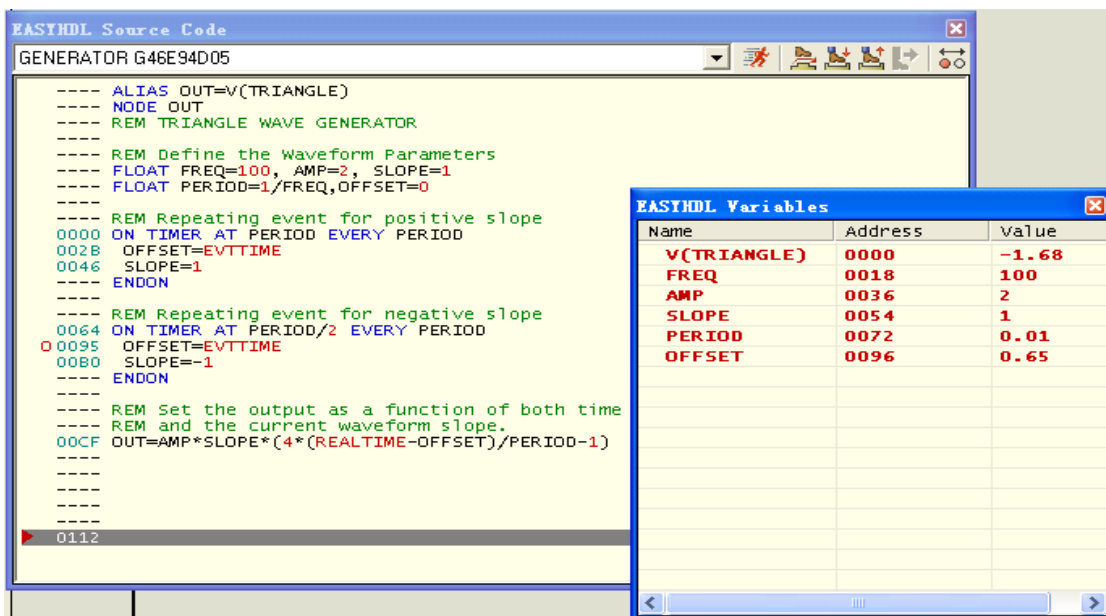
1. 以常规方法放置脚本化信号源，设置信号源类型“EasyHDL”并写脚本。

2. 检查对话框窗口中“Debug this generator”的选项使之处于调试之中(见下图所示)。



选中“Debug this generator”选项图示

3. 暂停仿真以观察脚本的源代码及变量。



暂停仿真时弹出的代码窗及变量窗

4. 在代码框中，设置断点然后仿真运行（在源代码窗的问题行中双击可设置断点）。
5. 当断点被命中时，仿真将暂停，你可以常规的方式单步运行脚本，监视变量的变化等。规定一个 EasyHDL 三角波脚本发生器被激活用来仿真。

从用户接口的视角来看，调试一个 EasyHDL 接口是完全等同于在 Proteus VSM 中调试一个微控制器固件。假如你对这个基本的操作不能明确的话，我们建议你在这点上可以参考上述的调试（微控制器的调试）。除此之外，你必须牢记在大脑里的有一点点不同的是：

●EASYHDL 描述执行 0 时间。当你单步执行 EASYHDL 代码时，电子仿真在每一步之后。这与微控制器的单步执行代码显著不同。在那里，每一步指令花费的时间有限，仿真接近每一步代码。

●由于时间落后于每一个描述之后，任何在脚本中所做的指定在电路仿真时都不会立即看到，直到每一个脚本的最后后一个描述或 “a SLEEP” 或 or “WAIT” 描述到达时。

●一旦仿真开始时，所有的脚本被立即执行。因此从脚本的开始处开始可以到达的任何断点在电路的操作点被计算以前都可以到达。这可能导致不正常的电压及逻辑状态显示在电路上。在这种情形下，操作点将在每一步之后计算，直到脚本的结束为止。当脚本被定时器或其它调用事件调用时调试才唯一激活。

●当一个模拟脚本在正常的模拟时间点被调用时，断点及单步并不使能。当脚本被定时器或其它呼叫事件调用时调试才唯一激活。

几个简单的 “EasyHDL scripts” 可以在你的 Proteus 被安装后的样本例子目录下的 “Generator Scripts ” 子目录中发现。它们的范围覆盖了从一个简单的正弦波到一个完整的 QPSK 调制器等。它们是值得很好研究的。

9.3 程序结构

EasyHDL 由声明、指定及命令组成。声明用来声明变量的名字及可能的别名，指定用来指定表达式对变量的结果，命令用来控制程序脚本的流向。

每一个声明或命令有一个或多个关键字与之相联系，不过并不是所有的关键字都需要规定，也就是说，命令的某些部分是可选的，例如 FOR...TO...STEP...

NEXT 命令有四个关键字，在其中，只有关键字 FOR, TO 及 NEXT 是必须的 (STEP 关键字及紧跟其后的表达式是可选的。

某些命令有块语法及线语法，块语法容许几个命令在命令的开始及结束关键字之间被嵌套（见下面的例子）。线语法只容许其他的命令在同一行作为命令本身被规定。也没必要规定结束命令关键

字，它假定命令行本身为结束行。下面的例子显示了 IF...ELSE...ENDIF 命令在块语法及线语法中的格式。

块语法格式：

```
IF i=0 THEN
    j=1      // IF... 块
    k=2
ELSE
    j=3      // ELSE... 块
    k=4
ENDIF
```

线语法格式

IF i=0 THEN j=1:k=2 ELSE j=3:k=4 // 不需要 ENDIF 作为结尾。

9.4 语法图解析

下面的术语及语法符号在随后的命令参考材料部分被使用，强烈建议用户在理解后面的参考材料时需要熟悉这些术语。

在命令参考部分的每一个命令，其语法以一个简单的“BNF”格式被给，解释如下：

1. 在语法定义及例子中，所有的命令关键字以“courier”字体表示，在语法定义内所有的命令关键字也处于加粗状态，以示命令关键字同一般文本描述的区别。例如：

IF expression **THEN** ...

2. 圆点(...) 指示一个或多个其他命令，由冒号分开

3. 方括号('[' 及 ']') 指示在括号内的文本是可选的。在那里，括号中的文字被嵌套，内部的文字只有在外部的被输入后才容许提供，也就是，假如你省略了外部的文字，那么你必须省略紧跟其后的括号内的所有内部的文字。例如：

COMMAND [[A[[,B]]]]

意指：假如你选择了可选的命令“A”，那么你可以选择使用可选的逗号及“B”，假如你没有选择命令“A”，则其后的任何命令你就不能选择。

4. 符号 (|) 用来指示多个中的一个，互斥的选项可以输入，例如：

COMMAND A | B | C

意指：命令可以按照中的'A', 'B', or 'C'任何一个。

5. 字母 **etc** 用来指示前面参数的重复。在 **etc** 前面的特性意指，在前面的文本中，从那里，你可以重复参数，例如：

```
INT variable=expression[[,variable=expression]][[,etc]]
```

“etc ” 意指前面的参数 [[,variable=expression]] 可以无限的重复。

6. 字或短语以更低情形的 Times Roman 斜体表示，用于某些任意的表达式或变量，常常以文本的形式出现，其后紧跟语法定义，在下面的例子中：

```
IF test_expression THEN ...
```

短语 “test_expression” 表示任意数字的表达式。

9.5 使用 EASYHDL 规定模拟信号

声明输出

模拟脚本化发生器的输出脚应该被声明为一个节点变量，例如：

```
NODE V(OUT)      // 单电压输出 Single voltage output

NODE V(POS,NEG)  // 差分电压输出 Differential voltage output
NODE I(OUT)      // 单点电流输出 Single ended current output
NODE I(POS,NEG)  // 电流输出 Current output
```

一个局部的发生器脚本包含了一个隐式声明，**NODE V(OUT)** 用于电压发生器，**NODE I(OUT)** 用于电流发生器。

程序流

当一个 EasyHDL 脚本用于规定一个模拟信号时，整个脚本在每一个时间点，或每一次调用事件发生时都被从头到尾地执行。命令像 “SLEEP” 不容许，也假定程序将对所有的节点变量进行指定。

事件调用可以通过使用“CALLBACK”或“TIMER”命令来安排。这些命令在规定的时间内暂停一个模拟的时间点，脚本以一个相应的事件 ID 号方式被调用。

典型地，你可以使用数学公式规定在特殊的时间点输出，使用调用事件以规定的次数来改变某些参数，假如这听起来有点让人糊涂，下面的例子将澄清这一概念：

REM TRIANGLE WAVE GENERATOR

REM Define the Waveform Parameters

FLOAT FREQ=100, AMP=2, SLOPE=1

FLOAT PERIOD=1/FREQ, OFFSET=0

REM Repeating event for positive slope

ON TIMER AT PERIOD EVERY PERIOD

OFFSET=EVTTIME

SLOPE=1

ENDON

REM Repeating event for negative slope

ON TIMER AT PERIOD/2 EVERY PERIOD

OFFSET=EVTTIME

SLOPE=-1

ENDON

REM Set the output as a function of both time

REM and the current waveform slope.

OUT=AMP*SLOPE*(4*(REALTIME-OFFSET)/PERIOD-1)

当脚本在任意时间点被调用时，只有最后一行被执行。这一行规定了输出的形式为电压，它将按照“SLOPE”的值，随时间线形变化，升高或降低。“SLOPE”的值被交替地设为 1 或-1，它通过两个“ON TIMER”块实现，这两个“ON TIMER”交替地全或半周触发以便三角波被产生。

从上述的例子可以明显看到，变量在调用之间保持它们自己的值。不过，你必须小心的，不能假定在仿真时间内你的脚本被调用多少次。例如：

REM !!!BAD CODE!!!

FLOAT VOUT=0

VOUT = VOUT + 1.0

OUT=VOUT

这将产生不可预料的结果，因为 VOUT 的上升速率完全取决于由 SPICE 内核所选择的时间步长。

规定输出转换

你能够以以下两种方式中的一种规定模拟信号产生器的输出：

- 作为一个连续值。

OUT=vout

- 作为一个时间量化的转换。

OUT=vout AFTER time_interval

在时间量化的转换中，输出将按照规定的时间间隔，从它的初始值到终止值线性上升。你也可以规定一个绝对的目的时间用于转换。

OUT=vout AT time

下面的例子，显示了输出转换如何用利用存储的数据来产生一个分段线性的波形。

REM PIECEWISE LINEAR GENERATOR

REM Define time/value pairs for the output

DATA 2m,5

DATA 3m,5

DATA 5m,0

DATA 0,0 // End of data

FLOAT PERIOD=10m

FLOAT v,t,td

TIME OFFSET=0

REM Reset the data pointer to

REM start a new cycle

```
ON TIMER AT PERIOD EVERY PERIOD
```

```
    RESTORE
```

```
    OFFSET=EVTTIME
```

```
ENDON
```

```
ON EVENT
```

```
    READ t,v
```

```
    IF t > 0
```

```
        t = t + OFFSET
```

```
        OUT=v AT t // 输出在时间“t”被设置为电压“v” The output is set to voltage v at time t
```

```
        CALLBACK AT t
```

```
    ENDIF
```

```
ENDON
```

9.6 使用 EASYHDL 规定数字信号

声明输出

数字脚本化发生器的输出脚应该被声明为一个引脚变量，例如：

```
PIN Q           //单数字输出 Single bit output
```

```
PIN D[0..7] //8 位数据输出 8 bit data bus
```

一个局部的发生器脚本包含了一个隐式声明 **PIN OUT**。

在那里，发生器有一个名字，但它是不合理的 EasyHDL 变量名，你可以使用一个别名命令。

```
ALIAS OUT=Test Signal
```

```
PIN OUT
```

程序流

当一个 EasyHDL 脚本用于规定一个数字信号时，整个脚本都被从头到尾地执行直到下列之一条件满足：

- 一个 **END** 描述被执行
- 在脚本中最后的描述被执行。
- 一个“**SLEEP**”或“**WAIT**”命令引起脚本执行的终止。

一旦“END”描绘或脚本的结尾到达，脚本将不再被调用，直到一个“CALLBACK”或“TIMER”事件被计划或安排。这两个命令将引起脚本被重新执行（假如被暂停）或从一个对应的事件 ID 从头开始执行。注意：只有一个唯一的执行线索；假如一个事件时脚本是暂停的或睡眠的，那么执行总是从紧跟的描述之后开始的。

典型地，一个数字化信号产生器脚本将由输出指定组成，这些指定中包含了“SLEEP”或“WAIT”描述，用以产生所需的定时。例如下面的代码将会对一个变量激发出一个串行的数据流。

```
// Bit Bang a single character

// 1 Start bit, 1 Stop bit and no
// parity are assumed.
// Start bit
OUT = 0
SLEEP FOR BITTIME

// Data bits
FOR j=0 TO 7
  OUT = d & (1 << j)
  SLEEP FOR BITTIME
NEXT j

// Stop bit
OUT = 1
SLEEP FOR BITTIME
```

规定输出转换

你能够以以下几种方式中的一种规定数字信号产生器的输出：

- 作为一个连续值（转换附加 0 延时）。

OUT=dout

- 作为一个将来值

OUT=vout AT time

多个将来的指定可以立即被安排。例如一个脉冲可以被描述为：

OUT=1 AT time

OUT=0 AT time+width

● 作为一个时间量化的转换

OUT=dout AFTER delay

OUT=dout AFTER tdlh,tdhl

在时间量化转换的情况下，有两个参数，当输出上升时(low->high transition)，tdlh 被使用，当输出下降时(high->low transition)，tdhl 被使用。

9.7 变量

变量声明

变量可以被声明为单一的或数组的，数组变量提供了一个产生查找表的方法。更重要的是，支持总线。

变量的声明有以下几种形式：

TYPE name[[lower..upper]] [[=expression]] [[,etc]]

TYPE name[[width]] [[=expression]] [[,etc]]

声明表现为你希望声明的变量类型、名字，紧接着由一个逗号分隔一系列变量名。假如变量代表一个数组，则在名字之后需有数组包含的两周期分离的高、地边界值或数组的宽度，它们被包含在方括号之中。数组的宽度或高低边界必须是整型常数，表达式不容许。假如你通过宽度规定了一个数组，那么低边界为 0、高边界为 1 被假定。变量的初始化可以通过其后的一个等号及之后的初始化表达式来实现。

例如，

PIN data[2..5]=10, output

INT i

INT j=0,k[2]

TIME t1,t2=10n

声明两变量的类型 **PIN** (data and output), 三变量的类型 **INT** (i, j, and k)及两变量的类型 **TIME** (t1 and t2). 数据 “data” 和 “k” 变量都被声明为数组, “data” 有四个元素 data[2], data[3], data[4] and data[5], 而 “k” 有两个元素 k[0] and k[1]. 在被声明的变量中, “data”, “j”, 及 “t2 ” 被相应地初始化为 10, 0 及 1E-8。

带有初始化表达式的变量的值只是在导入阶段一次被指定初值, 之后, 变量将依据调用脚本来决定其值, 相对每一次调用来讲, 而不再被指定为它们的初始化值。

关于声明 **PIN** 变量数组, 请看 **BOOL** 及 **PIN** 类型获取更多的信息。
请看变量中有关指定变量为表达式的解释。

数据类型

在 EasyHDL 中有三种数据类型:

INT 无符号或有符号的整数, 范围: 0 ~ 2,147,483,648.

FLOAT 无符号或有符号的浮点数, 范围: $2.225074 \times 10^{-308}$ ~ 1.767693×10^{308} .

TIME 无符号或有符号的整数, 有能力维持一个事件的时间, 有 1ps 的分辨率。在模拟仿真中的当前时间可以来自系统变量 “REALTIME”, 它将返回一个浮点数。

字符串类型

EasyHDL 支持简单字符串类型。

STRING 简单的 ASCII 字符串.

不同的操作可以通过字符串变量实现:

STRING S1 = "Hello", S2 = "World", S3

INT C

S3 = S1+" "+S2 // S3 <= "Hello World"

S3 = S1[1,4] // S3 <= "Hell"

C=ASC(S1) // C <= 48h ('H')

C=ASC(S1[2]) // C <= 65h ('e')

S3 = CHR(C+1) // S3 <= "f"

BOOL 及 PIN 类型

EasyHDL 的 **PIN** 是用来访问数字信号发生器的引脚,该数字信号发生器由其所连接的脚本所驱动。被声明为这种类型的变量被连接到被命名为相同的数字信号发生器上,该数字信号发生器由其所指定的脚本所驱动。

看声明输出获取有关如何声明 **PIN** 变量的更多信息。

变量声明的顺序是不重要的,对于其它的 EasyHDL 类型,被声明的变量的初始化也是容许的。

当一个 **PIN** 类型的变量被读或测试时,它返回这个引脚的以前值及当前值。在这里,引脚的活动取决于引脚被连接的网络状态、引脚的电器类型、逻辑器件系列的转换表、以及通过拥有的器件或发生器所指定的引脚的任何特性。类型为 **PIN** 的变量只能通过等价于下列的系统常数之一来测试:

- TRUE, ACTIVE** - 引脚当前是激动的,它以前的状态不管。
- FALSE, INACTIVE** - 引脚当前是非激动的,它以前的状态不管。
- FF** - 引脚当前是非激动的,并且保持非激动的。
- FT, POSEDGE** - 引脚是非激活的,已被激活。
- TE, NEGEDGE** - 引脚是激活的,变为非激活的。
- TT** - 引脚是激活的,保持为激活的。

直接驱动输出引脚连接的网络是可能的,而不须考虑引脚的电器类型,或所指定的任何其他特性。这可以通过指定 **PIN** 变量为下列系统常量之一来实现。

- FLT** - 使用漂浮态驱动网络。
- SLO** - 使用强低态驱动网络。
- SHI** - 使用强高态驱动网络。
- WLO** - 使用弱低态驱动网络。
- WHI** - 使用弱高态驱动网络。

由于引脚活动状态的存储是复杂的,特殊类型的“**BOOL**”被用来存储引脚变量的状态。**BOOL** 类型等同于 **PIN**, 除非在声明的 **BOOL** 类型没有连接到一个外部引脚的情况下例外。

节点类型

EasyHDL 中的类型 **NODE** 用来访问模拟信号发生器的管脚，该信号发生器被脚本所连接。声明为这种类型的变量被连接到由脚本所指定的拥有相同名字的信号发生器上。

看声明输出获取有关如何声明 **NODE** 变量的更多信息。

变量声明的顺序是不重要的，对于其它的 EasyHDL 类型，被声明的变量的初始化也是容许的。

当一个 **NODE** 类型的变量被读或测试时，它返回这个变量正在驱动电压或电流值。在时间量化转换的情况下，它将是当前仿真时间点的内插值。

指定变量

指定单一变量有常规的语法：

```
[[LET]] variable = expr'n[[,variable=expr'n]][[etc]]
```

例如：

```
INT VAR1, VAR2, VAR3
```

```
...
```

```
VAR1=10, VAR2=20, VAR3=30
```

通常，表达式的结果总是在分配数据之前，指派变量的类型。例外的是，指定一个网络状态给一个 **PIN** 类型的变量，这个变量连接到了一个外部的发生器或输出器件的管脚上，在那里状态被直接应用到发生器或管脚的网络上。

数组可以被指定。假如表达式产生一个数组结果，并且在数组表达式中的元素的数目同所指定的数组变量中的元素的数目一样，那么指定可以从每一个数组的低界开始，依次利用表达式来实现对数组变量中各个元素的指定工作。假如表达式中的元素不足以指定数组变量中的元素，那么一个实时运行错误将导致。

例如：

```
INT A[10..19]
```

```
A[10..12]=A[15..19]    // A[10]=A[11], A[11]=A[16], A[12]=A[17]
```

```
A[10..15]=A[16..17]    // Error, insufficient elements in
```

```
// A[16..17].
```

假如表达式的结果是一个单一值，数组变量被指定为非“**PIN**”类型，那么这个值被首先传给数组变量的类型，然后被指定给其中的每一个元素。对于“**PIN**”类型的数组变量，这个值首先传送的是一个“**INT**”，然后 **PIN** 数组中的每一个元素被设置为激活或非激活，这取决于该 **INT** 的结果的相应位是“1”还是“0”。在数组中的最低位元素被设置为表达式值的

最低位（第 0 位），下一位元素被设置为表达式值的第一位，依次类推。指定并不是依据位的值所做的（该位对应的元素号）。例如：

```
INT A[3]
```

```
PIN D[2..3]
```

```
A=4          // A[0]=4, A[1]=4, A[2]=4
```

```
D=10         // D[2]=Bit 0 of 10, D[3]=Bit 1 of 10.
```

对一个 **PIN** 类型的变量进行指定时可以使用扩展语法以提供定时信息，如下：

```
pin_var = expression AFTER time_expression
```

```
pin_var = expression AFTER tlh_expression,thl_expression
```

```
pin_var = expression AT|@ time_expression
```

第一个语法使用关键字“**AFTER**”和一个表达式来强制引脚的输出在规定的时间内改变。第 2 个语法也是用关键字“**AFTER**”，但有两个时间表达式，“*tlh_expression*”和“*thl_expression*”表达式强制输出引脚或者在“*tlh_expression*”后发生改变，假如引脚输出涉及一个从低到高的转变；或者在“*thl_expression*”后发生改变，假如引脚输出涉及一个从高到低的转变。注意术语“**low-to-high**”及“**high-to-low**”通常是：一个输出开关从一个悬浮态到一个弱低态被认为是完成一个高到低的转换。最后一句语法使用“**AT**”或“**@**”符号规定一个绝对时间，在该时间点，输出引脚被改变。

应该注意到，对一个引脚的指定在规定的时间内到达前不会发生，在脚本结束后以前也不会发生，被暂停的情况也是如此，甚至在指定的时间同脚本调用的时间一样的情况下也是如此。因此，在同一个脚本调用期，对引脚的指定或随后的测试将不产生指定值。例如：

PIN p

```
p=TRUE // Assume pin was inactive, we assign it active.
```

```
IF p=TRUE THEN // Pin is still inactive, therefore test fails.
```

```
...
```

关于指定 PIN 变量的更多信息请查看 **BOOL** 及 **PIN** 类型。

9.8 表达式

操作符

下列操作符在 EasyHDL 中是合理的:

Parentheses ()

Unary minus -

Unary logical NOT NOT or !

Unary bitwise NOT~

Logical AND AND or &&

Logical OR OR or ||

Bitwise AND &

Bitwise OR |

Bitwise XOR ^

Less than or equal <=

Less than <

Equal = or ==

Not Equal <> or !=

Greater than >

Greater than or equal >=

Left Shift>>

Right Shift <<

Addition +

Subtraction -

Multiplication *

Division /

Modulus %

在减过程中，下列操作符优先：

Unary operators (!, NOT, -).

Parentheses.

Modulus, Division, Multiplication.

Addition, Subtraction.

Left and Right Shift.

Bitwise AND, OR and XOR.

Less than, Less than or equal, Greater than, Greater than or equal.

Equals, Not equals

Logical AND and OR.

值

值可以是变量，系统变量、系统常量、函数的结果、子表达式的结果（通过在括号中的表达式估计）、数字。数字可以是十进制、十六进制（在数字前面加\$符号）、二进制（在数字前面加%符号）。所有的值可以通过一个一元的操作符修改。下面的例子给出了这些情况。

23 - 十进制值 Decimal value, 23 decimal

%1010 - 二进制值 Binary value 1010 (10 decimal)

\$010A - 十六进制值 Hexadecimal value 010A (266 decimal)

(10*5) - 子表达式 Sub-expression, 10*5 (50 decimal)

TRUE - 系统常量 System constant, TRUE

!TRUE - 一元的操作符及常数 Unary operator and constant, NOT TRUE

max_count - 变量 Variable, max_count

EVTTIME - 系统变量 System variable, EVTTIME

~%1010 - 一元的按位取反 Unary Bitwise NOT (5 decimal)

!\$A - 一元的逻辑非 Unary Logical NOT (0 decimal)

"Hello" - 字符串常数 String constant

评估优先操作的例子

下面是几个表达式的例子：

2 * -3 + 4 - 结果为 -2

2 * -(3 + 4) - 结果为 -14

5 << 3 - 结果为 40

10 >> 2 - 结果为 2
3 << 3+4 - 结果为 28
20-3*4 - 结果为 8.
2<1+3 - 结果为 1 (i.e. TRUE, 2<4)
1+2>=8 - 结果为 0 (i.e. FALSE, 3>=8)
40%16 - 结果为 8
!(1<3) - 结果为 0 (i.e. FALSE, !TRUE)
!5<2- 结果为 1 (i.e. TRUE, 0<2)

函数

EasyHDL 支持一些常用的数学函数。所有的函数由函数名及随后的圆括号中的参数（可以是表达式）所表达。函数的结果作为值被返回。下面的表达式将平方根表达式的结果指定给变量 Z。

```
z = sqrt (a*b*c)
```

EasyHDL 支持下列表达式：

DEG	-	将弧度转换为度
RAD	-	将度转换为弧度
SIN	-	返回参数的正弦值，参数为弧度
COS	-	返回参数的余弦值，参数为弧度
TAN	-	返回参数的正切值，参数为弧度
ASIN	-	反正弦函数，结果单位为弧度
ACOS	-	反余弦函数，结果单位为弧度
ATAN	-	反正切函数，结果单位为弧度
SQRT	-	返回平方根。
ROUND	-	-将参数整形为最近的整数
FRAC	-	返回浮点数的小数部分
ASC	-	返回在字符串中的第一个字符的 ASCII 码
CHR	-	将 ASCII 码转换为字符
LEN	-	返回字符串的长度
SUBSTR	-	返回一个子字符串，第二个参数规定其开始位置，可选的第三个参数规定其长度

9.9 系统常量及变量

系统常量是使用 DSIM 定义的常数值，以便使脚本简洁。其对于一个特定版本的模拟引擎是固定的，它不会随脚本的变化而变化。系统变量是只读的，它返回仿真脚本运行时某个特定的值，因此，系统变量是可以随运行状况而改变的。DSIM 系统常数定义如下：

数据常量

下列的数据常量被定义：

PI - 3.141582654?

事件 ID 常量

常量 **EI_ANY**, **EI_BOOT**, **EI_SETTLE**, **EI_INPUT** 及 **EI_CALLBACK** 用于测试系统变量 **EVTID**，以确定当前的脚本调用是否是系统事件的结果，以及使用在使用命令“**SLEEP** 及 **WAIT**”的情况下。

当脚本被调用时，引起调用的事件 ID 被放置到系统常量 **EVTID** 中，**EI_INPUT** 值指示是调用网络状态改变的结果，该网络是脚本正在模型化的器件的管脚之一。**EI_CALLBACK** 值指示调用是用户定义调用事件的结果，该事件在没有用户提供事件 ID 号时被初始化。

有关 **SLEEP** 命令的信息查看 **SLEEP** 命令

有关 **WAIT** 命令的信息查看 **WAIT** 命令

网络状态常量

常量 **SHI**, **SLO**, **FLT**, **WLO**, **WHI** 用来指定 **PIN** 类型的变量，以直接驱动连接在发生器或相应器件引脚上的网络。

有关这些常量的使用信息，请查看 **BOOL** 及 **PIN** 类型。

PIN 及 BOOL Pin 活动常量

引脚活动状态 **FALSE**, **INACTIVE**, **FF**, **FT**, **TRUE**, **ACTIVE**, **TF**, **FT**, **POSEDGE**, 及 **NEGEDGE** 用来测试 **PIN** 类型的变量，以获得引脚以前及当前活动的状态。

有关这些常量的使用信息，请查看 **BOOL** 及 **PIN** 类型。

TDSCALE 系统变量

TDSCALE 变量返回仿真引擎 **TDSCALE** 仿真控制特性的当前值，它提供了定时的度量值。假如 **TDSCALE** 特性被初始化以产生一个随机的时间度量值，那么 **TDSCALE** 变量将在每次被读时返回一个新的随机时间度量值。

RANDOM 系统变量

RANDOM 变量从 **DSIM** 脚本随机数发生器返回下一个随机数。随机数的范围为 1~32767 (215-1)。

当由发生器产生的数字序列看起来是随机的时候，序列本身是确定的，它由种子的值确定。你可以在脚本的任何地方通过使用种子命令放置种子放生器。**DSIM** 保证：对一个给定的种子值，由后来的 **RANDOM** 所读的的数据序列是确定的。

有关使用种子命令种植脚本的随机数发生器，请查看 **SEED** 命令。

REALTIME 及 EVTTIME 系统变量

REALTIME 变量在模拟仿真时以浮点类型返回当前时间值，这不同于 **EVTTIME**，它在数字仿真模拟时，以时间类型返回当前事件时间，事件时间以 1ps 的分辨率量化。

9.10 命令参考

ABORT 命令

语法

ABORT idcode_expr

用法

ABORT 命令提供一个放弃当前仿真的方法，这个命令的执行引起仿真引擎送出一个 EasyHDL 过程运行时间错误。错误信息被插入到仿真日志中，它包括紧随 **ABORT** 关键字之后的由表达式

“idcode_expr” 所给出的 ID 码。

ABORT 关键字

你应该注意到 **ABORT** 命令不仅放弃了当前的 EasyHDL 调用，而且放弃了整个仿真会话。

例子

```
IF input=TRUE THEN PRINT "Input Is TRUE!" :ABORT 0
```

ALIAS 命令

语法

```
ALIAS new_name=existing_name [[,new_name=existing_name]] [[,etc]]
```

用法

ALIAS 命令提供一个对一个变量名提供另外一个名字的方法。别名在某些地方是非常有用的，在这些地方器件的管脚名字使用一些字符，而这些字符在 EasyHDL 变量名中是不容许的；而对于发生器上 **PIN** 或 **NODE** 类型的变量也是不可能直接声明的。**new_name** 是你希望使用变量或外部管脚，它必须符合 EasyHDL 变量命名规则。**existing_name** 是一个存在的变量、属性、或管脚名-所有的字符包括空格或括号都是合理的。假如你需要别名某个事情，在其名字中使用了空格或括号，你需要在 **existing_name** 上加双引号。在接下来的别名声明中，**new_name** 可以认为是 **existing_name** 的同义词，可以用在你必须使用 **existing_name** 的任何地方

例，如 7493 TTL 计数器，有复位管脚包含括号- **R0(1)**及 **R0(2)**，你不能在 EasyHDL 中包含带有括号的变量名，你必须给 7493 管脚一个别名，例如 **RS0** 及 **RS1**。之后，声明一个带有这些别名的 **PIN** 类型变量，看下面的例子：

有关使用 **ALIAS** 命令声明 **PIN** 变量的情况，请查看 **BOOL** 及 **PIN** 类型获取更多的信息。

例子

```
ALIAS RS1=R0(1),RS2=R0(2)
...
PIN RS1,RS2      // RS1 is the synonymous with R0(1), etc.
...
IF RS1 AND RS2 THEN ...
```

BREAK 命令

语法

BREAK

用法

BREAK 命令引起当前 **FOR...TO...NEXT**, **WHILE...WEND**, **REPEAT...UNTIL** 循环的终止或 **MAP ON** 或 **SWITCH** 当前情况的终止。

- 在 **WHILE...WEND** 循环内的命令执行引起 **WEND** 关键字之后第一个命令连续执行。
- 在 **REPEAT...UNTIL** 循环内的命令执行引起 **UNTIL** 关键字之后第一个命令连续执行。
- 在 **FOR...TO...NEXT** 循环内的命令执行引起 **NEXT** 关键字之后第一个命令连续执行。
- 在 **MAP ON...ENDMAP** 循环内的命令执行引起 **ENDMAP** 关键字之后第一个命令连续执行。
- 在 **SWITCH...ENDSW** 循环内的命令执行引起 **ENDSW** 关键字之后第一个命令连续执行。

例子

```
WHILE pin_enable=TRUE
...
IF EVTTIME > 1 THEN BREAK
...
WEND
```

CALLBACK 命令

语法

CALLBACK *[[eventid]]* **AFTER** *time_expression*
CALLBACK *[[eventid]]* **AT** *time_expression* *[[EVERY repeat_expression]]*

用法

CALLBACK 命令容许一个回调事件被当作消息，当事件时间到达时，EasyHDL 脚本被调用。

这个命令也容许事件的 ID 值被规定。当事件被处理及脚本被调用时，这个 ID 值将被放置到系

统变量 **EVTID** 中,当所有系统事件 ID 码是零或负时,只有正的非零 ID 码可以使用 **CALLBACK** 命令。假如没有 ID 值被规定,当脚本被调用时,**EVTID** 将会包含系统变量值 **EI_CALLBACK**。

事件 ID 表达式,总是其后紧跟关键字 **AFTER** 或关键字 **AT**,然后是一个时间表达式,它规定在那一个时间回调将发生。**AFTER** 关键字指示时间表达式是一个添加到当前事件时间中的延时值。**AT** 指示时间表达式是绝对时间,它指出了回调事件将发生的时间。

一个重复的回调通过使用 **AT... EVERY** 结构实现。

有关 **EVTID** 系统变量及事件 ID 码的解释,请看事件 ID 常量

例子

```
IF EVTID = EI_BOOT THEN
    REM BOOT Sequence; post call back for 1u.
    CALLBACK 2 AT 1u
ELIF EVTID = 2 THEN
    REM Process out call-back.
    ...
ENDIF
```

CONTINUE 命令

语法

CONTINUE

用法

CONTINUE 命令引起当前的循环表达式 **WHILE...WEND**, **REPEAT...UNTIL** 循环被重新评估,或同命令限定的表达式相比,当前的 **FOR...TO...NEXT** 循环变量被增加或重新测试。

- 在 **WHILE...WEND** 循环内的命令的执行引起 **WHILE** 命令的连续执行。
- 在 **REPEAT...UNTIL** 循环内的命令的执行引起 **UNTIL** 命令的连续执行。
- 在 **FOR...TO...NEXT** 循环内的命令的执行引起 **NEXT** 命令的连续执行。

例子

```
REPEAT
```

```
...  
IF EVTTIME <= 100n THEN CONTINUE  
...  
UNTIL pin_enable=F
```

DATA 命令

语法

DATA *constant* *[[,constant]]* *[[,etc]]*
DATA REPEAT

用法

DATA 命令规定了一个或多个常数值，它们可以通过 **READ** 命令依次读出。典型地，存储的数据将会决定测试信号发展过程，其通过脚本在时间上的控制实现。例如为了定义分段线性的波形，你可以定义时间/电压对，或你可以存储数据及地址以输出到总线上。

常数值可以是任何合理的 EASYHDL 数据类型包括字符串。

关键字 **REPEAT** 引起数据表被循环考虑。也就是说，在最后的数项被读后，下一个命令将会在表的开始又重新 **READ**。

例子

```
DATA "Hello World ", 250m  
DATA "- EASYHDL Test Program\r", 500m  
DATA REPEAT
```

END 命令

语法

END

用法

END 命令引起当前 BASIC 脚本的终止及对模拟引擎控制的返回。没有必要使用一个 **END** 命令结束一个 BASIC 脚本，这里是暗指。

例子

```
IF EVTTIME < 100n THEN END
```

FOR...TO...STEP...NEXT 命令

语法

```
FOR variable=init_expr'n TO limit_expr'n [[STEP step_expr'n]]  
...  
NEXT variable
```

用法

FOR...TO...STEP...NEXT 命令容许在 **FOR** 及 **NEXT** 之间的命令执行给定的次数。**FOR** 后的变量用于循环，并且必须事先声明为一个 **INT** 或 **FLOAT** 类型，并且在 **NEXT** 之后也必须被规定。

循环变量被初始化为的 *init_expression* 值，在 **FOR...NEXT** 块中命令于是被执行。当到达 **NEXT** 关键字时，循环变量的值被增加，其或是表达式规定的值，或加 1，假如新的循环变量的值小于或等于 *limit_expression* 的值，那么将连续执行 **FOR** 关键字之后的第一个命令，否则，将连续执行 **NEXT** 关键字之后的第一个命令。

在 **FOR...NEXT** 块中的命令总是至少执行一次。而不管初始 *init_expression* 及 *limit_expression* 表达式的值。在 **FOR...NEXT** 块中的循环变量是合理的，也可以象其他变量一样操作，不过注意，可能象这样操作循环变量以至于 **FOR...NEXT** 循环从不终止。例如：

```
FOR i=0 TO 10 STEP 2  
    i=0  
NEXT i
```

循环变量总是设置为 0，尽管当 **NEXT** 相遇时，它在通过步长指令被增加，循环变量的值永远不会到达 10。

例子

```
INT v  
FOR v=0 TO 10 STEP 2
```

```
PRINT v;"*";v;"=";v*v
NEXT v
```

GOTO 命令

语法

GOTO label_name

用法

GOTO 容许程序执行顺序被切换到由标号 label_name 所规定的标号之后的命令行开始执行。在 BASIC 脚本中同 label_name 有一样名字的标号必须被声明，前后方向的跳转都是容许的。

标号必须在一行的开始处被声明，并且由不超过 30 个阿拉伯字符及下划线('_')组成的字符串组成，其不应同 EasyHDL 脚本中的任何关键字相同，进一步，命令之后紧跟终止冒号，如果希望的话。

例子

```
LBL_START:           // Declare the label.
PRINT "START OF SCRIPT"
...
IF !done THEN GOTO LBL_START // Jump to PRINT... command.
```

IF...ELIF...ELSE...ENDIF 命令

语法

```
IF expression [[THEN]]
...
...
[[ELIF expression [[THEN]]
...
...]]
[[ELIF expression [[THEN]]
...
...]]
etc
```

```
[/ELSE
```

```
...
```

```
...]]
```

```
ENDIF
```

```
IF expression [[THEN]] ... [[ELIF expression [[THEN]] ... ]][[ELSE ... ]][[ENDIF]]
```

用法

IF...ELIF...ELSE...ENDIF 命令容许在每一个块中的命令条件执行。有一个唯一的 **IF** 关键字及可选的一个 **ELSE** 关键字。在 **IF** 及 **ELSE** 或 **ENDIF** 之间可能会有多个 **ELIF** 关键字。

跟在 **IF** 及 **ELIF** 之后的表达式被连续地评估，假如这些中的任何一个评估为非零，在 **IF** 及 **ELIF** 之后的命令，在下一个 **ELIF**, **ELSE** 或 **ENDIF** 关键字之前的命令被执行。假如没有 **IF** 或 **ELIF** 表达式的值被评估为非零，那么在 **ELSE** 关键字之后的命令被执行。执行将连续 **ENDIF** 命令之后的命令。

例子

```
IF i=0 THEN
```

```
    PRINT "Value is zero"
```

```
    zero = TRUE
```

```
ELSE
```

```
    PRINT "Value is non-zero"
```

```
    zero = FALSE
```

```
ENDIF
```

```
IF i THEN j=0 : WHILE (i>0) j=j*2:i=i-1 : WEND : i=0 ELSE j=-1
```

MAP ON...ENDMAP 命令

语法

```
MAP ON property_name
```

```
CASE string1 :    ...
```

```
    ...
```

```
    [[BREAK]]
```

```
[[CASE string2    :    ...
```



```
...
    [[BREAK]] ]]
[[DEFAULT :    ...
...
    [[BREAK]] ]]
```

ENDMAP

用法

MAP ON...ENDMAP 命令容许一个置位命令按照一个或多个条件被选择执行。条件将按照 **MAP ON** 命令后的属性值同标号值的比较结果而被选择。

在 **MAP ON...ENDMAP** 块内的是多个条件。 每一个条件有由：要么是 **DEFAULT+冒号**，要么是 **CASE +标号+冒号**组成。在 **CASE** 关键字之后的前十五个字符直到一个空格、一个 **TAB** 字符、或冒号被认为是 **CASE** 的标号。冒号之后是 **CASE** 相联系的命令。

在 **MAP ON** 关键字之后的文本属性值同声明的 **CASE** 标号值依次比较，**Case** 是无关的。假如匹配找到了，然后连续执行 **CASE** 标号之后的命令。假如匹配没找到，一个默认 **DEFAULT** 的条件被设置，然后连续执行 **DEFAULT** 标号之后的命令。执行命令直到 **ENDMAP** 关键字位置；假如执行只到下一个下一个 **CASE** 被要求，那么在 **CASE** 中的最后一个命令应该是 **BREAK**。假如没有任何的 **CASE** 被发现且只有一个 **DEFAULT case**，执行 **ENDMAP** 之后的命令将被连续执行。

MAP ON...ENDMAP 一般用于按照器件的特性状态初始化模型脚本时使用。 为了这个目标，只是在 **MAP ON...ENDMAP** 块内特性被指定。在该块内的命令及有关的条件只是在脚本的导入期间被调用。对于所有其它的调用，命令被自动忽略，没有必要把它放置在一个 **IF...ENDIF** 块内。

例子

```
// Assign timing properties depending on what family
// of device we are modelling.
MAP ON VALUE
```

```
CASE 74259 :   TDRQ=16n   : TDLHDQ=14n
            TDHLDQ=11n : TDLHAQ=15n
            BREAK
CASE 74LS259 :   TDRQ=12n   : TDLHDQ=19n
            TDHLDQ=13n : TDLHAQ=17n
            BREAK
```

ENDMAP

ON BOOT...ENDON 命令

语法

ON BOOT

...

...

ENDON

用法

ON BOOT 容许一个置位命令在仿真的开始被执行一次，准确地说，等价于写。

```
IF EVTID = EI_BOOT THEN
```

```
...
```

```
...
```

```
ENDIF
```

例子

```
ON BOOT
```

```
    REM BOOT Sequence; post call back for 1u.
```

```
    CALLBACK AT 1u
```

```
ENDON
```

ON EVENT...ENDON 命令

语法

ON EVENT *[[event_id]]*

...

...

ENDON

用法

ON EVENT 命令容许无论什么时候一个特殊的事件发生时，一个置位命令被执行。准确地说，等价于：

```
IF EVTID = event_id THEN
```

```
...
```

```
...
```

```
ENDIF
```

假如一个 `event_id` 没有定义，那么它假定为 `EI_ANY`。

在正常的模拟时间点期间，在 **ON EVENT** 块内的任何代码将不被执行。

例子

```
ON TIMER AT PERIOD EVERY PERIOD
```

```
    RESTORE
```

```
    OFFSET=EVTTIME
```

```
ENDON
```

```
ON EVENT
```

```
    READ t,v
```

```
    IF t > 0
```

```
        t = t + OFFSET
```

```
        OUT=v AT t
```

```
        CALLBACK AT t
```

```
    ENDIF
```

```
ENDON
```

ON EVENT 句子在 `startup` (`EI_BOOT`)，每一个 **TIMER** 周期，以及每一次 `callback` (`EI_CALLBACK`)时被执行。

ON TIMER...ENDON 命令

语法

```
ON TIMER event_id AFTER delay_expr
```

```
...
```

```
...
ENDON

ON TIMER event_id AT time_expr [[EVERY period_expr ]]
...
...
ENDON
```

用法

ON TIMER 命令设置定时器并且容许当它被触发时一个置位被运行。准确地说，等价于：

```
TIMER event_id AT time_expr EVERY period_expr
IF EVTID = event_id
...
...
ENDIF
```

假如 *event_id* 没有被指定，那么一个唯一的 id 被自动定位。

当使用 **TIMER** 命令时，一旦一个事件已被安排，则 **ON TIMER** 命令不能被触发。

例子

```
ON TIMER AT PERIOD EVERY PERIOD
    OFFSET=EVTTIME
    SLOPE=1
ENDON
```

PRINT 命令

语法

```
PRINT expression | "text" [[,|; expression | "text"]][[etc]][[,|; ]]
```

用法

PRINT 命令提供了插入文本、变量/表达式进入仿真日志中的方法。

PRINT 关键字后面由一系列参数组成，它们由逗号、分号等隔离。每一个参数由一个表达式或文本加上双引号。每一个参数以从左到右的方式被插入到仿真日志中。由逗号分割的参数插入到仿真日志中以空格分开，由分号分割的参数插入到仿真日志中直接位于前一个参数之后。假如 **PRINT** 命令参数单以参数结束，那么在仿真日志中，在最后一个参数被插入时将换行。假如 **PRINT** 命令参数单以逗号或分号结束，那么没有新行被插入，来自随后的 **PRINT** 命令的任何输出将在最后的一个参数之后，它们都将在一行内。

例子

```
PRINT var1;"+";var2;"=";var1+var2;".";
```

EAD 命令

语法

```
READ var1 [[,var2]] [[,etc]]
```

用法

READ 命令从常数值中获取下一个项目，这些常数值在程序中以 **DATA** 描述方式被声明，并以规定的变量存储。

例子

```
REM Define data to be output
```

```
DATA "Hello World", 250m
```

```
REM Read the data
```

```
READ s,td
```

REM 命令

语法

```
REM text
```

用法

REM(remark) 命令一些解释信息包含在 EasyHDL 脚本中。在 **REM** 关键字之后的所有内容将被编译器所放弃。

注意：由于在 **REM** 关键字之后的所有内容将被编译器所放弃，**REM** 命令应该在由冒号分开的顺序命令结尾之前。

也应该注意：你也可能需要“//”，“//”这样的优点是不需关键字，它不要求在它之前有一个冒号。对于 **REM** 命令，“//”之后的所有内因容将被 EasyHDL 编译器放弃。

例子

```
REM This is a comment, and will be ignored.
```

```
REM So is this.
```

```
PRINT "A=",a      // Display the value of A
```

REPEAT...UNTIL 命令

语法

REPEAT

...

...

UNTIL *expression*

用法

REPEAT...UNTIL 命令容许在 **REPEAT** 及 **UNTIL** 关键字之间的命令被重复执行，直到在 **UNTIL** 关键字之后的表达式的值为非零值为止。

在 **REPEAT** 及 **UNTIL** 关键字之间的命令被执行，当 **UNTIL** 关键字到达时，循环表达式的值被评估，假如结果为非零，执行将在 **UNTIL** 关键字之后的第一个命令处继续，否则将在 **REPEAT** 关键字的第一个命令处继续执行。

注意：在循环的末尾，当循环表达式被测试时，在 **REPEAT** 和 **UNTIL** 之间的命令总是被至少执行一次。

例子

REPEAT

```
WAIT  
count=count+1  
UNTIL enable_pin=TRUE
```

RESTORE 命令

语法

RESTORE

用法

RESTORE 命令复位由命令所使用的数据指针，以便它指向 程序中的第一个数据描述。这也容许数据再次被读。

例子

```
// Define data to be output  
  
DATA "Hello World", 250m  
// Read the data  
READ s,td  
// Read the data again  
RESTORE  
READ s,td
```

SEED 命令

语法

SEED (*value*)

用法

SEED 命令被用来种植脚本的随机数发生器。对一个给定的种子值,DSIM 保证:当由 **RANDOM** 系统变量的并发读所返回的系列值看起来是随机的情况下, 系列本身是有限和确定的。

例如, 规定一个种子值为 1, 则由 **RANDOM** 系统变量所产生的随机系列的前 10 个随机数总是 909, 3365, 5030, 24170, 3305, 10539, 20191, 14244, 15844, 21839, 9277. 类似地, 假如一个种子值为

500, 其产生的前 10 个数的随机系列总是 14616, 21006, 7672, 12031, 9888, 2966, 3985, 24555, 30757, 30317, 27030.

有关 **RANDOM** 系统变量的解释, 请看 **RANDOM** 系统变量

例子

```
SEED (500)
FOR i=1 TO 10
    PRINT "Random Value",i,"=",RANDOM
NEXT I
```

SLEEP 命令

语法

```
SLEEP [[eventid]] FOR time_expression
SLEEP [[eventid]] UNTIL time_expression
```

用法

SLEEP 命令使得当前被调用的数字脚本被暂停要么一个固定的延时, 要么暂停到直到一个设定的时间为止。当结束时间到达时, 在 **SLEEP** 命令之后的第一个命令将继续执行。

SLEEP 命令容许一个事件的 ID 值被规定, 这个表达式的值决定脚本的暂停是否可以被其他事件所中断。如, 使得以前的被声明的回调变为正常, 或使得输入引脚所连接的网络中的一个或多个发生改变。事件的 ID 表达式, 假如规定了, 应该是一个唯一的正非零值, 或系统常量中的一个: **EI_ANY**, **EI_INPUT**, 或 **EI_CALLBACK**. 使用唯一的整数、或 **EI_CALLBACK** 常数保证暂停将不被中断。**EI_ANY** 或 **EI_INPUT** 容许暂停被任何事件或在输入引脚上的变化所中断。当暂停完成时, **EVTID** 变量将包含所规定的事件 ID 的值。默认的 ID 值, 假如没有表达式所指定, 则是 **EI_CALLBACK**。

事件 ID 表达式, 在任何情况下, 总是其后为关键字 **FOR** 或关键字 **UNTIL**, 并且表达式用于表达在那一个时间点暂停结束。**FOR** 关键字指示时间表达式是一个被添加到当前时间中的延时值, **UNTIL** 关键字指示时间表达式是绝对时间, 在该时间点暂停结束。

SLEEP 命令不能用在模拟信号产生器脚本中。

有关 **EVTID** 系统变量及事件 ID 码的解释请看 Event Identification Constants 部分

例子

PIN output

```
...
// Generate 100 periods of mark=20n, space=10n clock
FOR i=0 TO 100
    output = TRUE      // Output active
    SLEEP 1 FOR 20n    // Wait 20ns (mark time)
    output = FALSE     // Output inactive
    SLEEP 1 FOR 10n    // Wait 10ns (space time)
NEXT i
...
```

SWITCH...ENDSW 命令

语法

SWITCH *switch_expression*

CASE *value1*:

```
...
    [[BREAK]]
[[CASE value2 : ...
...
    [[BREAK]] ]]
[[DEFAULT : ...
...
    [[BREAK]] ]]
```

ENDSW

用法

SWITCH...ENDSW 命令容许一个置位命令按照 CASE 中的一个或多个条件执行。条件的选择将依据评估 SWITCH 表达式的值而定。

在 SWITCH...ENDSW 块中是一个或多个条件。每一个条件由关键字 CASE+固定的浮点数值+冒号或关键字 DEFAULT+冒号。在冒号之后是同 CASE 有关的命令。

在 SWITCH 关键字之后的 switch 表达式被作为一个浮点值被评估并且依次同每一个所声明的 CASE 值相比较。假如相匹配，那么将在相匹配的 CASE 值之后的命令处继续执行，假如不匹配，并且一个 DEFAULT 条件被规定，那么将在 DEFAULT 之后的命令处继续执行。执行继续到 ENDSW 命令为止。假如要求只执行到下一个 CASE 处，那么在 CASE 中的最后一个命令将是 BREAK 命令。假如在任何 CASE 中没有发现匹配，并且有一个 DEFAULT 条件，或假如一个 BREAK 在任何 CASE 中被碰到，那么执行将在 ENDSW 关键字后的第一个命令处继续执行。

例子

```
SWITCH bus[0..2]
CASE 0 : REM Fall through to next case...
CASE 1 : REM Fall through to next case...
CASE 2 : PRINT "Bus is 0,1,or 2" : BREAK
DEFAULT : PRINT "Unknown bus value!" : BREAK
ENDSW
```

TIMER 命令

语法

```
TIMER [[event_id]] AFTER delay_expr
TIMER [[event_id]] AT time_expr [[EVERY period_expr]]
```

用法

TIMER 命令以类似于 CALLBACK 命令的流行方式安排事件。不过，不同的是，当以前的计划事件发生时定时器才唯一被触发。换句话说，一旦定时描述已经被执行且发出了一个事件通知后，在事件的时间到达以前，定时器将不做任何事情。

例子

```
PIN OUT = 0
TIMER 1 AT PERIOD EVERY PERIOD
OUT = !OUT
```

WAIT 命令

语法

WAIT *[[eventid]]*

用法

WAIT 命令引起当前脚本的调用暂缓执行一个新的事件。当且仅当当前这个事件发生后，WAIT 命令之后的第一个命令才继续执行。

可选地，命令之后可以是用于等待的事件 ID 值，假如这样的值被规定的话。当表达式对应的事件到达以后，脚本才继续执行。假如没有 ID 值被规定的话，默认值即为系统常量 EI_ANY，脚本将在任何事件到达以后继续执行。

有关事件 ID 码的解释，请看事件 ID 常量。

例子

```
REM Post call-back, and then wait for it and only it - input
REM  pin transitions are ignored.
CALLBACK 2 AFTER 100n
WAIT 2
...
REM Post call-back; wait for it or an input pin transition
CALLBACK 3 AFTER 200n
WHILE EVTID != 3 AND EVTID != EI_INPUT WAIT
```

WHILE...WEND 命令

语法

WHILE *expression*

...

...

WEND

WHILE *expression ...* **[[WEND]]**

用法

WHILE...WEND 命令容许在 **WHILE** 和 **WEND** 关键字之间的命令，在 **WHILE** 关键字之后的循环表达式的值为非零时重复执行。

在 **WHILE** 关键字之后的循环表达式的值被评估，假如结果为零时，在 **WEND** 关键字之后的第一个命令将继续执行。否则在 **WHILE** 和 **WEND** 关键字之间的命令被执行。当 **WEND** 关键字被遇到时，执行返回到 **WHILE** 命令且重新评估表达式的值。

注意，当在循环顶部的的循环表达式的值被评估时，假如第一次测试时表达式的值为零值，则在在 **WHILE** 和 **WEND** 关键字之间的命令可以不被执行。

例子

```
WHILE i<j
    PRINT "i is less than j"
    i=i+2:j=j+1
WEND
```

```
WHILE enable_pin=F WAIT
```

第十章 SPICE 模型的应用

10.1 前言

目前许多主流器件厂商都提供产品线的与 SPICE 兼容的仿真模型。由于 PROSPICE 是基于 Berkley SPICE 源码的，因此使用这些模型几乎没有问题。然而，在使用第三方模型时，有一些问题必须明白：

- ◆ 一个 SPICE 模型是一个 ASCII 网表文件。里面不含任何图形信息，故不能直接放在原理图上。这就意味着应该使用 ISIS 器件库中相关 SPICE 模型的器件。遗憾的是，对于原理图器件库的部件没有标准的文件格式，因此通常需要自己绘图设计。
- ◆ 在一个 SPICE 网表中，模型是参考使用 X 卡（X card）的子电路来调用的。电路节点与模型节点的关联由 X 卡中电路节点的序号决定。

例如：

```
XU1 46 43 32
```

意思是电路节点 46 连接到模型节点 1。类似的电路节点 46 连到模型节点 2、电路节点 32 连接到模型节点 3。不幸的是，这种结构意味着模型节点未被命名。更糟的是没有将模型节点号和实际器件引脚号相联系，假如实际的封装含有 NC（空脚），这在 SPICE 模型中是无法描述的。对于多元器件如 TL074 也很复杂：模型是实现单个运放还是四个？

实际上，这意味着需要通过某种准确的交叉参考信息告诉 ISIS 器件引脚所使用的 SPICE 模型节点号，因为 ISIS 的引脚名和引脚号都不能用于表达此信息，所以引入 SPICEPINS 这个器件属性。

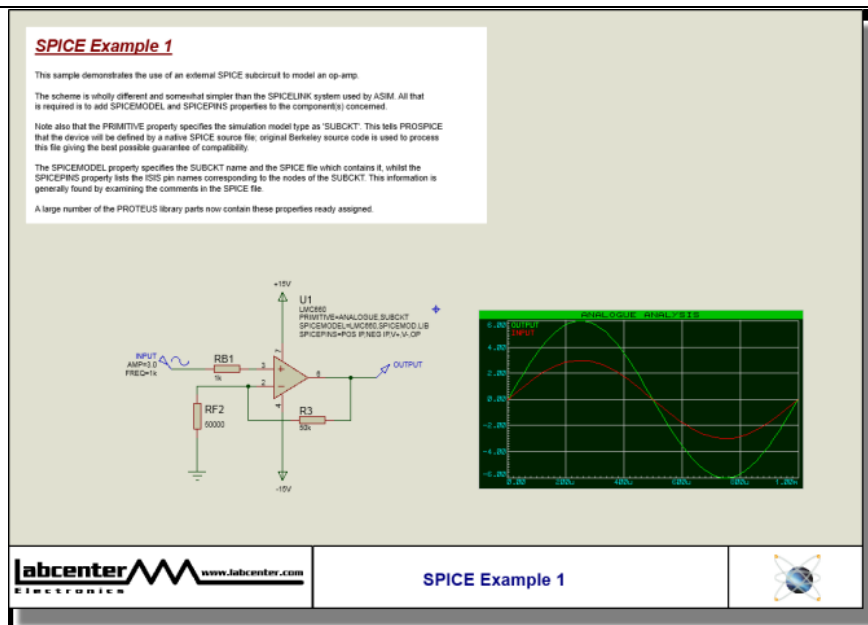
- ◆ SPICE 存在诸多变种。最早期的是 SPICE 2，许多模型依据此写成。然而，较多厂家公布了他们兼容 PSPICE 的模型，这是一个在基本标准下扩展的专有版本。PSPICE 同时支持 SPICE3F5 的很多功能，它也有些不同的原型类别，并且使用了不同的语法描述一些新的内容。因此为 PSPICE 设计的模型可能不适用于 PROSPICE

❗注意：简单地说，如果第三方模型工作不正常，首先应该检查它是不是基于 SPICE2 或 SPICE3。一个好消息是我们自己搜集了大量（目前超过 1500 个）模型并用此生成了 ISIS 器件。

您必须明白，我们并不对这些模型的精度甚至功能负责，不论是我们还是厂家都不会承担由于使用这些模型而造成的任何损失。任何情况下都不能保证电路仿真能完全准确地反映实际硬件的性能，所以我们忠告在量产之前最好搭试实际电路进行试验。

10.2 使用 SPICE 子电路（SUBCKT 定义）

我们的第一个示例是 SPICE 文件 SPICEMOD.LIB 中保存的 LMC660 OPAMP 模型。这是由示例目录中的文件 SPICE1.DSN 提供的，如下所示。



SPICE1.DSN 原理图的截图，位于“..\samples\Simulation 目录”中

运算放大器组件已获得三个属性分配—PRIMITIVE，SPICEMODEL 和 SPICEPINS。依次进行以下操作：

PRIMITIVE=ANALOGUE, SUBCKT

对于由 SPICE 子电路建模的组件，此分配始终相同。它表示该组件将由 SPICE3F5 直接仿真，并且由 SPICE 子电路表示，而不是实际的原始 SPICE 模型。

SPICEMODEL = LMC660, SPICEMOD.LIB

此行指示要使用的子电路的名称，以及包含子电路定义的 ASCII 文件的名称。或者，您可以使用：

SPICEMODEL = LMC660

SPICEFILE = SPICEMOD.LIB

一些制造商在一个文件中提供许多模型，其他制造商在每个模型中提供一个文件。这些区别没关系。子电路名称必须与模型文件中使用的名称完全匹配，因此，如果子电路名为 LMC660 / NS，则必须在 SPICEMODEL 属性中完全包含此文本。

在当前目录中以及在“设置路径”对话框中指定的“模块路径”中搜索 SPICE 模型文件。

SPICEPINS = POS IP, NEG IP, V+, V-, OP

SPICEPINS 属性处理将 ISIS 引脚名称绑定到 SPICE 模型节点编号的问题。要了解如何使用此属性，您需要查看原始 SPICE 模型文件中的注释：

```

*////////////////////////////////////
*LMC660AM/AI/C CMOS Quad OP-AMP MACRO-MODEL
*////////////////////////////////////
*

```

```

* connections:          non-inverting input
*                        |
*                        |      inverting input
*                        |      |
*                        |      |      positive power supply
*                        |      |      |
*                        |      |      |      negative power supply
*                        |      |      |      |
*                        |      |      |      |      output
*                        |      |      |      |      |
*                        |      |      |      |      |
*                        |      |      |      |      |
.SUBCKT LMC660          1      2      99      50      41
*
    
```

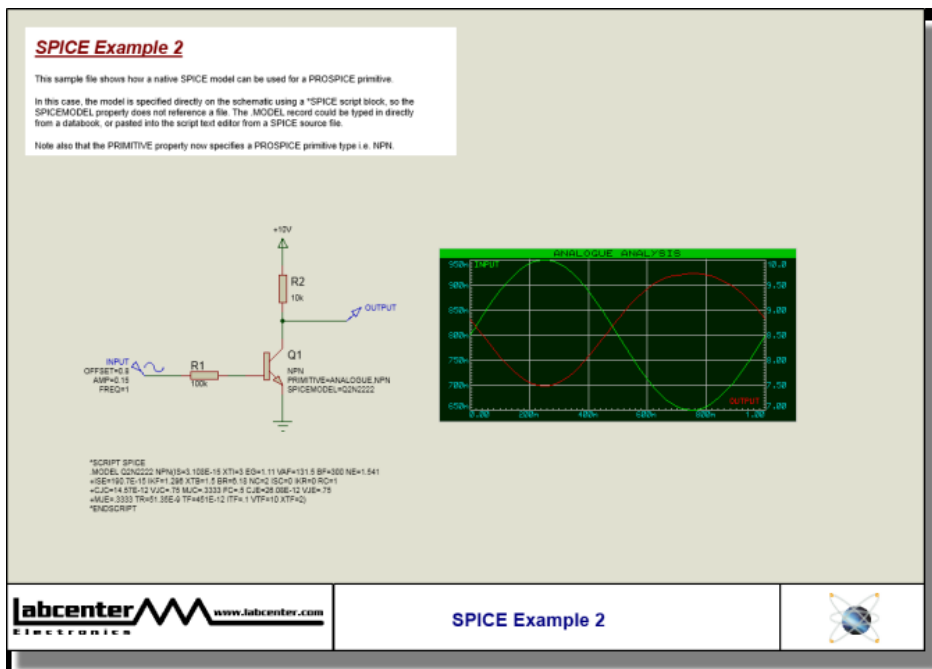
“图表”列出了引脚功能及其在 SUBCKT 线上的顺序。重要的是要认识到分配的数字与模型的内部定义有关，而实际上与我们的目的无关。关键信息是同相输入是第一个节点，反相输入是第二个节点，依此类推。这决定了如何构造 SPICEPINS 属性的值，因为您以相应的顺序指定了 ISIS 引脚名称。可以使用带空格的引脚名称，但是请注意不要在逗号前留下空格。或者，您可以将每个引脚名称都用引号引起来-如果您的引脚名称包含逗号，则这是必不可少的。

当然，您需要知道经常被隐藏的 ISIS 引脚名称。可以通过将鼠标移到组件的引脚端上并观察状态线来建立它们。

10.3 使用一个 SPICE 模型（模型卡）

对于半导体器件，特别是二极管，三极管，JFET 和 MOSFET，SPICE 模型通常用单个模型卡来定义。它列出各个 SPICE 原型的参数值。这个过程类似于 SUBCKT 模型，只是由于没有引脚名的转换而简单一些。需要补充的是，在 ISIS 器件库中已经存在不同类型的 SPICE 原型——存在于 ASIMMDLS.LIB 中。

第二个例子是一个 2N2222 三极管（ASCII 文件 SPICEMOD.LIB 中的器件），设计文件是 SPICE2.DSN，在 samples 目录下。



三极管有两个属性——PRIMITIVE 和 SPICEMODEL。

分别加以解释：

◆ PRIMITIVE=ANALOGUE,NPN

这个属性表示该器件由 PROSPICE 使用 NPN 型原型直接仿真。如果你从 ASIMMDLS.LIB 中选取的一个二极管，三极管，JFET 或 MOSFET，就已经有了 PRIMITIVE 的属性赋值。

◆ SPICEMODEL=Q2N2222,SPICEMOD.

LIBSPICEMODEL 的赋值与使用 SUBCKT 模型时一样。它指定模型的名称以及包含该名称的 SPICE 网表文件。大多数厂家在一个文件中提供很多的模型定义。SPICE 模型文件搜索路径是本级目录或者在 Set Paths 对话框中的 Module Path 所设置的路径。

10.4 SPICE 模型库

PROSPICE 提供的第三方 SPICE 模型库是与 ISIS 器件和符号库一样的二进制文件。这样做有两个理由：

- ◆ 许多文件非常小且是数字表示，对于大容量硬盘，如果分别存放会浪费很多硬盘空间。
- ◆ 如果许多模型保存于一个 ASCII 文件，PROSPICE 会因被迫分析整个文件而变慢。

根据 SPICE 模型在 SPICE 模型库 (SML) 中的位置，ISIS 库器件与模型绑定的语法略有不同。例如，第一个例子中的 LMC660 可以替换为如下属性：


```
PRIMITIVE=ANALOGUE,SUBCKT (as before)
SPICEMODEL=LMC660
SPICEPINS=POS IP,NEG IP,V+,V-,OP(as before)
SPICELIB=NATSEMI
```

SPICELIB 属性给出包含该器件模型的 SPICE 库文件。它的搜索路径是本级目录或者在 Set Paths 对话框中的 Module Path 所设置的路径。

SPICE 模型库可以使用命令行工具 PUTSPICE.EXE 和 GETSPICE.EXE 来管理,虽然我们不期望初学者用很多创建或者搜集的模型来获取授权。这两个程序不带参数运行会出现有用的信息。

10.5 *SPICE 脚本

实际上也可以在 ISIS 中直接键入 SPICE 模型定义,然后调用原理图中的相关器件。例如,三极管 2N2222 的模型可以用如下 ISIS 脚本表示:

```
*SCRIPT SPICE
.MODEL Q2N2222 NPN(IS=3.108E-15 XTI=3 EG=1.11 VAF=131.5 BF=300 NE=1.541
+ISE=190.7E-15 IKF=1.296 XTB=1.5 BR=6.18 NC=2 ISC=0 IKR=0 RC=1
+CJC=14.57E-12 VJC=.75 MJC=.3333 FC=.5 CJE=26.08E-12 VJE=.75
+MJE=.3333 TR=51.35E-9 TF=451E-12 ITF=.1 VTF=10 XTF=2)
*ENDSCRIPT
```

三极管的 SPICEMODEL 属性改为: SPICEMODEL=Q2N2222

不需要指定文件名称。

这个功能非常适用于书面得到的模型或者想试验一下参数值的影响的情形。

10.6 模型仿真失败的处理

由于子电路模型具有很高的复杂性和设计难度。结果导致某些模型比其他模型易于仿真。作为一般性准则,如果 PROSPICE 不能通过它翻译的模型仿真你的电路,那就没更多其他工具能够仿真了。

我们经常发现一些问题是因为较差的电路设计所致。如果你有器件的应用笔记,可以对照检查一下自己的电路,例如,你是否成功地在输入级馈入了足够的偏置电流。模型的设计者总是参照应用笔记设计而不是一般情形来设计模型。

通过提高 GMIN 可以使 ProSPICE 可以变得更加稳定(但是牺牲了精度)。默认值是 1E-14,因此最好试试 1E-13 和 1E-12。如果高于 1E-6,仿真结果将与实际情况相去甚远,所以应该尽量取最小值。

第十一章 进阶

11.1 温度模型

SPICE3F5 提供了温度效应的扩展模型支持。运行机制如下：

有一个应用于电路中所有器件的全局属性 TEMP。

单个器件的温度属性可以由设置其自己的 TEMP 属性来设定。

如果器件参数与测试温度密切相关，可以全局或单独地通过 TNOM 属性设置为该温度。

电阻、二极管、JFET、MOSFET、BJT 和 1、2、3 级 MOSFET 都具有温度模型。BSIM 模型是在特定的温度下测得。数字原型没有应用温度特性。

然而，重要的是要意识到大多数 IC 的等效电路模型没有正确地对温度效应建模。因为这些模型通常使用理想受控源和其他宏建模原型，而不是包含实际的器件。一旦使用这些原型，模型是不可能显示正确的温度特性的，除非有人不厌其烦地使其能做到。

11.2 固化模型数据

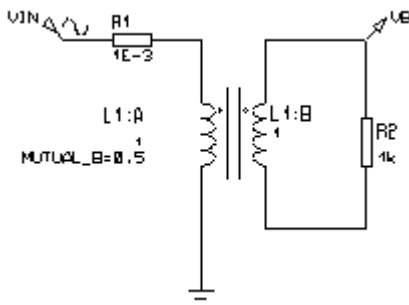
特定的仿真模型，如 EPROM 和带 EEPROM 的单片机或者 Flash 存储器，可以在仿真中“记忆”数据，正如它们在实际中那样。这个动作来自于 MODDATA 属性。固化模型数据块保存在 DSN 文件中，因此在保存设计之前请不要切换 Proteus 任务。

要将固化模型数据还原到初始值，可以使用 Debug 菜单下面的 Reset Persistent Model Data 命令。

11.3 地和电源范围

为什么需要一个地

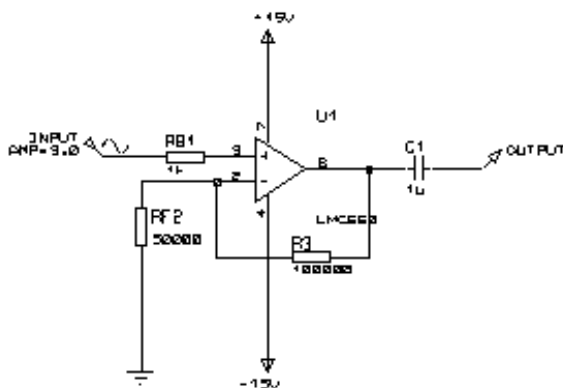
所有仿真都要求定义一个地网络——否则在特定网络上放置探针就没有意义，因为该网络的电压必有须固定的参考点。实际上，更深一层的要求是所有电路元件都应该有对地直流通路，用探头接在浮空的网络上同样没有意义。例如，下面电路中的 VB 是没有意义的：



因为次级是浮空的。理论上，我们可以使用双端探头（比如实际的万用表），但是对没有地的电路求解在数学上是非常困难的，再者，伯克莱大学在 SPICE3F5 中也没有考虑这个。

关键就在于电路中的各个部分都必须有对地直流通路。值得庆幸的是 ProSPICE 会检查这个问题，当网络中存在违反规则将发出警告。多数情况下仿真会因此失败。

下面是另一个导致过问题的电路：



这里，由于耦合电容 C1 的存在，使得 OUTPUT 探针没有对地直流通路，因而仿真器无法求解输出的工作点，因为工作点是在所有电容开路的情形计算的。我们使电容轻微漏电来解决此问题。因此，在没有任何直流通路的情况下，工作点基于 C1 完全充电状态计算。

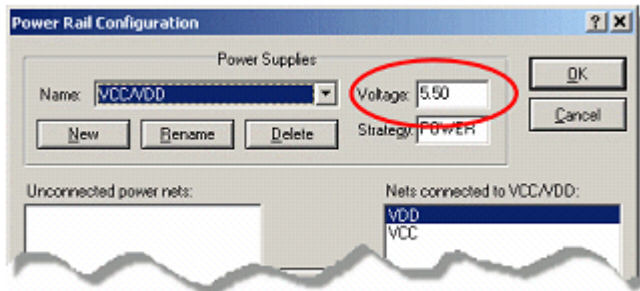
电容的漏电特性由仿真控制属性 GLEAK 定义，默认值是 1E-12Mho。传统的 SPICE 仿真器中，GLEAK 设置为 0 表示电容不漏电。

除了仿真 DRAM 存储器内部构造，我们无法发现这个电路有什么问题，这也避免初学者面对许多奇怪的错误信息。任何情况下，实际的电容器通常有愈一百万兆欧的漏电电阻。

地线网络在电路中既可以显式定义也可以隐含定义。显式定义是放置一个无网络标号的地线端子。比如上图 RF2 的下端。如果空间紧张，也可以通过放置 GND 网络标号来定义。隐含定义在使用电源范围、单端信号发生器、负载探头或数字输出时产生。在使用包含内部接地节点的模型也会用到隐含接地。

电源

一般可以使用 ISIS 中的 Design 菜单中的 Power Rail Configuration 对话框来指定仿真电源电压。



使用这个表格有两个重要步骤:

1. 使用 NEW 按钮生成一个或多个电源。每生成一个电源,可以在 Voltage 栏设置其电压。
2. 连接一个或多个电源网络到电源,在左边列表选中高亮的网络名并点击 Add 按钮。

同时需注意:

电源网络通过在原理图中放置电源端子或者设计中使用隐含电源脚的器件时形成。

不能对一个存在未连接电源网络的设计进行仿真——仿真将导致网表编译器错误。

也可以放置名称为+5, -5 或+10,-10 的电源端子来定义电源。“+”和“-”符号是必需的,例如 5.0V 是不允许的。

ISIS 提供了默认电源范围: VCC/VDD 为+5V, VEE 为-5V。VSS 和 GND 设定为内部短路的。这样设定使得像多数 TTL、CMOS 和处理器件仿真时不需要显式定义任何电源连接。

更多关于电源范围设置的信息可以查阅 ISIS 手册。

11.4 初始条件

前言

无论任何时候进行任何仿真,PROSPICE 首先做的是计算电路的工作点——那是加入任何输入信号前应用的稳态条件。这些值的计算有两种不同的操作模式:

工作点在所有电容充电和所有电感放电情形获得。这时,瞬态分析将显示电路加电足够时间以后的行为,仿真开始时电路已处于稳态。这是默认的操作模式,我们可以指定特定器件和(或)节点电压的初始条件,这是常见的情形。

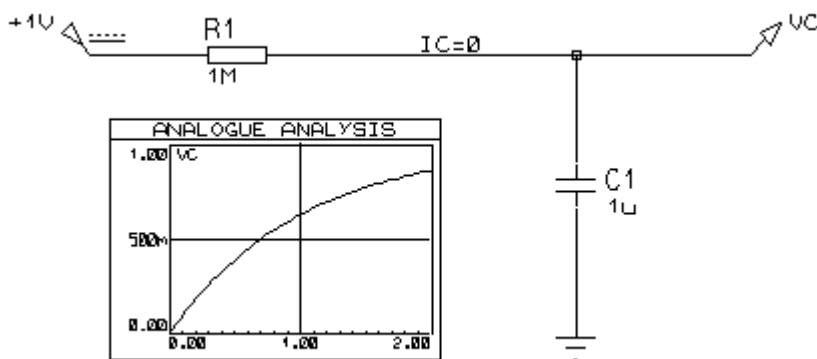
工作点在所有电容短路和所有电感开路情形获得。这时,瞬态分析将显示电路加电瞬间的行为。

这种操作模式可以通过不选图表属性的 Compute Operating Point 选项来实现。

在两种情况下，都可以指定特定器件或节点电压的初始条件。这在电路为振荡器，或者某些依赖于特定电容处于放电状态而工作的电路中特别有用。确实，对于振荡器来说，稳态工作点是没有意义的，仿真也会因为缺少某些初始条件而完全失败。

设定一个网络的初始条件

最容易的做法是标示一个网络的起始电压。



上面电路中，初始条件用加在探针网络的 $IC=0$ 的网络标号实现。如果没有这个设置，PROSPICE 会计算 C1 的稳态电压（如 1V），图表中 VC 就会是一条水平直线。

对于纯数字器件的网络，应该使用逻辑状态作为初始条件，例如：1,0,H,L,HIGH,LOW,SHI,WHI,SLO,WLO 或 FLT，并赋予 BS (Boot State 启动状态) 属性。对于混合网络，应该设定初始电压——对于数字器件会自动转换成逻辑电平。

设定器件的初始条件

该选项只用于 PROSPICE 不计算初始工作点的情形——如图标中 Initial DC Solution 选项未被选择时。此时除了用上述方法设定了初始条件的外所有节点电压初始为 0。在此条件下，就有可能设定特定器件的初始条件。例如，可以在前面例子中添加属性

$IC=1$

到 C1 使得 C1 从它的稳态电压 1V 开始仿真。

关于 IC 属性所支持的不同 SPICE 原型参见 Modelling 有关章节。

工作点的计算方面没有这个选项是比较难堪，但是 Berkeley 的 SPICE3F5 确实这样编程的，我们增加 PRECHARGE 属性对此作为补救。

NS (NODESET)属性

有时候，当因为无法确定工作点而失败，有益的做法是给 SPICE 一个特定网络初始值的提示。这于设置初始条件不同，这些值只用于首次迭代运算，然后该网络“浮动”到矩阵求解的收敛值上。这样有助于收敛并且不会对实际的工作点求解产生影响。

这样的收敛提示可以通过 NS 网络属性设定，因此放置一个 NS=10 的网络标号就假定该网络起始值为 10V。

PRECHARGE 属性

PRECHARGE 是设定初始条件的另一个选择。它可以设置给电路中的电流或电感，用以分别设定跨越器件的电压或流经器件的电流。

PRECHARGE 属性是 Labcenter 对 SPICE 专门补充的，与 IC 属性不同，PRECHARGE 属性的应用与 Initial DC Solution 选项的选择与否无关。

11.5 数字仿真范例

九态逻辑

你也许认为数字仿真器只需要对高、低电平建模，而事实上 DSIM 是对总共 9 种状态进行建模的。

State Type	Keyword	Description
Power High	PHI	Logic 1 power rail.
Strong High	SHI	Logic 1 active output.
Weak High	WHI	Logic 1 passive output.
Floating	FLT	Floating output - high-impedance.
Undefined	WUD	Mid voltage from analogue source.
Contention	CON	Mid voltage from digital conflict.
Weak Low	WLO	Logic 0 passive output.
Strong Low	SLO	Logic 0 active output.
Power Low	PLO	Logic 0 power rail.

一般来说，一个给定的状态包含的信息有它的极性——高、低或中间态，和它的强度。它的强

度是以两个或更多输出接在同一网络上时的灌出电流或吸入电流的量来计算的。

例如，如果一个集电极开路输出经一电阻接 VCC，那么当输出被拉低时，网络会同时出现弱高电平和强低电平。强低电平获胜，网络变为低电平。另一方面，如果两个三态输出在一个网络上同时生效，并且分别向相反的逻辑方向驱动，那么双方都不能获胜，并导致一个竞争状态。

这种设计保证 DSIM 可以仿真有集电极开路或发射极开路的电路以及上拉电阻，也可以仿真通过电阻相连的三态电路。重要的是要记住 DSIM 只是一个数字仿真器，它不能对模拟行为建模。比如用很大的电阻接到 TTL 输入时，DSIM 仿真是正确的，而可能在实际中因为没有足够的驱动电流而失败。

不确定状态

何时一个数字模型的输入是不确定状态，这是数字模型根据共同的准则而产生的。例如，如果一个与门有一个低电平输入，那么它的输出将是低电平，然而当除了一个输入外其他输入为高电平时，它的输入是不确定状态，其输出也将是不确定状态。

- 边缘触发器件需要一个从逻辑 0 到逻辑 1（或相反）的边沿。从逻辑 0 或 1 到不确定状态并不是一个有效的边沿。
- 更加复杂的时序逻辑器件（计数器、锁存器等）由他们的内部逻辑的设计决定了会经常不确定输入是 0 还是 1，这与实际中是不同的。

浮空输入行为

这是共同的，如果不是把未接的 TTL 输入都看成接逻辑 1。这种情况发生在忽略连线或者一个输入接在一个未激活的三态输出上。因为内部模型设定需要高电平或低电平的真实逻辑行为，DSIM 不得不做出一些行为。

这个通过 FLOAT 属性来设置。它既可以赋给一个器件，也可以赋给一个接口模型。特别的，对 TTL 部件的赋值是：

FLOAT=HIGH

这样将浮空输入状态翻译成逻辑电平 1。

要设定浮空输入状态为逻辑 0，可赋值：

FLOAT=LOW

否则，浮空输入会被识别为不确定状态。参见前述。

毛刺的处理

在设计 DSIM 过程中，关于如何在非常窄的脉冲下的模型进行仿真，我们讨论了很久。根本的问题是，在这种情况下，一个主要的 DSIM 范例的假设即模型纯的数字行为——开始被破坏了。例如，对一个真实的 7400 施加一个 5ns 的输入脉冲会在其输出产生某种脉冲，但是不符合 TTL 逻辑电平的定义。

该输出脉冲是否能够驱动下一级的计数器，这是一个模拟现象的问题了。

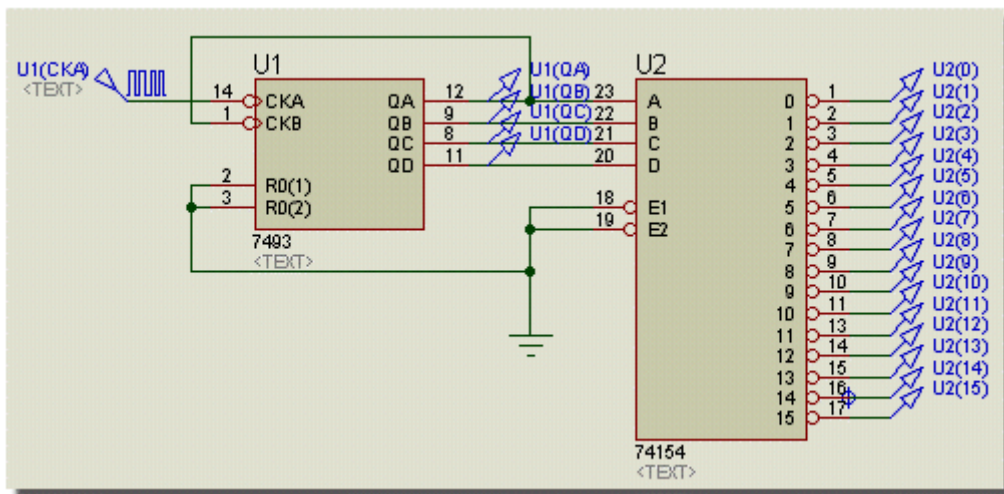
最好的办法是考虑极端情况，命名：

一个 1ns 的输入脉冲将不产生输出。

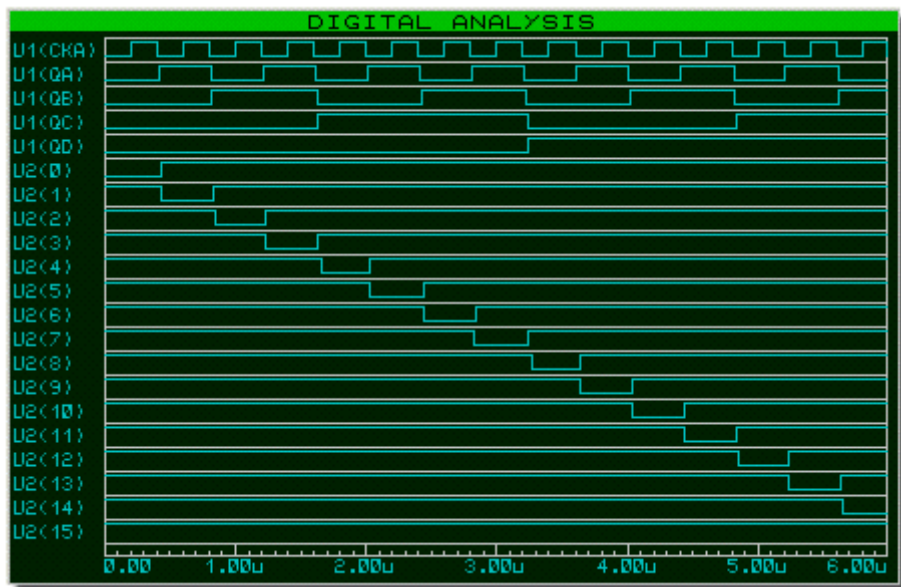
一个 20ns 输入脉冲可以完美地输出。

在其间，门电路停止输出脉冲，可以被认为是抑制毛刺。这给我们一个 Glitch Threshold Time 的概念，作为模型除 TDLH 和 TDHL 外的补充属性。

模型是否抑制毛刺的值得关注。为了解决这个问题，考虑下面一个由跳跃计数器驱动的 4-16 译码器：



计数器的输出是阶梯状的，出现这样的可能：译码器会在中间状态产生杂散的脉冲输入，见下面图表：



以这种现象的第一个毛刺为例，在 U1 (QA) 第一个下降沿，它驱动 U1 (QB) 的上升沿，一个中间输入状态 0 送到译码器约 10ns。问题在于译码器是否对此响应，更重要一点，如果输入步长变成 1ns 或 1ps 后会出现什么情况？显然，在后面两种情况实际器件是不会作出反应的，这告诉我们必须在输出处理毛刺而不是在输入。这是因为在上面例子中，输入脉宽都是相对较长而不会被当作毛刺。

特别有趣的是，如果你创建上述电路，也许没有毛刺。当然这是一个差劲的设计，但是 74154 的 TDLH 和 TDHL 大约 22ns 使其阻止的对 10ns 的输入的响应。对于我们单独试过的器件都没有输出，除了电源的细微抖动是可测的。

为了提供对毛刺的控制，所有 DSIM 原型提供给用户可定义的毛刺门限时间属性，叫做 TGxx，xx 表示相对应的输出。我们的 TTL 模型定义这些属性可以在 TTL 器件上取其默认值，因此毛刺门限时间是高一低和低一高传输延时的平均值。把毛刺门限时间设置为零将允许所有毛刺通过。上图中，设置了 74154 的属性 TGQ=0。

最后需要重点指出，如果毛刺门限时间比低一高或高一低传输延时大，那么它将被忽略。这是因为在一个输入边沿后，当相对传输延时过了以后，门输出必须转换其状态——不可能看到将来是否有另外一个输入事件（那将会取消当前输出）。考虑一个对称的门电路，其产生的延时是 10ns，毛刺门限时间是 20ns。在 $t=0\text{ns}$ 输入变高， $t=15\text{ns}$ 时输入变低。你也许期待能有这样的输出：输出在 $t=10\text{ns}$ 时为高然后在 $t=25\text{ns}$ 变低。因此产生一个 15ns 将会被抑制的脉冲输入，因为它小于毛刺门限时间。这个脉冲没有被抑制，因为在 $t=10\text{ns}$ 时输出必须变高——无法在后续的 20ns 保持为低。

11.6 混合模式接口模型（ITFMOD）

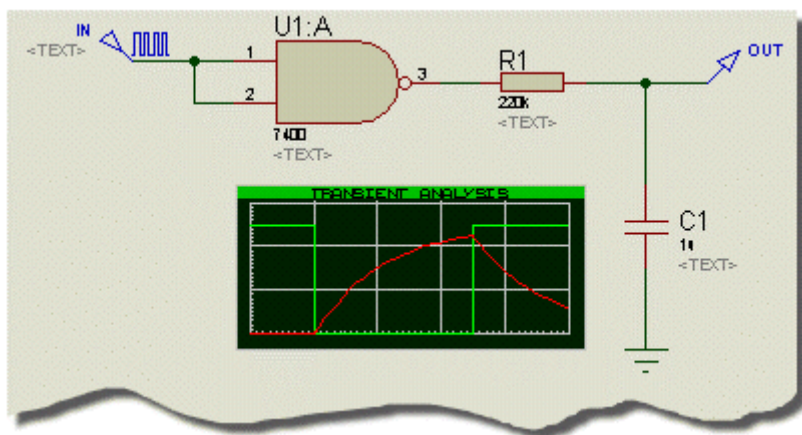
概述

在设计 PROSPICE 下混合模式仿真架构时,我们深思了如何确定数字器件的模拟特性这个问题,这些特性包括:

- 器件的输入输出阻抗
- 器件输入的逻辑门限
- 高低电平电压
- 器件输出的上升和下降时间
- 浮空输入的默认逻辑状态

对于 TTL 库中所有器件都能涵盖以上特性的架构是极其臃肿的。

再者,一个关于电源指标的严重问题产生了(至少对于初学者)——如果低于某个敏感值将会使电路失效。该问题在于我们隐含了 7400 有通过隐含电源脚接到 VCC/GND 而获得 5V 电源。



所有这些问题因 ITFMOD 属性的引入迎刃而解。很像 MODEL 属性,它提供了一系列属性参考值,还在网表编译器中激活一个特殊的机制。它实质上是这样工作的:

- 对于任何具有 ITFMOD 属性的器件,一个附加的模型定义在网表中被调用,用以设定 ADC 和 DAC 对象的控制参数,甚至正负电源的引脚名称。在上面电路中 U1:A 具有属性 ITFMOD=TTL。
- 获得电源引脚名称后(本例中的 VCC, GND),ISIS 生成一个特殊的原型跨接在电源引脚上。ISIS 像子电路或模型那样命名此对象,因此在上电路中,电源对象被命名为 U1:A_#P。
- 当 PROSPICE 仿真一个混合模式电路时,生成 ADC 和 DAC 对象并认为它们属于相连接的器件。在上面电路中,将产生一个名为 U1:A_DAC#0000 的 DAC 对象以形成 U1:A 的输出接口。
- 这是一个明智的做法,它以名称前缀来定位电源接口对象如 U1: A 的对象是 U1: A_#P。然后指引 U1:A_DAC#0000 从 U1:A_#P 中依次获得其属性,这些属性继承了模型在原始 ITFMOD 设定的属性。这样 DAC 对象便以 TTL 逻辑定义的参数运作。

- 电源接口对象也像在上层部件那样连接在其他同电压的电源引脚，从器件的电源引脚吸入或流出电流，使得数字器件的阶梯状输出的建模更加接近真实情形。

使用 ITFMOD 属性

已有的接口模型如下：

Property	Description
TTL	Standard TTL (74 series)
TTLs	Low power Schottky TTL (74LS series)
TTLs	Standard Schottky TTL (74S series)
TTLHC	High Speed CMOS TTL (74HC series)
TTLHCT	High Speed CMOS TTL with TTL outputs (74HCT series)
CMOS	4000 series CMOS.
NMOS	Microprocessor type MOS circuits with TTL logic levels.
PLD	PLD type MOS circuits.

此后任何新的数字模型通过设置以下属性赋予一个器件系列：

ITFMOD=TTL

各器件系列的定义在 models 目录中的文件 ITFMOD.MDF。

每个定义可以包含任何或所有 ADC 和 DAC 接口原型的属性定义。补充一点，下述属性可能需要给定：

Primitive	Value Example	Description
V+	-	Name of the positive power supply pin.
V-	-	Name of negative power supply pin.
VOLTAGE	5V	Specifies the default operating voltage.
RINT	1mΩ	Specifies the impedance of the internal battery. A value of zero will disable the battery.
FLOAT	-	Specifies HIGH or LOW value for floating inputs.

最后值得指出的是任何特定的属性，如 TRISE，可以被上层器件改写，因此如果你想以低上升

时间仿真一个 4000 系列的 IC，可以在器件属性列表添加属性 $TRISE=10u$ 。

11.7 录音机和电路分区

概述

VSM 系统的一个基本功能是可以把一个大型的设计划分为一个或多个部分或单元并分别进行仿真。

这样做有以下两个主要优点：

- 在完全集成的 CAD 实验中，产品整个设计电路中可能含有某些你不想或者不能仿真的部分。为了避免割裂整个设计，需要某些机制来定义哪些元件需要参与仿真。

ISIS 通过判断测量点来决定仿真区域，测量点由测试信号源或/和电源驱动。

- 如果设计是多级的，一般要求观察后级根据前级的输出而作出的表现。当然可以让所有级电路一起仿真，但这样会非常的慢。电路分区允许录音机捕获前级的仿真结果，然后作为后级电路的输入重方出来。

ISIS 包含可通过人工或自动的方式这样做的逻辑。在后一种情形，它检测给定区间的电路变化，并对变化部分或者受变化影响的部分进行再仿真。

单个部分的仿真

许多仿真实验需要摆脱其他器件测试设计中的局部电路。一般可以通过在该部分注入信号，然后观察信号通过的情况。

显然，信号可以通过放置激励源和相应的导线来注入，电路行为可以通过放置探针来监视。然而，这些并不能使该部分电路脱离设计中的其他部分。

要想做到这样，你需要设置激励源的 **Isolate Before** 对话框和探针的 **Isolate After** 对话框。一旦这样做了，那么仅对分区出的部分电路建立网表并仿真。

了确定对那些部分仿真，ISIS 分析放置在图表的探针，依据以下条件提取器件和导线（包括内部端口连接）：

- 没有其他需要处理的器件。
- 遇到一个隔离的探针或激励源。
- 遇到录音机的输入或者输出。
- 遇到特定电源范围。一个网络通过连接到 POWER 或 GROUND 端口被划分到特定电源范围。GND 和 VCC 网络标号没有此功能。

录音机对象

录音机对象在系统中有两个独特的用法：

- 确定仿真点，只对左边的电路进行仿真，隔离右边电路。这些仿真点一般是低阻抗输出驱动高阻抗输入。这个也可以通过放置隔离探针实现。
- 表示可捕获设计中一级电路的输出，用来驱动后一级电路，使后级电路的仿真时就没有前级仿真的开销。

放置一个录音机：

1. 选择录音机图标；
2. 使用旋转和镜像按钮根据需要确定录音机的方向。录音机上的小箭头符号表示其方向；在正常方向下，输入在左输出在右。
3. 单击左键进入编辑窗，移动并再次单击将录音机放置在需要的位置上。
4. 可以直接把录音机放在现有导线上，录音机的连接点与导线相触。你也可以把若干录音机放在设计窗的空余地方，然后用导线连接到电路具体位置。

录音机必须放置在敏感位置——必须是低阻抗驱动高阻抗。如果你将录音机放在其他位置，将会改变电路行为而导致无效的仿真结果。

录音机模式

为了提供用户最大限度的可控性，录音机有三中工作模式：AUTO, PLAY and RECORD 在后面的讨论中，我们使用左、右来描述电路分区的逻辑关系。设计的输入、输出描述为左、右。

AUTO 模式

这是默认架构，根据 ISIS 判定哪个部分需要再仿真来选择操作模式，可以使用预先保存的数据。在大多数需要使用录音机的情形，AUTO 模式是最有用的。

自动检测是基于相关部分的子网表中出现的文本。如果任何部件名称、值、属性、连线等发生变化，就进行再仿真，除非电路分区文件已经含有那些信息。

注意所有模型、脚本、全局设计属性等都包括在子网表中，因此这些对象的改变会导致自动分区单元的再仿真。ISIS 不会考虑分区部分的器件是否使用特定的模型、脚本等等。如果想克服此问题，可以使用人工的 RECORD 和 PLAY 模式。

还要注意 AUTO 模式的录音机在电路的交互仿真时会被移除。

PLAY 模式

这个模式是你可以重放先前录制的文件，该文件既可以通过录音机，也可以通过带 RECORD 属性的探针录制。将需要重放的文件名设置到录音机的 Filename 中；只有输入文件名后才能使用 PLAY 模式。

当录音机处于 PLAY 模式，其左边电路被断开并忽略，除非存在应用于当前图表的探针。

记住你只能播放几路的电路分析数据，尝试播放其他文件将导致错误。

RECORD 模式

这个模式使得出现在录音机输入的数据以 **Filename** 中设定的文件名被记录下来；只能在设定了文件名才能使用 **RECORD** 模式。

RECORD 模式的另一个影响是强制对录音机左边的部分进行再仿真，而与其是否发生变化无关。如果录音机右边电路被放置探针，那么它也将被再仿真，因为隐含认为它依赖于左边部分。

当你想记录一段仿真波形，作为未来后级电路仿真的输入信号时，录音机的 **RECORD** 模式显得非常有用。

11.8 仿真控制属性

概述

有大量的参数影响着仿真运行的细节。包括搜索工作点的最大允许迭代次数、判定结果收敛的误差、使用的积分方法等等。

这是所有分析的通用的选项，可以分别在每个图表中通过点击 **SPICE Option** 来进行调整。

误差属性

这一类参数决定了 **SPICE** 求解的精度。精度越高则仿真时间越长，如果要求误差太高经常会导致无法收敛。

这里最为有用的值是截断误差系数，在传统 **SPICE** 术语中叫 **TRTOL**。如果你的结果显得非常糟糕或者粗糙，你可以尝试减小这个值。

最小电导值 **GMIN** 定义了反偏 **PN** 结和一些理论上电阻无穷大的漏电特性。减小这个值也许会有助于仿真失败电路的收敛，但是以牺牲仿真精度为代价。参见 **Convergence Problems** 。

漏电导 **GLEAK** 我们提到过是用来求解含隔直电容的电路，它定义电容器的直流漏电，通常可以不用管它，除非你仿真像 **CMOS** 存储器单元那样的特殊的情形。

MOSFET 属性

MOSFET 的 **SPICE** 仿真是基于假定你在做 **IC** 设计，必然应用规则几何图案的框架。实际上这意味着有很多的参数来定义 **MOSFET** 器件单元的默认物理尺寸，这些就是这里定义的值。

补充一点，某些模型根据旧版本 **SPICE** 行为来创建，如果你使用它们，可以开关切换其 **MOSFET** 行为。

迭代属性

Iteration 选项卡下的属性定义 **SPICE** 如何处理难以收敛的电路。

积分方法可以是 Gear 或者 Trapezoidal。后者主要向下兼容早期版本的 SPICE，Gear 积分方法则在给定步长下获得更高的精度的结果。在 Gear 积分中，高于 2 的阶数是允许的，意味着 SPICE 使用某个时间点更多的历史数据来预测下个时间点。

当 SPICE 在工作点无法收敛时，它做两个尝试：Gmin 单步和 Source 单步。每个算法的尝试的步数在这里设置。

跟着 3 个选项定义每个工作点的最大迭代次数、传输分析步长、瞬态分析时间点。这些值的提高有助于求解难以稳定或接近不稳定的电路。

最后，有两个选项用来获得快速的仿真。LTRA 压缩仅用于含传输线损耗（LOSSYLINE 模型）的电路。数据点因数据流水线上舍弃接近相同的数据而被处理。绕开未变化的单元以免 SPICE 再计算那些节点电压从上次分析后没发生变化的器件。

温度属性

有两个全局的温度属性：TEMP — 默认的工作温度，TNOM — 参数测量温度。TEMP 定义电路的实际温度，TNOM 则是器件参数测试条件时的温度。关于 PROSPICE 下温度建模的讨论参见温度建模部分。

数字仿真器属性

TDSCALE, TDSEED, TDLOWER 和 TDUPPER

TDSCALE 变量用于控制仿真运行时模型的全部定时属性值的大小，这些属性并没有被模型明确地引用为非量化的。TDSCALE 变量既可以赋予一个浮点常数，也可以赋予关键字 RANDOM。

如果指定了一个常数值，那么模型设定的所有温度属性值将乘以此值；小于 1.0 的常数减小了定时属性值，反之提高属性值。例如，变量赋值：

TDSCALE = 1.1

仿真时扩大了所有定时属性值 10%。TDSCALE 的默认值是 1.0；这使得所有定时属性为原值。

如果 TDSCALE 被赋予关键字 RANDOM，DSIM 引擎将使用一个随机的浮点数乘以每个定时属性值进行随机定标。仿真引擎选择的定时定标数值范围受限于 TDLOWER 和 TDUPPER 变量；它们分别定义随机浮点数的允许最大和最小值。TDLOWER 和 TDUPPER 的默认值分别是 0.9 和 1.1，因此限制了随机定标值在 10%之内。

产生的随机数序列被称为伪随机，因为每产生一个成功的值，看上去是随机的，其实是从前面数值预测而来的（用复杂的公式）。任何随机序列都是由一个种子值决定的，也就是说，对于同样的种子值，产生的随机序列都是一样的。TDSEED 变量允许你指定一个种子值来产生随机的定时定标序列，以保证各个仿真运行一致。这样避免在成功的仿真运行时由于随机产生延时的出现和消失带来定时设计错误。

TDSEED 变量只能设置 1—32767 以内的正整数。TDSEED 的默认值本身是一个随机数（由时间和日期决定），这意味着一个仿真运行产生的一系列值相对于下一仿真是随机的。

例如对如下赋值：

TDSCALE = RANDOM

TDLOWER = 2.00

TDUPPER = 3.00

TDSEED = 723

使所有定时属性值随机地增加了 200—300%。种子值是 723，它使得每次仿真运行都产生同样的伪随机序列。

INITSEED

INITSEED 属性用于作为随机初始化值发生器的种子，被那些初始化属性设定为 RANDOM 关键字的 DSIM 原型所使用。

与前面提到的 TDSCALE 和 TDSEED 属性一起，由随机初始化值产生器生成的序列是随机的，序列整体是有限和确定的。INITSEED 属性意味着 DSIM 仿真器选择哪个序列随机数。

INITSEED 属性只能被赋予 1—32767 范围内的正整数。其默认值本身是一个随机数（由时间和日期决定），这意味着一个仿真运行产生的一系列值相对于下一仿真是随机的。

11.9 仿真模型的类型

如何描述器件含有模型

PROTEUS 有超过 8000 个库部件，其中约 6000 个含有仿真模型。没有仿真模型的器件适用于 PCB 设计，要求所有器件都有仿真模型是不现实的。也就是说，设计类似 68020 处理器模型是没有太多实际价值的。

为了达到电路仿真的目的，你必须明白器件是否具有仿真模型。在第一个实例中，如果仿真就会失败，并有如下提示信息：

ERROR [PSM] : No model specified for 'U1'.

该错误是在电路分区阶段（PSM）检测出的，因为仿真电路部件中不允许存在没有模型的器件。有时也会出现一下信息：

ERROR [U1] : Value '74F00' of VALUE not found in parameter mapping table.

这意味着该器件存在模型文件，但是你改变的器件类型值与 MDF 文件不相符。在上例中，我们把 74LS00 门电路（已经建模）改成了 74F00。

器件适用的仿真模型的类型显示在器件库浏览器预览窗口的右上角，例如，如果你打开器件库浏览器并在 74LS 库中选择 74LS00 器件，你会看到

Schematic Model [74NAND.MDF]

更多关于不同的模型类型的信息将在后面讨论。

原型模型（Primitive Models）

PROSPICE 中有大量的基本器件模型。这些器件类型被称为原型（Primitives），包括电阻、电容、二极管、三极管、逻辑门、计数器、锁存器、存储器等等。

原型模型（Primitive models）仿真时不需要额外的文件，只需要一个 PRIMITIVE 属性来识别。其他属性直接在器件上指定并通过网表传递给 PROSPICE。例如一个电阻的属性：

PRIMITIVE=ANALOG,RESISTOR

这定义此部件是一个 SPICE 电阻原型。

仿真原型的标准集存在于 ASIMMDLS 和 DSIMMDLS 库。这些部件都有属性的文字帮助，它们使用的实例参见 VSM SDK 文档。

原理图型模型（Schematic Models）

当仿真一个更加复杂的器件时，通用的做法是用仿真原型画一个描述其行为的电路。这个电路可以是器件的内电路，但是为了加速仿真，我们通常使用理想电流源、电压源和开关。

原理图模型由 MODFILE 属性来设定，为了方便我们创建了这个只读的属性。例如，对于 741 运放有如下设置：

MODFILE=OA_BIP

从这里你会发现一个模型文件可用于若干个器件——通过参数映射表（Parameter Mapping Table）来区分它们的功能。

原理图模型是在 ISIS 中绘制原理图创建的，通过 Model Compiler 生成 MDF 文件。更多的信息参见 VSM SDK 文档。

VSM 模型（VSM Models）

VSM 模型也是原型模型，不过它们使用 PROSPICE 以外的 DLL。这使得我们可以使用特定的编程语言来描述仿真器件的功能，通常最容易的是使用 C++。

一个 VSM 模型既有 PRIMITIVE 属性（因为 ISIS 和 PROSPICE 均把它当作原型），又有指定器件相关 DLL 文件名的 MODDLL 属性。

例如，8052 模型属性：

```
PRIMITIVE=DIGITAL,8052
```

```
MODLL=MCS8051
```

注意模型 DLL 文件可以用于多个原型类型——MCS8051.DLL 可应用于许多 8051 系列的处理器。

VSM 模型还可以应用于与动画相关的模型，这样使得器件操作上电气的和图形方面的表现惊人的一致。LCD 显示器就是最好的印证。

创建 VSM 模型涉及一系列 C++ 接口类（与 COM 类似）。这些参见 VSM SDK 文档。

SPICE 模型（SPICE Models）

由于 PROSPICE 基于伯克莱的 SPICE3F5，因此直接兼容标准的 SPICE 模型，PROTEUS 中许多器件都是由制造商提供的 SPICE 文件来创建的。SPICE 模型既可以通过 SUBCKT 块设定，也可以通过 MODEL 记录中一系列参数来设定。以 CA3140 为例，SUBCKT 模型将具有以下属性设置：

```
PRIMITIVE=ANALOG,SUBCKT
```

```
SPICEMODEL=CA3140
```

三极管 BC108 的 SPICE 原型模型有以下属性：

```
PRIMITIVE=ANALOG,NPN
```

```
SPICEMODEL=BC108
```

模型本身可以保存在一个 ASCII 文件中或一个 SPICE 模型库中。这些文件的名字在 SPICEFILE 或 SPICELIB 属性中设定。

更多内容参见“SPICE 模型的应用”一节。

11.10 如何提高交互仿真的速度

前言

虽然 Proteus VSM 可以进行许多实时交互仿真，但是显然不是对所有电路均如此，我们可以很完美地画出一个振荡在 1GHz 的电路，但是它不可能在一台机器指令周期超过 1ns 的计算机上实时仿真。在这个单元，我们将解释仿真复杂性的细节以及如何优化电路来最大限度地加速仿真。

使用数字的电阻和二极管模型

首先，我们必须知道 ProSPICE 中模拟和数字仿真的差别。因为数字电路仿真比模拟电路仿真快 2~3 个数量级（几乎 1000 倍）。因此，ProSPICE 包含一个数字仿真器，它以事件驱动过程来表示数字器件的操作，从而避免了大量不必要的计算。

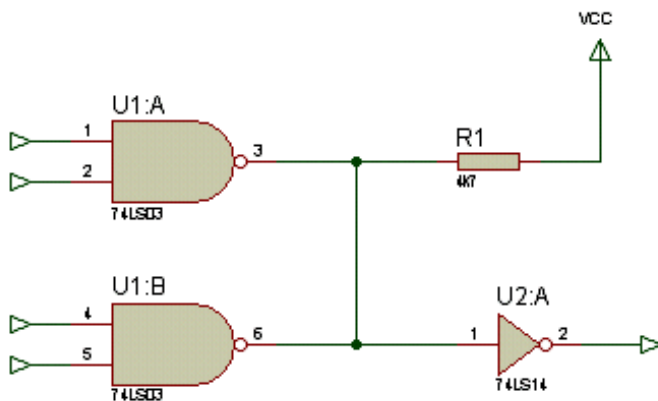
例如，一个 600MHz 的奔腾 III 处理器可以每秒仿真大约二百万个数字事件，同样的电脑在 CPU 负荷达到 100% 时却只能仿真 2kHz 下的正弦波振荡器。这样的波形需要每秒计算 60,000 个模拟测试点，每个测试点又需要通过电路节点方程的一组收敛解来获得——这比处理数字事件复杂很多。

对于许多器件，要认真考虑需要的是数字的还是模拟的仿真。例如，几乎所有的 TTL 和 CMOS 部件都是用数字模型表示的，而模拟 IC 如运放、比较器等等是用模拟模型表示的。所有用标准 SPICE 模型表示的器件都需要模拟仿真。

然而却出现一个灰色地带，某些器件现实是模拟器件，但是在特定情形可以表示为数字模型。二极管甚至电阻就落在了这个范围。这与线或逻辑、上拉电阻、集电极开路输出器件以及二极管——电阻逻辑网络密切相关。

实例 1 线或逻辑

下面电路表示了一个典型的线或逻辑网络：



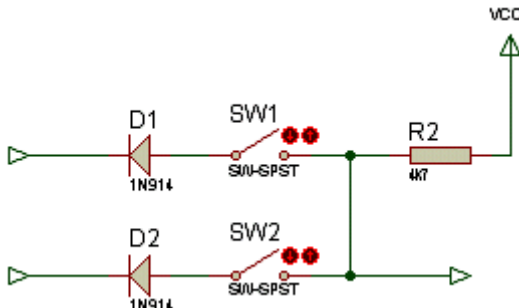
U1:A 和 U1:B 具有集电极开路输出，只能吸入电流。逻辑 1 输出导致一个高阻状态。按照 DSIM，这意味着门输出驱动一个 SL0（强低电平）或者一个 FLT（浮空）逻辑状态。如果电阻 R1 是模拟模型，PROSPICE 必须在电阻和逻辑门之间插入一个数模接口对象，并在电阻和 U2:A 的输入之间插入模数接口对象。这将导致对该节点起伏波形和流经 R1 电流的精确仿真，但是也会造成每当 U1:A 或 U1:B 输出状态改变时巨大的计算量。

所有这些可以通过对 R1 使用数字模型来避免。在这种情形，它的行为是将 VCC 的 SHI 逻辑状态转换为 WHI（弱高电平）逻辑电平。当两个与非们拉低时，SL0 压倒电阻上的 WHI 状态，网络状态变成低电平。但是当两个门都不吸入电流，WHI 状态击败 FLT 状态，网络升高为逻辑高电平。所有这些都受控于数字仿真阶段，不需要模拟仿真。

因此，任何时候的上拉电阻都应该是数字模型的。

实例 2 二极管电阻网络

另一种模拟电路可以使用数字模型的情况就是下面所示的二极管电阻网络。这经常出现在矩阵扫描键盘电路中，二极管用于防止多个按键同时按下时列驱动电路之间发生短路。



就像在线或逻辑例子那样，ProSPICE 会很自然地用模拟的方式建模仿真，但是必将导致巨大的计算量。如果上面使用数字电阻模型，二极管被当作只从阴极传递逻辑低电平到阳极和只从阳极传递高电平到阴极，那么上面电路同样就以数字模型建立了。

由于键盘扫描程序是以固定时间间隔进行的，常常使用大量的二极管和电阻，因此这种优化是极其重要的。

如何选择数字电阻模型

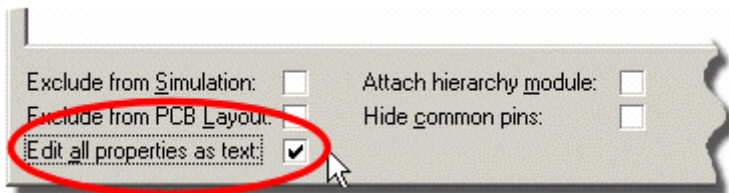
1. 鼠标指向需要改变属性的电阻，按 CTRL+E。
2. 点击 Edit All Properties as Text 选择框。
3. 改变 PRIMITIVE 属性为：

PRIMITIVE=DIGITAL,RESISTOR

如果是新建电路并且知道那些电阻需要数字模型的，你也可以通过从元件库中直接使用 PULLUP 或 PULLDOWN 模型。这些器件已经具有上述属性。

如何选择数字二极管模型

1. 鼠标指向需要改变属性的二极管，按 CTRL+E。
2. 点击 Edit All Properties as Text 选择框。

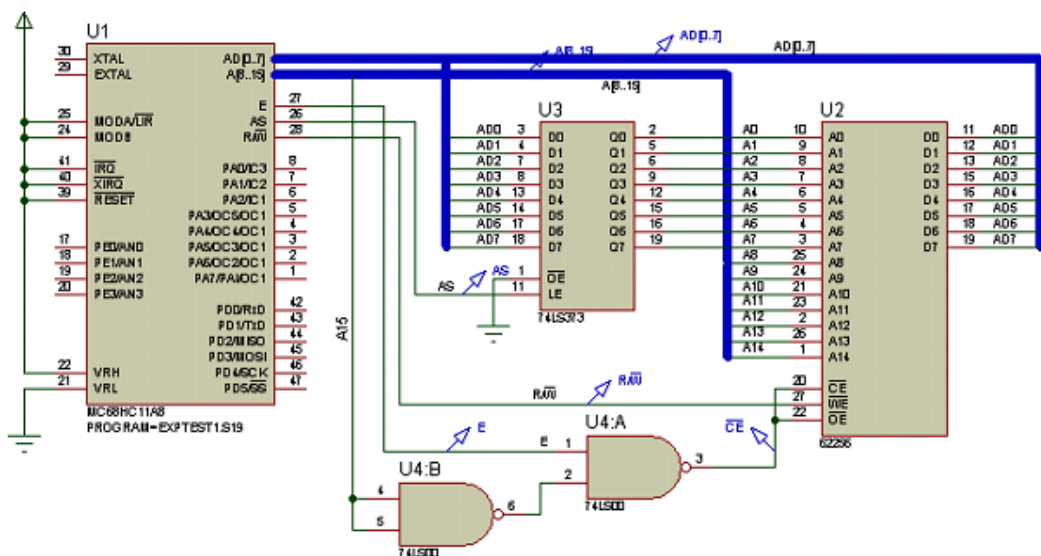


3. 改变 PRIMITIVE 属性为：
4. PRIMITIVE=DIGITAL,DIODE
5. 删除所有 MODEL 属性，因为 SPICE 参数此时没有意义。

如果是新建电路并且知道那些二极管需要数字模型的，你也可以通过从元件库中直接使用 DIODE-DIGITAL 器件。这个器件已经具有上述属性。

优化外部 RAM 和 ROM 的访问

许多较大型的单片机设计会用到外部的 ROM、RAM 或 EEPROM。它们用于存储代码或扩展单片机的 SRAM。给定一个正确的地址译码电路，假设存储器有仿真模型，Proteus VSM 可以正确仿真如下电路。当 CPU 访问外部存储器时，模型将驱动相应的地址、数据和控制线，外部译码逻辑和存储器将对相应位置的读写出反应。



如果你希望校验你的译码电路工作，那么这是非常有用的，但是同样会带来很大的计算量开销。设置一个 16 位地址将产生最少 16 个数字事件，从总线上读或写一个字节的数据又会产生 8 个数字事件。由于地址线和数据线是复用的，如上 HC11 的设计，将会产生更多的数字事件。所有这些与 VSM CPU 每机器周期运行一条指令的能力相比是极不适宜的。

因此，我们提供的 CPU 模型能够以内部访问的形式来仿真外部存储器的访问。目前应用在 8051、HC11 和较大的 AVR CPU 模型。外部存储器可以用 EXTRAM 或 EXTROM 属性加以声明，这些属性设定了存储器的每个存储块的范围。全部细节见特定芯片的相关模型帮助文档，可以通过 Edit Component 对话框或开始菜单访问。

一旦外部存储器这样定义了，对特定区间外部存储器访问的指令就可以不需要产生大量的数字事件来仿真。对存储器映射的外设访问也仍然可以完全仿真，因为它们位于 EXTRAM 和 EXTROM 设定的存储范围之外。

第十二章 错误处理

12.1 仿真日志

每当运行一个仿真就会生成一个仿真日志 (Simulation Log)。仿真日志文件包括所有有关仿真器和个别模型的信息、警告和错误消息。当一个模型产生一个消息，消息前缀是以方括号包括的模型器件参考名称，因此你也许会看到：

[U1] Loaded 26 files from PROGRAM.HEX

在交互仿真期间，这个文件的内容可以从 Debug 菜单下的弹出窗口看到，在图形仿真期间，这个文件可以通过鼠标指向图表并按下 CTRL+V 看到。

当仿真失败时，日志文件会自动出现使你可以立即知道问题所在。

12.2 网络表错误

网络表错误发生在 ISIS 为原理图创建网络表时出现问题，你也会在试图从原理图到 ARES 进行 PCB 设计时遇到，一般问题在于：

- ◆ 有两个同名的器件，或未命名器件，例如两个电阻为 R？
- ◆ 脚本文件格式（如 MAP ON 表）不对。如果你不能立即判断出问题，请参考 ISIS 手册中的语法定义。

12.3 链接错误

模型链接发生在 ISIS 为基于等效电路建模的器件调用 MDF 文件。目前共同的问题是指定的器件没有模型文件。模型文件必须在当前设计目录下，或在 Module Paths 中 Set Paths 设置的路径下。

其他常见的链接错误包括：

- ◆ 值在参数映射表中找不到。意思是部件类型——如 CA3140 没有列在特定模型文件的映射表中。像 OA_MOS.MDF 这样的模型文件是为若干个有相同电路但不同参数的电路设计的。这个错误意味着该模型文件没有包括你指定的器件类型的参数，你可以用文本编辑器打开 MDF 文件看看建模了哪些器件。
- ◆ 无法解释的模块引脚。这是个警告而不是错误，意味着父器件体有一个引脚在模型中不存在。这个一般没有问题，例如，许多运放没有对调零和空脚建模。但是对一个新模型不能工作，这也许是个常见的错误，特别是同时出现 “No DC path to Ground” 的警告时。

12.4 分区错误

分区是一种机制，通过分区 ISIS 决定哪些部分电路需要仿真。这部分可能出现的问题有：

- ◆ 检测到循环依赖。意思是录音机的布置使得前后的分区相互依赖。假定你已经正确地设置了探针和激励源的 Isolate Before 和 Isolate After 标志，最简单的处理办法就是删去录音机，对全电路仿真。
- ◆ 没有指定模型。意思是提示的器件没有 PRIMITIVE、MODFILE 属性并出现在你的仿真电路中。如果器件与仿真无关（如连接器），那么你可以指定其属性 PRIMITIVE=NULL，否则你就需要一个模型！

详细信息参见[仿真模型的类型](#)。

12.5 电源范围定义错误

所有设计中的电源网络必须通过原理图或 Power Rail Configuration 绑定到电源上。如果存在未绑定的电源网络，仿真日志就会出现相应的错误信息。

请注意，在以下情况下，网络被定义为电源网络：

- ◆ 连接到电源端子的任何网络均被视为电源网络。
- ◆ 任何带有隐藏电源引脚的设备都将创建与引脚名称相同的电源网络。例如，一个 74LS00 将创建电源网络 VCC 和 GND。

注意连接电池或 SPICE 电压源到电源网络也不足以避免这个错误，你必须定义一个电源并绑定到 Power Rail Configuration 对话框中的网络。

12.6 仿真错误

仿真错误产生于 PROSPICE 而不是 ISIS，故常发生在网表文件成功产生之后。常见问题包括：

- ◆ 器件类型无法识别—意味着你设定了一个不支持的原型类型，或者一个模型文件使用了那样的原型。
- ◆ 没有对地直流通路—在[地和电源范围](#)中讨论。
- ◆ 没有发现探针—你试题参考一个不存在的探针或电压发生器。记住你必须使用 ASIMMDLS.LIB 中的 IPROBE 对象—你不能参考一个电流探针元件。
- ◆ 无法打开 SPICE 源文件—SPICEMODEL 属性设定的源文件无法找到。它必须在当前设计目录下或在 Module Path 中 the Set Paths 设置的路径下。
- ◆ 找不到模型库—你设定的 SUBCKT 或 MODEL 在磁盘指定的目录中找不到。
- ◆ 模型 DLL 文件找不到—设定的 VSM 模型 DLL 无法定位。它必须在当前设计目录下或在 Module Path 中 the Set Paths 设置的路径下。

12.7 收敛问题

最后一些问题与 SPICE 自己仿真电路时失败的原因有关。有一下三个基本的错误消息：

- ◆ 奇异矩阵。等价于未知数数目超过方程数，通常和绘图错误有关，或者某些需要设定初始值来确定起始状态的场合。

②注意：这个错误经常出现在“**No DC path to ground**”警告之前，你需要调查消息后面列出的引脚的连线情况。如果你的某部分电路没有接地，仿真器会解释为其电压是对地的——就那么简单。

- ◆ 多次迭代也不能收敛。这意味着电路的解是不稳定的。带有 VSWITCH 或 CSWITCH 原型的电路容易出现此问题，但是任何具有不连续传输函数的电路都会造成 SPICE 严重的问题。
- ◆ 时间步长太小。这意味着电路切换在这样一种状态：在充分小的时间步长下（典型如 $1\text{E}-18$ 秒）仍然无法产生可接受的足够小的电路电压变化。

②注意：这个经常由模型设计不当产生，或者没有提供给二极管或三极管足够的参数。特别是，如果没有正确地选择结电容值，这些器件会呈现零开关时间，那将直接导致这个错误消息。

大多数收敛错误是因为绘图错误或错误的模型——我们收到很多“不能仿真”的电路，却只发现某些地方少了连线。请以仿真日志为线索，在得出 PROSPICE 出错的结论前重新检查你的电路。

 **警告：**当使用第三方 SPICE 或 VSM 模型时，除非你提供简单的说明电路，否则我们是不可能花时间去调试它们的。

还要防止使用那些功能不被 SPICE2 或 SPICE3 支持的第三方 SPICE 模型。为 PSPICE 开发的模型具有的元件风格和语法结构都不是标准的 SPICE。

振荡器因为工作点的初始解失败而产生特殊的问题。总之，振荡器是没有稳态的！在[初始条件](#)部分讨论了使用 IC、NS 和 OFF 属性确定初始状态的方法。

如果问题确实在于数值收敛上，你可以尝试一下办法：

- ◆ 增加 GMIN 的值。这是反偏 PN 结的漏电阻，降低该值使电路看上去更接近于电阻网络（肯定有解）。但是以降低精度为代价。默认值是 $1\text{E}-12$ ；超过 $1\text{E}-9$ 的值将导致没有意义的结果。注意在任何情况下如果开始电路不收敛，SPICE3F5 会尝试 GMIN stepping。意思是较大的 GMIN 用于寻找初始解，然后 GMIN 回到原值以获得精度。
- ◆ 增加 ABSTOL 和/或 RELTOL 的值。这些值控制着仿真收敛的精度。然而，你使得误差变大，结果的精度实际上就降低了。
- ◆ 如果电路中使用了运放，尝试设置 MODFILE=OA_IDEAL，取代早期器件设定的类型。
- ◆ 降低 TRTOL 的值。这将使 SPICE 使用更小的步长，便不容易漏掉收敛解，但是代价是更长的运行时间。这个方法只用于瞬态分析仿真部分失败的情形。
- ◆ 如果绘出来的图线过于“锯齿”或含有数学噪声，你也可以尝试降低 TRTOL。这经常表现为某个值在快速电平变化时产生的振荡。